



1ST EDITION

Efficient Algorithm Design

Unlock the power of algorithms to optimize
computer programming

MASOUD MAKREHCHI



Efficient Algorithm Design

Unlock the power of algorithms to optimize
computer programming

Masoud Makrehchi



Efficient Algorithm Design

Copyright © 2024 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

The author acknowledges the use of cutting-edge AI, such as ChatGPT, with the sole aim of enhancing the language and clarity within the book, thereby ensuring a smooth reading experience for readers. It's important to note that the content itself has been crafted by the author and edited by a professional publishing team.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author nor Packt Publishing or its dealers and distributors will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Group Product Manager: Kunal Sawant

Publishing Product Manager: Samriddhi Murarka

Book Project Manager: Manisha Singh

Senior Editor: Esha Banerjee

Technical Editor: Kavyashree K S

Copy Editor: Safis Editing

Proofreader: Esha Banerjee

Indexer: Hemangini Bari

Production Designer: Vijay Kamble

DevRel Marketing Coordinator: Shrinidhi Manoharan

First published: October 2024

Production reference: 1111024

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83588-682-3

www.packtpub.com

To my daughter, Ava.

Contributors

About the author

Masoud Makrehchi is an associate professor at Ontario Tech University in Canada. He earned his Ph.D. in electrical and computer engineering from the University of Waterloo. With more than two decades of combined experience in industry and academia, Masoud approaches the dynamic fields of AI, machine learning, and natural language processing with a humble curiosity. Over the past 12 years, he has found deep satisfaction in guiding students through the challenging world of algorithms. His teaching philosophy emphasizes creating a collaborative learning environment where the exchange of ideas is as vital as the material itself. Masoud is committed to making the subject matter accessible and engaging, helping students develop a genuine appreciation for the art and science of algorithmic problem-solving.

I would like to extend my heartfelt thanks to my friends who knew I was writing this book and offered their unwavering support. I am also deeply grateful to all my students from the past 12 years. Your challenging questions and fresh perspectives in every class pushed me to continually explore new ways of teaching, understanding, and learning algorithms.

About the reviewer

Chase Thompson is a machine learning engineer, consultant, and accomplished researcher with experience in implementing machine learning (ML) and artificial intelligence (AI) solutions for several industries. With an education in computer science and business administration from North Carolina State University, he is adept at identifying and implementing business strategies in the field of AI/ML. Further, Thompson has contributed to research in computer science education by creating an automatic feedback system to help students improve their understanding of Java programming. He has worked for two innovative startups and helped develop AI-powered evaluation systems and training algorithms.

Table of Contents

Preface

xiii

Part 1: Foundations of Algorithm Analysis

1

Introduction to Algorithm Analysis 3

Understanding algorithms and problem-solving	4	The dual dimensions of algorithm analysis – efficiency and correctness	8
The rationale for algorithm analysis	6	Summary	10

2

Mathematical Induction and Loop Invariant for Algorithm Correctness 11

Mathematical induction	12	Summary	19
Loop invariants for the correctness of algorithms	16	References and further reading	20

3

Rate of Growth for Complexity Analysis 21

Unpacking the rate of growth in algorithms	22	Non-linear growth	26
Constant growth	23	Asymptotic notations	32
Sub-linear growth	23	Simplification rules in asymptotic notation	33
Linear growth	25	Asymptotic bounds	34
		Asymptotic upper bound (O notation)	35

Asymptotic lower bound (Ω -notation)	37	Complexities of NP, NP-complete, and NP-hard	42
Asymptotic tight bound (θ -notation)	39		
Confronting the unsolvable NP-hard problems	41	Summary	43
		References and further reading	43

4

Recursion and Recurrence Functions 45

Recursive algorithms	46	Divide-and-conquer recurrence functions	60
The basics of recursion	46	Unfolding recurrence functions	63
Types of recursion	48	Summary	65
Recurrence functions	56	References and further reading	65
Subtractive recurrence functions	57		

5

Solving Recurrence Functions 67

The substitution method	68	Case 3 – dominance of non-recursive work or root-heavy recursion trees	81
Iteration approach or unrolling the recurrence	68	Modified master theorem to solve subtracting recurrence functions	82
Guessing and induction approach	71	Master theorem limitations	84
Variable change approach	73	Alternative approaches	85
Recursion tree as a visualization technique	75	Beyond the master theorem – the Akra-Bazzi method	89
The master theorem	78	Why does it work? Intuition behind Akra-Bazzi	90
Case 1 – dominance of recursive calls or leaf-heavy recursion trees	79	Summary	91
Case 2 – balanced growth or balanced recursion trees	80	References and further reading	92

Part 2: Deep Dive in Algorithms

6

Sorting Algorithms 95

The taxonomy of sorting algorithms	96	Recursive sorting algorithms	114
Comparison	96	Merge sort	114
Recursion	99	Quick sort	117
Adaptability	102	Non-comparison-based sorting	121
Inversion	103	Counting sort	121
Memory usage	104	Radix sort	125
Iterative sorting algorithms	106	Bucket sort	127
Bubble sort	106	Summary	130
Selection sort	109	References and further reading	131
Insertion sort	111		

7

Search Algorithms 133

Properties of search algorithms	134	Hashing	148
Linear-time and logarithmic search algorithms	136	Hash functions	149
Linear or sequential search	137	Constant time search using hashing	150
Sub-linear search	139	Summary	170
		References and further reading	171

8

Symbiotic Relationship between Sort and Search 173

Striking the right balance between sorting and searching	173	Think like a computer scientist	178
Symbiotic link between sorting and searching	174	Summary	180
The efficiency dilemma – to organize or not?	177	References and further reading	180

9

Randomized Algorithms 181

A review of probabilistic algorithms	182	Case studies	195
Non-deterministic algorithms	184	Optimal selection in an online dating app	195
Analysis of randomized algorithms	186	Finding the closest parking spot	197
Monty Hall problem	187	Summary	199
Birthday paradox	188	References and further reading	199
Hiring secretary problem	191		

10

Dynamic Programming 201

Dynamic programming versus divide-and-conquer	202	Limitations of dynamic programming	219
Optimal substructure	203	Greedy algorithms – an introduction	220
Overlapping subproblems	207	Traveling salesman problem	221
Exploring dynamic programming	208	Heuristics and their role in greedy algorithms	222
Top-down versus bottom-up approaches for dynamic programming	211	Summary	225
Solving the 0/1 knapsack problem using dynamic programming	215	References and further reading	225

Part 3: Fundamental Data Structures

11

Landscape of Data Structures 229

Taxonomy of data structures	229	Dictionaries	237
Physical versus logical data structures	230	Insertion	239
Primitive versus composite data structures	231	Search	239
Linear versus non-linear data structures	232	Update	240
Static versus dynamic memory allocation	233	Deletion	240
Sequential versus random access	234	Summary	241
Abstract data types	235	References and further reading	241

12

Linear Data Structures 243

Lists	243	Stacks	261
Arrays	244	Queue	263
Linked lists	247	Deque	265
Skip lists	254	Summary	267
Insertion in skip lists	255	References and further reading	267
Search in skip lists	259		

13

Non-Linear Data Structures 269

Introduction to non-linear data structures	269	Tree representation	286
Graphs	271	BSTs	290
Graphs representation	274	Heaps	299
Traversing graphs	277	Heap operations	300
Trees	284	Heapsort	303
Different types of trees and their properties	284	Summary	304
		References and further reading	305

Part 4: Next Steps

14

Tomorrow's Algorithms 309

Learning from the past	309	Moral consciousness in software practice	319
Scalability	311	Final words	321
Context awareness	316	References and further readings	322
Moral responsibility	317		

Index	325
Other Books You May Enjoy	338

Preface

In the second chapter of this book, we explore the concept of the “loop invariant,” which refers to an attribute within a changing process that remains constant. If we look at human progress since the Industrial Revolution, despite the volatility and constant change, we can identify a few unchanging attributes. One is our drive to automate more and more, and the other is our systematic approach to problem-solving, which took a formal direction with the invention of the digital computer. This is what we now call algorithms.

This book is another contribution to the world of algorithms. It’s not meant to replace the existing and remarkable works that I have enjoyed reading and teaching. Instead, it offers an alternative approach to learning algorithms by merging theoretical concepts with practical applications. It also represents the culmination of 30 years of experience in research, industry, and academia, particularly in computer science and artificial intelligence, as well as 12 years of continuous teaching of algorithms to students.

If it were possible, I would add my students as co-authors, because I have learned more from their challenging questions than I could ever teach them. Their inquiries have pushed me to uncover the deeper explanations and logic behind every aspect of algorithms. Yes, I do mean every aspect. Algorithms are about making human-made machines function correctly and deliver the results we expect. Here, nothing short of a logical process and behavior is acceptable.

This book does not claim to cover every aspect or topic in the vast field of algorithms. Instead, it narrows its focus to the design and analysis of algorithms, seeking to address the essential questions we must always consider when evaluating them. First and foremost, can we prove that the algorithm functions correctly and as intended? It’s crucial to establish that the algorithm reliably produces the expected results for all possible inputs.

Next, we must ask how we can ensure the algorithm consistently behaves as we anticipate. This involves scrutinizing its logic, understanding its steps, and confirming that each part contributes to the overall goal. Finally, we need to consider whether the algorithm terminates within an acceptable timeframe. No matter how elegantly an algorithm is designed, if it doesn’t complete its task efficiently, it becomes impractical for real-world use. By concentrating on these fundamental questions, this book aims to guide you through the process of designing robust, reliable, and efficient algorithms.

Who this book is for

The primary audience for this book is mid-career software engineers, developers, information technology professionals, and research scientists. It serves as a practical manual for understanding and applying algorithms effectively in various professional contexts.

Although this book was not initially intended as a textbook, given its lack of exercises—which may be included in a future edition – it provides numerous examples that enhance learning. We believe it can be used as a primary or secondary textbook for courses such as Design and Analysis of Algorithms for students in computer science, software engineering, and other related engineering programs. The author, who has been teaching these courses for over a decade, wrote this book to tackle the challenges that often arise when explaining complex algorithmic concepts.

While the book is geared toward readers who have some familiarity with algorithms and a background in mathematics or engineering, certain chapters have broader appeal. *Chapters 1* through *5*, as well as *Chapters 8*, *9*, and *14*, discuss interesting real-life decision-making scenarios, making them accessible and engaging for a wider audience beyond the technical field.

What this book covers

Chapter 1, Introduction to Algorithm Analysis, offers a foundational understanding of algorithms as structured and systematic tools for problem-solving. It introduces the essential concepts of algorithm design and explores the dual nature of computer systems, distinguishing between hardware and software. The chapter also emphasizes the importance of algorithm analysis, focusing on both correctness and efficiency, setting the stage for a deeper exploration of algorithmic techniques in subsequent chapters.

Chapter 2, Mathematical Induction and Loop Invariant for Algorithm Correctness, discusses the mathematical foundations necessary for proving the correctness of algorithms. It introduces mathematical induction as a fundamental technique and extends this concept with loop invariants to ensure the reliability of iterative processes. The chapter provides practical examples to illustrate these principles, establishing a strong basis for validating algorithms and ensuring they produce correct results in all scenarios.

Chapter 3, Rate of Growth for Complexity Analysis, explores the concept of algorithm efficiency and how running times scale with input size. It introduces various growth rates, from constant to factorial time, and explains how to represent and compare them using asymptotic notations such as Big O, Omega, and Theta. This chapter lays the groundwork for understanding and predicting algorithm performance, helping you make informed decisions when designing or selecting algorithms for different tasks.

Chapter 4, Recursion and Recurrence Functions, introduces the concept of recurrence functions, essential for analyzing the complexity of recursive algorithms. The chapter explores two primary types of recurrence functions: subtractive and divide-and-conquer, detailing how each impacts the efficiency of algorithms. Through practical examples such as merge sort and binary search, it demonstrates the application of recurrence relations in real-world scenarios.

Chapter 5, Solving Recurrence Functions, discusses in detail the techniques for analyzing the performance of recursive algorithms. It introduces key methods such as the substitution method, the master theorem, and recursion trees to solve recurrence relations that arise in divide-and-conquer and other recursive algorithms. The chapter explains how to apply these methods to determine the asymptotic complexity of an algorithm, highlighting the strengths and limitations of each approach.

Chapter 6, Sorting Algorithms, explores the fundamental techniques for arranging data in a specific order, offering a comprehensive overview of both iterative and recursive sorting methods. The chapter begins with a detailed examination of comparison-based algorithms, highlighting their operational principles and time complexities. It further explores advanced sorting techniques such as merge sort, explaining the divide-and-conquer approach and its implications for stability and efficiency. Additionally, the chapter introduces non-comparison-based sorting algorithms.

Chapter 7, Search Algorithms, begins by examining different types of search algorithms, including linear and sublinear methods, highlighting their characteristics and performance. The chapter then introduces the concept of hashing, explaining how hash functions can be used to achieve constant-time search operations.

Chapter 8, Symbiotic Relationship Between Sort and Search, provides an overview of the interconnected nature of sorting and searching algorithms. It examines how sorting can significantly enhance the efficiency of search operations and explores scenarios where sorting is advantageous. The chapter presents a comprehensive analysis of the trade-offs between the cost of sorting and the benefits of faster searching, using real-world examples to highlight these dynamics.

Chapter 9, Randomized Algorithms, is about the use of randomness in algorithm design to tackle problems that deterministic approaches may struggle to solve efficiently. Through various case studies, including the hiring problem and the birthday paradox, this chapter demonstrates how randomized algorithms can effectively navigate uncertainty and enhance decision-making processes.

Chapter 10, Dynamic Programming, begins with the fundamental principles that distinguish dynamic programming from other methods, such as divide-and-conquer and greedy algorithms. The chapter presents dynamic programming through detailed examples. This chapter provides a thorough understanding of dynamic programming's advantages and limitations, preparing you for its practical application in various problem-solving scenarios.

Chapter 11, Landscape of Data Structures, explores how different types of data structures, such as linear and non-linear, static and dynamic, as well as those supporting sequential and random access, can significantly impact algorithm performance. This foundational knowledge sets the stage for a more in-depth exploration of specific data structures and their algorithmic applications in the following chapters.

Chapter 12, Linear Data Structures, provides an in-depth exploration of fundamental structures such as arrays, linked lists, stacks, queues, and deques. This chapter will guide you through the essential operations and characteristics of each data structure, highlighting their unique benefits and trade-offs. It also introduces advanced concepts such as skip lists.

Chapter 13, Non-Linear Data Structures, discusses non-linear data structures. The chapter begins with an exploration of the general properties that distinguish non-linear structures from linear ones. It covers two major categories – graphs and trees – examining their types, properties, and applications. The chapter concludes by introducing heaps, a specialized form of binary trees, demonstrating their significance in sorting algorithms and priority queues.

Chapter 14, Tomorrow's Algorithms, explores the cutting-edge trends that are defining the future of computing. The chapter categorizes these emerging trends into three key areas: scalability, context awareness, and moral responsibility. It explores how algorithms are evolving to address the demands of large-scale data processing, adapt to dynamic environments, and operate ethically.

To get the most out of this book

To effectively use the code in this book, you will need the latest version of Python and a suitable **integrated development environment (IDE)** for developing and testing the code. We recommend using the Anaconda Python environment due to its ease of use and comprehensive package management. Anaconda comes with many pre-installed libraries and tools that are useful for algorithm development and analysis. You can download and install Anaconda from the official website: <https://www.anaconda.com/download>.

Once installed, you can use Anaconda's IDE (Spyder) or Jupyter Notebook to run the code examples provided in this book. Both tools offer an interactive environment that is particularly helpful for learning and experimenting with algorithms. Additionally, you may want to explore other IDEs such as PyCharm or Visual Studio Code, which offer powerful features for Python development.

If you are using the digital version of this book, we advise you to type the code yourself or access the code from the book's GitHub repository (a link is available in the next section). Doing so will help you avoid any potential errors related to the copying and pasting of code.

Download the example code files

You can download the example code files for this book from GitHub at <https://github.com/PacktPublishing/Efficient-Algorithm-Design>. If there's an update to the code, it will be updated in the GitHub repository.

We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Conventions used

There are a number of text conventions used throughout this book.

`Code in text`: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter/X handles. Here is an example: "In the preceding function, the running time depends on the number of times the `for i in range(1, n + 1) : loop` is executed."

A block of code is set as follows:

```
def dp_fib(n, memo={}):  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        return n  
    memo[n] = dp_fib(n-1, memo) + dp_fib(n-2, memo)  
    return memo[n]
```

Any command-line input or output is written as follows:

```
pip install networkx matplotlib
```

Bold: Indicates a new term, an important word, or words that you see onscreen. For instance, words in menus or dialog boxes appear in **bold**. Here is an example: “In *Chapter 13*, we will discuss a search algorithm based on a specific data structure called **Binary Search Trees (BSTs)**.”

Tips or important notes

Appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book, email us at customer@packtpub.com and mention the book title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you would report this to us. Please visit www.packtpub.com/support/errata and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit authors.packtpub.com.

Share Your Thoughts

Once you've read *Efficient Algorithm Design*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



`https://packt.link/free-ebook/9781835886823`

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly

Part 1:

Foundations of

Algorithm Analysis

This part lays the groundwork for understanding and analyzing algorithms. It begins with fundamental concepts, focusing on how to prove algorithm correctness and measure efficiency. You'll learn crucial techniques such as mathematical induction, loop invariants, and complexity analysis, setting the stage for more advanced topics.

This part includes the following chapters:

- *Chapter 1, Introduction to Algorithm Analysis*
- *Chapter 2, Mathematical Induction and Loop Invariant for Algorithm Correctness*
- *Chapter 3, Rate of Growth for Complexity Analysis*
- *Chapter 4, Recursion and Recurrence Functions*
- *Chapter 5, Solving Recurrence Functions*

Introduction to Algorithm Analysis

The goal of this book is to demystify algorithms, making them accessible and actionable for readers who wish to enhance their understanding of algorithmic design, analysis, and application within various fields of technology. Although designed for software engineers, computer scientists, and other professionals familiar with algorithms and eager to enhance their skills, this book is equipped with sufficient depth and resources to provide a head start for early-career professionals through more practice and effort. By exploring both the theoretical underpinnings and practical implementations, this book aims to bridge the gap between academic study and real-world technological applications.

In this opening chapter, we explore the essential nature of algorithms, defining them as structured, systematic tools crucial for problem-solving in computing and other fields. We explore the significant role of algorithms through a detailed examination of the hardware-software dichotomy and their unique characteristics, highlighting the importance of algorithm analysis for both academic and practical applications in a rapidly evolving tech landscape where hardware is increasingly affordable. The chapter sets a foundational roadmap, guiding you through complex algorithmic concepts toward a comprehensive understanding that prepares you for advanced topics and practical applications. This introduction marks the beginning of a deeper journey into mastering algorithm analysis and its applications.

In this chapter, we will cover the following primary topics:

- Algorithms and problem-solving
- The rationale for algorithm analysis
- The dual dimensions of algorithm analysis – efficiency and correctness

Understanding algorithms and problem-solving

René Descartes (1596-1650), the French philosopher and mathematician, is renowned for his theory of mind-body dualism. He proposed that the mind and body are two fundamentally distinct substances: the mind as a non-extended, thinking entity and the body as an extended, non-thinking entity. Descartes believed in the interaction between these substances while maintaining their separateness and independent existence. This dualistic view emphasizes the separation of the mental (mind) and physical (body) aspects, positing that they differ in nature. His theory has had a significant influence on philosophical discussions about consciousness and the relationship between mind and body.

But why are we starting our discussion about algorithms with Descartes' *theory of dualism*, despite serious criticisms of it from philosophers? The answer lies in the model's ability to aid our understanding of the essence of algorithms and their uniqueness among all human inventions.

The historical narrative of computers, primarily seen as devices for automating problem-solving through mathematical representations, starts with a *distinct separation between hardware and software*. Hardware is the tangible, physical component that executes software code, producing the desired results. Conversely, software is the systematic solution articulated through a formal language known as a computer program, ranging from high-level languages such as Python and C++ to low-level ones such as assembly language and machine code.

However, hardware and software are fundamentally different, a distinction particularly notable in computer systems. The primary difference lies in the disciplines that govern each. Computer hardware is governed by the laws of physics, dictating how the physical components operate and interact. In contrast, computer software operates within the domain of mathematics, which dictates the logic, algorithms, and functions that software can perform. This dichotomy sets computer systems apart from other human-made technologies. For example, a car is governed entirely by physical laws at both microscopic and macroscopic levels. A significant distinction between hardware and software is that hardware is mortal. It is prone to decay, defects, and expiration. In contrast, software is immortal, not subject to depreciation, aging, defects, or expiration. This concept mirrors Descartes' theory of dualism.

Software, at its core, embodies an abstract concept known as an **algorithm**. An algorithm represents an abstract set of rules or procedures, which can be realized and expressed in many ways across various programming languages. Despite this diversity in representation, all these different implementations of the same algorithm are designed to produce a singular, consistent output. This characteristic of algorithms, being abstract yet versatile in their implementation, is what makes them a fundamental element in software development and design. In the real world, the concepts most akin to algorithms are food recipes and musical notes. Both represent step-by-step implementations of a *plan* to create food or music.

The indication of a good recipe, aside from yielding delicious food, is in its expression in a quantitative and abstract form, allowing any cook, regardless of experience level, to execute it. However, this is often not entirely feasible, as recipes are typically written in natural language, making them prone to various interpretations. Another desired trait of a good recipe is its independence from specific

kitchen equipment, although this too is not always realistic. Recipes, much like algorithms, aim for a level of universality in their application, but the nuances of language and specific contexts can affect their reproducibility and outcomes.

The situation with musical notes is slightly more favorable. Since these notes resemble formal language, they offer a clearer, more standardized method of instruction. However, the actual music produced is still subject to the interpretation of the player, the characteristics of the instruments, and a myriad of acoustic factors. While musical notes provide a more precise guide compared to the natural language of recipes, the variability in performance and environmental conditions means that the outcome can still vary significantly. This underscores the challenge of translating abstract, formalized instructions into consistent real-world results, much like the execution of algorithms in different computing environments.

These two examples help us infer key properties of algorithms:

- **Programmer independence:** Ideally, the final product of an algorithm should be largely independent of who implements it. This means that regardless of the programmer, the algorithm should consistently produce the same correct result, and the computational cost or resource usage should be comparably similar across different implementations.
- **Hardware independence:** An effective algorithm should, as much as possible, be independent of the hardware on which it runs. It should be capable of producing consistent results across various hardware platforms without significant modifications or dependence on specific hardware features.
- **Abstraction and clarity:** Algorithms should be abstract and unambiguous, leaving no room for interpretation. This clarity ensures that the algorithm can be understood and implemented consistently, regardless of the programmer's subjective understanding or approach.
- **Quantifiable correctness and cost:** The correctness of an algorithm – its ability to produce the intended result – and its cost, in terms of computational resources such as time and memory, must be quantifiable. This allows for the objective assessment and comparison of different algorithms based on their efficiency and effectiveness.

Algorithms are defined as step-by-step, procedural, and often iterative methods for solving problems. It is crucial to understand, however, that they are designed to address only **computable** problems. This means that for a problem to be solvable by an algorithm, it must be one that can be resolved through a sequence of logical and mathematical steps. In essence, a computable problem is one that can be systematically worked through using algorithms, which provide a clear and organized approach to finding a solution.

One classic example of a **non-computable problem** is the **halting problem**. Posed by Alan Turing, this problem asks whether it is possible to create an algorithm that can determine whether any given program and its input will halt (stop running) or continue to run indefinitely. *Turing proved that no such algorithm can exist*; it is impossible for a general algorithm to predict the behavior of all possible

program-input combinations and determine whether they will halt. This is because the algorithm would have to account for an infinite number of possible program behaviors, which is not feasible.

A classic example of a computable problem is sorting a list of numbers. For instance, given a list of numbers, such as [3, 1, 4, 1, 5, 9, 2], a sorting algorithm can rearrange these numbers in a specific order, such as ascending: [1, 1, 2, 3, 4, 5, 9]. Sorting problems are computable because they have a well-defined procedure or set of steps that can be followed to achieve the sorted list, and this process will work for any finite list of numbers.

Having established a basic understanding of the types of problems that can be solved using algorithms, we are now ready to explore the major problem-solving approaches employed in algorithm design. However, it is important to address a common misconception first. Some textbooks present heuristics and algorithmic approaches as opposites, but the relationship between them is more complementary than contradictory.

Heuristic methods, as we will discuss in *Chapter 10*, provide practical, often quicker solutions grounded in experience, intuition, or common-sense rules. Their main advantage lies in their speed and practicality, but this comes with a trade-off: heuristics do not always guarantee an optimal or correct solution. On the other hand, an algorithmic approach follows a set of defined, structured steps leading to a definite solution, which is often the most optimal for the problem at hand. Based on mathematical and logical procedures, algorithmic methods offer predictability, repeatability, and guaranteed outcomes, making them reliable in situations where accuracy is paramount. As we will explore in detail in *Chapter 10*, heuristics and algorithms have a symbiotic relationship. While each has its own strengths and weaknesses, they can often complement each other to effectively tackle a wide range of problems. Understanding when and how to use each approach is a key skill in the art of algorithm design.

The rationale for algorithm analysis

In the last two decades, we have witnessed extraordinary advancements in advanced computational systems, particularly in fields such as Artificial Intelligence (AI), Machine Learning, Deep Learning, Robotics, and Computer Vision. This progress owes more to two significant revolutions in the tech world and information society than to improvements in algorithm design. The first revolution was the availability of vast amounts of data following the public release of the Internet in 1991. The second was the tremendous advancements in hardware, including the development of powerful yet affordable processors such as **Graphical Processing Units (GPUs)** and the creation of ultra-high-capacity memory and storage solutions.

Despite these remarkable improvements in computational resources, the study and analysis of algorithms remain critically important for several reasons:

- **Algorithm correctness:** Irrespective of the abundance of computational resources, it is vital to mathematically prove an algorithm's correctness in all scenarios. Demonstrating effectiveness through examples is not sufficient; a mathematical framework is needed for rigorous proof. This ensures the reliability of the algorithm across various conditions and inputs.

- **Algorithm efficiency:** Algorithm analysis is key to understanding the efficiency of different algorithms. By examining time and space complexity, we can select or design algorithms that are not only faster but also more resource-efficient. This is especially important in environments with limited resources or when dealing with large datasets. Furthermore, when several algorithms are available for the same task, analysis aids in making informed decisions about which to use.
- **Better problem-solving skills:** Deep knowledge of how algorithms function and how to analyze them improves problem-solving capabilities. This involves methodically breaking down problems into smaller components and devising optimal solutions, a skill valuable in many areas beyond computing.
- **Understanding limitations:** Understanding an algorithm's limitations is as important as finding the most efficient solution. Algorithm analysis helps recognize what problems an algorithm can or cannot solve effectively, which is crucial when dealing with time, memory, or specific data structure constraints.
- **Preparing for future challenges:** The landscape of technology and data is continuously evolving, growing in size and complexity. A solid foundation in algorithm analysis equips us to effectively face and tackle these emerging computational challenges.

Hence, we can say that while technological advancements have significantly boosted computational power, the relevance of algorithm analysis persists, ensuring that we continue to develop solutions that are not just fast but also correct, efficient, and suited to the ever-evolving complexities of modern problems.

Let's conduct a thought experiment. Imagine a scenario where computational resources are limitless, with incredibly fast processors and virtually free memory. Consider an array, A , containing a very large number of elements, n . Our objective is to sort these elements in ascending order. To minimize the complexity of algorithm design and analysis, we might opt to generate all possible permutations of A and check each one to see whether it is sorted. This brute-force approach requires generating $O(n!)$ permutations. However, even the most basic sorting algorithms, such as bubble sort, operate at a complexity of $O(n^2)$, and more advanced sorting algorithms can offer even greater efficiency. This example illustrates how a deep understanding of algorithm analysis can significantly alter our approach to solving problems, emphasizing the importance of efficient algorithm design even in a world abounding with computational resources.

In this book, we explore four primary algorithmic problem-solving methods, each with its unique advantages, limitations, and applicability to different types of problems:

- **Sequential or straightforward approach:** This method is the most fundamental form of problem-solving. It involves a linear sequence of instructions, often including loops and decision points. While sequential algorithms are relatively simple to test and debug, they can be computationally expensive, making them less efficient for complex tasks.

- **Divide-and-conquer:** This approach tackles a problem by breaking it down into smaller, more manageable subproblems. Each subproblem is solved independently, and then their solutions are combined to address the original, larger problem. However, converting a sequential algorithm into a divide-and-conquer strategy is not always advantageous. A classic example of where it might not be beneficial is the factorial problem, where the sequential approach is more straightforward (see *Chapter 4*).
- **Dynamic programming:** Dynamic programming addresses a problem by decomposing it into smaller subproblems, solving each of these recursively. A key requirement for dynamic programming is that the subproblems must overlap, allowing the method to efficiently reuse previously computed solutions. The limitation of this approach lies in its necessity for overlapping subproblems, which may not always be present (see *Chapter 10*).
- **Greedy algorithms:** Greedy algorithms focus on finding the optimal solution from a set of possible solutions. They make the best choice at each step, aiming for a global optimum. The challenge with greedy algorithms is that they may not always lead to the best overall solution, as making the locally optimal choice at each step doesn't necessarily result in the globally optimal solution (see *Chapter 10*).

In the subsequent chapters of this book, we will explore each of these problem-solving methods in detail. Our exploration will focus on the benefits, limitations, and real-world applications of each approach. It's important to note that a key criterion for comparing these algorithms is their computational cost.

Now that we've established our expectations from this book, the primary question remains: what do we seek from algorithm analysis? While "analysis" can encompass a broad spectrum of concepts, in this context, it specifically pertains to the crucial goals of designing efficient algorithms. Firstly, we aim to develop algorithms that are absolutely correct, meaning they consistently produce the expected results. Secondly, we strive to design algorithms that are as cost-effective as possible. Therefore, in the process of algorithm analysis, we concentrate on two key dimensions: correctness and cost.

The dual dimensions of algorithm analysis – efficiency and correctness

The objective of algorithm analysis is indeed twofold. Firstly, it seeks to ascertain the correctness of an algorithm. An algorithm is considered correct if it consistently solves the intended problem and produces the right output for all valid inputs. This correctness hinges on two critical criteria:

- **Termination:** An algorithm must always conclude after a finite number of steps. It should not fall into perpetual loops or continue indefinitely, regardless of the input provided.
- **Validity:** The algorithm must yield the expected outcome or a valid solution for every possible input. It needs to adhere to the problem's specifications and requirements precisely.

Interestingly, demonstrating an algorithm's correctness cannot be conclusively achieved merely by testing it with numerous positive examples. While many successful test cases may suggest correctness, it takes only a single counterexample to disprove it. This approach, known as **indirect proof**, plays a crucial role in algorithm analysis.

The second approach of algorithm analysis involves **direct proof methods**, such as inductive reasoning or mathematical induction. This approach entails confirming that the algorithm functions correctly for a basic case (usually the simplest input) and then proving that if it works for an arbitrary case, it will continue to work for subsequent cases. Another key method for proving correctness is the loop invariant condition, which establishes certain conditions that hold true during each iteration of a loop within the algorithm. This is often the primary method for demonstrating an algorithm correctness.

In *Chapter 2*, we will deeply explore the concept of algorithm correctness. We'll explore these proof methods in detail, providing a thorough understanding of how to establish and verify the correctness of algorithms.

The second crucial aspect of algorithm analysis is the evaluation of an algorithm's efficiency, which is pivotal in determining how effectively an algorithm performs under varying conditions. Efficiency in algorithms is primarily gauged by two main criteria: time and space. Time efficiency refers to the amount of computational time required by an algorithm to complete its execution. Space efficiency concerns the amount of memory an algorithm needs to complete its tasks:

- **Time efficiency or computational complexity:** This relates to how much time an algorithm takes to solve a problem, particularly as the size of the input data grows. It's essential to understand the time complexity of an algorithm to determine how efficiently it can handle increasingly large datasets.
- **Space efficiency:** This pertains to the amount of memory an algorithm requires during its execution. Estimating memory usage is crucial, especially for data-intensive tasks or environments with limited memory resources.

To analyze these aspects, we employ the theory of algorithms, also known as **asymptotic analysis**. Asymptotic notations such as Big O , Θ , and Ω provide a framework to describe the behavior of an algorithm's time and space complexity in relation to the size of the input. These notations offer a way to express the upper and lower bounds of the algorithm's resource requirements, allowing us to make theoretical assessments of its efficiency under different conditions.

In *Chapter 3*, we will explore the theory of algorithms and asymptotic notations in detail. This will equip us with the tools to evaluate and compare the efficiency of different algorithms, an essential skill for selecting the right algorithm for a given problem based on its performance characteristics.

It is important to acknowledge that there are indeed other dimensions to consider when assessing an algorithm's efficiency, especially in the context of modern computing and diverse application environments:

- **Battery and energy consumption:** In mobile applications, the efficiency of an algorithm can significantly impact battery life. Algorithms that demand less processing power can help conserve battery, a crucial factor in mobile computing.
- **Data transfer and network access:** For algorithms requiring data transfer and network connectivity, the amount of data communicated and the frequency of network access become key efficiency factors. This is particularly relevant in applications where network bandwidth is limited or costly.
- **Cloud-based services:** In scenarios where algorithms rely heavily on cloud-based services, the cost associated with these services must be considered. This includes not just computational costs but also data storage and transfer costs in the cloud environment.
- **Human annotation in AI and Machine Learning:** Some algorithms, particularly in AI and Machine Learning, may require human annotation or intervention. The time and effort involved in this process can also be a critical factor in the overall efficiency and practicality of these algorithms.

While these aspects are indeed significant in various contexts, the primary focus of this book is to address algorithmic efficiency in terms of time and space. This focus allows for a more in-depth exploration of these two fundamental and universally applicable criteria, providing a solid foundation for understanding and evaluating the performance of algorithms across a wide range of applications.

Summary

This chapter introduced the fundamental nature of algorithms, emphasizing their role as structured and systematic tools for effective problem-solving across various fields. It began with a comprehensive definition and explored the distinct aspects of algorithms, including their dependence on the dual nature of hardware and software, influenced by physical laws and mathematical principles, respectively. The chapter focused on the critical importance of algorithm analysis, which revealed the necessity of evaluating algorithms, particularly as hardware becomes more accessible and cost-effective. It sets the stage for you to explore the complexities and applications of algorithms through guided and structured learning paths. This introductory chapter served as a foundational step toward mastering the art of algorithm analysis and application. The next chapter will establish the essential mathematical foundation that every software practitioner needs for effective algorithm analysis.

2

Mathematical Induction and Loop Invariant for Algorithm Correctness

Rene Descartes famously said, “*With me, everything turns into mathematics*,” emphasizing the profound connection between mathematics and various intellectual pursuits. This sentiment is especially relevant in the field of computer science, where the design, efficiency, and validation of algorithms are deeply rooted in mathematical principles. In this chapter, along with the next, we will explore the mathematical foundations that underpin algorithm analysis. We will discuss how these mathematical concepts not only guide the development of algorithmic solutions but also ensure their effectiveness and accuracy. This discussion aims to provide a comprehensive understanding of how integral mathematics is to the art and science of algorithm design and analysis.

In this chapter, we explore the concept of mathematical induction and its crucial role in establishing loop invariants, a framework for evaluating algorithm correctness forming the foundation for proving algorithm correctness. Through a series of detailed examples, we explain the concept of loop invariants, depicting how they function and their significance. This discussion sets the groundwork for applying these concepts in subsequent chapters, enhancing your ability to rigorously verify the techniques for algorithm correctness. This exploration not only serves as a theoretical framework but also provides practical insights, equipping you with essential tools for effective algorithm design and analysis. In this chapter, the following topics will be explored:

- Mathematical induction
- Loop invariants for the correctness of algorithms

Mathematical induction

Searching for counterexamples is one of the strategies when questioning the correctness of certain propositions or algorithms. This approach is particularly useful in cases where conventional methods of proving correctness are challenging. A counterexample is essentially a specific instance or case that demonstrates the falsehood of a claim or the incorrectness of an algorithm. When a proof seems complex or the logic behind an algorithm is convoluted, identifying a counterexample can provide a more straightforward means of disproving the claim. This approach is also called **indirect proof**.

Often, a counterexample can be easily observed. In such instances, it serves as a quick and efficient way to invalidate a hypothesis, bypassing the need for a lengthy and intricate proof. However, there are situations where finding a counterexample is a challenging task. If after diligent search, a counterexample remains difficult, it might indicate that approaching the problem with a direct proof of correctness is more appropriate. It's crucial to note, though, that the inability to find a counterexample does not inherently prove the correctness of an algorithm. Just because a counterexample is hard to find or non-apparent, does not mean the algorithm is valid. This underscores the importance of rigorous analysis and testing in algorithm development and evaluation, ensuring that claims of correctness are not prematurely accepted without thorough examination. Now, let's look at the following example.

Example 2.1:

Prove or disprove: $\forall a > 1 \forall b > 1: (a + b)! = a! b!$

Let $a = 2$ and $b = 3$. Then, we must show the following:

$$(2 + 3)! = 2! * 3!$$

This is false.

As previously highlighted, indirect proofs do not guarantee the validity of a relationship or algorithm. This underscores the necessity for **direct proof techniques** in algorithm analysis. In contrast to indirect proofs, where we seek counterexamples to disprove a hypothesis, direct proofs involve affirmatively demonstrating that a relationship or algorithm holds true under all possible conditions.

For instance, in an indirect proof, finding just one counterexample is sufficient to invalidate the claim. However, in direct proofing, especially when dealing with natural numbers, it is necessary to consider all natural numbers from $n=0$ to ∞ . This exhaustive approach requires a clear definition of the range and type of n . In the context of algorithm analysis, n typically represents the number of cases we examine as we progress through the proof. Since the number of cases cannot be fractional and must always be a whole number, n is defined as a natural number. Furthermore, the range of n extends from 0 to infinity to ensure that every possible scenario is accounted for, verifying that no case exists that could falsify the algorithm's correctness. This rigorous method ensures a comprehensive validation of the algorithm's reliability across all conceivable cases. This is the exact objective of mathematical induction.

Flashback

The concept of **mathematical induction** has a rich history stretching back over 3,000 years. The principle behind mathematical induction is that if a hypothesis holds true at the boundary of a range and continues to hold true within that range, then the hypothesis can be considered valid throughout. This method is particularly efficient for proving mathematical sequences and has a direct relevance to computer algorithms.

In the context of computer algorithms, mathematical induction can be seen as a powerful tool for validating algorithm correctness. An algorithm can be viewed as a sophisticated sequence where the input, n , represents the case number or the index of a loop, and the sequence embodies the functionality expected from the algorithm.

In the process of mathematical induction, the proof begins at a boundary, typically when $n = 0$. From there, the task is to demonstrate that if the hypothesis is valid for some arbitrary natural number $n = k$, then it is also true for $n = k + 1$. Once this has been established for k (which could be any natural number), it effectively proves the hypothesis for all natural numbers $n \in \mathbb{N}$. This step-by-step approach, progressing from one natural number to the next, underlines the reliability and applicability of mathematical induction in both pure mathematics and algorithm design.

Mathematical induction can be succinctly described in two straightforward steps:

1. **Base case (initial step):** This involves evaluating and proving that the hypothesis holds true for the smallest value of n , typically $n = 0$ or $n = 1$, depending on the specifics of the problem. In the context of algorithm design, the base case is usually $n = 1$ since testing an algorithm typically begins with the first possible, non-void instance. This step establishes the validity of the hypothesis at the starting point of the sequence.
2. **Induction step:** This step requires proving that if the hypothesis is valid for some arbitrary case number n , then it remains valid for $n + 1$. This process, often called the assumption or inductive hypothesis, involves showing that the truth of the hypothesis for one case logically extends its truth to the next case. By successfully demonstrating this, we establish that the hypothesis holds for all subsequent cases, up to infinity.

In Example 2.2, we demonstrate the process of mathematical induction step by step.

Example 2.2:

Use mathematical induction to prove the following proposition.

The sum of the first natural numbers is given by the formula:

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Proof:

1. **Base case:** For $n = 1$, the sequence contains only one element, which is 1. According to the formula:

$$1 = \frac{1(1+1)}{2} = \frac{2}{2} = 1$$

The base case holds true as both sides of the equation are equal.

2. **Induction step:** Assume the proposition holds for $n = k$; that is, assume that $1 + 2 + 3 + \dots + k = \frac{k(k+1)}{2}$.

Now, prove it for $n = k + 1$. Thus, consider the sum:

$$1 + 2 + 3 + \dots + k + (k + 1)$$

According to the induction hypothesis, the sum of the first k numbers is $\frac{k(k+1)}{2}$.

Adding $k + 1$ to both sides give the following:

$$1 + 2 + 3 + \dots + k + (k + 1) = \frac{k \times (k + 1)}{2} + (k + 1) = \frac{k \times (k + 1) + 2(k + 1)}{2} = \frac{(k + 1)(k + 2)}{2}$$

This shows that if the proposition is true for $n = k$, it is also true for $n = k + 1$.

You might wonder about the link between mathematical induction and algorithm correctness. To address this, we must first examine the architecture of an algorithm. Fundamentally, an algorithm is a "sequence" – much like a cooking recipe, it is a series of commands designed to solve a problem. Every algorithm comprises various types of commands, which can be broadly categorized as follows:

- **Simple commands:** These are instructions not dependent on the input size. An example of this could be a basic assignment, such as $x = 0$. Despite potentially involving complex arithmetic or logic operations, these commands remain independent of the input size. We refer to these as **scale-free commands**. Generally, the correctness of these commands is straightforward and often assumed, given their simplicity.
- **Compound commands:** This category includes blocks of other commands, which could be simple commands or additional compound commands. A compound command might be encapsulated within a function call or consist of a block of simpler commands. However, the most notable forms of compound commands are **selection blocks** and **iterations**:
 - **Selections:** These commands in an algorithm modify the control flow or the direction of logic within the program. The most commonly used selection commands are the `if-then` instruction blocks. The correctness of these blocks is crucial and depends on the logical structure of the algorithm, necessitating careful analysis to ensure they behave as intended.

- **Iteration:** If we analogize an algorithm to a vehicle, then simple commands represent the body and all stationary parts of the vehicle, while selection components are akin to the steering mechanisms. The movement of the vehicle, however, depends on the engine, similar to how the operation of an algorithm relies on its iterative components. While all other components may be scale-free, loops define the scale of the algorithms – in other words, their costs and complexities. Furthermore, the correctness of an algorithm heavily relies on these iterative components.

In essence, the relationship between mathematical induction and algorithm correctness becomes apparent when considering iterative components. Mathematical induction is akin to proving that if a vehicle (algorithm) operates correctly over a given distance (for a certain number of iterative steps), then it will also operate correctly for the next increment of distance (the next step in the iteration). This method of proof is ideally suited for verifying the behavior of loops and recursive procedures within algorithms, ensuring they meet their specifications for all potential scenarios.

The following Python code implements an iterative binary search algorithm, which is designed to find the position of a target value within a sorted array. In the following code, we provide a detailed walk-through of the code and highlight the specific roles of the simple commands, iteration block, and selection block. Finally, we will demonstrate how to test the function:

```
def binary_search(ar, x):
    low = 0
    high = len(ar) - 1
    mid = 0
    while low <= high:
        mid = (high + low) // 2
        if ar[mid] < x:
            low = mid + 1
        elif ar[mid] > x:
            high = mid - 1
        else:
            return mid
```

Let's analyze each command block in the binary search algorithm:

- **Simple commands:**

- `low = 0`: Initializes the lower boundary of the search area
- `high = len(ar) - 1`: Sets the upper boundary based on the length of the array
- `mid = 0`: Initializes the midpoint variable

These commands set up the initial conditions for the search and do not depend on the input size directly. They're executed once before the loop begins.

- **Iteration block:** The `while` loop (`while low <= high:`) continues as long as there is a portion of the array left to consider, defining the complexity and also impacting the correctness of the algorithm by ensuring every element within the search boundaries is considered.
- **Selection block:** Inside the loop, based on the comparison of `ar[mid]` with `x`, the `low` and `high` bounds are adjusted, narrowing down the search space. The `if` and `elif` conditions adjust the search boundaries (`low` and `high`), while the `else` condition handles the scenario where the target is found, directly affecting the flow and decision-making within the algorithm.

The binary search example was used to demonstrate why the correctness of an algorithm (and its cost, which will be discussed in the next chapter) is often assessed based on the algorithm's iterative components. With this in mind, a significant challenge arises when applying mathematical induction to verify an algorithm's correctness: ensuring that the algorithm eventually terminates rather than running indefinitely. This necessity to confirm that the algorithm still functions correctly upon termination is why we introduce a concept specifically tailored to computer algorithms known as loop invariant. This adaptation of mathematical induction helps ensure that despite the iterative nature of an algorithm, it consistently produces the correct output and terminates as expected.

Loop invariants for the correctness of algorithms

The **loop invariant** is a concept that extends mathematical induction, tailored specifically for analyzing algorithms. It adjusts the traditional steps of mathematical induction to better suit the structure of iterative processes in computer algorithms, as outlined in *Table 2.1*.

Mathematical induction	Steps in loop invariant
Base case	Initialization: Corresponds to the base case in mathematical induction. Here, it is verified that the relation of interest holds true before the iterative loops begin.
Induction step	Maintenance: Analogous to the induction step in mathematical induction. This step ensures that if the relation holds true at the beginning of an iteration, it must also hold true at the end of that iteration.
<i>In mathematical induction, we prove the hypothesis for an infinite number of cases, and it does not involve a termination step.</i>	Termination: An additional step not typically part of mathematical induction. This step confirms that the relation of interest continues to hold true after the loop terminates.

Table 2.1: Comparison of steps in mathematical induction and loop invariant

These three steps — *initialization*, *maintenance*, and *termination* — form the core of loop invariant theory and are crucial for proving the correctness of algorithms involving loops. The approach ensures that the algorithm not only starts off correctly but also maintains its correctness throughout the execution and concludes with the desired outcome, thus verifying both its functional and operational integrity.

To better understand the concept of a loop invariant, let's use a practical example involving a school bus journey from a starting station to the school, picking up students at each station without any students getting off. We'll use this scenario to demonstrate how some elements change (the station) and others remain constant (the number of passengers) during the loop, and how this relates to loop invariants in algorithmic loops.

Example 2.3

A school bus starts its route with all seats empty. At each station, it stops to pick up students, with no students getting off. This process repeats at each station until the bus reaches the school.

Loop invariant analysis: In this scenario, we analyze the number of passengers as our loop invariant. The invariant we are testing is: *"The number of passengers on the bus only increases from the beginning to the end of the journey."*

Now, let's examine each step of the loop invariant in our school bus example.

- **Initialization:** At the initial stop (starting station), the bus is empty, with zero students onboard. This sets our initial condition for the loop invariant, where the number of passengers (students) on the bus is initially zero.
- **Maintenance:** As the bus travels from one station to the next (from station $i-1$ to station i), students only board the bus and none get off. Therefore, the number of students on the bus either increases or remains the same, but never decreases. This maintains the loop invariant that the number of passengers can only increase.
- **Termination:** Upon arrival at the school, the process of picking up students ceases. The loop invariant holds true as the number of students has only increased or remained constant throughout the journey. At this final point, the invariant confirms that no students have been dropped off and only additional students could have boarded at each stop.

This example illustrates that the loop invariant – the assertion that the number of students on the bus never decreases – remains true throughout the entire journey. It is initialized correctly, maintained through each iteration of the loop (each station stop), and still holds when the loop terminates at the school. This analogy helps clarify how loop invariants are applied to ensure certain conditions remain unchanged throughout the execution of an algorithmic loop, providing a method to verify the correctness of the process.

An important aspect to clarify is the distinction between the loop condition and the loop invariant. To better illustrate this, let's revisit the binary search algorithm example from the previous section, focusing specifically on the loop segment of the algorithm:

```
while low <= high:
    mid = (high + low) // 2
    if ar[mid] < x:
        low = mid + 1
    elif ar[mid] > x:
        high = mid - 1
    else:
        return mid
```

The loop condition is a logical statement that determines whether the loop should continue executing or terminate. It is evaluated before each iteration of the loop. If the condition evaluates to true, the loop continues to execute; if it evaluates to false, the loop terminates. The loop condition ensures that the loop runs for the correct number of iterations and is crucial for controlling the flow of the algorithm. For example, in a binary search algorithm, the loop condition might be `low <= high`, which ensures that the search continues as long as there are elements to consider.

On the other hand, a loop invariant is a property or condition that holds true before and after each iteration of the loop. It is used to prove the correctness of the algorithm. By establishing that the invariant holds at the start and throughout the loop, we can ensure that the algorithm maintains its intended properties and ultimately produces the correct result. The process of maintaining a loop invariant involves three steps: initialization (showing that the invariant holds before the loop starts), maintenance (showing that if the invariant holds before an iteration, it also holds after), and termination (showing that when the loop terminates, the invariant and the loop condition together imply the correctness of the algorithm).

In summary, the loop condition controls the execution of the loop, while the loop invariant ensures the correctness of the algorithm by maintaining a consistent property throughout the loop's execution.

In the binary search algorithm, the loop condition is `low <= high`. This condition dictates the execution of the loop, ensuring it continues if there is a valid search space. Essentially, it checks whether there are still potential elements within the designated range (`low` to `high`) where the target might be found. When this condition is not met (i.e., `low` exceeds `high`), it indicates that the search space has been exhausted without finding the target, and the loop (and thus the search) terminates. Therefore, the loop condition is crucial for defining the operational boundaries of the loop. The question is why is the loop condition not a loop invariant? The loop condition is not a loop invariant for a couple of key reasons:

- **Validity at different stages:** The loop invariant must hold true before the loop starts, throughout each iteration, and remain true when the loop exits. In contrast, the loop condition needs to be true exclusively during the loop to permit the continuation of the loop. Once the loop exits (when `low` exceeds `high`), the `low <= high` condition is false, which is necessary for stopping the loop but does not align with the requirements of a loop invariant.

- **Role of condition:** The loop condition's primary role is to check the feasibility of continuing the loop based on the current state of `low` and `high`. It is a control mechanism, not a statement of correctness or a guarantee about the algorithm's effectiveness or the integrity of the process. On the other hand, a loop invariant relates to maintaining a certain property or condition that validates the correctness of the algorithm's logic throughout its execution.

Now, let's take a look at a loop invariant.

The loop invariant, in contrast with the loop condition, must hold true at the beginning and end of each iteration of the loop for the algorithm to be correct. For a binary search, a suitable loop invariant could be: *"At the start of each iteration, the target must be within the subarray bounded by low and high if it is present in the array."* This invariant assures that the search space is correctly narrowed down after each iteration, by adjusting either the `low` or `high` index based on the comparison of the target with the middle element of the current search range (`ar[mid]`).

In this context, the loop invariant is crucial for ensuring the correctness of the algorithm, guaranteeing that the target element is not overlooked and that the search space is reduced effectively. Conversely, the loop condition governs the continuation of the search process. Understanding the distinct roles of each is essential for effectively analyzing and establishing the correctness of algorithms such as binary search, where it is critical to accurately maintain and narrow the search boundary. In subsequent chapters, as new algorithms are introduced, we will consistently revisit the loop invariant to evaluate the correctness of each algorithm.

Summary

In this chapter, we discussed the important concepts of mathematical induction and loop invariants as foundational tools for verifying the correctness of algorithms. The chapter started by explaining mathematical induction, describing its two primary steps: the base case, which establishes the initial validity of the proposition, and the induction step, which demonstrates the proposition's maintenance across successive iterations. It then introduces loop invariants, expanding on mathematical induction by including the additional step of termination to ensure that the algorithm's properties hold from initiation through termination. Through detailed examples, such as a binary search algorithm, the chapter illustrates how these principles are applied to prove that algorithms perform correctly and efficiently, emphasizing their importance in developing reliable software. The chapter aimed to equip you with the methodologies to rigorously assess the integrity and operational correctness of various algorithms.

As previously mentioned, algorithm analysis involves two key aspects: proving the correctness of the algorithm and analyzing its complexity. The next chapter will address the second aspect, focusing on the complexity of algorithms and how to estimate this complexity.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022.
- *Algorithm Design*. By Jon Kleinberg and Éva Tardos. First Edition. Pearson. 2005.
- *The Art of Computer Programming*, Volume 1: Fundamental Algorithms. Donald E. Knuth. Third Edition. Addison-Wesley Professional. 1997.
- *Algorithms Unlocked*. By Thomas H. Cormen. MIT Press. 2013.
- *Discrete Mathematics and Its Applications*. By Kenneth H. Rosen. McGraw-Hill Science/Engineering/Math. Twelfth Edition. McGraw-Hill. 2012.
- *Concrete Mathematics: A Foundation for Computer Science*. By Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. Second Edition. Addison-Wesley. 1994.
- *How to Prove It: A Structured Approach*. By Daniel J. Velleman. Third Edition. Cambridge University Press. 2019.

Rate of Growth for Complexity Analysis

In the domain of algorithm design and analysis, understanding the rate at which an algorithm's running time increases with the size of the input data is paramount. This concept, known as the **rate of growth**, allows us to predict and compare the performance of various algorithms, ensuring that we select the most efficient solutions for our computational problems. As input sizes expand, the efficiency of an algorithm becomes increasingly critical, particularly in fields such as data processing, machine learning, and artificial intelligence where large datasets are the norm.

The rate of growth is described using asymptotic notations, which provide a framework for categorizing algorithms based on their running time or space requirements relative to the size of the input. These notations, including big O, Ω , and Θ , help us formalize the efficiency of algorithms by expressing their upper, lower, and tight bounds, respectively. Through this formalism, we can better understand the scalability of algorithms and make informed decisions about their suitability for specific tasks and environments.

In this chapter, we will explore the intricacies of these asymptotic notations, providing detailed explanations and practical examples to illustrate their application. We will explore common growth rates, such as constant, logarithmic, linear, log-linear, polynomial, exponential, and factorial, highlighting their implications for algorithm performance. By the end of this chapter, you will have a solid grasp of how to analyze and interpret the complexity of algorithms, equipping you with the knowledge to design more efficient and effective computational solutions.

The following topics will be covered in this chapter:

- Unpacking the rate of growth in algorithms
- Asymptotic notations
- Confronting the unsolvable – non-deterministic polynomial time (NP)-hard problems

Unpacking the rate of growth in algorithms

Let's revisit the initial discussion on the goal of studying algorithms. One of the primary objectives is to analyze algorithms to predict their behavior and performance. This analysis is crucial for two main reasons:

- To make informed design decisions
- To understand how to design efficient and low-cost algorithms

The cost of an algorithm depends on many parameters throughout its life cycle. However, our focus here is on the cost of running an algorithm. While the costs associated with designing, maintaining, testing, and retiring an algorithm are also important, they fall outside the scope of our current discussion.

The cost of running an algorithm, or its complexity, is generally measured or estimated in two dimensions: *time* and *space*. **Time complexity** refers to the duration needed to produce the expected output. This assumes that the algorithm will eventually terminate, although there are instances where algorithms may never terminate due to design flaws or other errors. **Space complexity**, on the other hand, refers to the memory required to run the algorithm. Both time and space complexities are crucial considerations due to the inherent limitations of our computational resources. If we hypothetically had access to instantaneous computing, allowing us to run any algorithm with any amount of data in constant time, and unlimited memory, the study of computational complexity would be unnecessary. However, given the current state of technology, these constraints make the study of computational complexity essential for designing efficient algorithms.

Both time and space complexities are measured as functions of the size of the input data, commonly represented by $n \in \{0, 1, 2, 3, \dots\}$. This is straightforward because the goal of every algorithm is to produce an expected output by processing input data. Regardless of whether the input data is numerical, textual, images, or videos, we are interested in understanding how the complexity changes as the amount of input data increases. This concept is known as the rate of growth and is denoted by big O, which describes the upper bound or worst-case scenario of time complexity relative to the input size. The complexity of an algorithm, denoted by $T(n)$, is then estimated using big O analysis. We will discuss in detail techniques to calculate $T(n)$, especially for recursive algorithms, which are more challenging. It might seem trivial, but it's worth mentioning that a higher or faster growth rate generally means the algorithm takes more time to produce the expected results.

The rate of growth is a function of data size and can take various mathematical forms. However, we focus on certain well-known patterns that facilitate the comparison of algorithms. By comparing algorithms that perform the same task, we aim to identify the most efficient one in terms of time and space. While the primary focus will be on time complexity, the concepts can also be generalized to space and memory complexity.

Constant growth

The root of all challenges in computational complexity lies in the iterative nature of computation. As discussed in the previous chapter, the iteration component, which we likened to the engine of algorithms, can be implemented either through **loops** or **recursion**. Regardless of the specific implementation – though it is true that the choice between loops and recursion can significantly impact an algorithm's complexity – the number of iterations ultimately dictates the complexity of an algorithm. Understanding how these iterations contribute to overall complexity is crucial for designing efficient algorithms.

The slowest growth rate occurs when the complexity of an algorithm is independent of the size of the input data. This family of growth functions is called **constant**, represented by $O(1)$. An algorithm is said to have constant time complexity, denoted as $O(1)$, if the value of $T(n) = O(1) = C$, which means it is bounded by a constant that does not depend on the size of the input. The question is, in what situations might we have a constant growth rate, with an increasing number of input data? The answer is that this happens in very rare cases. Here are some examples of situations with a constant growth rate:

- **Reporting the first number entered:** This is independent of the number of inputs
- **Accessing an element of an array:** Retrieving an element by its index is a constant time operation
- **Inserting an element at the beginning of a linked list:** This operation involves updating a few pointers, which takes constant time

In none of these examples do we use a loop to implement the algorithm. However, it's important to note that the existence of any loop typically makes the complexity non-constant. If the iteration is controlled by the size of the input data n , the algorithm cannot be constant. The key is that for an algorithm to have a constant growth rate, the number of operations it performs must not depend on the size of the input.

Sub-linear growth

Except for constant growth functions, all other growth rates depend on the size of the input data, n . Between constant growth and linear growth (the complexity of linear time algorithms), there exists a family of sub-linear functions. **Sub-linear time algorithms** are those whose time complexity grows slower than the size of the input data, denoted as $O(n)$. In other words, these algorithms do not need to examine every element of the input to produce an output. The goal of these algorithms is to achieve efficiency by performing fewer operations than the total number of elements in the input.

As a rule of thumb, whenever a problem requires examining every single input element, such as in a sequential search algorithm, the complexity is at least linear time, $O(n)$. In these cases, we cannot achieve a better time complexity because each element must be checked to ensure correctness. Sublinear time algorithms are typically characterized by time complexities, such as $O(\log n)$ and $O(\sqrt{n})$. Here are some examples of sublinear time algorithms:

- **Binary search** ($O(\log n)$): Binary search is an efficient algorithm for finding an element in a sorted array by repeatedly dividing the search interval in half. This algorithm is particularly useful for large, sorted datasets. The following is a simple Python implementation of the binary search algorithm. The iterative component is implemented using a while loop:

```
def binary_search(a, x):
    low = 0
    high = len(a) - 1
    while low <= high:
        mid = (high + low) // 2
        if a[mid] < x:
            low = mid + 1
        elif a[mid] > x:
            high = mid - 1
        else:
            return mid
    return -1
print(binary_search([0,1,3,5,8,10],10))
```

Binary search has a worst-case time complexity of $T(n) = O(\log n)$ when the item being searched for is located at the bottom of the binary tree. In the best-case scenario, its time complexity is $O(1)$. The logarithmic rate of growth, $O(\log n)$, is a special case of polylogarithmic growth, denoted by $O((\log n)^k)$ or $O(\log^k n)$, where $k > 0$ and constant. In logarithmic growth, we have $K = 1$.

- **Jump search** ($O(\sqrt{n})$): Jump search (or block search) is an algorithm designed to efficiently search for an element in a sorted array by jumping ahead by fixed steps and then performing a linear search within the identified block. This method is particularly useful for large, sorted arrays where a linear search would be too slow. Presented here is the Python code implementation for the jump search algorithm:

```
import math
def jump_search(a, x):
    n = len(a)
    step = int(math.sqrt(n))
    prev = 0
    while a[min(step, n)-1] < x:
        prev = step
        step += int(math.sqrt(n))
    if prev >= n:
```

```

        return -1
    for i in range(prev, min(step, n)):
        if a[i] == x:
            return i
    return -1

```

- **Interpolation search** ($O(\log\log n)$): An example of an algorithm with complexity $O(\log\log n)$ is the interpolation search algorithm. Interpolation search is an improvement over binary search for instances where the values in a sorted array are uniformly distributed. The search can be performed in $T(n) = O(\log\log n)$ time complexity when the data is uniformly distributed.

Interpolation search works on the principle of estimating the position of the target value within the sorted array. It calculates the probable position of the target using the formula based on the array's range and the target value. If the distribution of the data is uniform, the position estimate is quite accurate, leading to a significant reduction in the number of comparisons. Presented in the following is the Python implementation of interpolation search for your examination:

```

def interpolation_search(a, x):
    low = 0
    high = len(a) - 1
    while low <= high and x >= a[low] and x <= a[high]:
        if low == high:
            if a[low] == x:
                return low
            return -1
        pos = low + ((high - low) // (a[high] - a[low])) * (x -
a[low]))
        if a[pos] == x:
            return pos
        if a[pos] < x:
            low = pos + 1
        else:
            high = pos - 1
    return -1

```

Both jump search and interpolation search exhibit sublinear growth rates, offering potential efficiency gains over linear search. In the next subsection, we will discuss the concept of linear growth patterns, illustrating their characteristics through pertinent examples.

Linear growth

The most straightforward rate of growth function in algorithm analysis is linear time, represented as $O(n)$. **Linear growth** serves as a crucial dividing line between fast algorithms, such as those with constant time ($O(1)$) and sublinear time (e.g., $O(\log n)$, $O(\sqrt{n})$), and slower or extremely slow algorithms, such as polynomial time ($O(n^2)$, $O(n^3)$) and exponential time ($O(2^n)$), which are considered non-linear.

An algorithm is said to have linear time complexity when its running time increases in direct proportion to the size of the input data. In other words, the running time of a linear time algorithm is $T(n) = cn$, where c is a constant. Linear time algorithms are essential for addressing problems where every single input element must be processed. In such scenarios, the number of operations grows directly in proportion to the size of the input data. This means that if the input size doubles, the time required to process the input also doubles. Examples of linear time algorithms include sequential search, finding the minimum or maximum element in an array, and summing up the elements in an array.

One notable linear time algorithm is to compute the factorial of a non-negative integer n (denoted as $n!$). Here is a simple iterative implementation of the factorial algorithm in Python:

```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Due to the nature of the problem, where every single number from 1 to n must be visited, the recursive implementation cannot achieve better complexity than linear time:

```
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial_recursive(n - 1)
```

Linear time algorithms are crucial because they provide a balance between efficiency and thoroughness. They ensure that every element is considered, making them suitable for problems where skipping elements could lead to incorrect or incomplete results. However, it is important to recognize that while linear time algorithms are efficient for moderate-sized inputs, they can become impractical for very large datasets. In such cases, more sophisticated algorithms with sublinear time complexities may be required to achieve acceptable performance.

Non-linear growth

The **non-linear time family of algorithms** encompasses a vast array of well-known problems across various domains, including data processing, optimization, machine learning, and artificial intelligence. These algorithms exhibit growth rates that are more complex than linear time, ranging from relatively efficient algorithms with complexities such as $O(n \log n)$ to extremely costly ones with non-polynomial growth such as exponential $O(2^n)$ and factorial $O(n!)$ time.

The following are examples of non-linear algorithms:

- **$O(n \log n)$ algorithms:** Algorithms such as **merge sort**, **quick sort**, and **heap sort** fall into this category. These are classic sorting algorithms that have a time complexity of $O(n \log n)$. They are highly efficient for large datasets and are frequently used in practical applications due to their relatively manageable growth rates. In the upcoming chapters, we will discuss this family of algorithms in detail. These algorithms are fundamental in computer science due to their efficiency and widespread applicability in various sorting tasks.
- **Log-linear time:** Log-linear time algorithms, also referred to as **quasi-linear time algorithms**, have a time complexity of $T(n) = O(n \log^k n)$, where k is a positive constant. Many well-known algorithms fall into this category and are essential for efficiently processing large datasets. Examples of log-linear time algorithms are as follows:
 - **Merge sort:** A classic divide-and-conquer sorting algorithm that splits the input array into smaller subarrays, sorts them, and then merges them back together. Its time complexity is $O(n \log n)$.
 - **Heap sort:** This sorting algorithm builds a heap data structure from the input data and then repeatedly extracts the maximum element to build a sorted array. Its time complexity is $O(n \log n)$.
 - **Quick sort:** Another divide-and-conquer sorting algorithm that selects a pivot element and partitions the array around the pivot. In the average case, its time complexity is $O(n \log n)$.
- **Polynomial time algorithms:** Polynomial time algorithms have a complexity of $O(n^a)$, where a is a small constant. These algorithms are generally considered efficient for moderate-sized inputs, but their performance can degrade significantly as the input size increases. Here are some examples:
 - **Matrix multiplication:** The standard algorithm for multiplying two matrices has a time complexity of $O(n^3)$. More advanced algorithms, such as Strassen's algorithm, can reduce this complexity to approximately $O(n^{2.81})$, but it is still polynomial.
 - **Floyd-Warshall shortest path:** This algorithm finds the shortest path from a source node to all other nodes in a graph with non-negative weights. Its time complexity is $O(V^3)$, where V is the number of vertices in the graph.

Polynomial time algorithms are essential for many practical applications due to their relatively predictable performance. However, their efficiency can become problematic as input sizes grow, making them less suitable for very large datasets. Despite this, they remain a cornerstone of algorithm design, providing feasible solutions for a wide range of problems in computer science and engineering.

- **Exponential time algorithms:** These algorithms are characterized by their extremely slow and computationally expensive nature. The general form of their complexity is $O(a^n)$, where a is a positive constant. A common special case of these algorithms is $O(2^n)$. One well-known example of an exponential time algorithm is recursive solutions to the **Tower of Hanoi problem**.

The Tower of Hanoi problem involves moving n disks from one peg to another, using a third peg as an auxiliary, following these rules:

- Only one disk can be moved at a time
- A disk can only be placed on top of a larger disk

The recursive solution to the Tower of Hanoi has a time complexity of $O(2^n)$, as each move involves recursively solving two subproblems of size $n-1$. The time complexity of the Tower of Hanoi problem, which is $O(2^n)$, will be thoroughly explored in subsequent chapters when we discuss recurrence functions. The recurrence function for this problem, $T(n) = 2T(n-1) + O(1)$, will be analyzed using techniques such as substitution and the master method to derive the complexity. Through these methods, it will be shown that $T(n) = O(2^n)$. For a comprehensive understanding, please refer to *Chapters 4* and *5*.

Here is a recursive Python implementation of the Tower of Hanoi problem:

```
def tower_of_hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    tower_of_hanoi(n - 1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    tower_of_hanoi(n - 1, auxiliary, target, source)
```

Exponential time algorithms are often impractical for large input sizes because their running time increases dramatically as the input size grows. These algorithms are typically used for problems where no efficient solution is known, and they serve as a baseline for comparing the performance of more sophisticated algorithms.

- **Factorial time algorithms:** These algorithms generate all permutations of a set, making them extremely computationally expensive. One notable example is the brute force solution to the **traveling salesman problem (TSP)**, which has a factorial time complexity of $O(n!)$.

The TSP is a classic optimization problem where a salesman must find the shortest possible route that visits a set of cities exactly once and returns to the starting city. Given a list of cities and the distances between each pair of cities, the objective is to determine the most efficient tour that minimizes the total travel distance.

The brute force approach to solving TSP involves generating all possible permutations of the cities and calculating the total distance for each permutation to find the shortest one. This results in a time complexity of $O(n!)$. The number of permutations of n cities is $n!$ (n factorial) where n is the number of cities. The following is a simple implementation of TSP problem:

```
from itertools import permutations
def calculate_distance(permutation, distance_matrix):
    distance = 0
    for i in range(len(permutation) - 1):
        distance += distance_matrix[permutation[i]][permutation[i + 1]]
    distance += distance_matrix[permutation[-1]][permutation[0]] # Return to the start
    return distance
def tsp_brute_force(distance_matrix):
    n = len(distance_matrix)
    cities = list(range(n))
    min_distance = float('inf')
    best_permutation = None
    for permutation in permutations(cities):
        current_distance = calculate_distance(permutation, distance_matrix)
        if current_distance < min_distance:
            min_distance = current_distance
            best_permutation = permutation
    return best_permutation, min_distance
```

Factorial time algorithms are highly impractical for all but the smallest input sizes because the runtime increases extraordinarily fast. As the number of cities (or elements) grows, the number of permutations and thus the computational effort required to solve the problem increases factorially. Due to their factorial time complexity, these algorithms become infeasible for larger datasets, underscoring the need for more efficient heuristic or approximation algorithms in practical applications.

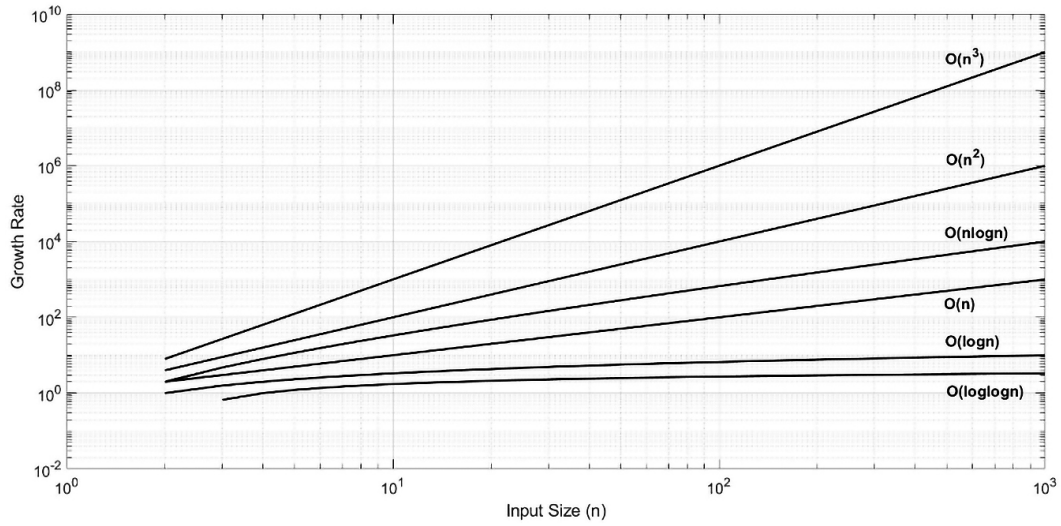
In summary, non-linear time algorithms are fundamental in tackling complex and computationally intensive problems across various fields. While they offer powerful solutions, their performance can vary significantly based on the input size and the specific growth rate of the algorithm. Understanding these complexities helps in choosing the appropriate algorithm and optimizing performance for specific applications.

Table 3.1 provides examples of actual growth rates for various input data sizes. It clearly demonstrates that non-linear algorithms, particularly those with exponential and factorial growth rates, experience an extremely rapid increase in computational complexity as the input size grows. This rapid escalation in resource requirements makes such algorithms computationally impractical for all but the smallest datasets.

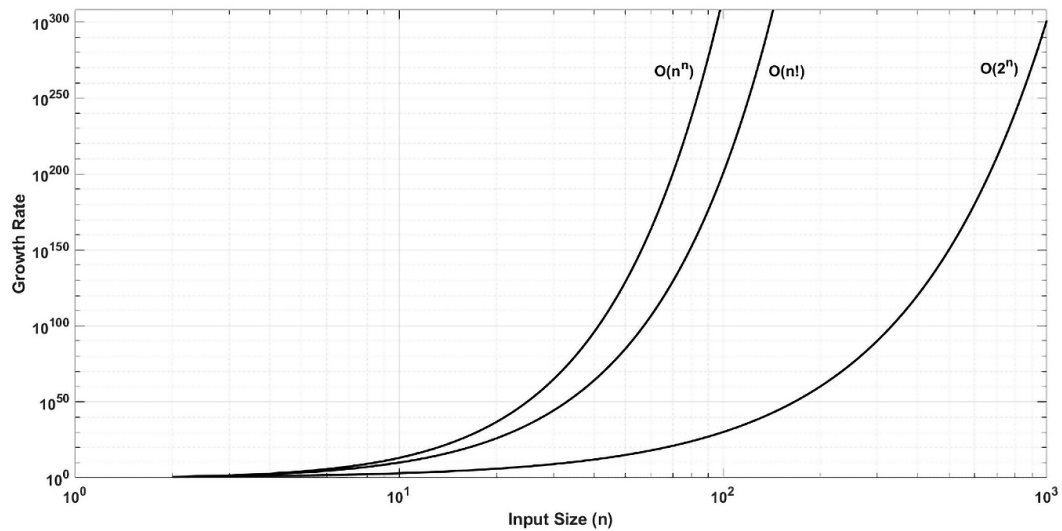
	The size of input data (n)			
Growth rate	16	256	2^{20}	2^{30}
$O(1)$	C	C	C	C
$O(\log \log n)$	2	3	4.32	4.32
$O(\log n)$	4	8	20	30
$O(n)$	16	256	1,048,576	1.073×10^9
$O(n \log n)$	64	2.048×10^3	20.971×10^6	32.212×10^9
$O(n^2)$	265	65,536	1.099×10^{12}	1.152×10^{18}
$O(n^3)$	4,096	16.777×10^6	1.152×10^{18}	1.237×10^{27}
$O(2^n)$	65×10^3	11.57×10^{76}	E	E
$O(n!)$	20.92×10^{12}	E	E	E
$O(n^n)$	18.44×10^{18}	E	E	E

Table 3.1: Examples of actual growth rates for various input sizes (**C**: A constant number and computationally negligible; **E**: Extremely large number and computationally impractical)

Figure 3.1 (a) and (b) present graphs that illustrate the rate of growth for various functions with input sizes ranging from $n = 1$ to $n = 1000$. These graphs provide a visual comparison of different time complexities, demonstrating how they scale with increasing input sizes. These figures collectively illustrate the impact of different time complexities on algorithm performance, emphasizing the importance of selecting efficient algorithms for large-scale problems.



(a)



(b)

Figure 3.1 (a) and (b): Graphs illustrating the rate of growth for $n = 1$ to $n = 1000$

This section discussed the importance of understanding how the complexity of algorithms scaled with the size of input data, commonly represented by n . It emphasized that both time and space complexities were crucial metrics for evaluating algorithm performance. The section explained that growth rates could vary significantly, ranging from constant time $O(1)$ to factorial time $O(n!)$, with intermediate complexities such as logarithmic $O(\log n)$, linear $O(n)$, and log-linear $O(n \log n)$. The rate of growth was critical for making informed design decisions and ensuring efficient algorithm implementation, especially for large datasets. By comparing different growth rates, one could better understand the trade-offs between algorithm efficiency and computational resource requirements. Now, it is time to introduce a mathematical framework for studying the behavior of algorithms, which is directly connected to the rate of growth of the algorithms.

Asymptotic notations

Asymptotic notation is a mathematical framework used to describe the behavior of algorithms in terms of their time and space complexities as the input size approaches infinity. It classifies algorithms according to their growth rates, allowing for the comparison of efficiency across different algorithms independently of hardware or implementation details. Adapted from the field of mathematical analysis, asymptotic notation describes the limiting behavior of mathematical functions. As its name suggests, asymptotic notation does not provide specific solutions but rather a formal representation of the behavior of functions as they scale.

Let's assume we have a function like the following:

$$f(n) = n^3 + 1000n^2 + 6n + 5 \times 10^5$$

We want to predict the behavior of $f(n)$ as $n \rightarrow \infty$. Although the term $1000n^2$ has a very large coefficient and there is a large constant 5×10^5 , all terms except n^3 become insignificant when n grows very large. In mathematical analysis, we symbolically write $f(n) \sim n^3$ and read it as " $f(n)$ is asymptotically n^3 ."

In algorithm analysis, $f(n)$ is replaced by $T(n)$, which represents the complexity of the function describing the running time of the algorithm under study. $T(n)$ can be estimated directly by analyzing the algorithm when it is implemented using an iterative (non-recursive) approach, or it can be described by a recurrence relation, which is specific to the recursive implementation of algorithms. Once we have $T(n)$, we can predict the behavior of the algorithm as n (the number of data elements or the problem size) grows very large using asymptotic notations.

Before describing the asymptotic notations in algorithm analysis, we need to explain the simplification guideline to deal with equations and inequalities when using asymptotic notations.

Simplification rules in asymptotic notation

Asymptotic notation provides a way to describe the behavior of functions as the input size grows toward infinity. When working with asymptotic notation, it's important to simplify the expressions to focus on the most significant terms. Here are the key simplification rules:

- **Drop constants:** Ignore constant coefficients because they do not affect the growth rate.
Example: $T(n) = 5n^3 + 3n^2 + 2n + 7$ simplifies to $O(n^3)$.
- **Keep the most significant term:** Retain the term with the highest growth rate, as it dominates the function as n becomes large.
Example: $T(n) = n^3 + 1000n^2 + 50n$ simplifies to $O(n^3)$.
- **Combine similar terms:** When adding functions, keep the term with the highest growth rate.
Example: $T(n) = O(n^2) + O(n)$ simplifies to $O(n^2)$.
- **Product of terms:** When multiplying functions, multiply their growth rates.
Example: $T(n) = O(n) \times O(\log n)$ simplifies to $O(n \log n)$.
- **Nested functions:** For nested functions, the outer function's growth rate dominates.
Example: $T(n) = O(2^{O(n)})$ simplifies to $O(2^n)$.

Let's practice the simplification rules in the following examples:

- **Example 3.1:**
Simplify $T(n) = 3n^4 + 2n^3 + n^2$.
Solution: Drop constants and lower-order terms: $T(n) = O(n^4)$.
- **Example 3.2:**
Simplify $T(n) = 100n \log n + 50n$.
Solution: Drop the lower-order term: $T(n) = O(n \log n)$.
- **Example 3.3:**
Simplify $T(n) = O(n) + O(\log n)$.
Solution: Retain the term with the highest growth rate: $T(n) = O(n)$.
- **Example 3.4:**
Simplify $T(n) = O(n^2)O(n \log n)$.
Solution: Multiply the growth rates: $T(n) = O(n^3 \log n)$.

- **Example 3.5:**

Simplify $T(n) = O(\log n^2)$.

Solution: Simplify the inner function first:

$$T(n) = O(\log n^2) = O(2 \log n) = O(\log n).$$

Having established a foundation in asymptotic notation and simplification techniques, we will now turn our attention to a crucial aspect of this chapter: the asymptotic bounds denoted by $O(n)$, $\Theta(n)$, and $\Omega(n)$.

Asymptotic bounds

The key to analyzing the complexity of an algorithm lies in studying its running time. But which running time should we consider? Algorithms can behave differently depending on the nature of the input data and its distribution. Generally, we aim to understand the worst-case scenario, as it provides a guarantee on the upper limit of the algorithm's running time for any input. However, we may also be interested in the average-case scenario, which gives a more practical expectation of the algorithm's performance over a typical set of inputs. Ultimately, our goal is to draw general conclusions about the algorithm's behavior that are independent of specific input data and its distribution. Let's discuss these different scenarios in detail:

- **Worst-case scenario:** The worst-case scenario considers the maximum time an algorithm can take to complete, given the most challenging possible input of size n . This analysis provides a guarantee that the algorithm will not exceed this time, making it particularly important for applications where performance guarantees are critical. The asymptotic notation used to describe the worst-case scenario is O notation (big O notation). This is called the **asymptotic upper bound**.
- **Average-case scenario:** The average-case scenario evaluates the expected running time of an algorithm over all possible inputs of size n , assuming a certain probability distribution of the inputs. This analysis provides a more realistic measure of an algorithm's performance in typical use cases, giving insights into its efficiency under normal operating conditions. The asymptotic notation used to describe the average-case scenario is typically Θ notation (big Theta notation). This is known as the **asymptotic tight bound**.
- **Best-case scenario:** The best-case scenario examines the minimum time an algorithm takes to complete, given the most favorable input of size n . While less commonly used in performance analysis, it can highlight the efficiency of an algorithm under optimal conditions. The asymptotic notation, called **asymptotic lower bound**, which is used to describe the best-case scenario, is typically Ω -notation (big Omega notation).

Having explored the three scenarios of time complexity (best, average, and worst case), we will now introduce three fundamental asymptotic bounds: upper, tight, and lower bounds.

Asymptotic upper bound (O notation)

An algorithm is classified as $O(g(n))$ if there exist positive constants c and n_0 , such that the time complexity of the algorithm or its running time $T(n)$ satisfies the inequality $T(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Formally speaking, $O(g(n))$ represents a set of functions or running times that share a similar asymptotic upper bound:

$$O(g(n)) = \{T(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, 0 \leq T(n) \leq c \cdot g(n)\}$$

Figure 3.2 illustrates hypothetical graphs for $T(n)$ and $g(n)$. The asymptotic upper bound, represented by $O(g(n))$, provides a way to describe the worst-case growth rate of the algorithm's running time as the input size increases, ensuring that $T(n)$ grows no faster than $c \cdot g(n)$. The following example demonstrates how to determine the upper asymptotic bound of a function and the method used to prove it:

Example 3.6:

Prove that $4n^2 = O(n^3)$ by determining the constants c and n_0 .

Solution: From the formal definition of O notation, we need to show the following:

$$4n^2 \leq c \cdot n^3 \text{ for all } n \geq n_0.$$

To simplify, we divide both sides by n^2 : $4 \leq c \cdot n$

This inequality has two unknowns, c and n_0 , giving us multiple possible solutions. For example, if we choose $c = 1$, this simplifies to $4 \leq n$. So, $n_0 = 4$ or any larger value. Therefore, for $c = 1$ and $n_0 = 4$, the inequality holds. There are other possible pairs of c and n_0 that satisfy the inequality, but this is one straightforward solution.

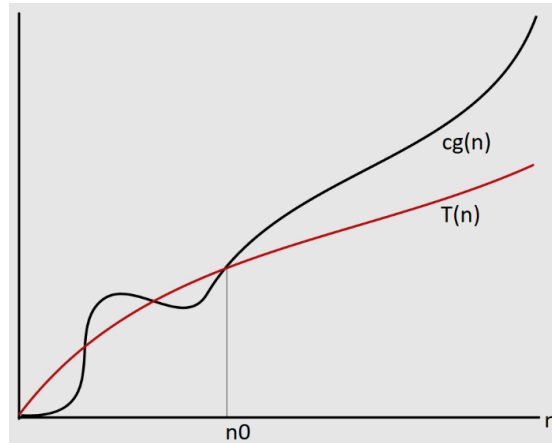


Figure 3.2: Demonstrating the concept of **O** notation. The running time $T(n)$ is asymptotically bounded by $c \cdot g(n)$, where c is a positive constant. This means that for sufficiently large input sizes n , specifically for all $n \geq n_0$, the value of $T(n)$ will not exceed $c \cdot g(n)$.

Example 3.7:

Demonstrating members of $O(n^2)$.

As mentioned earlier, O notation represents a set of functions. Table 3.2 provides examples of functions that are members of $O(n^2)$ along with their corresponding constants c and n_0 .

$T(n)$	c	n_0
n	1	0
$60n + 4$	60	2
$\frac{n}{\log n}$	0.5	2
n^2	1	0
$n^2 + n$	2	1
$n^2 + 5000n$	5001	1

Table 3.2: Demonstrating some members of $O(n^2)$

Example 3.8:

Proving $O(n!)$ grows faster than $O(2^n)$.

We need to show that $O(n!) > O(2^n)$. To do this, we use **Stirling's approximation**:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

Using this approximation for $n!$, we need to prove the following:

$$O\left(\sqrt{2\pi n} \left(\frac{n}{e}\right)^n\right) > O(2^n)$$

Taking the logarithm of both sides, we get the following:

$$\log\left(\sqrt{2\pi n} \left(\frac{n}{e}\right)^n\right) > \log(2^n)$$

This simplifies to the following:

$$0.5(\log 2 + \log \pi + \log n) + n \log(n - e) > n \log 2$$

Further simplifying both sides, we get the following:

$$n \log n > n$$

As n grows large, the $n \log n$ term dominates. Therefore, $O(n!)$ grows faster than $O(2^n)$, which completes the proof.

We encourage the reader to apply the same approach to prove that $O(n^n) > O(n!)$.

In this subsection, we introduced the concept of the upper asymptotic bound, explaining how to describe and prove it for specific growth functions. In the following subsection, we will explore the lower asymptotic bound.

Asymptotic lower bound (Ω -notation)

An algorithm is considered to be $\Omega(g(n))$ if there are positive constants c and n_0 such that the time complexity of the algorithm or its running time $T(n)$ meets the condition $T(n) \geq c \cdot g(n)$ for all $n \geq n_0$. Formally speaking, $\Omega(g(n))$ represents a set of functions or running times that share a similar asymptotic lower bound:

$$\Omega(g(n)) = \{T(n) : \exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, 0 \leq c \cdot g(n) \leq T(n)\}$$

The asymptotic lower bound, represented by $\Omega(g(n))$, provides a way to describe the best-case growth rate of the algorithm's running time as the input size increases, ensuring that $T(n)$ grows no slower than $c \cdot g(n)$. Figure 3.3 illustrates hypothetical graphs for $T(n)$ and $g(n)$.

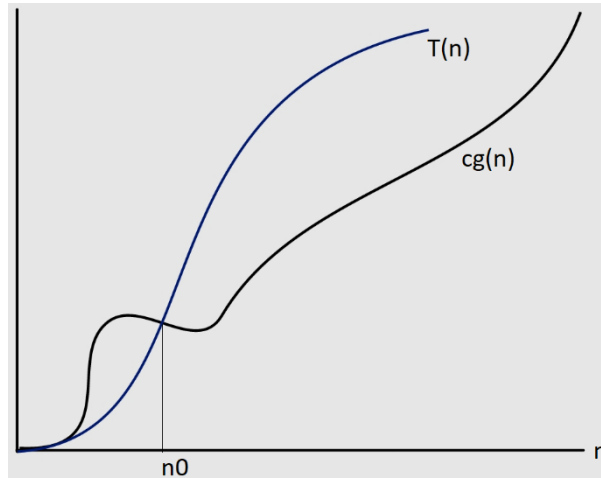


Figure 3.3: Demonstrating the concept of Ω notation. The running time $T(n)$ is asymptotically bounded by $c \cdot g(n)$, where c is a positive constant. This means that for sufficiently large input sizes n , specifically, for all $n \geq n_0$, the value of $T(n)$ will not be lower than $c \cdot g(n)$.

Example 3.9:

Prove that $\sqrt{n} = \Omega(\log n)$ by determining the constants c and n_0 .

Solution: From the formal definition of Ω notation, we need to show that: $\sqrt{n} \geq c \log n$ for all $n \geq n_0$. We divide both sides of the inequality by $\log n$, $\frac{\sqrt{n}}{\log n} \geq c$, and then we find suitable c and n_0 . To determine appropriate values for c and n_0 , we need to find values such that the inequality holds for all $n \geq n_0$. Let's start with a specific value for c . We choose $c = 1$:

$$\frac{\sqrt{n}}{\log n} \geq 1$$

This simplifies to $\sqrt{n} \geq \log n$. We need to find n_0 such that this inequality holds for all $n \geq n_0$. Let's square both sides of the inequality to make it easier to compare:

$$n \geq \log^2 n$$

For large values of n , the term $\log^2 n$ grows much slower than n . Thus, there will be a point n_0 beyond which the inequality $n \geq \log^2 n$ holds. To find a rough estimate of n_0 , we can solve this equation: $n \geq \log^2 n$.

This is a transcendental equation, and solving it analytically is complex. However, we can employ numerical methods to find an approximate solution. From practical calculations, n_0 is approximately 16. Thus, we can choose $n_0 = 16$.

Example 3.10:

Demonstrating members of $\Omega(n^3)$.

As mentioned earlier, Ω notation represents a set of functions. *Table 3.3* provides examples of functions that are members of $\Omega(n^3)$ along with their corresponding constants c and n_0 .

$T(n)$	c	n_0
n^3	1	0
$n^3 \log n + n^2 + 4$	5	1
2^n	1	4
$n^{\sqrt{10}}$	1	0
2^{2^n}	1	0
$n^{\sqrt{n}}$	1	9

Table 3.3: Demonstrating some members of $\Omega(n^3)$

Asymptotic tight bound (θ -notation)

An algorithm is said to be $\theta(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that the algorithm's running time $T(n)$ satisfies $c_1 \cdot g(n) \leq T(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. Formally speaking, $\theta(g(n))$ represents a set of functions or running times that share a similar asymptotic lower bound:

$$\theta(g(n)) = \{T(n) : \exists c_1 > 0, \exists c_2 > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, 0 \leq c_1 \cdot g(n) \leq T(n) \leq c_2 \cdot g(n)\}$$

Figure 3.4 illustrates hypothetical graphs for $T(n)$ and $g(n)$.

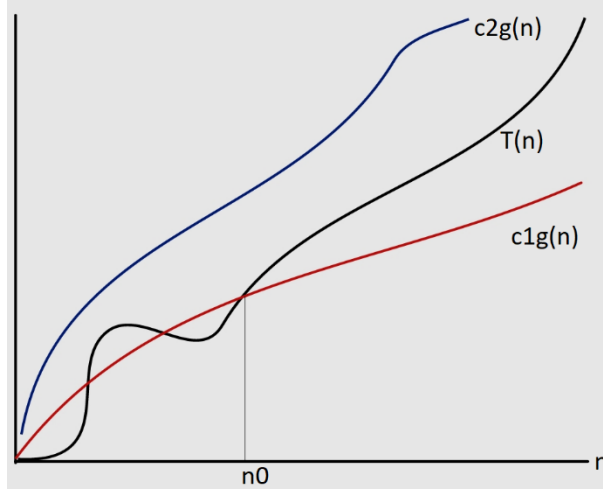


Figure 3.4: Demonstrating the concept of θ notation. The running time $T(n)$ is asymptotically bounded between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$, where $c_1, c_2 > 0$. This means that for sufficiently large input sizes n , specifically, for all $n \geq n_0$, the value of $T(n)$ will not be lower than $c_1 \cdot g(n)$ and will not exceed $c_2 \cdot g(n)$.

Example 3.11:

Prove that $n^2 - 4n = \theta(n^2)$ by determining the constants c_1 , c_2 , and n_0 .

Solution: From the formal definition of θ notation, we need to show that

$c_1 \cdot n^2 \leq n^2 - 4n \leq c_2 \cdot n^2$ for all $n \geq n_0$. We divide both sides of both inequalities by n^2 :

$$c_1 \leq 1 - \frac{4}{n} \leq c_2$$

For $n_0 = 8$, we will have $c_1 = \frac{1}{4}$ and for large n , $c_2 = \frac{1}{2}$.

Example 3.12:

Prove that $6n^3 \neq \theta(n^2)$.

Solution: First, we attempt to prove that $6n^3 = \theta(n^2)$. To do this, we need to show that there exist positive constants c_1 and c_2 such that the following is the case:

$$c_1 n^2 \leq 6n^3 \leq c_2 n^2$$

Let's consider the second inequality and divide both sides by n^2 :

$$6n \leq c_2$$

For large n , this inequality does not hold because c_2 is a constant. Therefore, it is impossible to find a constant c_2 that satisfies this inequality for all large n . This disproves that n^2 is a tight bound for $6n^3$. Hence, we conclude that $6n^3 \neq \theta(n^2)$.

As mentioned earlier, θ notation represents a set of functions. Table 3.4 provides examples of functions that are members of $\theta(n^3)$ along with their corresponding constants c_1 , c_2 , and n_0 .

$T(n)$	c_1	c_2	n_0
n^3	1	1	0
$2n^3 + 4$	1	2	0
$n^3 + \log n$	1	2	1
$10n^3 + 1$	1	10	0

Table 3.4: Demonstrating some members of $\theta(n^3)$

Example 3.13:

Illustrate the difference between $\theta(g(n))$ and $O(g(n))$.

Solution: The major difference between these two asymptotic bounds is that $O(g(n))$ describes an upper bound that holds for all inputs, while $\theta(g(n))$ represents a tight bound, meaning it provides both upper and lower bounds and thus does not hold for every input. Table 3.5 illustrates this difference using the insertion sort algorithm as an example.

	$O(g(n))$	$\theta(g(n))$
Worst case scenario (unsorted input)	n^2	n^2
Sorted input	n^2	n

Table 3.5: Illustrate the difference between $\theta(g(n))$ and $O(g(n))$ in insertion sort algorithm

In this section, we explored various scenarios of time complexities before transitioning to asymptotic bounds. We provided formal definitions of asymptotic bounds and included several examples to demonstrate how to solve them and estimate their parameters.

Confronting the unsolvable NP-hard problems

In previous sections, we explored various rates of growth and learned how to describe them using formal representations, particularly asymptotic notations. While studying these growth rates, we encountered problems that appeared highly challenging, yet we did not address their solvability. In this section, we will focus on problems that go beyond our computational capabilities, discussing NP-hard problems that are inherently difficult, if not impossible, to solve efficiently.

In computer science, there exists a category that challenges our ability to find efficient solutions. These problems, known as **NP-hard problems**, represent some of the most complex and difficult challenges in computer science. Before exploring NP-hard problems, it is essential to understand why some problems cannot be solved efficiently, or in some cases, at all:

- **Inherent complexity:** Certain problems are intrinsically complex, requiring an astronomical number of steps to solve as the size of the input grows. This complexity often makes it impractical, if not impossible, to find a solution within a reasonable timeframe. For example, problems that involve checking all possible configurations or combinations, such as TSP, can have a factorial or exponential number of possible solutions.
- **Resource constraints:** Even with modern computing power, the resources required (time, memory, or both) to solve some problems can exceed feasible limits. Problems that demand more computational resources than are available become effectively unsolvable.
- **Non-deterministic nature:** Some problems can be verified quickly if a solution is given, but finding the solution itself requires exploring an exponentially large search space. These problems fall into the NP class, where solutions can be checked in polynomial time, but finding them is not feasible with current algorithms.
- **Undecidability:** Certain problems are undecidable, meaning there is no algorithm that can determine the solution for all possible inputs. The most famous example is the **Halting problem**, which asks whether a given computer program will eventually halt or run forever. Alan Turing proved that no algorithm can solve this problem for all possible programs and inputs.

Complexities of NP, NP-complete, and NP-hard

In computational complexity theory, the classes **NP**, **NP-complete**, and **NP-hard** are used to categorize problems based on their computational difficulty and the resources required to solve them. These classifications have significant connections to asymptotic bounds, helping to understand the theoretical limits of algorithmic performance. The following is a short description of these problems:

- **NP:** NP is the class of decision problems for which a given solution can be verified as correct or incorrect in polynomial time by a deterministic Turing machine. If a problem is in NP, there exists an algorithm that can verify a solution in $O(n^k)$ time for some constant k .
- **NP-complete:** NP-complete problems are a subset of NP problems that are both in NP and as hard as any problem in NP. A problem is NP-complete if every problem in NP can be reduced to it using a polynomial-time reduction. Solving an NP-complete problem efficiently (in polynomial time) would imply that every problem in NP can be solved in polynomial time, effectively collapsing the complexity classes P and NP. The existence of a polynomial-time algorithm for any NP-complete problem remains one of the most significant open questions in computer science.
- **NP-hard:** NP-hard problems represent a class of computational problems that are at least as challenging as the most difficult problems within the NP complexity class. In contrast to NP-complete problems, NP-hard problems are not restricted to decision problems (yes/no questions) and are not necessarily solvable within NP itself. NP-hard problems are often characterized by exponential time complexity. If a problem is NP-hard, there is no known polynomial-time algorithm to solve it, and its asymptotic bounds are typically represented by super-polynomial functions such as $O(2^n)$ or $O(n!)$.

NP-hard problems are among the most challenging and complex problems in computational theory. They are defined as problems to which any NP problem can be reduced in polynomial time. However, unlike NP-complete problems, NP-hard problems do not need to be decision problems and do not have to belong to the NP class. Solving an NP-hard problem efficiently would mean solving all NP problems efficiently, which is currently beyond our reach.

The study of NP-hard problems is crucial because it helps us understand the limits of computational power and the boundaries of what can be feasibly solved. By confronting these unsolvable or intractable problems, researchers strive to develop better heuristics, approximations, and theoretical insights that can lead to more efficient algorithms for a wide range of applications.

The classifications NP, NP-complete, and NP-hard provide a framework for understanding the computational difficulty of problems. These classes are closely related to asymptotic bounds, as they help to identify the limits of algorithm efficiency and the potential for finding polynomial time solutions. Understanding these complexity classes is essential for determining the feasibility of solving problems within practical time constraints and for guiding the development of efficient algorithms.

Summary

In *Chapter 3*, we examined the concept of the rate of growth in algorithmic complexity, highlighting its importance for understanding how an algorithm's running time scaled with increasing input size. This understanding was crucial for predicting algorithm behavior and making informed design decisions. We covered a range of growth rates, from constant time $O(1)$ to factorial time $O(n!)$, and discussed how these rates impacted the efficiency and practicality of algorithms, particularly when dealing with large datasets.

We also introduced various asymptotic notations, such as Big O , Ω , and Θ , to formally describe the upper, lower, and tight bounds of an algorithm's running time. Through examples and comparisons, we demonstrated how different growth rates influenced computational resources and performance. This chapter laid the foundation for recognizing and analyzing the complexity of algorithms, providing the necessary tools to evaluate and compare their efficiency in real-world applications.

In the next chapter, we deep dive into recursive algorithms and discuss how we model the running time using recurrence functions and then study the behavior of the algorithms using methods to estimate their complexities.

In the next chapter, we will take a deep dive into recursive algorithms, exploring how their running time can be modeled using recurrence relations. Recurrence relations provide a powerful tool for expressing the time complexity of recursive processes by breaking down the problem into smaller subproblems and establishing a relationship between them. We will discuss various techniques to solve these recurrence relations, such as the master theorem and substitution method. These methods will help us to accurately estimate the complexities of recursive algorithms and understand their behavior as the input size grows. This deeper understanding of recursive algorithms will further enhance our ability to design efficient and effective solutions to complex computational problems.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022.
- *Algorithm Design*. By Jon Kleinberg and Éva Tardos. 1st Edition. Pearson. 2005.
- *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Donald E. Knuth. Third edition. Addison-Wesley Professional. 1997.
- *Algorithms in a Nutshell: A Practical Guide*. By George Heineman, Gary Pollice, Stanley Selkow. 2nd Edition. O'Reilly Media. 2016.
- *Algorithms*. By Robert Sedgewick and Kevin Wayne. Fourth Edition. Addison-Wesley Professional. 2011.
- *Data Structures and Algorithm Analysis in C++*. By Mark Allen Weiss. Fourth Edition. Pearson. 2013.

- *Discrete Mathematics and Its Applications*. By Kenneth H. Rosen. McGraw-Hill Science/Engineering/Math. Twelfth edition. McGraw-Hill. 2012.
- *Concrete Mathematics: A Foundation for Computer Science*. By Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. Second Edition. Addison-Wesley. 1994.

4

Recursion and Recurrence Functions

Estimating the complexity or running time of iterative algorithms is relatively straightforward due to their linear and predictable nature. However, recursive algorithms, which involve the function calling itself one or more times during execution, present a unique challenge in complexity estimation. These self-referential structures often lead to intricate and non-intuitive running times that cannot be easily discerned through simple observation or traditional iterative analysis.

To address this challenge, we introduce the concept of **recurrence functions**. Recurrence functions are mathematical models that describe the running time of a recursive algorithm in terms of its input size. By expressing the running time as a function that recurs upon itself, we can systematically analyze and solve these recurrences to obtain a precise estimate of the algorithm's complexity.

This chapter explores the various aspects of recurrence functions, including their formulation, components, and the techniques used to solve them. We will explore how these functions capture the essence of recursive calls and their subsequent impact on the overall computational cost of the algorithm. Understanding recurrence functions is crucial for accurately predicting the performance of recursive algorithms and optimizing their implementation.

The chapter is structured as follows:

- Recursive algorithms
- Recurrence functions
- Unfolding recurrence functions

Recursive algorithms

Imagine a traditional Matryoshka doll, commonly found in Russia. When you open the outermost doll, you find a smaller doll inside. Opening this smaller doll reveals an even smaller one within, and this process continues until you reach the smallest, indivisible doll at the core. This fascinating nested structure is a perfect analogy for understanding recursion in algorithms.

Just like the Matryoshka doll, a **recursive algorithm** solves a problem by breaking it down into smaller instances of the same problem. Each instance is simpler than the last, until a base case is reached, which can be solved directly without further recursion. This self-referential approach is a fundamental concept in computer science and is used to solve complex problems in a straightforward and elegant manner.

In this section, we will explore the principles of recursive algorithms, starting with their basic definition and properties. We will examine how recursion works, its advantages and potential pitfalls, and the types of problems for which it is particularly well suited. By understanding the core concepts of recursion, you will be able to design and implement recursive solutions effectively, leveraging their power to simplify and solve intricate problems.

The basics of recursion

The evolution of computer science has been marked by a profound understanding of how to model intelligence and problem-solving through computation. At its core, computation involves breaking down complex tasks into simpler, repeatable steps. This foundational concept has given rise to the technology that powers our modern computers.

One fundamental way to implement repetition in computing is through **loops** or iterative processes. **Loops** are constructs that repeat a block of instructions a number of times or until a specific condition is met. They are straightforward to understand, their complexity is easy to estimate, and they are relatively simple to debug. However, while loops are efficient for many tasks, they can sometimes be computationally expensive and less intuitive for certain types of problems.

An alternative approach to managing repetition is through **recursion**. Unlike loops, which iterate over a sequence of steps, **recursion** is a function calling itself to solve smaller instances of the original problem which is called a subproblem. This method is rooted in the **divide and conquer** strategy, where a problem is divided into smaller subproblems, each of which is solved recursively until a base case is reached. This approach can often lead to more elegant and simpler solutions, especially for problems that naturally fit a recursive structure.

The following two Python code examples demonstrate how to implement the factorial(n) algorithm using two different methods: the iterative approach (using loops) and the recursive approach:

```
def factorial_iterative(n):  
    result = 1  
    for i in range(1, n + 1):
```

```
        result *= i
    return result
```

In the iterative implementation, the repetition is linear and straightforward. One advantage of iterative, non-recursive implementations is that their complexity and running time are easy to estimate. In the preceding function, the running time depends on the number of times the `for i in range(1, n + 1):` loop is executed. Given the loop's range, it is clear that the time complexity is $T(n) = n$, which simplifies to $O(n)$.

Now, let's have a look at the recursive implementation of `Factorial(n)`:

```
def factorial_recursive(n):
    # Base case
    if n == 0:
        return 1
    # Recursive case
    else:
        return n * factorial_recursive(n - 1)
```

In the recursive implementation, the first noticeable difference is the absence of loops. Instead, repetition is achieved through nested self-calls of the function. This makes tracking the flow of a recursive algorithm more complex. Additionally, estimating the running time of a recursive algorithm is not as straightforward as it is for non-recursive algorithms.

To estimate the running time, we need to model the recursion process using a computational function called a recurrence function, which will be discussed in detail in the next section. For the recursive `Factorial(n)` function, the recurrence function is $T(n) = T(n - 1) + c$, where $T(n)$ represents the running time for computing the factorial of n , and c is a constant representing the time for the non-recursive operations. In the next chapter, we will see that solving this recurrence function shows that the computational complexity of the recursive `Factorial(n)` is $O(n)$, which is no better than the iterative implementation.

From the recursive algorithm of `Factorial(n)`, we can identify three components that are common in all recursive algorithms:

- **Recursive calls:** A function that calls itself within its definition, such as `factorial_recursive(n - 1)`. Each call works on a smaller or simpler version of the original problem.
- **Base case:** The condition under which the recursive calls stop. It prevents infinite recursion and provides a straightforward solution to the simplest version of the problem. In the recursive factorial, this component is as follows:

```
    if n == 0:
        return 1
```


- **Recursive case:** The part of the function where the recursion occurs. It breaks the problem into smaller subproblems and calls the function itself to solve these subproblems. In the recursive factorial, this component is `return n * factorial_recursive(n - 1)`.

Unlike `factorial_recursive(n)`, the recursive implementation can often offer better running time than the non-recursive one. Let's consider the following example of calculating the power of a number using a recursive approach that leverages this technique to achieve improved efficiency:

```
def power_recursive(base, exponent):  
    # Base case  
    if exponent == 0:  
        return 1  
    # Recursive case  
    elif exponent % 2 == 0:  
        half_power = power_recursive(base, exponent // 2)  
        return half_power * half_power  
    else:  
        return base * power_recursive(base, exponent - 1)
```

Let's explain the code:

- **Base case:** If the exponent is 0, the result is 1, since any number raised to the power of 0 is 1.
- **Recursive case:** If the exponent is even, the problem is divided into two smaller subproblems by halving the exponent. We recursively calculate `half_power` for base raised to the power of half the exponent and then square the result.

If the exponent is odd, the problem is reduced to an even exponent by subtracting 1 from the exponent, resulting in an additional multiplication by base after the recursive call.

- **Recursive call:** The function calls itself recursively in `power_recursive(base, exponent // 2)` and `power_recursive(base, exponent - 1)`.

This method ensures that the number of recursive calls grows logarithmically with respect to the exponent, achieving a time complexity of $O(\log n)$. This is a significant improvement over the iterative approach, which has a linear time complexity of $O(n)$.

Types of recursion

Recursion can be broadly classified into two types: direct recursion and indirect recursion. Understanding these types helps in identifying the nature of recursive calls and their potential impacts on performance and complexity.

Direct recursion

Direct recursion occurs when a function calls itself directly. This is the most common form of recursion, where the function is self-referential. All examples of recursive algorithms so far in this book have been direct recursive types. Some major use cases of direct recursion are as follows:

- Simplifies problems that can naturally be divided into identical subproblems
- Easier to understand and debug compared to indirect recursion
- Commonly used in problems such as **factorial calculation**, **Fibonacci series**, and **tree traversals**

There are several types of direct recursion, each suited to different kinds of problems and algorithmic strategies. Here are the main types of direct recursion:

- **Tail recursion:** This is a special case of recursion where the recursive call is the very last action a function takes before returning. This means no further operations are performed on the result of the recursive call. The advantage of tail recursion is that it can often be optimized by compilers to avoid stack overflow. Instead of creating a new stack frame for each recursive call, the compiler can reuse the existing frame, effectively turning the recursion into a loop. An illustrative example is the `factorial_tail` function, which calculates the factorial of a number by directly calling itself as its final step:

```
def factorial_tail(n, accumulator=1):
    if n == 0:
        return accumulator
    else:
        return factorial_tail(n - 1, n * accumulator)
```

One application of tail recursion is in managing linked lists. Although readers may already be familiar with linked list data structures, we will explore them in detail in *Chapter 12*. In brief, linked lists are a fundamental data structure made up of nodes, where each node contains a value and a reference (or link) to the next node in the sequence. The inherent recursive structure of linked lists – where each node can be viewed as a smaller version of the entire list – makes recursion an ideal approach for performing operations such as traversal, insertion, and deletion.

Here is one example of using tail recursion in the recursive traversal of a linked list. Traversing a linked list is typically done recursively by processing the current node and then moving to the next node through a recursive call. In a tail-recursive approach, this call to process the next node is the final operation in the function. Here is a simple piece of Python code:

```
def traverse_linked_list(node):
    if node is None:
        return
    print(node.value)
    traverse_linked_list(node.next)
```

- **Head recursion:** This is characterized by a function's initial action being a recursive call to itself. This means all other operations within the function are postponed until after this recursive call completes. While less prevalent than tail recursion, head recursion does have its uses. However, it is generally considered less efficient due to the need to maintain a stack of pending operations until the recursive calls unwind. The following `head_recursive` is an example of this type of recursive function:

```
def head_recursive(n):  
    if n == 0:  
        return  
    else:  
        head_recursive(n - 1)  
        print(n)
```

- **Linear recursion:** In this type of recursion, a function makes at most one recursive call during each invocation. This leads to a simple chain of recursive calls, resembling a straight line. The following is a simple example of linear recursion:

```
def linear_recursive(n):  
    if n == 0:  
        return 0  
    else:  
        return n + linear_recursive(n - 1)
```

- **Tree recursion:** Unlike linear recursion, in tree recursion, a function calls itself multiple times within a single activation. This results in a branching structure of recursive calls, resembling a tree. Examples include the Fibonacci sequence calculation, tree traversals, and the quick sort algorithm:

```
def fun(n):  
    if (n > 0):  
        print(n, end=" ")  
        fun(n - 1)  
        fun(n - 1)
```

- **Binary recursion:** This is a pattern where a function makes two recursive calls to itself within a single invocation. This approach is frequently employed in divide-and-conquer algorithms, which break down a problem into two smaller subproblems that are solved recursively. The solutions to these subproblems are then combined to obtain the solution to the original problem:

```
def fibonacci_binary_recursive(n):  
    if n <= 1:  
        return n  
    else:  
        return fibonacci_binary_recursive(n - 1)  
+         fibonacci_binary_recursive(n - 2)
```

- **Multiple recursion:** This is a type of recursion where a function makes more than two recursive calls to itself within a single invocation. This pattern is less common than linear or binary recursion but can be useful for problems that inherently decompose into more than two subproblems. Each of these subproblems is then solved recursively, and their solutions are combined to obtain the final result. A simple multiple recursion has been implemented in the next example (multiple_recursive):

```
def multiple_recursive(n):  
    if n <= 1:  
        return 1  
    else:  
        return multiple_recursive(n - 1) + multiple_recursive(n  
- 2) + multiple_recursive(n - 3)
```

A more sophisticated example is as follows:

```
def ternary_search(arr, target, start, end):  
    if start > end:  
        return -1 # Target not found  
    else:  
        mid1 = start + (end - start) // 3  
        mid2 = start + 2 * (end - start) // 3  
        if arr[mid1] == target:  
            return mid1  
        elif arr[mid2] == target:  
            return mid2  
        elif arr[mid1] > target:  
            return ternary_search(arr, target, start, mid1 -  
1) # First recursive call  
        elif arr[mid2] < target:  
            return ternary_search(arr, target, mid2 + 1, end) #  
Second recursive call  
        else:  
            return ternary_search(arr, target, mid1 + 1, mid2 -  
1) # Third recursive call
```

In this example, the ternary_search function performs a ternary search on a sorted array. It divides the array into three roughly equal parts and makes three recursive calls to search each part, illustrating the concept of a multiple recursion.

- **Nested recursion:** This is a more intricate form of recursion where a function's recursive call doesn't simply pass a modified argument, but includes another recursive call as part of the argument itself. This means the depth of recursion can increase rapidly, making it a less common and more complex type of recursion to analyze and understand:

```
def nested_recursive(n):  
    if n > 100:  
        return n - 10  
    else:  
        return nested_recursive_function(nested_recursive(n +  
11))
```

Indirect recursion

Indirect recursion occurs when a function, let's say function A, calls another function, function B, and then function B (or a sequence of other functions) eventually calls function A again. This creates a cycle or chain of function calls where the original function is indirectly invoked through other functions. In the following example, functionA calls functionB, and functionB calls functionA, forming an indirect recursion:

```
def functionA(n):  
    if n <= 0:  
        return "End"  
    else:  
        return functionB(n - 1)  
def functionB(n):  
    if n <= 0:  
        return "End"  
    else:  
        return functionA(n - 2)
```

In the example of indirect recursion, functionA calls functionB, which then calls functionA again. This cycle continues until the base condition in either function is met ($n \leq 0$), terminating the recursion. This form of recursion can involve more than two functions, creating a complex chain of calls.

Some major use cases of indirect recursion are as follows:

- Useful for problems that require multiple stages of transformation or processing
- Can be more difficult to trace and debug due to the involvement of multiple functions
- Commonly seen in mutually recursive algorithms and certain state machines

Understanding the differences between direct and indirect recursion allows for better selection of the appropriate recursive approach for a given problem. Direct recursion is straightforward and widely used, while indirect recursion, though more complex, can be powerful in specific scenarios requiring interdependent function calls.

Recursive problem-solving

Recursive problem-solving often employs the *divide, conquer, and combine* strategy. This framework is highly effective for breaking down complex problems into simpler subproblems, solving these subproblems independently, and then combining their results to form the final solution. Let's explore this framework in detail:

- **Divide:** In the divide step, the problem is broken down into smaller subproblems that are easier to solve. This step involves identifying how to partition the original problem into smaller parts. The key is to ensure that the subproblems are of the same nature as the original problem, but simpler or smaller in size.
- **Conquer:** In the conquer step, the subproblems are solved recursively. If the subproblems are still too large, they are further divided using the same divide, conquer, and combine approach. This process continues until the subproblems reach a base case, which can be solved directly without further recursion.
- **Combine:** In the combine step, the solutions to the subproblems are merged to form the solution to the original problem. This step involves integrating the results of the recursive calls to obtain the final answer.

It is worth mentioning that the key element in this framework is to identify the *subproblem*. A **subproblem** is similar to the main problem but smaller in size. Essentially, a subproblem is a smaller instance of the original problem.

This framework will be explored in detail in upcoming chapters, especially in sorting algorithms such as merge sort. To give the reader a taste of this framework, consider the following merge sort algorithm, which will be discussed in *Chapter 6*:

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]
    sorted_left = merge_sort(left_half)
    sorted_right = merge_sort(right_half)
    return merge(sorted_left, sorted_right)
def merge(left, right):
    sorted_array = []
```

```
i = j = 0
while i < len(left) and j < len(right):
    if left[i] < right[j]:
        sorted_array.append(left[i])
        i += 1
    else:
        sorted_array.append(right[j])
        j += 1
# Append any remaining elements
sorted_array.extend(left[i:])
sorted_array.extend(right[j:])
return sorted_array
```

Let's briefly explain the divide, conquer, and combine framework in the merge sort algorithm:

- **Divide:** The `merge_sort` function splits the array into two halves, `left_half` and `right_half`. This is done using the midpoint of the array.
- **Conquer:** The `merge_sort` function is called recursively on both halves. Each recursive call further divides the halves until the base case is reached, where the array length is 0 or 1 (already sorted).
- **Combine:** The merge function is used to merge the sorted halves. It iterates through both halves, comparing elements and appending the smaller element to the sorted array. Any remaining elements from either half are then appended.

In the upcoming chapters, we will illustrate how identifying these three components is crucial for the design and analysis of recursive algorithms.

Advantages and challenges of recursion

Recursion is a fundamental concept in computer science and algorithm design. While it offers several advantages, it also comes with its own set of challenges. Understanding both can help you decide when and how to use recursion effectively.

The following are three major advantages of recursive solutions:

- **Simplicity and clarity:** Recursion often provides a more straightforward and intuitive solution for problems that have a repetitive or self-similar structure, such as tree traversals, factorial calculations, and the Fibonacci sequence. Recursive solutions can be also more concise and easier to read compared to iterative solutions. This can make the code more maintainable and easier to understand.

- **Reduction of complex problems:** Recursion simplifies the process of breaking down complex problems into simpler subproblems. This is particularly useful in divide-and-conquer algorithms, such as merge sort and quicksort. Recursive functions can also lead to elegant and clean code, especially when the problem is naturally recursive, such as in dynamic programming and combinatorial problems.
- **Implicit state management:** Recursive calls inherently manage states through the call stack, eliminating the need for explicit state management in many cases. This can simplify the logic and reduce the chance of errors related to state handling.

On the other side, recursive algorithms come with some challenges. The following are some major challenges and disadvantages:

- **Performance concerns:** Each recursive call adds a new frame to the call stack, which can result in significant overhead compared to iterative solutions, especially for deep recursion. Deep or unbounded recursion can lead to stack overflow errors if the recursion depth exceeds the maximum stack size. This is a common issue in languages with limited stack memory.
- **Complexity in debugging:** Debugging recursive functions can be challenging because it involves tracing multiple layers of function calls. Understanding the flow of execution and the state at each level of recursion can be difficult. Incorrectly defined base cases can also lead to infinite recursion or incorrect results. Ensuring that all base cases are correctly handled is crucial for the correctness of the algorithm.
- **Space complexity:** Recursive algorithms can have high space complexity due to the additional memory required for the call stack. This can be problematic for problems with large input sizes or deep recursion. Recursive functions often require additional memory for each recursive call, which can lead to increased auxiliary space usage compared to their iterative counterparts.

In addition, when designing a recursive algorithm, we need to pay attention to the following practical issues. Some of them will be discussed in detail in upcoming chapters:

- **Tail recursion:** Some languages and compilers optimize tail-recursive functions to iterative ones, reducing the call stack overhead. When possible, design recursive functions to be tail-recursive.
- **Memoization:** Use memoization to store the results of expensive recursive calls and avoid redundant computations. This technique is especially useful in dynamic programming. For more information, see *Chapter 10*.
- **Iterative alternatives:** Consider iterative solutions when recursion leads to performance or memory issues. Iterative algorithms can often achieve the same results more efficiently.

In this section, the concept of recursion was introduced, highlighting its importance in computer science and algorithm design. Recursion, a method where a function calls itself to solve smaller instances of the same problem, was explained with examples such as factorial calculations and the Fibonacci sequence. The section emphasized the divide, conquer, and combine framework, illustrating how problems are broken down into simpler subproblems, solved independently, and then combined to form the final solution, with merge sort as a key example.

Various types of recursion, including direct and indirect recursion, were examined, along with their respective examples. The advantages of recursion, such as simplicity, clarity, and effective problem decomposition, were discussed, alongside its challenges, such as function call overhead, potential stack overflow, and debugging complexity. Practical considerations, including tail recursion, memoization, and iterative alternatives, were provided to optimize recursive functions.

In conclusion, this section offered a comprehensive overview of recursion, detailing its principles, advantages, and challenges. It laid a strong foundation for understanding and applying recursive problem-solving techniques in algorithm design. The next section will discuss running time models for recursive algorithms, called **recurrence functions**, essential for complexity analysis.

Recurrence functions

The function representing the running time of an incremental (non-recursive) algorithm can be determined straightforwardly due to the linear, sequential nature of these algorithms. For example, consider the incremental implementation of the factorial of n . *Table 4.1* illustrates the algorithm along with the associated computational cost in the second column.

The function describing the running time of recursive algorithms is not as straightforward as it is for incremental algorithms. To analyze the running time of recursive algorithms, we use **recurrence functions** or **recurrence relations**. These concepts are adapted from mathematics.

In mathematics, a **recurrence function** is an equation that defines the n th term of a sequence in terms of its preceding terms. Typically, only the previous k terms of the sequence are involved in the equation, where k is a parameter that does not depend on n . This parameter k is known as the order of the recurrence function. Once the values of the first k terms are known, the remaining terms of the sequence can be determined by repeatedly applying the recurrence function.

Instruction	Cost
<code>def factorial_incremental(n):</code>	-
<code>result = 1</code>	c
<code>for i in range(1, n + 1):</code>	$n + 1$
<code>result *= i</code>	n
<code>return result</code>	c
Running time function $T(n)$	$T(n) = 2n + 1 + 2c$
Complexity	$O(n)$

Table 4.1: Incremental implementation of factorial of n and its running time

For example, the Fibonacci sequence can be defined by a recurrence function where each term is the sum of the two preceding terms. By knowing the first two terms, the entire sequence can be generated (See *Example 4.2*). Similarly, in algorithm analysis, recurrence functions help us model the running time of recursive algorithms by expressing the time complexity of a problem in terms of smaller subproblems.

In the context of recursive algorithms, a recurrence function captures the essence of how an algorithm divides a problem into subproblems, solves them recursively, and combines their solutions. This approach allows us to systematically analyze and predict the performance of recursive algorithms, even though their execution paths and computational costs are not as immediately apparent as those of incremental algorithms.

Two major categories of recurrence functions are **subtractive recurrence** and **divide-and-conquer recurrence** functions. Both subtractive and divide-and-conquer recurrences are ways to define problems recursively, where the solution to a larger problem is expressed in terms of solutions to smaller subproblems. However, they differ in how they break down the problem and how the solutions to the subproblems are combined. Let's explore these two types of recurrences in detail.

Subtractive recurrence functions

Subtractive recurrence functions (also called **subtract-and-conquer** and **decrease-and-conquer**) are a type of linear recurrence function where the next term in a sequence is a linear combination of previous terms. These functions are crucial in both mathematics and computer science for modeling problems that have recursive structures.

A linear recurrence function is an equation that relates a term in a sequence to its preceding terms using linear coefficients. The general form of a linear recurrence function of order k is as follows:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$$

Here, $c_1, c_2, \dots, c_k$ are constants, and k is the order of the recurrence function.

The general form of a subtractive recurrence function is as follows:

$$T(n) = T(n - k_1) + T(n - k_2) + \dots + f(n)$$

Here's a detailed breakdown of the components:

k_i : This is a positive integer where $0 < k_i < n$. It represents the step size by which the problem size is reduced in each recursive call. If $k_1 = k_2 = \dots = k$, the linear recurrence function can be written as follows:

$$T(n) = aT(n - k) + f(n)$$

Here, a is the number of subproblems to be solved.

$T(n - k_i)$: This is the recursive component of the recurrence function. It indicates that the function calls itself with a problem size reduced by k_i . This term captures the essence of the recursion, showing how the current problem depends on the solution to a smaller subproblem.

$f(n)$: This represents the running time of the non-recursive component of the algorithm. It encompasses the time taken by all the operations that are performed in the function, excluding the time taken by the recursive call. This could include operations such as initialization, the combination of results, or any other processing that occurs outside of the recursive calls.

The following are the main properties of subtractive recurrence functions:

- A subtractive approach breaks the problem down by subtracting a constant value (such as k) from the original problem size
- The solution to the smaller problem is then used to solve the original problem, often without needing to solve the remaining part of the original problem entirely
- This approach is less common than divide-and-conquer, but it can be effective for specific problems where the solution can be derived from a solution to a slightly smaller problem

Let's examine this type of recurrence function in two well-known Factorial and Fibonacci algorithms.

Example 4.1

Implementing factorial of n using linear recurrence: The factorial of a number n can be described using a linear recurrence function. In this case, the problem size is reduced by $k = 1$ in each recursive step, making it a straightforward linear recurrence.

The recurrence function for the **factorial function** can be written as follows:

$$T(n) = T(n - 1) + O(1)$$

Here's the breakdown:

- $a = 1$ is the number of subproblems in each recursion is 1.
- $k = 1$ means the problem size is reduced by 1 in each recursive step.
- $T(n - 1)$ is the recursive component. It shows that the factorial function calls itself with $n - 1$.
- $O(1)$ is the non-recursive component. It represents the constant time operations performed at each step, such as the multiplication and the function call overhead.

The following code is the Python implementation of the factorial function using linear recursion:

```
def factorial_recursive(n):  
    # Base case: if n is 0, the factorial is 1  
    if n == 0:  
        return 1  
    # Recursive case: multiply n by the factorial of (n - 1)  
    else:  
        return n * factorial_recursive(n - 1)
```

This recurrence function indicates that the time complexity is $O(n)$, as the function makes n recursive calls and performs a constant amount of work for each call.

The running time of the factorial algorithm for both the incremental and recursive approaches cannot be better than $\theta(n)$. This limitation arises from the nature of the problem itself. To compute the factorial of n , it is necessary to process every single number from 2 to n . There are no shortcuts or optimizations that can reduce the computational cost below this linear time complexity.

Example 4.2

The **Fibonacci sequence** is one of the most well-known examples of a linear recurrence function. It is defined by the following recurrence function:

$$F(n) = F(n - 1) + F(n - 2)$$

It is also defined with the initial conditions:

$$F(0) = 0$$

$$F(1) = 1$$

This means that each term in the Fibonacci sequence is the sum of the two preceding terms. The Fibonacci sequence can be implemented using a recursive algorithm as follows:

```
def fibonacci_recursive(n):  
    if n <= 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

The recurrence function for the linear recursive algorithm for the Fibonacci sequence is as follows:

$$T(n) = T(n - 1) + T(n - 2) + O(1)$$

Here is the breakdown:

- $T(n - 1)$ represents the time complexity of computing the $(n - 1)$ th Fibonacci number. The function calls itself with $n - 1$ as the argument.
- $T(n - 2)$ represents the time complexity of computing the $(n - 2)$ th Fibonacci number. The function calls itself with $n - 2$ as the argument.
- $O(1)$ accounts for the constant time operations performed in each recursive call, such as adding the results of the recursive calls and any other constant-time operations.

The recursive implementation of the Fibonacci sequence has exponential time complexity, $O(2^n)$. This inefficiency arises because the same calculations are repeated multiple times.

Linear recurrence functions, such as the Fibonacci sequence, are not just theoretical constructs; they have practical applications in various fields in addition to algorithm design in computer science, such as predicting stock prices, analyzing investment strategies, and studying population growth in ecology and epidemiology.

Divide-and-conquer recurrence functions

In divide-and-conquer recurrence functions, the problem size is divided into smaller subproblems, which are solved independently and then combined. These are typically represented as follows:

$$T\left(n\right) = aT\left(\frac{n}{b}\right) + f\left(n\right)$$

Here, a is the number of subproblems, b is the factor by which the problem size is divided, and $f(n)$ is the time complexity of dividing the problem and combining the results.

It's important to note that we generally have some constraints on a and b , which will be discussed in the next chapter when introducing the master theorem for solving recurrence functions. In simple terms, the constraints on the parameter are:

- $a \geq 1$ means the number of subproblems cannot be smaller than one
- $b > 1$ means each subproblem should be smaller than the original problem, ensuring that the algorithm eventually terminates

Additionally, both a and b should be constants and independent of n .

The following are the main properties of divide-and-conquer recurrence functions:

- A divide-and-conquer approach breaks the problem into two or more smaller subproblems of roughly equal size
- The solutions to these subproblems are then combined to get the solution to the original problem
- This approach is often used for problems that can be naturally divided into independent subproblems

Let's examine the recurrence function of merge sort as an example, which demonstrates the divide-and-conquer approach.

Example 4.3

An example is the recurrence function seen in algorithms such as merge sort (see previous section), where the problem size is halved at each step, and the subproblems are solved recursively before being combined.

The recurrence function for the merge sort algorithm is as follows:

$$T(n) = 2T(n/2) + O(n)$$

Here is the breakdown:

- $a = 2$ is the number of subproblems in each recursion.
- $T(n/2)$ means that the problem is divided into two subproblems ($b = 2$), each of size $n/2$. The time complexity for solving each subproblem is $T(n/2)$.
- $O(n)$ accounts for the time taken to merge the sorted subarrays, which is linear with respect to the size of the array.

In the previous example, a typical divide-and-conquer recurrence was demonstrated along with its key parameters. In the next example, we will examine the recurrence function for binary search, which also follows a divide-and-conquer approach similar to merge sort.

Example 4.4

Binary search is a classic example of a divide-and-conquer algorithm. Binary search works by repeatedly dividing the search interval in half and checking whether the target value is in the left or right half. Here's the Python implementation of binary search using a recursive approach:

```
def binary_search(arr, target, low, high):
    if low > high:
        return -1 # Target is not in the array
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, high)
    else:
        return binary_search(arr, target, low, mid - 1)
```

The binary search algorithm splits the problem size in half with each recursive call. This can be expressed with the following recurrence function:

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Here is the breakdown:

- $T\left(\frac{n}{2}\right)$ represents the recursive call to search within one-half of the array. From the recurrence function, we know that $a = 1$ and $b = 2$.
- $O(1)$ accounts for the constant time operations, such as comparing the target with the middle element and determining which half to search next.

In this section, we explored recurrence functions, focusing on both subtractive and divide-and-conquer approaches. We learned how to identify each type and determine the parameters of their components. Through this analysis, we gained the ability to classify recurrence functions accurately.

Additionally, we applied our understanding of recurrence functions to well-known algorithms, such as search and sort algorithms. We examined specific examples, such as merge sort and binary search, to see how recurrence functions operate in practice, allowing us to observe the direct impact of these concepts on the efficiency and complexity of these algorithms. This comprehensive study provided a solid foundation for analyzing and designing recursive algorithms effectively.

Unfolding recurrence functions

So far, we have discussed the recursive structures in algorithm design and introduced different types of recursions. We then focused on two types of recurrence functions: subtractive and divide-and-conquer. *Table 4.2* provides a summary of the properties of these two recurrence functions. As the table illustrates, divide-and-conquer recurrences generally offer more efficient solutions, although the efficiency highly depends on the specific problem being solved.

In this section, we will demystify recurrence functions. This understanding will be essential in the next chapter, where we will solve recurrence functions and estimate their computational complexity, or in other words, their rate of growth. By unfolding these functions, we can gain insights into how recursive algorithms operate and how their performance scales with input size. This knowledge will enable us to analyze and optimize algorithms more effectively.

Regardless of the type of recurrence function, they consist of two main elements: the recursive component and the non-recursive component. The **recursive component** reduces the problem size, either through subtraction or division. On the other hand, the **non-recursive component**, also known as the **driving function**, is expressed as a function of n , representing the size of the problem.

The recursive component defines how the problem is broken down into smaller subproblems. For subtractive recurrence functions, this reduction is typically by a constant amount, such as $T(n) = T(n - k)$. For divide-and-conquer recurrence functions, the problem is divided by a factor, such as $T(n) = a T(\frac{n}{b})$. Since division reduces the problem size more rapidly than subtraction, divide-and-conquer recursion generally leads to more efficient algorithms. This is analogous to how multiplication grows faster than addition; in the same way, dividing the problem size tends to reduce it more quickly than subtracting a constant. Consequently, algorithms based on divide-and-conquer strategies are often more efficient than those using simple subtractive recursion.

Feature	Subtractive Recurrence	Divide-and-Conquer Recurrence
Problem breakdown	Decreases by a constant amount.	Divided into roughly equal-sized subproblems.
Subproblem combination	A solution from one subproblem; others may be ignored.	Solutions combined to find the overall solution.
Number of subproblems	One (in most cases).	Two or more.
Applicability	Specific problems where the solution can be derived from a slightly smaller problem.	Problems that are naturally divisible into independent subproblems.

Feature	Subtractive Recurrence	Divide-and-Conquer Recurrence
Efficiency	Can be less efficient than divide-and-conquer for problems where dividing into subproblems leads to a faster overall solution. This is because the problem size decreases slowly in subtractive recurrence.	Generally, more efficient for problems that can be broken down into multiple subproblems. This is because it can leverage parallel processing and often leads to logarithmic or linearithmic time complexities. However, the overhead of dividing and combining subproblems can sometimes make it less efficient than subtractive recurrence for very small problem sizes.
Example algorithms	Factorial, binary search, and Fibonacci.	Merge sort, quick sort, and Strassen's matrix multiplication algorithm.

Table 4.2: Summarizing the key differences between subtractive and divide-and-conquer recurrence functions

The non-recursive component, or driving function, $f(n)$, represents the work done outside the recursive calls. It encompasses all the operations performed to divide the problem and combine the results of the subproblems. This function plays a crucial role in determining the overall time complexity of the algorithm.

Let's imagine the recurrence function as a mechanical vehicle. The recursive component acts as the engine. The larger the value of a , the slower the engine, meaning the recursive algorithm has to solve more subproblems. On the other hand, b functions as the gears. The higher the value of b , the faster the engine works. This implies that with each step (or recursion), the problem size is reduced more rapidly into smaller subproblems.

Now, you might wonder about the role of $f(n)$. The driving function, though the name might not seem very intuitive, represents the load of the vehicle. The larger $f(n)$ is in terms of its rate of growth, the more work the recursive algorithm must do to complete the task. This means that if $f(n)$ grows quickly, the algorithm will need to exert more effort at each step, akin to a vehicle carrying a heavier load.

Recursive algorithms offer elegant solutions to many problems, but their time complexity can be tricky to analyze. A helpful guideline is that the driving function, the work done outside the recursive calls, sets a lower bound on the overall complexity. In other words, a recursive algorithm cannot be faster than the time it takes to perform the non-recursive operations.

However, the driving function is not the only factor determining the final complexity. The number of subproblems created in each recursive step (often denoted by a) and the reduction factor by which the problem size shrinks (denoted by b) play crucial roles. These parameters interact with the driving function, leading to different time complexities. For example, if the number of subproblems is small and the problem size reduces significantly at each step, the impact of the driving function might be overshadowed. On the other hand, if there are many subproblems or the problem size shrinks slowly, the driving function might become the dominant factor.

To systematically analyze the interplay of all these factors and accurately estimate the time complexity of recursive algorithms, we turn to a powerful tool called the **master theorem**. This theorem provides a framework for solving recurrence functions, which are mathematical expressions that define the runtime of recursive algorithms in terms of their subproblems.

Summary

In *Chapter 4*, we explored the intricacies of recurrence functions and their crucial role in analyzing the complexity of recursive algorithms. We began by examining the structure of recursive algorithms, distinguishing between subtractive and divide-and-conquer recurrence functions. These concepts were illustrated through various examples, highlighting how different types of recurrence functions impact the overall efficiency of an algorithm.

We then explained the components of recurrence functions, emphasizing the importance of both the recursive and non-recursive (driving) components. The chapter introduced the master theorem as a powerful tool for solving recurrence functions. By applying this theorem, we demonstrated how to estimate the computational complexity of recursive algorithms, taking into account the number of subproblems, the reduction scale, and the driving function. The detailed analysis and examples provided a comprehensive understanding of how to approach and solve recurrence functions, setting the stage for more advanced topics in algorithm design and complexity analysis.

In the next chapter, we will explore how to solve recurrence functions. We will discuss several approaches to analyze recursive algorithms and estimate their running time, including the master theorem. Additionally, we will examine the applications of these methods to various recursive algorithms, gaining a deeper understanding of their time complexity.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022:
 - *Chapter 4, Divide-and-Conquer*
 - *Chapter 34, Advanced Topics*

- *Algorithm Design*. By Jon Kleinberg and Éva Tardos. First Edition. Pearson. 2005:
 - *Chapter 5, Divide and Conquer*
 - *Chapter 6, Recurrence Relations and the Master Theorem*
- *Algorithms*. By Robert Sedgewick and Kevin Wayne. Fourth Edition. Addison-Wesley Professional. 2011:
 - *Chapter 2, Principles of Algorithm Analysis*
 - *Chapter 4, Divide-and-Conquer Algorithms*
- *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. By Donald E. Knuth. Third edition. Addison-Wesley Professional. 1997:
 - *Chapter 1, Basic Concepts*
 - *Chapter 2, Information Structures*
- *Introduction to the Design and Analysis of Algorithms*. By Anany Levitin. Third Edition. Pearson. 2011:
 - *Chapter 5, Divide-and-Conquer*

Solving Recurrence Functions

In the previous chapter, we discussed the challenges of analyzing recursive algorithms, particularly in estimating their computational complexity. In this chapter, we will explore three primary methods for solving recurrence functions: substitution method, master theorem, and visualization techniques using recursion trees.

The substitution method involves constructing rigorous proofs to solve recurrence functions. This method, while sometimes intricate, is versatile and can handle a wide range of recurrence functions. In the substitution method, we employ various techniques, including mathematical induction, to validate our solutions.

The master theorem, also known as the master method, provides a systematic approach to determining the complexity of a recursive algorithm based on the parameters of its recurrence function. This theorem offers a set of straightforward rules, making it a powerful tool for analyzing many common recurrence functions.

Lastly, recursion trees help visualize the complexity of a problem by breaking down the recurrence into a tree structure. Although recursion trees do not provide direct proofs, they offer valuable insights and intuitive understanding that can guide us toward a formal solution.

Throughout this chapter, we will thoroughly examine the limitations of each method and present a comprehensive set of examples for practical application. By the end, readers will have a deeper understanding of how to effectively solve recurrence functions using these techniques.

We will discuss the following topics in this chapter:

- The substitution method
- Recursion tree as a visualization technique
- The master theorem
- Beyond the master theorem – the Akra-Bazzi method

The substitution method

The **substitution method** encompasses a variety of techniques, including induction, to provide proofs for recurrence functions. Often, we need to be innovative in making variable substitutions to transform the recurrence function into a form for which we already know the solution. One key characteristic of this method is its flexibility – there can be multiple approaches to solving the same recurrence. Although it does not offer a uniform solution for all problems, the substitution method is a powerful tool. In fact, even the master theorem (discussed in the next section) is proven using this method.

By employing the substitution method, we can address a wide range of recurrence functions. The process typically involves hypothesizing a solution, substituting it into the original recurrence, and then using induction to prove that the hypothesis is correct. This approach allows for creativity and adaptability, making it a valuable technique in algorithm analysis. While the substitution method may require careful thought and ingenuity, it provides a robust framework for tackling complex recurrences. This method's versatility and power underscore its importance in the broader context of algorithm design and analysis.

Perhaps the best way to explore the substitution method is through practical application with several examples. By working through a variety of recurrence functions, we can better understand the nuances and techniques involved in this method. This hands-on approach will not only illustrate the versatility and power of the substitution method but also demonstrate how it can be applied to solve different types of problems effectively. Let's examine some examples to see this method in action and gain a deeper appreciation of its utility in algorithm analysis. Readers should note that in this chapter, and throughout the rest of the book, when we refer to *solving a recurrence function*, we mean estimating the complexity or rate of growth. This generally involves determining $\theta(n)$ or $O(n)$, and in rare cases, $\Omega(n)$.

Iteration approach or unrolling the recurrence

In the upcoming examples, the substitution method is applied by using the iterative approach, also known as **unrolling the recurrence function**. This technique involves expanding the recurrence step by step to reveal a pattern, which helps in deriving a closed-form solution. By iteratively substituting back into the original recurrence, we can systematically identify how the terms evolve and accumulate, leading to a clearer understanding of the function's growth behavior. This method is particularly useful for solving and simplifying complex recurrence functions.

Example 5.1

Solve the following subtractive recurrence function: $T(n) = T(n - 2) + n$.

Solution: To solve the recurrence function, we can use the iteration method to find a pattern and derive a closed-form solution. Let's start by expanding the recurrence step by step:

$$T(n) = T(n - 2) + n$$

$$T(n - 2) = T(n - 4) + (n - 2)$$

$$T(n-4) = T(n-6) + (n-4)$$

...

Substitute the following back into the original recurrence:

$$T(n) = T(n-2) + n$$

$$T(n) = [T(n-4) + (n-2)] + n$$

$$T(n) = [T(n-6) + (n-4)] + (n-2) + n$$

...

Continuing this pattern, we get the following:

$$T(n) = T(n-2k) + (n + (n-2) + (n-4) + \dots + (n-2(k-1)))$$

We stop when $n-2k$ reaches the base case, which we assume is $T(0)$ or $T(1)$. For simplicity, let's assume $T(0) = 0$. So, the pattern becomes the following:

$$T(n) = T(0) + \sum_{i=0}^{k-1} (n-2i)$$

Since $T(0) = 0$, we have the following:

$$T(n) = \sum_{i=0}^{k-1} (n-2i)$$

To find the number of terms in the sum, we solve $n-2k = 0$: $k = \frac{n}{2}$.

Thus, the sum becomes the following:

$$T(n) = \sum_{i=0}^{(n/2)-1} (n-2i)$$

This is an arithmetic series where the first term $a = n$ and the last term $l = n - 2((n/2) - 1) = 2$. The number of terms is $\frac{n}{2}$.

The sum of an arithmetic series is given by the following:

$$S = \frac{\text{number of terms}}{2} \times (\text{first term} + \text{last term})$$

So, the following is the case:

$$T(n) = \frac{n/2}{2} \times (n + 2)$$

Therefore, the closed-form solution for the recurrence function is as follows:

$$T(n) = \frac{n^2}{4} + \frac{n}{2}$$

This provides us with the asymptotic complexity $T(n) = \theta(n^2)$.

In the previous example, we demonstrated how to prove a subtractive recurrence function using the substitution method.

Example 5.2

Solve the following subtractive recurrence function: $T(n) = T(n - 2) + n^2$.

Solution: This recurrence function describes an algorithm where the problem of size n is reduced to a subproblem of size $n - 2$. The work done outside of the recursive call is n^2 . We can solve this recurrence using the substitution method using iteration or unrolling the recurrence.

Let's expand the recurrence a few times:

$$T(n) = T(n - 2) + n^2$$

$$T(n) = T(n - 4) + (n - 2)^2 + n^2$$

$$T(n) = T(n - 6) + (n - 4)^2 + (n - 2)^2 + n^2$$

...

If we continue this pattern, we eventually reach the base case $T(1)$ or $T(0)$ (depending on how the base case is defined). Notice that we get a sum of squares:

$$T(n) = T(\text{base case}) + 1^2 + 2^2 + \dots + (n - 2)^2 + n^2$$

The sum of squares from 1 to n is a well-known formula:

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n + 1)(2n + 1)}{6}$$

Therefore, the following is the case:

$$T(n) = T(\text{base case}) + \frac{n(n + 1)(2n + 1)}{6}$$

The exact solution depends on the value of $T(\text{base case})$, which is in constant time ($O(1)$). Then, asymptotically, we can say the following:

$$T(n) = \Theta(n^3)$$

In this example, to implement the substitution method, we iterate or unroll the recursion to identify the growth pattern. The recurrence function $T(n) = T(n - 2) + n^2$ has a time complexity of $\Theta(n^3)$. This indicates that the algorithm's runtime grows cubically with the input size.

Example 5.3

Solve the following subtractive recurrence function: $T(n) = T(\log n) + n$.

Solution: We can solve it by unrolling the recurrence and finding a pattern.

Let's explain the recurrence function. This recurrence function involves a logarithmic decrease in the subproblem size. The recurrence function $T(n) = T(\log n) + n$ describes an algorithm where the following is the case:

- The problem of size n is reduced to a subproblem of size $\log n$
- The work done outside of the recursive call is n

Let's use an iterative approach by unrolling the recurrence to understand the behavior of the function:

$$T(n) = T(\log n) + n$$

$$T(n) = T(\log \log n) + \log n + n$$

$$T(n) = T(\log \log \log n) + \log \log n + \log n + n$$

...

Note that each iteration applies the logarithm function to the argument of T . The number of iterations before we reach a base case (such as $T(1)$) is roughly the number of times we can take the logarithm of n before it becomes a constant. This is approximately $\log^* n$ (the iterated logarithm).

The total work done in all iterations is approximately the following:

$$n + \log n + \log \log n + \log \log \log n + \dots (\log^* n \text{ terms})$$

The dominant term in the preceding sum is n . Each subsequent term grows much slower than the previous one due to the repeated application of the logarithm. Therefore, we can say the following:

$$T(n) = O(n)$$

In the previous example, the algorithm's runtime is primarily dominated by the work done outside the recursive calls (n). The recursive calls do not significantly contribute to the overall runtime because the problem size decreases very rapidly with each recursion (logarithmically). The iterated logarithm ($\log^* n$) is a very slow-growing function. For all practical purposes, it can be considered a constant. This is why we can say that the time complexity is $O(n)$ even though there are $\log^* n$ terms in the summation.

Guessing and induction approach

Example 5.3 can be solved by guessing and induction approach. We solve $T(n) = T(\log n) + n$ using the substitution method by guessing that $T(n) = O(n)$ and proving it by induction. In *Example 5.4*, we demonstrate how to solve a recurrence function using the guessing and induction approach.

Example 5.4

Solve $T(n) = T(\log n) + n$ by guessing $T(n) = O(n)$.

We solve $T(n) = T(\log n) + n$ using the substitution method by guessing that $T(n) = O(n)$ and then proving it by induction.

Here is the step-by-step solution:

1. **Guess the form:**

We guess that $T(n) = O(n)$. Specifically, let's assume $T(n) \leq cn$ for some constant $c > 0$.

2. **Base case:**

We need to establish a base case. For small values of n (e.g., $n = 1$), the recurrence function simplifies. Assume $T(1)$ is a constant c_0 . Since we are concerned with asymptotic behavior, we focus on larger values of n .

3. **Induction hypothesis:**

Assume that $T(k) \leq ck$ holds for all $k \leq n$. We need to show that $T(n) \leq cn$ also holds.

4. **Induction step:**

Using the recurrence function, we have $T(n) = T(\log n) + n$.

By the induction hypothesis, $T(\log n) \leq c \log n$. Substituting this into the recurrence function, we get $T(n) \leq c \log n + n$.

For our assumption $T(n) \leq cn$ to hold, we need $c \log n + n \leq cn$.

5. **Simplify the inequality:**

To satisfy the inequality, we need $\frac{c \log n}{n} \leq c - 1$.

As n grows large, $\frac{\log n}{n}$ approaches 0. Therefore, there exists a sufficiently large n such that $\frac{\log n}{n} \leq \frac{c-1}{c}$.

6. **Choosing the constant:**

We can choose c such that the preceding condition holds for sufficiently large n . For instance, for $c = 2$: $\frac{2 \log n}{n} \leq 1$.

This inequality holds for sufficiently large n . Therefore, our assumption that $T(n) = O(n)$ is valid. Thus, we have shown that the solution to the recurrence is $T(n) = T(\log n) + n$ is $T(n) = O(n)$.

By this method of guessing and using induction, we confirmed that our initial guess was correct and that the complexity of the recurrence is linear in terms of n .

The guessing step in the guessing and induction method is a critical part of solving recurrence functions. It involves making an educated hypothesis about the form of the solution based on the structure of the recurrence function. Here's how the guessing step is typically done:

- **Understanding the recurrence function:** We begin by closely examining the recurrence function. We look at the components, such as how the problem size reduces in each recursive call and the non-recursive cost function $f(n)$. Then, we compare the recurrence to familiar patterns, such as those seen in common algorithms (e.g., divide-and-conquer algorithms such as merge sort or binary search).

- **Analyzing the growth rate:** We consider the terms in the recurrence function to infer how they might contribute to the overall complexity. For instance, if the recurrence involves terms such as $T(\frac{n}{2})$ or $T(\frac{n}{4})$, we might guess a logarithmic component in the solution since each step reduces the problem size by a factor. If the recurrence includes an additive linear term, such as n , this suggests the solution might involve a linear or non-linear growth $n \log n$.
- **Leveraging experience and patterns:** We use our knowledge of common recurrence functions and their solutions. For example, if the recurrence looks like $T(n) = 2T(\frac{n}{2}) + n$, we might guess $T(n) = O(n \log n)$ because this form is typical for divide-and-conquer algorithms. For the recurrence $T(n) = T(n-1) + 1$, we might guess $T(n) = O(n)$ because each step reduces the problem size by one with a constant cost.
- **Making an educated guess:** Based on our analysis, we hypothesize a potential form for $T(n)$. This could be $T(n) = O(n^2)$, $T(n) = O(n \log n)$, or even $T(n) = O(\log n)$. The guess should reflect the expected behavior of the recurrence's growth. For example, if we see a recursive call with $T(\frac{n}{2})$ and a linear term, $O(n \log n)$ might be a reasonable guess.
- **Refining the guess:** Sometimes, we might need to refine our guess after initial attempts to validate it using induction. For example, if our guess $T(n) = O(n)$ doesn't satisfy the induction step, we might need to consider a higher-order term, such as $O(n \log n)$.

By following these steps, we systematically arrive at a plausible solution that can be rigorously validated. The guessing step combines intuition, experience, and analysis to propose a solution that you can then confirm through induction.

Variable change approach

There are irregular recurrence functions that conventional methods, such as those outlined in *The master theorem* section or even the *Recursion tree as a visualization technique* section, are unable to solve. In these cases, the substitution method offers an alternative solution, alongside advanced, generalized methods, such as the **Akra-Bazzi method** (see the *Beyond the master theorem – the Akra-Bazzi method* section).

The **substitution method** involves transforming the original recurrence into a more manageable form through variable change. This approach can reveal patterns and simplify the analysis, making it easier to find a closed-form solution. By carefully choosing new variables, we can convert complex recurrence functions into forms that are more straightforward to solve.

In the next example, we demonstrate how to implement the substitution method using variable change to tackle irregular recurrence functions effectively. This example will illustrate the power and flexibility of the substitution method in dealing with complex recurrence functions that other methods cannot handle.

Example 5.5

Solve $T(n) = 2T(\sqrt{n}) + \log n$.

In the previous chapter, we categorized recurrence functions into two groups: subtracting (decreasing) and divide-and-conquer (dividing) recurrence functions. While the function $T(n) = 2T(\sqrt{n}) + \log n$ could be considered a sub-category of divide-and-conquer recurrence functions, it more accurately belongs to a family of **substitution recurrence functions**.

To solve this recurrence function, we can use a change of variables to simplify the recurrence and make it easier to analyze.

Let $n = 2^m$.

Then, $\log n = m$.

Rewriting the recurrence in terms of m , we get the following:

$$T(2^m) = 2T(2^{m/2}) + m$$

To simplify notation, let $S(m) = T(2^m)$. Then we have the following:

$$S(m) = 2S\left(\frac{m}{2}\right) + m$$

This is a familiar form of recurrence function for algorithms such as the merge sort algorithm (see Chapter 6), which has the recurrence $T(n) = 2T\left(\frac{n}{2}\right) + n$ with an asymptotic bound of $\Theta(n \log n)$. Therefore, $S(m) = \Theta(m \log m)$. By substituting m with $\log n$, we get $T(n) = \Theta(\log n \log \log n)$. However, for the sake of thoroughness, let's assume we do not know this result and solve it in detail.

Now, we solve this new recurrence function $S(m)$. Let's unroll the recurrence a few steps to identify a pattern:

$$S(m/2) = 2S\left(\frac{m}{4}\right) + \left(\frac{m}{2}\right)$$

Substitute $S\left(\frac{m}{2}\right)$ into the first equation:

$$S(m) = 2\left[2S\left(\frac{m}{4}\right) + \left(\frac{m}{2}\right)\right] + m$$

$$S(m) = 4S\left(\frac{m}{4}\right) + 2m$$

Unroll it one more step:

$$S\left(\frac{m}{4}\right) = 2S\left(\frac{m}{8}\right) + \left(\frac{m}{4}\right)$$

Substitute $S\left(\frac{m}{4}\right)$:

$$S(m) = 4\left[2S\left(\frac{m}{8}\right) + \left(\frac{m}{4}\right)\right] + 2m$$

$$S(m) = 8S\left(\frac{m}{8}\right) + 3m$$

By unrolling this pattern, we can see that at each level, the coefficient of S decreases exponentially, while the coefficient of m increases linearly.

We can generalize this pattern after k steps:

$$S(m) = 2^k S\left(\frac{m}{2^k}\right) + km$$

When $\frac{m}{2^k}$ becomes 1, we have $m = 2^k$, which means $k = \log m$. Substitute

$$k = \log m:$$

$$S(m) = 2^{\log m} S(1) + (\log m)m$$

$$S(m) = mS(1) + m \log m$$

Given that $S(1) = T(2^0) = T(1)$, a constant, we denote it as c .

Thus, we have the following:

$$S(m) = m c + m \log m$$

$$S(m) = m (c + \log m)$$

Substitute back $m = \log n$:

$$S(m) = T(2^m) = T(n) = \log n (c + \log \log n)$$

Therefore, the solution to the recurrence function $T(n) = 2T(\sqrt{n}) + \log n$ is as follows:

$$T(n) = \Theta(\log n \log \log n)$$

In this section, we explained a powerful technique for solving recurrence functions by hypothesizing a solution and proving its correctness through mathematical induction. The method involves substituting the guessed solution back into the recurrence function to verify if it satisfies the original equation. This approach allows for flexibility and creativity in handling a wide variety of recurrence forms, including those that are not easily addressed by other methods, such as the master theorem or recursion trees. By iteratively refining the guess and employing induction, substitution methods provide a structured way to derive closed-form solutions and understand the growth behavior of recursive algorithms.

Recursion tree as a visualization technique

The **recursion tree method** is a powerful technique for solving and visualizing recurrence functions, particularly those that arise in the analysis of divide-and-conquer algorithms. It involves visualizing the recursive process as a tree, where each node represents a subproblem, and the edges represent the recursive calls. By summing the costs at each level of the tree, we can determine the overall complexity of the algorithm.

Here's the step-by-step explanation of the recursion tree method:

1. **Construct the recursion tree:**
 - Begin by writing down the original recurrence function
 - Each node in the tree represents a call to the recurrence
 - The root of the tree corresponds to the original problem $T(n)$
 - The children of a node represent the subproblems generated by the recursive calls
2. **Identify the costs:**
 - Determine the cost at each node. This cost typically corresponds to the non-recursive work done at that step, often denoted as $f(n)$.
 - Write down the cost at the root and propagate it through the tree.
3. **Expand the tree:**
 - Continue expanding the tree by breaking down each subproblem into its constituent parts according to the recurrence function
 - This process continues until the subproblems reach the base case of the recursion
4. **Calculate the total cost at each level:**
 - Sum the costs of all nodes at each level of the tree
 - Identify the number of nodes at each level and the cost per node
5. **Sum the costs across all levels:**
 - Add the costs from all levels of the tree to get the total cost
 - Analyze the sum to determine the overall asymptotic complexity

The recursion tree method involves visualizing the breakdown of a recursive problem into smaller subproblems in the form of a tree to analyze its complexity. Starting with the original problem at the root, the method entails drawing each level of recursion, where each node represents a subproblem and the edges represent recursive calls. At each level, the cost of solving all subproblems is calculated and summed. By expanding the tree until the base case is reached, and summing the costs across all levels, the total complexity of the algorithm can be determined. This method provides a clear and systematic way to understand the distribution of costs and the overall behavior of recursive algorithms, often resulting in identifying the asymptotic complexity of the recurrence. Let's explore the method in the next example.

Example 5.6

Solve $T(n) = 4T(\frac{n}{2}) + n$ using the recursion tree method.

Here is the solution:

1. **Construct the recursion tree:**

- The root of the tree is $T(n)$
- This breaks down into 4 subproblems, each of size $T(\frac{n}{2})$

...

Level 0: $T(n)$

/ | | \

Level 1: $4 * T(n/2)$

/ / / / | | | | \ \ \ \ \

Level 2: $4 * T(n/4)$ (16 subproblems)

...

2. **Identify the costs:**

- The cost at the root (Level 0) is n
- At Level 1, each of the 4 subproblems $T(\frac{n}{2})$ has a cost of $\frac{n}{2}$
- At Level 2, each of the 16 subproblems $T(\frac{n}{4})$ has a cost of $\frac{n}{4}$

...

3. **Expand the tree:**

- Continue expanding until the subproblems reach the base case (e.g., $T(1)$)
- The number of levels in the tree is $\log n$ because the problem size is halved at each level

4. **Calculate the total cost at each level:**

- **Level 0:** Cost = n
- **Level 1:** Cost = $4 \cdot (\frac{n}{2}) = 2n$
- **Level 2:** Cost = $16 \cdot (\frac{n}{4}) = 4n$
- **General Level i :** Cost = $4^i \cdot (\frac{n}{2^i}) = 2^i n$

5. Sum the costs across all levels:

- The total cost $T(n)$ is the sum of the costs at all levels:
 - $T(n) = n + 2n + 4n + 8n + \dots + 2^{\log n} n$
- This is a geometric series with the first term $a = n$ and the common ratio $r = 2$:
 - $S = a \frac{r^{k+1} - 1}{r - 1}$
- In our case, $a = n$, $r = 2$, and $k = \log n$:
 - $T(n) = n \frac{2^{(\log n)+1} - 1}{2 - 1} = n(2n - 1)$
- Simplifying, we get the following: $T(n) = 2n^2 - n$
- Asymptotically, the dominant term is $2n^2$, so we have the following: $T(n) = \Theta(n^2)$

The recursion tree method provides a clear visual representation of how a recursive algorithm breaks down a problem into subproblems. By summing the costs at each level of the tree, we can determine the total complexity of the algorithm. This method is particularly useful for understanding and solving complex recurrence functions, offering insight into the behavior and efficiency of divide-and-conquer algorithms.

The master theorem

In the analysis of algorithms, the **master theorem** plays a crucial role in solving recurrences for divide-and-conquer algorithms. Introduced in 1980, it has become a mainstream approach for estimating the complexity of a wide range of recurrence functions. The master theorem provides a straightforward framework for determining the asymptotic behavior of recurrences of the following form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Here, $a \geq 1$ and $b > 1$ are constants, and $f(n)$, the driving function, is an asymptotically positive function bounded by polynomial functions. This means there exist two polynomial functions $g(n)$ and $h(n)$ such that the following is the case:

$$g(n) \leq f(n) \leq h(n)$$

The importance of the master theorem lies in its ability to simplify the complexity analysis of many common algorithms, such as merge sort, quicksort, and binary search, among others. By categorizing the behavior of the recurrence based on the relationship between $f(n)$ and $n^{\log_b a}$, the master theorem allows for quick and accurate complexity estimation without requiring detailed, case-by-case analysis.

Although we do not provide proof of the master theorem here, readers can refer to the references at the end of this chapter for detailed proof and further reading. Understanding the master theorem and its applications is essential for anyone studying algorithm design and analysis, as it forms the foundation for evaluating the efficiency of many recursive algorithms.

Let's explore the key concepts of the master theorem, which are essential for understanding and describing the different cases of this method:

- **Critical exponent (w):** This value, $w = \log_b a$, represents a threshold for comparing the growth rates of the driving function $f(n)$ and the recursive part of the recurrence function.
- **Watershed function (n^w):** This function, $n^w = n^{\log_b a}$, serves as a *dividing line* between the cases of the master theorem. It tells us the growth rate of the recursion if the driving function $f(n)$ were negligible.

Case 1 – dominance of recursive calls or leaf-heavy recursion trees

In *Case 1*, the work done outside the recursive calls (the driving function $f(n)$) is dominated by the work done within the recursive calls. The driving function $f(n)$ grows polynomially but at a slower rate than the watershed function n^w . More formally, this is as follows:

$$f(n) = O(n^c) \text{ where } c < w$$

Or, equivalently, this is as follows:

$$f(n) = O(n^{w-\epsilon}) \text{ for some } \epsilon > 0$$

This means that $f(n)$ is upper-bounded by a polynomial with a smaller exponent than the watershed.

Why is *Case 1* leaf-heavy? To understand why *Case 1* is called *leaf-heavy*, imagine a recursion tree representing the algorithm's execution. Each node represents a recursive call, and its children are the subproblems. In this case, the work done at each level of the tree (represented by $f(n)$) is relatively small compared to the number of recursive calls. As a result, the majority of the work is done on the leaves of the recursion tree, hence the term **leaf-heavy**.

When the recursive calls dominate, the overall time complexity of the algorithm is determined by the number of leaves in the recursion tree. This number grows exponentially with the depth of the tree. Since the depth of the tree is logarithmic in the input size ($\log_b n$), the overall time complexity becomes the following:

$$T(n) = \Theta(n^w) = \Theta(n^{\log_b a})$$

In other words, the time complexity is dominated by the recursive splitting of the problem into smaller subproblems. If the work done outside the recursive calls ($f(n)$) is small compared to the work done inside ($aT(\frac{n}{b})$), then most of the time is spent in the recursive calls, making the recursion tree leaf-heavy. In this scenario, the overall runtime is primarily determined by how many times you can divide the problem before reaching the base case. This is captured by the exponent $w = \log_b a$.

Example 5.7

Consider this recurrence: $T(n) = 2T(\frac{n}{2}) + n$.

Here, $a = 2$, $b = 2$, and $f(n) = n$. We have $w = \log_2 2 = 1$. Since $f(n) = n = O(n^1)$, *Case 1* applies, and the time complexity is as follows:

$$T(n) = \Theta(n^1) = \Theta(n)$$

Case 2 – balanced growth or balanced recursion trees

Case 2 of the master theorem deals with scenarios where the work done outside the recursive calls ($f(n)$) grows at roughly the same rate as the work done within the recursive calls. This leads to a balanced recursion tree, where each level contributes a comparable amount of work. In *Case 2*, the driving function $f(n)$ grows polynomially at the same rate as the watershed function, potentially with an additional logarithmic factor. More formally, this is as follows:

$$f(n) = \Theta(n^w \log^k n) \text{ for some } k \geq 0$$

This means that $f(n)$ is tightly bounded (both above and below) by a function proportional to n^w multiplied by a logarithmic factor.

Why is *Case 2* *balanced*? In a recursion tree for *Case 2*, the work done at each level is roughly proportional to the number of nodes at that level. Since the number of nodes at each level increases exponentially with depth, the work done at each level also increases exponentially. However, the logarithmic factor in $f(n)$ slows down this increase, leading to a more **balanced** distribution of work throughout the tree.

When the work is balanced across the levels of the recursion tree, the overall time complexity is determined by the total work done across all levels. This can be calculated by summing the work at each level, which turns out to be the following:

$$T(n) = \Theta(n^w \log^{k+1} n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

In other words, the time complexity is the product of the watershed function and an additional logarithmic factor that accounts for the work done at each level.

If the work done outside the recursive calls ($f(n)$) grows at roughly the same rate as the work done inside ($aT(\frac{n}{b})$), then the recursion tree is balanced. Each level contributes significantly to the overall runtime. The logarithmic factor in $T(n)$ represents the extra work done at each level due to the balanced nature of the recursion.

Example 5.8

Consider this recurrence: $T(n) = 2T(\frac{n}{2}) + n \log n$.

Here, $a = 2$, $b = 2$, and $f(n) = n \log n$. We have $w = \log_2 2 = 1$. Since $f(n) = n \log n = \Theta(n^1 \log^1 n)$, *Case 2* applies with $k = 1$, and the time complexity is as follows:

$$T(n) = \Theta(n^1 \log^2 n) = \Theta(n \log^2 n)$$

Case 3 – dominance of non-recursive work or root-heavy recursion trees

Case 3 of the master theorem addresses scenarios where the work done outside the recursive calls ($f(n)$) significantly dominates the work done within the recursive calls. This leads to a “root-heavy” recursion tree, where the majority of the work is concentrated at the top levels.

In *Case 3*, the driving function $f(n)$ grows polynomially at a faster rate than the watershed function, potentially with an additional logarithmic factor. More formally, this is as follows:

$$f(n) = \Omega(n^{w+\varepsilon}) \text{ for some } \varepsilon > 0$$

This means that $f(n)$ is lower-bounded by a polynomial with a larger exponent than the watershed.

In addition to the growth rate condition, *Case 3* also requires a regularity condition to hold: $af(\frac{n}{b}) \leq cf(n)$ for some constant $c < 1$ and sufficiently large n .

This condition ensures that the work done at each level of the recursion tree does not grow too quickly as we move down the tree.

Why is *Case 3* root-heavy? Imagine the recursion tree again. In *Case 3*, the work done at the root of the tree ($f(n)$) is much larger than the work done in the subproblems. As we go down the tree, the work done at each level decreases significantly. This concentration of work at the top levels is why *Case 3* is called **root-heavy**.

When the non-recursive work dominates, the overall time complexity of the algorithm is primarily determined by the work done at the root of the recursion tree, which is $f(n)$. Therefore, the time complexity becomes the following:

$$T(n) = \Theta(f(n))$$

If the work done outside the recursive calls ($f(n)$) is much larger than the work done inside ($aT(\frac{n}{b})$), then most of the time is spent at the beginning, making the recursion tree “root-heavy.” In this scenario, the overall runtime is primarily determined by how much work is done before you start splitting the problem into subproblems.

Example 5.9

Consider the following recurrence:

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Here, $a = 2$, $b = 2$, and $f(n) = n^2$. We have $w = \log_2 2 = 1$. Since $f(n) = n^2 = \Omega(n^{1+1})$ and if the regularity condition holds, *Case 3* applies, and the time complexity is as follows:

$$T(n) = \Theta(n^2)$$

To evaluate the regularity condition, we check if $2\left(\frac{n}{2}\right)^2 \leq cn^2$ where $c \leq 0.5$. The regularity condition holds and *Case 3* applies to this problem.

The regularity condition is crucial for *Case 3*. Without it, the theorem's conclusions might not hold. Some functions, such as $\frac{n}{\log n}$, might appear to fit *Case 3* at first but fail the regularity condition.

Modified master theorem to solve subtracting recurrence functions

The master theorem can be extended to handle a special case of subtracting recurrence functions of the following form:

$$T(n) = aT(n - b) + f(n)$$

Here, $f(n) = \theta(n^k)$ with $k \geq 0$. Let's break this down further:

- $a > 0$ represents the number of subproblems in the subtracting recursive function
- $b > 0$ is the size of the reduction for each subproblem, with $n - b$ being the size of each subproblem

In this type of problem, we have three distinct cases:

Case 1: If $a < 1$, then $T(n) = \theta(n^k)$.

When the number of subproblems a is less than 1, the recurrence is dominated by the driving function $f(n)$. Thus, the overall complexity is directly given by $\theta(n^k)$.

Case 2: If $a = 1$, then $T(n) = \theta(n^{k+1})$.

When a equals 1, each subproblem contributes equally to the overall complexity, leading to an additional factor of n in the final solution. Therefore, the complexity is $\theta(n^{k+1})$.

Case 3: If $a > 1$, then $T(n) = \theta(a^{n/b} \cdot f(n))$.

When the number of subproblems a is greater than 1, the recurrence function grows exponentially. The overall complexity is then influenced by the combination of the exponential growth factor $n^{n/b}$ and the driving function $f(n)$, leading to $\theta(n^{n/b} f(n))$.

Example 5.10

Solve $T(n) = 0.5T(n-3) + n$.

Solution: Let's identify the key parameters:

- $a = 0.5$ is the factor by which the number of subproblems decreases with each recursion
- $b = 3$ is the amount by which the problem size decreases with each recursion
- $k = 1$ is the exponent of the polynomial term n

Since $a < 1$, we can apply *Case 1*:

$$T(n) = \Theta(n^k)$$

In our example, we have $a = 0.5$, $b = 3$, and $f(n) = n = \Theta(n^1)$. Since all the conditions are met, *Case 1* tells us the following:

$$T(n) = \Theta(n^1) = \Theta(n)$$

This means that the time complexity of the algorithm described by the recurrence function is linear (i.e., it grows proportionally to the input size n). The cost of the recursive calls diminishes rapidly due to the factor of 0.5, and the linear term n ultimately dominates the runtime.

Example 5.11

Solve $T(n) = T(n-2) + n^2$.

Solution: Let's identify the key parameters: $a = 1$, $b = 2$, and $k = 2$.

Since $a = 1$, we can apply *Case 2*:

$$T(n) = \Theta(n^3).$$

Example 5.12

Solve $T(n) = 4T(n-1) + n$.

Solution: Let's identify the key parameters: $a = 4$, $b = 1$, and $k = 1$.

In our problem, we have $a = 4$, which is greater than 1. Therefore, we fall into *Case 3* of the modified master theorem. Substituting our values, we get the following:

$$T(n) = \Theta(4^{(n/1)} * n) = \Theta(4^n * n)$$

The time complexity of this algorithm is exponential, specifically $\Theta(4^n * n)$. This means that the runtime grows incredibly quickly as the input size n increases. The exponential term 4^n dominates the growth, and the linear term n becomes negligible as n gets larger.

Master theorem limitations

The master theorem is a powerful tool for analyzing divide-and-conquer algorithms. However, its effectiveness is confined to a specific class of recurrence functions. Recognizing these limitations is crucial for choosing the right approach when analyzing more complex algorithms:

- **Restrictions on the driving function $f(n)$:**
 - **Non-polynomial functions:** The master theorem assumes that the work done outside the recursive calls ($f(n)$) is a polynomial function (e.g., n^2 , $n \log n$). If $f(n)$ is exponential (2^n), logarithmic ($\log n$), factorial ($n!$), or another non-polynomial form, the master theorem cannot be directly applied.
 - **Non-positive functions:** The master theorem requires $f(n)$ to be strictly positive for all relevant input sizes. If $f(n)$ becomes negative or zero, the theorem's assumptions are violated.
 - **Non-smooth functions:** The master theorem assumes a certain level of smoothness in $f(n)$. Functions with abrupt changes, discontinuities, or piecewise definitions may not fit within the theorem's framework.
 - **Irregular functions:** The theorem's effectiveness hinges on $f(n)$ having a regular growth pattern compared to the recursive part of the function. Functions with oscillatory behavior or variable exponents can challenge the theorem's applicability.
- **Constraints on recurrence parameters (a and b):**
 - **Non-constant parameters:** The master theorem assumes that the number of subproblems (a) and the factor by which the input size decreases (b) are constants. If either of these depends on the input size n , the master theorem is no longer applicable.
 - **Non-integer divisors:** The theorem assumes that the input is divided evenly into subproblems (n/b). Fractional or irrational values for b can lead to complications that the theorem is not equipped to handle.
- **Limitations on the recurrence function $T(n)$:**
 - **Non-monotonic functions:** The master theorem assumes that the time complexity function $T(n)$ is monotonically increasing, meaning the algorithm's runtime doesn't decrease as the input size increases. Functions such as $T(n) = \cos(n)$ violate this assumption.

Alternative approaches

When the master theorem's conditions are not met, alternative methods can be used to analyze the recurrence function:

- **The Akra-Bazzi method:** This method generalizes the master theorem and can handle a wider range of recurrence functions, including those with non-polynomial differences. It provides a systematic way to calculate the asymptotic complexity of these recurrences.
- **The substitution method:** This involves guessing a solution for the recurrence and then proving its correctness using mathematical induction.
- **The iteration method:** This involves repeatedly expanding the recurrence function to uncover a pattern that leads to a closed-form solution.
- **The recursive tree method:** This visual approach helps understand the structure of the recursive calls and estimate the overall work done at each level.

In the next section, we introduce the Akra-Bazzi method, which is a generalization of the master theorem. This method extends our capability to solve a broader range of recurrence functions, addressing many of the limitations found in the master theorem.

Let's review several examples that demonstrate the limitations of the master theorem.

Example 5.13

Solve $T(n) = 2^n T(\frac{n}{2}) + n^n$.

Solution: The master theorem does not apply here since $a = 2^n$ is not constant.

To solve this recurrence, we need to explore other methods such as the substitution method, the iteration method, or the Akra-Bazzi method, which can handle the variable nature of a and the complex form of $f(n) = n^n$.

Example 5.14

Solve $T(n) = 2T(\frac{n}{2}) + \frac{n}{\log n}$.

Solution:

Before examining each case of the master theorem, let's review the components of the recurrence function:

$$a = 2$$

$$b = 2$$

The critical exponent is n^w where $0 < w < \log_2 2 = 1$.

$$f(n) = \frac{n}{\log n}$$

Now, let's evaluate each case in the master theorem:

Case 1:

Check if $\frac{n}{\log n} = O(n^{1-\epsilon})$.

This is not true because $\frac{n}{\log n}$ grows faster than $n^{1-\epsilon}$.

Case 2:

Check if $\frac{n}{\log n} = \theta(n \log^k n)$. We need to find a k such that the following is true:

$$n \log^k n \leq \frac{n}{\log n} \leq n \log^{[k+1]} n$$

There is no such k that can sandwich $\frac{n}{\log n}$ between these functions.

Case 3:

Check if $\frac{n}{\log n} = \Omega(n^{1+\epsilon})$. This might seem applicable because $\frac{n}{\log n}$ grows faster than $n^{1+\epsilon}$. However, we must also evaluate the regularity condition:

$$af\left(\frac{n}{b}\right) \leq cf(n)$$

Substituting, we get the following:

$$2 \frac{n/2}{\log(n/2)} \leq c \frac{n}{\log n} \text{ for } c < 1$$

$$\frac{1}{\log n - 1} \leq \frac{c}{\log n}$$

For large n , there is no constant $c < 1$ that satisfies this condition. Thus, *Case 3* does not apply.

The recurrence function of $T(n) = 2T(n/2) + \frac{n}{\log n}$ in the previous example does not fit any case of the master theorem. This demonstrates a limitation of the master theorem: it requires $f(n)$ to be bounded by polynomial functions and to have a polynomial difference with the critical exponent $n^{\log_b a}$. Here, $f(n)$ does not fit within these bounds, highlighting the need for alternative methods, such as the substitution method, the iteration method, or the Akra-Bazzi method to solve this recurrence.

Example 5.15

Solve $T(n) = 0.5T(\frac{n}{2}) + \frac{1}{n}$.

Solution: The master theorem does not apply since $a < 1$.

Example 5.16

Solve $T(n) = 64T(\frac{n}{8}) - n^2 \log n$.

Solution: The master theorem does not apply because $f(n) < 1$.

Example 5.17

Solve $T(n) = T(\frac{n}{2}) + n(2 - \cos n)$.

Solution:

This recurrence function describes a divide-and-conquer algorithm where the following is the case:

- The problem of size n is divided into one subproblem of size $n/2$
- The work done outside the recursive call is $n(2 - \cos n)$

Why doesn't the master theorem directly apply?

The master theorem works best with functions that have simple polynomial growth. In this case, the term $n(2 - \cos n)$ isn't a straightforward polynomial due to the presence of the cosine function.

One alternative approach is to simplify the non-recursive part of the recurrence function. We can leverage the fact that the cosine function oscillates between -1 and 1:

$$-1 \leq \cos n \leq 1$$

Multiplying by -1 and adding 2, we get the following:

$$1 \leq 2 - \cos n \leq 3$$

Therefore, the following is the case:

$$n \leq n(2 - \cos n) \leq 3n$$

This tells us that $n(2 - \cos n)$ is bounded above and below by linear functions. Now, we apply the master theorem to the function with bounds. We can create two new recurrence functions based on the lower and upper bounds of the non-recursive term:

- **Lower bound:** $T^-(n) = T^-(\frac{n}{2}) + n$
- **Upper bound:** $T^+(n) = T^+(\frac{n}{2}) + 3n$

Now, both of these recurrences fit the form of the master theorem:

- $a = 1$ (number of subproblems)
- $b = 2$ (factor by which the input size decreases)
- $f(n) = n$ (for the lower bound) and $f(n) = 3n$ (for the upper bound)

In both cases, we have $n^{(\log_2 a)} = n^{(\log_2 1)} = n^0 = 1$.

Applying *Case 2* of the master theorem to both $T^-(n)$ and $T^+(n)$, we get the following:

$$T^-(n) = \Theta(n \log n)$$

$$T^+(n) = \Theta(n \log n)$$

Since $T^-(n) \leq T(n) \leq T^+(n)$ and $T(n)$ are sandwiched between two other functions, and both $T^-(n)$ and $T^+(n)$ are $\Theta(n \log n)$, we can conclude the following:

$$T(n) = \Theta(n \log n)$$

You should note that the oscillating nature of the cosine function in the previous example does not significantly affect the overall growth rate of the recurrence function. In the long run, the linear term n dominates the behavior, and the algorithm's time complexity is primarily determined by the recursive splitting and the linear work done in each step.

Example 5.18

Solve $T(n) = 4T\left(\frac{n}{2}\right) + \frac{n}{\log n}$.

Solution: At first glance, the master theorem seems like the right tool. We have the following:

- $a = 4$ (number of subproblems)
- $b = 2$ (factor by which the input size decreases)
- $f(n) = \frac{n}{\log n}$ (work done outside of the recursive calls)

This gives us a **watershed** function of $n^{(\log_b a)} = n^{(\log_2 4)} = n^2$.

Now, we compare $f(n) = \frac{n}{\log n}$ to n^2 .

Similar to *Example 5.17*, the function $\frac{n}{\log n}$ doesn't fit neatly into any of the three cases of the master theorem:

- **Case 1:** $f(n) = O(n^{(\log_b a - \epsilon)})$ but $\frac{n}{\log n}$ grows faster than the watershed function
- **Case 2:** $f(n) = \Theta(n^{(\log_b a)} \log^k n)$ but $\frac{n}{\log n}$ doesn't fit this form
- **Case 3:** $f(n) = \Omega(n^{(\log_b a + \epsilon)})$ and $\frac{n}{\log n}$ grows slower than the watershed function

Even though *Case 3* seems like a potential fit, the regularity condition fails. The condition requires that $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and sufficiently large n . In this case, it becomes the following:

$$4 \left(\frac{n}{2} \right) / \log \left(\frac{n}{2} \right) \leq c \left(\frac{n}{\log n} \right)$$

Simplifying, we get the following:

$$2n / (\log n - \log 2) \leq cn / \log n$$

For large values of n , we cannot find a constant $c < 1$ that satisfies this inequality. Therefore, *Case 3* of the master theorem doesn't hold.

This section explains a systematic approach for solving recurrences that commonly appear in the analysis of divide-and-conquer algorithms. The master method provides a set of straightforward rules to determine the asymptotic behavior of recurrences. It categorizes the recurrences into three cases based on the relative growth rates of $f(n)$ and $n^{\log_b a}$, allowing for quick and accurate complexity estimation. The method's utility lies in its ability to handle a wide range of problems without requiring detailed, case-by-case analysis, making it a valuable tool in algorithm design and analysis.

Beyond the master theorem – the Akra-Bazzi method

In the previous section, we discussed the limitations of the master theorem in solving recurrence functions. Although the master theorem can address a wide range of problems and is the most common approach, it does have its constraints. For example, it only applies to a specific class of divide-and-conquer recurrence functions:

$$T(n) = aT(n/b) + f(n)$$

Here, the subproblem sizes are equal (n/b). As we showed in the previous section, we can solve special cases of subtracting recurrence functions using the master theorem. It also has limitations on the form of the function $f(n)$. Specifically, the master theorem struggles with the following:

- **Unequal subproblem sizes:** The recurrence splits into subproblems of significantly different sizes
- **More complex splitting:** There are more than two subproblems in the recursion
- **Non-polynomial work:** The function $f(n)$ representing the work done outside the recursive calls isn't easily categorized as polynomial or logarithmic

For cases where the master theorem falls short, the **Akra-Bazzi method** offers solutions for a broader spectrum of recurrence functions. This method extends our ability to solve more complex recurrences, providing a valuable tool for algorithm analysis. The Akra-Bazzi method can handle recurrences of the following more general form:

$$T(n) = f(n) + \sum_{i=1}^k a_i T(b_i n + h_i(n))$$

Here, the following is the case:

- k is the number of subproblems (can be greater than 2)
- $n > n_0$ and n_0 is a constant
- $a_i > 0$, a constant for all i , is the number of times the i th subproblem appears
- $0 < b_i < 1$, a constant for all i and $\frac{n}{b_i}$ is the size of the i th subproblem (different subproblems can have different sizes)
- $|f(n)| \in O(n^c)$, where c is a constant
- $|h_i(n)| \in O\left(\frac{n}{\log^2 n}\right)$, for all i

- $f(n)$ is the non-recursive part of the recurrence or driving function and represents the work done outside the recursive calls, which has a wider range of allowable forms

The following are the steps to solve a recurrence function using the Akra-Bazzi method:

1. **Find p :** Solve the equation:

$$\sum_{i=1}^k a_i b_i^p = 1$$

This p is crucial; it represents a “balancing point” for the recurrence.

2. **Calculate the integral:** Find the value of the following integral:

$$I = \int_1^n \left[\frac{g(u)}{u^{p+1}} \right] du$$

3. **Asymptotic bound:** The asymptotic complexity of $T(n)$ is then the following:

$$T(n) \in \theta \left(n^p \left(1 + \int_1^n \frac{g(u)}{u^{p+1}} du \right) \right)$$

Please note the following about the Akra-Bazzi method:

- The Akra-Bazzi method is a generalization of the master theorem
- It handles recurrences with unequal subproblem sizes, more complex splitting, and broader types of work functions
- The method involves finding a balancing exponent p and integrating the work function to determine the asymptotic complexity

Why does it work? Intuition behind Akra-Bazzi

Think of the recurrence function as a tree. Each node represents a recursive call, and its children are the subproblems. The value p acts like a weight balancing the contributions of different branches of the tree. The integral then aggregates the work done at each level of the tree, taking into account this balancing.

Example 5.19

Let's consider the following recurrence:

$$T(n) = 2T(n/3) + T(n/4) + n$$

Find p : $(2 \times 3^p) + (1 \times 4^p) = 1$. Solving this (numerically) gives us $p \approx 1.207$.

Calculate the integral:

$$\int_1^n \left(\frac{u}{u^{1+1.207}} \right) du \approx 0.828 n^{0.207}$$

Asymptotic Bound:

$$T(n) = \Theta(n^{1.207}(1 + 0.828 n^{0.207})) = \Theta(n^{1.207})$$

Example 5.20

$$\text{Solve } T(n) = 4T\left(\frac{n}{2}\right) + \frac{n}{\log n}.$$

Solution: In the previous section, we saw that this recurrence function cannot be solved using the master theorem. On the other hand, the Akra-Bazzi method is a generalization of the master theorem that handles a broader class of recurrences. Let's apply it here:

Find p : Solve the equation: $\frac{4}{2^p} = 1$. This gives us $p = 2$.

Calculate the integral:

$$\int_1^{n^{u/\log u}} \frac{1}{u^3} du = \int_1^n \frac{1}{u^3 \log u} du$$

This integral is a bit tricky, but it evaluates to approximately $\log \log n$.

Asymptotic bound:

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{[u/\log u]}{u^3} du\right)\right)$$

$$T(n) = \Theta(n^2(1 + \log \log n))$$

$$T(n) = \Theta(n^2 \log \log n)$$

This section detailed an advanced and generalized approach for solving more complex recurrence functions that cannot be addressed by the master theorem. This method is particularly useful for recurrences where the subproblem sizes are not uniform or when dealing with more intricate forms of $f(n)$. The Akra-Bazzi method extends the applicability of traditional recurrence-solving techniques by providing a framework that accommodates a broader range of functions and splitting ratios. By leveraging this method, one can derive precise asymptotic bounds for complex recurrences, making it a powerful tool for analyzing the performance of sophisticated recursive algorithms. The section includes practical examples to illustrate the method's application and effectiveness in resolving recurrences beyond the reach of simpler methods.

Summary

In *Chapter 5*, the methods for solving recurrence functions in the context of algorithm analysis were discussed. The chapter outlined three primary techniques: the substitution method, the master theorem, and recursion trees. The substitution method involves constructing proofs through variable substitution and induction to solve recurrence functions. The master theorem provides a systematic approach to determining the complexity of recursive algorithms based on their recurrence functions. Recursion trees help visualize the breakdown of problems into subproblems, providing insights into their complexity without direct proof.

The substitution method was elaborated upon, highlighting its flexibility and power in addressing various recurrence functions. The method involves hypothesizing a solution, substituting it into the recurrence, and using induction to prove its correctness. Practical examples illustrated the iterative approach, showing how unrolling the recurrence can reveal patterns and lead to closed-form solutions. This method's versatility is emphasized through multiple examples, demonstrating its utility in solving complex recurrences that other methods might not handle effectively.

The chapter also explored the limitations of the master theorem, especially in dealing with non-polynomial, non-positive, non-smooth, or irregular driving functions, and when a or b are not constants. The Akra-Bazzi method was introduced as a generalization of the master theorem, capable of handling a wider range of recurrence functions, including those with unequal subproblem sizes or more complex splitting. Examples were provided to show the application of these methods, emphasizing the need for alternative approaches when the master theorem's conditions are not met. The chapter concluded with the application of these advanced methods, ensuring a comprehensive understanding of solving recurrence functions in algorithm analysis.

In the next chapter, we will discuss one of the most fundamental families of algorithms: sorting. The skills we learned in this chapter, particularly in analyzing recurrence functions, will be utilized to analyze sorting algorithms and estimate their complexities.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022:
 - *Chapter 4, Recurrences*
- *Algorithms*. By R. Sedgewick, K. Wayne. Fourth Edition. Addison-Wesley. 2011:
 - *Chapter 2, Principles of Algorithm Analysis*
 - *Chapter 5, Sorting*
- *The Algorithm Design Manual*. By S. S. Skiena. Second Edition. Springer. 2008:
 - *Chapter 3, Data Structures and Recursion*
 - *Chapter 5, Graph Algorithms*
- *Algorithm Design*. By Jon Kleinberg and Éva Tardos. First Edition. Pearson. 2005:
 - *Chapter 5, Divide and Conquer*
 - *Chapter 7, Recurrences*
- *On the solution of linear recurrence equations*. Mohamad Akra, Louay Bazzi. *Computational Optimization and Applications*. Vol. 10, No 2, pp.195–210. 1998.

Part 2:

Deep Dive in Algorithms

In this section, we explore specific types of algorithms in detail, from fundamental sorting and searching techniques to more advanced topics, such as dynamic programming and randomized algorithms. This part deepens your understanding of algorithm design, demonstrating how various approaches can solve complex problems efficiently.

This part includes the following chapters:

- *Chapter 6, Sorting Algorithms*
- *Chapter 7, Search Algorithms*
- *Chapter 8, Symbiotic Relationship between Sort and Search*
- *Chapter 9, Randomized Algorithms*
- *Chapter 10, Dynamic Programming*

Sorting Algorithms

A staggering 25% of global computing power is devoted to sorting data, underscoring its critical role in modern computational processes. This significant allocation of resources highlights the essential nature of efficient sorting algorithms in diverse applications, ranging from information retrieval and database management to scientific simulations and machine learning. This chapter systematically explores the most significant sorting algorithms, ranging from the basic bubble sort to the advanced quick sort, both in iterative and recursive approaches. Each algorithm is assessed for correctness and complexity. The chapter concludes with a discussion on sorting in linear time, providing context about data assumptions.

Sorting algorithms are implemented using different approaches. Each algorithm possesses unique characteristics described by properties such as stability, in-place sorting, or adaptability, which determine its suitability for different tasks. This chapter explores these properties, elucidating the advantages and disadvantages of each sorting mechanism. By understanding these nuances, you will be equipped to choose the optimal sorting approach for your specific use cases and datasets.

Building upon this foundational overview, the chapter delves into the intricacies of both iterative and recursive sorting algorithms. It begins with the simpler, more intuitive iterative methods, such as selection and insertion sort, gradually progressing toward the more efficient recursive approaches, such as merge sort and quick sort. Additionally, it explores sorting algorithms capable of achieving linear time complexity under specific conditions, highlighting the significance of assumptions about the input data. This comprehensive exploration ensures that you develop a robust understanding of various sorting techniques, equipping you to efficiently tackle a wide array of computational challenges.

This chapter covers the following topics:

- The taxonomy of sorting algorithms
- Iterative sorting algorithms
- Recursive sorting algorithms
- Non-comparison-based sorting algorithms

The taxonomy of sorting algorithms

To begin, let's examine the key properties that distinguish different sorting algorithms, offering a thorough framework for understanding their distinct characteristics and practical applications. We will explore six critical properties: comparison, recursion, adaptability, inversion, memory usage, and stability. Comparison determines whether an algorithm relies on pairwise element comparisons to sort data, influencing its general applicability and time complexity bounds. Recursion involves breaking down the sorting process into smaller, more manageable sub-problems, often leading to elegant, divide-and-conquer solutions. Adaptability measures an algorithm's ability to efficiently handle already partially sorted data, enhancing performance in real-world scenarios. Inversion counts the number of element pairs that are out of order, serving as a metric for the algorithm's efficiency in different contexts. Memory usage examines the additional space required by an algorithm, distinguishing between in-place and non-in-place methods. Finally, stability ensures that equal elements retain their original relative order, which is crucial for multi-level sorting tasks. By understanding these properties, we can better appreciate the strengths and limitations of each sorting algorithm and make informed decisions about their application.

Comparison

Comparison is a fundamental operation in both sorting and searching algorithms. In sorting, comparison determines the relative order of elements through pairwise evaluations. We can categorize sorting algorithms based on their use of comparisons to establish order.

Sorting algorithms based on comparison, such as merge sort and quick sort, are versatile. We can apply them to any data type with a defined comparison function. These algorithms generally achieve $O(n \log n)$ time complexity. On the other hand, some algorithms are not based on comparisons. Examples of linear time-sorting algorithms include counting sort and Radix sort. They employ alternative techniques for sorting, often achieving linear time complexity ($O(n)$) under specific conditions.

Comparison-based sorting

Comparison plays a central role in many sorting algorithms. In a comparison-based sorting algorithm, the primary operation used to determine the order of elements is the comparison between pairs of elements. The algorithm makes decisions about the positions of elements based on the outcomes of these comparisons, which typically involve relational operators such as $<$, $>$, or $==$.

These algorithms are based on comparing elements to decide their order. Every comparison operation determines whether one element should be placed before or after another. Comparison-based sorting algorithms can be applied to any data type for which a comparison function is defined. This makes them versatile and widely applicable.

We can show that there is a lower bound of $O(n \log n)$ for the time complexity of comparison-based sorting algorithms. This is due to the decision tree model of comparisons. This means that, on average,

at least $O(n \log n)$ comparisons are required to sort n elements. Let's explain why the time complexity of any comparison-based sort cannot be better than $O(n \log n)$.

These algorithms are implemented based on comparing every pair of elements. We can represent the decisions made by these algorithms as a binary decision tree. The following are the elements of this decision tree:

- Each internal (non-terminal) node represents a comparison between two elements (e.g., $A < B$?)
- Each branch represents the outcome of that comparison (yes or no)
- Each leaf (terminal) node represents a possible final sorted order of the input array

Let's explain the decision tree representation of comparison-based sorting by an example.

Example 6.1

Let's consider an array with three random numbers: $A = [3, 1, 4]$. To sort this array using comparisons, we might start with the decision tree illustrated in *Figure 6.1*.

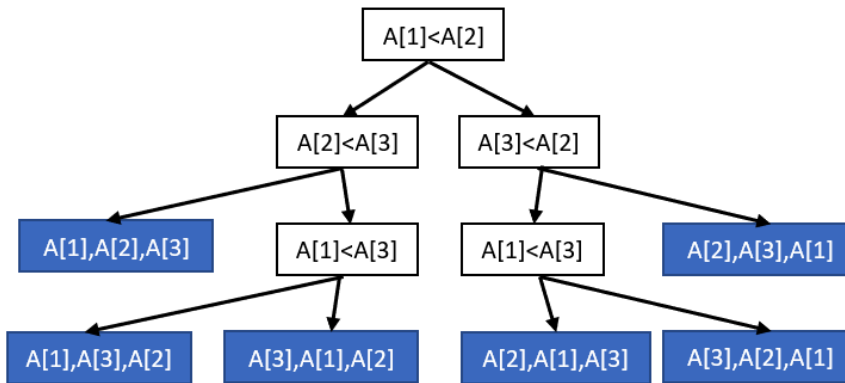


Figure 6.1: Decision tree to implement comparison-based sorting of an array with three elements (white boxes are internal nodes; the left branch is “yes” and the right branch is “no”)

The tree will continue branching out, representing all possible comparison outcomes and leading to leaf nodes that represent different sorted orders of the input array. Let's estimate the lower-bound time complexity of sorting using this decision tree:

- For an array of size n , there are $n!$ possible permutations (different orderings). Each of these permutations could potentially be the correctly sorted order.
- In the decision tree, each leaf node represents one of these possible permutations. Therefore, the tree must have at least $n!$ leaves to cover all possibilities. In our example, we will have $3! = 6$ possible permutations or leaf nodes.

- A binary tree with L leaves has a minimum height of $\log L$. Since our decision tree needs at least $n!$ leaves, its minimum height is $\log(n!)$. Using Stirling's approximation, we know that $\log(n!)$ is approximately equal to $n \log n$ (see *Example 3.8* from *Chapter 3*).

Therefore, there is a lower-bound time complexity of $\Omega(n \log n)$ for all sorting algorithms based on comparison. This means no algorithm can consistently sort an array faster than $O(n \log n)$.

The following are well-known comparison-based sorting algorithms:

- **Bubble sort:** The adjacent elements in the array are repeatedly compared, and if they are in the wrong order, they are swapped. With each pass, larger elements are gradually moved to the end of the array.
- **Insertion sort:** This algorithm progressively constructs a sorted array by iteratively comparing and inserting elements into their appropriate places within the already sorted portion.
- **Quick sort:** In the quick sort algorithm, comparisons are used to partition the array around a pivot element, followed by recursive sorting of the resulting partitions.
- **Merge sort:** In this sorting algorithm, the array is split into two partitions of nearly the same size. Each partition is recursively sorted, and then the sorted partitions are merged using comparisons.
- **Heap sort:** In this sorting algorithm, a max-heap (or min-heap) tree is built from the original array, and then the maximum (or minimum) element is repeatedly extracted. Throughout the process, comparisons are used to maintain the heap property (see *Chapter 13*).

Non-comparison-based sorting

Unlike comparison-based methods, in non-comparison-based algorithms, we do not directly compare elements to determine their order. Instead, we use alternative techniques such as counting frequencies, employing hash functions, or leveraging specific properties of the data. These methods make use of assumptions about the input data to achieve efficient sorting, often reaching linear time complexity, $O(n)$, under certain conditions. The main properties of non-comparison-based sorting algorithms include the following:

- **Data-specific techniques:** These algorithms frequently utilize specific properties of the data, such as the range (particularly for numerical, integer data) or the number of digits, to carry out the sorting process
- **Linear time complexity:** Non-comparison-based sorting algorithms can achieve better than $O(n \log n)$ time complexity, often $O(n)$, by avoiding comparisons and using more direct methods of ordering elements
- **Memory usage:** Although these algorithms can achieve linear time complexity, they often require temporary memory that is typically proportional to the input size or the range of the data values

In this chapter, we introduce three well-known non-comparison-based sorting algorithms. The first is counting sort, which counts the occurrences of each distinct element to place them in their correct positions, making it effective for integers within a known range. Next is Radix sort, which processes each digit or character of the elements in a specific order (e.g., least significant digit first) and uses a stable sub-routine such as counting sort to sort the elements by each digit. Finally, bucket sort distributes the elements of the input list into several buckets or bins based on their values. Each bucket or bin is then sorted individually (often using a simple comparison-based sort), and all the sorted buckets are concatenated to build the final sorted list.

Recursion

Recursion plays a vital role in many sorting algorithms, enabling them to efficiently break down and conquer the sorting problem through the divide-and-conquer strategy. Recursive sorting algorithms offer clean and intuitive implementations but come with stack overhead and potential performance variability. On the other side, non-recursive sorting algorithms avoid the complexities of recursion and stack management, offering memory efficiency and simplicity, though they may not perform as well on large datasets.

Recursive sorting algorithms

Recursion is often employed in sorting algorithms to break down complex problems into simpler sub-problems. In the context of sorting, recursion allows algorithms to divide the input array into smaller segments and sort these segments independently. The final sorted array is produced by merging the sorted segments. Recursive sorting algorithms leverage the divide-and-conquer strategy, which helps in achieving efficient sorting with manageable code complexity. Recursive sorting algorithms have the following properties:

- **Divide-and-conquer strategy:** As the name suggests, this strategy consists of three steps: first, breaking the problem (array) into smaller sub-problems (sub-arrays). Second, recursively solving (sorting) each sub-problem (sub-array). Third, combining the solutions to address the original problem.
- **Base case and recursive case:** Each recursive algorithm has a base case that terminates the recursion when the sub-problem is small enough (e.g., a single element or an empty array). The recursive case continues to break down the problem and solve the sub-problems.
- **Stack usage:** Recursive calls consume stack space, which can lead to high memory usage, especially for deep recursion. However, tail-recursive optimization and iterative approaches can mitigate this issue in some cases.

Table 6.1 provides some examples of recursive sorting algorithms:

Sorting algorithm	Process	Time and space complexity
Merge sort	1. Divide: If the array has more than one element, split it into two roughly equal halves. 2. Conquer: Recursively apply merge sort to each of the two halves. This continues until each sub-array has only one element (a trivially sorted array). 3. Combine: Merge the two sorted halves back into a single sorted array by comparing the elements from each half, taking the smaller one first, and placing it in the new array. 4. Repeat this process until all elements from both halves have been merged.	Time: $O(n \log n)$ in all cases (best, average, and worst). Space: $O(n)$ for temporary memory of merge component.
Quick sort	1. Choose pivot: From the array, select an element to serve as the pivot. Common choices are the first, the last, or a randomly chosen element. 2. Partition: Divide the array into two sub-arrays. The elements of the first sub-array (left) are smaller than the pivot, and the elements of the second sub-array (right) are greater than the pivot. The pivot is now in its final sorted position. 3. Recursive sorting: Apply quick sort recursively to the left sub-array. Apply quick sort recursively to the right sub-array. 4. Continue this process until each sub-array is either empty or contains only one element (a trivially sorted array).	Time: $O(n \log n)$ on average, but $O(n^2)$ in the worst case. Space: $O(\log n)$ for the recursion stack in the average case.
Heap sort	Build max heap: Starting from the last non-leaf node, apply recursive heapify to each node in reverse level order. This builds a max heap from the input array. Extract and rebuild: Repeat the following steps until the heap is empty: ▪ Swap the root (maximum element) with the last element of the heap. ▪ Reduce the heap size by 1. ▪ Apply recursive heapify to the root to restore the max-heap property. (Heap sort will be discussed in detail in <i>Chapter 13</i>).	Time: $O(n \log n)$ in all cases. Space: $O(1)$ for the iterative version; the recursive heapify process uses $O(\log n)$ stack space.

Table 6.1: Examples of recursive sorting algorithms

Non-recursive sorting algorithms

Non-recursive sorting algorithms do not use recursion to achieve sorting. Instead, they rely on iterative techniques, using loops to manage the process of sorting. These algorithms often have simpler space complexity because they avoid the stack overhead that is a problem with recursive strategies. Non-recursive sorting algorithms use an iterative approach (implemented by loops) and they are generally memory efficient. Technically speaking, non-recursive sorting algorithms have the following characteristics:

- **Iterative approach:** Non-recursive sorting algorithms use loops (e.g., `for` loops or `while` loops) to sort the array, eliminating the need for recursive calls
- **Memory efficiency:** These algorithms are often more memory-efficient because they avoid the additional stack space required for recursive calls
- **Simpler stack management:** By using an iterative approach, non-recursive algorithms avoid the complexities of managing recursive stack depth, which can lead to stack overflow when dealing with large datasets

Table 6.2 provides a list of well-known non-recursive sorting algorithms:

Sorting algorithm	Process	Time and space complexity
Insertion sort	Start with the second element. Assume the first element is already sorted. Iterate through the unsorted portion: • Compare it with the elements in the sorted portion (to its left). • Shift the elements in the sorted portion to the right until the element is in the correct position. • Insert the current element into that position. Continue until the entire array is sorted. Repeat step 2 for each element in the unsorted portion.	Time: $O(n^2)$ for both the average and worst cases, $O(n)$ in the best case. Space: $O(1)$ since it is in-place sort.

Sorting algorithm	Process	Time and space complexity
Selection sort	Find the smallest element (initially the entire array). Swap: Swap the smallest element with the leftmost element of the unsorted portion. The leftmost element is now in its final sorted position. Repeat: Consider the remaining unsorted portion (excluding the elements already sorted). Repeat steps 1 and 2 until the entire array is sorted.	Time: $O(n^2)$ in all cases. Space: $O(1)$ since it is in-place sort.
Bubble sort	Compare and swap: • Compare every two elements. If they are in the wrong order, swap them. • Continue this process until the last element of the array is visited. Repeat: • After one pass through the array, the largest element will have “bubbled up” to the end. • Repeat step 1, but this time, stop one position before the end of the array (since the last element is already in its correct place). • Continue repeating step 1, reducing the comparison range by one element each time, until the entire array is sorted.	Time: $O(n^2)$ for both the average and worst cases, $O(n)$ in the best case. Space: $O(1)$ since it is in-place sort.

Table 6.2: Examples of non-recursive sorting algorithms

Recursion is a key factor that differentiates sorting algorithms. In the following sub-section, we will explore other aspects, such as adaptability.

Adaptability

In sorting, adaptability means an algorithm’s capacity to leverage pre-existing order within the data to enhance its efficiency. An adaptive sorting algorithm excels when the input is partially sorted or exhibits some underlying structure, requiring fewer operations to achieve complete sorting. In essence, the more ordered the input, the faster the algorithm can complete its task.

An adaptive sorting algorithm will have a faster average-case running time when the data is partially ordered compared to when the input is completely random. In addition, these sorting algorithms make fewer comparisons and swaps when the data is closer to the sorted order, resulting in reduced computational effort. Adaptive sorting algorithms are more efficient in real-world applications because many real-world datasets are not completely random and often have some existing order (e.g., time-series data or lists that are periodically updated). Adaptive sorting algorithms can handle such datasets more efficiently.

Let's consider two adaptive sorting algorithms. First is insertion sort, which is highly adaptive. In the best-case scenario, when the input data is already ordered, its running time is $O(n)$, as it only needs to make $n - 1$ comparisons. For small or nearly sorted data, insertion sort is efficient since it performs few swaps and comparisons. The next adaptive sorting algorithm is bubble sort, which is adaptive to some extent. If the input data is already ordered, bubble sort can complete the sorting in $O(n)$ time, though this requires an optimization to stop early when no swaps are made in a pass.

On the other hand, let's examine two non-adaptive sorting algorithms: quick sort and heap sort. The complexity of quick sort is heavily influenced by the choice of pivot and the initial arrangement of elements. However, it doesn't inherently improve with partially sorted data. Heap sort is another non-adaptive algorithm. As we will see in *Chapter 13*, pre-sorted data doesn't offer any advantage in this process.

Inversion

An inversion in a sequence is a condition where a pair of elements is out of order. Formally, given an array A , an inversion is defined as a pair of indices, i and j , such that $i < j$ and $A[i] > A[j]$. In simpler terms, an inversion occurs when a larger element precedes a smaller one in the array. Inversions are significant in sorting because they measure the degree of disorder within an array. The total number of inversions provides insight into how far an array is from being sorted. We can consider two scenarios:

- **Zero inversions:** For all $i < j$, we have $A[i] \leq A[j]$; in other words, the array A is fully sorted
- **Maximum inversions:** For all $i < j$, we have $A[i] > A[j]$; in other words, the array A is sorted in reverse order

The concept of inversions is essential to understanding sorting algorithms and their analysis. Inversions refer to pairs of elements in an array that are out of order. For instance, in bubble sort, the algorithm repeatedly swaps adjacent elements that are not in their correct order. The total number of swaps performed during bubble sort directly corresponds to the number of inversions present in the array.

In merge sort, inversions can be counted during the merging process. Specifically, when an element from the right half of the array is inserted before an element from the left half, this indicates a number of inversions equal to the remaining elements in the left half. This insight helps us quantify the disorder present within the array as it undergoes sorting.

Inversions also impact the complexity of sorting algorithms. The number of inversions in an array can be used to analyze the efficiency of these algorithms. For instance, algorithms with $O(n \log n)$ complexity, such as merge sort and quick sort, can handle a large number of inversions efficiently. In contrast, algorithms with $O(n^2)$ complexity, such as bubble sort and insertion sort, may become increasingly inefficient as the number of inversions rises.

Memory usage

Memory usage in sorting is measured by the amount of extra storage needed by the algorithm to process and arrange the input data. This is closely tied to whether a sorting algorithm is classified as in place or not.

In-place sorting

In-place sorting algorithms operate directly within the input array. They require only a fixed amount of additional storage. These algorithms rearrange the elements within the given array using a fixed amount of additional space, typically $O(1)$ extra space.

In-place sorting algorithms are memory efficient, requiring minimal extra space. This is particularly important in environments with limited memory resources or when dealing with large datasets. These algorithms often achieve sorting by swapping elements within the input array. Common techniques include partitioning and incrementally placing elements in their correct positions. Insertion sort, quick sort, and heap sort are examples of in-place algorithms. Quick sort, with its in-place partitioning scheme, typically sorts the array without requiring additional storage proportional to the input size. However, it uses $O(\log n)$ extra space for the recursive stack. Insertion sort inserts elements into their correct positions within the array, requiring no extra space beyond a few variables for looping and swapping. Finally, heap sort transforms the array into a heap structure and then sorts it in place. This is achieved by repeatedly removing the maximum element (or minimum, depending on the desired order) and then restoring the heap property to maintain the structure.

Non-in-place sorting

Sorting algorithms that operate outside the input data and require additional memory space, either linearly or non-linearly proportional to the input size, are referred to as non-in-place sorting algorithms. These algorithms use extra space to store copies of the data or intermediate results during the sorting process. Non-in-place sorting algorithms consume more memory because they need additional storage to hold temporary copies of the data or intermediate structures. On the other hand, these algorithms can be simpler to implement and understand, and they often preserve the stability of the sort more easily than in-place algorithms.

Examples of non-in-place sorting are merge sort, counting sort, and Radix sort. Merge sort uses $O(n)$ extra memory to hold the two partitions of the input being merged. Each recursive step requires additional space to store intermediate results, leading to higher overall memory usage. Counting sort uses additional memory proportional to the range of the input values, typically $O(n + k)$, where k is

the range of the input data. Radix sort, like counting sort, requires extra space to hold temporary data for each digit or character being processed, resulting in $O(n + k)$ memory usage.

When choosing between in-place and non-in-place sorting algorithms, there are several trade-offs to consider:

- In environments with memory constraints, in-place sorting algorithms are preferred due to their minimal memory footprint.
- Non-in-place algorithms may be simpler to implement and understand, particularly for complex sorting tasks.
- In some cases, non-in-place algorithms can offer better performance due to their ability to efficiently handle large or complex datasets, despite their higher memory usage.
- Non-in-place algorithms often preserve the relative order of equal elements more easily, making them stable. Ensuring stability in in-place algorithms can be more challenging.

Memory usage is a crucial consideration when choosing a sorting algorithm. In-place sorting methods are efficient in terms of memory consumption, as they manipulate the input array directly with minimal extra space. However, non-in-place methods, such as merge sort and counting sort, require extra memory proportional to the input size, although they can provide advantages in terms of simplicity and stability. The choice between in-place and non-in-place sorting depends on the specific problem. For instance, if memory is limited, in-place sorting should be considered.

Stability

Stability in sorting refers to maintaining the original relative order of elements with identical values. A sorting algorithm is considered stable if it ensures that elements with the same value keep their initial order relative to one another after sorting. Stable algorithms include insertion sort, merge sort, and bubble sort. In contrast, quick sort, heap sort, and selection sort are examples of unstable algorithms.

The distinction between stable and unstable sorting algorithms lies in their ability to maintain the original order of equal elements. Stable algorithms guarantee this preservation, ensuring that the relative positions of equal elements remain unchanged during sorting. In contrast, unstable algorithms may alter the relative positions of equal elements.

Consider the following list of tuples, where each tuple contains a letter and a number: [(‘A’, 3), (‘B’, 1), (‘C’, 3), (‘D’, 2), (‘E’, 1)]

If we sort this list by the number (the second element in each tuple) in ascending order using a stable sorting algorithm, the result might be as follows:

[(‘B’, 1), (‘E’, 1), (‘D’, 2), (‘A’, 3), (‘C’, 3)]

Observe that for tuples with identical numbers (such as ('A', 3) and ('C', 3)), their original order is maintained after sorting. In other words, ('A', 3) precedes ('C', 3) in the sorted list, just as it did in the original list. If an unstable sorting algorithm is used, the resulting order may be as follows:

[('E', 1), ('B', 1), ('D', 2), ('C', 3), ('A', 3)]

Here, the relative order of ('A', 3) and ('C', 3) is not preserved.

Stability is particularly important in the following scenarios:

- **Multiple key sorts:** When performing multi-level sorting (e.g., sorting first by one attribute and then by another), stability ensures that the order from the previous sort is preserved. For example, if you first sort a list of employees by their employment status and then by their employee numbers, a stable sort will maintain the employment status order while sorting by id.
- **Preserving input order:** In some applications, the order of input elements carries significance beyond their sorted position. Stability ensures that this significance is preserved.

Stability is crucial in applications requiring multiple levels of sorting or where the input order carries additional information. Stable sorting algorithms are preferred in such contexts to ensure data integrity and correctness.

Iterative sorting algorithms

Iterative sorting algorithms are renowned for their simplicity, making them easy to understand, implement, and debug. The major advantage of iterative sorting methods is their space efficiency; they typically require minimal additional memory, which can be a significant benefit in memory-constrained environments. However, these algorithms often exhibit poor performance in terms of time complexity, especially when handling large datasets. This limitation can be particularly problematic in scenarios where data size and processing time are critical factors. This section introduces three common iterative sorting algorithms: bubble sort, selection sort, and insertion sort.

Bubble sort

Bubble sort is a simple comparison-based algorithm that repeatedly scans through an array, comparing and swapping adjacent elements if they are in the wrong order. This process is repeated until the entire array is sorted. The analogy of “bubbling” arises from how smaller elements gradually ascend to the top while larger ones descend to the bottom.

Despite its simplicity, bubble sort is generally inefficient for sorting datasets, exhibiting a quadratic running time, $O(n^2)$, in both average and worst-case scenarios. Nonetheless, its ease of comprehension and implementation make it a valuable educational tool and a viable option for sorting small datasets. As we have seen in the previous section, bubble sort is a stable and in-place algorithm and its space complexity is $O(1)$.

Example 6.2

Table 6.3 illustrates the step-by-step process of bubble sort for the array [5, 3, 8, 4, 2, 7, 1, 6]:

Pass	Description of the operation	Content of the array
1	5 vs 3, swap them	[3, 5, 8, 4, 2, 7, 1, 6]
	5 vs 8, no swap	[3, 5, 8, 4, 2, 7, 1, 6]
	8 vs 4, swap them	[3, 5, 4, 8, 2, 7, 1, 6]
	8 vs 2, swap them	[3, 5, 4, 2, 8, 7, 1, 6]
	8 vs 7, swap them	[3, 5, 4, 2, 7, 8, 1, 6]
	8 vs 1, swap them	[3, 5, 4, 2, 7, 1, 8, 6]
	8 vs 6, swap them	[3, 5, 4, 2, 7, 1, 6, 8]
2	3 vs 5, no swap	[3, 5, 4, 2, 7, 1, 6, 8]
	5 vs 4, swap them	[3, 4, 5, 2, 7, 1, 6, 8]
	5 vs 2, swap them	[3, 4, 2, 5, 7, 1, 6, 8]
	5 vs 7, no swap	[3, 4, 2, 5, 7, 1, 6, 8]
	7 vs 1, swap them	[3, 4, 2, 5, 1, 7, 6, 8]
	7 vs 6, swap them	[3, 4, 2, 5, 1, 6, 7, 8]
	8 is now in its correct position	
3	3 vs 4, no swap	[3, 4, 2, 5, 1, 6, 7, 8]
	4 vs 2, swap them	[3, 2, 4, 5, 1, 6, 7, 8]
	4 vs 5, no swap	[3, 2, 4, 5, 1, 6, 7, 8]
	5 vs 1, swap them	[3, 2, 4, 1, 5, 6, 7, 8]
	5 vs 6, no swap	[3, 2, 4, 1, 5, 6, 7, 8]
	7 and 8 are now in their correct positions	

Pass	Description of the operation	Content of the array
4	3 vs 2, swap them	[2, 3, 4, 1, 5, 6, 7, 8]
	3 vs 4, no swap	[2, 3, 4, 1, 5, 6, 7, 8]
	4 vs 1, swap them	[2, 3, 1, 4, 5, 6, 7, 8]
	4 vs 5, no swap	[2, 3, 1, 4, 5, 6, 7, 8]
	6, 7, and 8 are now in their correct positions	
5	2 vs 3, no swap	[2, 3, 1, 4, 5, 6, 7, 8]
	3 vs 1, swap them	[2, 1, 3, 4, 5, 6, 7, 8]
	3 vs 4, no swap	[2, 1, 3, 4, 5, 6, 7, 8]
	5, 6, 7, and 8 are now in their correct positions	
6	2 vs 1, swap them	[1, 2, 3, 4, 5, 6, 7, 8]
	2 vs 3, no swap	[1, 2, 3, 4, 5, 6, 7, 8]
	4, 5, 6, 7, and 8 are now in their correct positions	
	Now the array is sorted.	

Table 6.3: Demonstrating an example of bubble sort

Here is the Python implementation of the bubble sort algorithm:

```
def bubble_sort_iterative(a):  
    n = len(a)  
    for i in range(n):  
        elements_swapped = False  
        for j in range(0, n - i - 1):  
            if a[j] > a[j + 1]:  
                a[j], a[j + 1] = a[j + 1], a[j]  
                elements_swapped = True  
        if not elements_swapped:  
            break  
    return a
```

The bubble sort algorithm is quite simple. It consists of two nested loops; the outer loop always executes n times, while the inner loop can terminate early if the array is already sorted or nearly sorted (indicating no swaps are necessary).

Now, let's analyze the correctness of bubble sort and evaluate its time complexity.

Proof of correctness

As discussed in *Chapter 2*, we utilize the concept of a loop invariant to prove the correctness of algorithms. A loop invariant is a condition that remains true before and after each iteration of a loop.

The loop invariant for bubble sort states that at the beginning of each iteration of the outer loop (for i in range(n)), the last i elements are sorted and in their final positions. To establish the correctness of the algorithm, we need to assess three conditions:

- **Initialization:** Before the first iteration ($i = 0$), no elements have been processed. The invariant trivially holds since the empty sub-array is sorted.
- **Maintenance:** Assume the invariant holds before the i^{th} iteration. During the i^{th} iteration, the adjacent elements are compared and swapped if they are not in the right order. By the end of the i^{th} pass, the largest unsorted element is “bubbled up” to its correct position in the array, ensuring that the last i elements are sorted. Thus, the invariant is maintained.
- **Termination:** The algorithm terminates after n iterations of the outer loop, where n is the size of input data. At this point, the invariant guarantees that the entire array is sorted, as the last n elements (i.e., the entire array) are in their correct positions.

Complexity analysis

To understand time complexity, we can analyze it in the following scenarios:

- **Best case:** $O(n)$ – This happens when the array is already sorted. The algorithm will make a single pass through the array without performing any swaps and terminate early, as no out-of-order pairs (“bubbles”) will be found.
- **Average case:** $O(n^2)$ – This occurs when input data is random. The algorithm will need to make multiple passes, each involving $n - i - 1$ comparisons.
- **Worst case:** $O(n^2)$ – This occurs when the array is sorted in reverse order. Bubble sort needs to perform the maximum number of comparisons and swaps during each pass through the array.

To determine space complexity, we need to evaluate the auxiliary or temporary memory used during sorting. Bubble sort is recognized as an in-place algorithm, meaning it works directly on the input array and only requires a constant amount of extra space for swapping elements.

Selection sort

Selection sort is a comparison-based method that divides a list into two parts: a sorted section, which starts empty and grows, and an unsorted section, which starts with all the elements and shrinks. In each step, the algorithm finds the smallest (or largest) item in the unsorted part and moves it to the end of the sorted part. This continues until the entire list is sorted.

Selection sort, characterized by its simplicity, consistently exhibits a quadratic running time, $O(n^2)$, in the best, average, and worst-case scenarios. While inefficient for large datasets, its straightforward nature makes it easy to grasp and implement. This algorithm is not adaptive, meaning it doesn't exploit any existing order within the data. However, selection sort operates directly within the input data, requiring only a small, constant amount of additional memory ($O(1)$). It is important to note that selection sort is unstable and does not guarantee maintaining the original order of identical elements. Despite its drawbacks, selection sort is useful for sorting small data sets and easy to implement.

Example 6.3

Table 6.4 illustrates the step-by-step process of selection sort for the array [29, 10, 14, 37, 13, 5]:

Pass	Description of the operation	Content of the array
1	Find the minimum in [29, 10, 14, 37, 13, 5] (which is 5), and swap the minimum with the first element	[5, 10, 14, 37, 13, 29]
2	Find the minimum element in [10, 14, 37, 13, 29] (which is 10), and swap the minimum with the second element (no swap needed as it's already in place)	[5, 10, 14, 37, 13, 29]
3	Find the minimum in [14, 37, 13, 29] (which is 13), and swap the minimum with the third element	[5, 10, 13, 37, 14, 29]
4	Find the minimum in [37, 14, 29] (which is 14), and swap the minimum with the fourth element	[5, 10, 13, 14, 29, 37]
5	Find the minimum in [37, 29] (which is 29), and swap the minimum with the fifth element	[5, 10, 13, 14, 37, 29]
	Now the array is sorted.	[5, 10, 13, 14, 29, 37]

Table 6.4: Demonstrating an example of selection sort

The following is the Python implementation of the selection sort algorithm:

```
def selection_sort_iterative(a):
    n = len(a)
    for i in range(n):
        min_id = i
        for j in range(i + 1, n):
            if a[j] < a[min_id]:
                min_id = j
        a[i], a[min_id] = a[min_id], a[i]
    return a
```

The selection sort algorithm is straightforward. The outer loop, `for i in range(n) :`, traverses the array, `a`, and the inner loop, `for j in range(i + 1, n) :`, finds the element with the minimum value. Then, the minimum and the element at the top of the sub-array are swapped: `a[i] , a[min_id] = a[min_id] , a[i]`.

Let's examine the correctness of the algorithm and also evaluate its complexities.

Proof of correctness

The loop invariant in selection sort states that at the beginning of each iteration of the outer loop (`for i in range(n) :`), the sub-array, `a[0 : i]`, consists of the smallest `i` elements from the original array, arranged in ascending order. We need to evaluate three conditions for the algorithm's correctness:

- **Initialization:** Before the first iteration ($i = 0$), the sub-array, `a[0 : 0]`, is empty. The invariant trivially holds since the empty sub-array is sorted.
- **Maintenance:** Let's assume the invariant holds true before the i^{th} iteration. During this iteration, the algorithm identifies the smallest element within the sub-array, `a[i : n]`, and swaps it with `a[i]`. Consequently, `a[i]` becomes the smallest element in `a[i : n]`, and the sub-array, `a[0 : i+1]`, is sorted. This ensures the invariant is maintained.
- **Termination:** The algorithm terminates after n iterations of the outer loop. At this point, the invariant guarantees that the entire array, `a[0 : n]`, is sorted, as all elements have been processed.

Complexity analysis

To evaluate the time complexity of selection sort, consider the two nested loops. The outer loop runs n times, and for each iteration, the inner loop runs $n - 1$ times, leading to a total of $\frac{n(n-1)}{2}$ comparisons. Since this operation dominates the algorithm's runtime, the time complexity for the best, average, and worst cases is $O(n^2)$.

For space complexity, we need to determine the auxiliary or temporary memory used to execute selection sort. Since it uses an in-place approach, it only needs a constant amount of extra space for swapping elements, making its space complexity $O(1)$.

Insertion sort

Insertion sort is a simple and intuitive algorithm that builds the final ordered list gradually, one element at a time. Insertion sort operates by splitting the array into a sorted section and an unsorted section. With each step, the algorithm takes the next element from the unsorted section and inserts it into its proper place within the sorted section. This process is analogous to how one might sort playing cards, placing each card in the correct place relative to the cards already sorted.

The time complexity of insertion sort is $O(n^2)$ for both average and worst-case scenarios. However, it demonstrates impressive efficiency with a time complexity of $O(n)$ when dealing with nearly sorted or small datasets.

A key advantage of insertion sort is its stability as a sorting method. Moreover, it is an in-place algorithm, requiring only a constant amount of additional memory. Its simplicity and efficiency with small datasets make insertion sort valuable in specific situations and an excellent educational tool for understanding fundamental sorting concepts.

Example 6.4

Table 6.5 illustrates the step-by-step process of insertion sort for the array [8, 3, 1, 7, 0, 10]:

Pass	Description of the operation	Content of the array
1	Key = 3, compare 3 with 8, and move 8 to the right. Insert 3 at the correct position.	[3, 8, 1, 7, 0, 10]
2	Key = 1, compare 1 with 8, and move 8 to the right. Compare 1 with 3, and move 3 to the right. Insert 1 at the correct position.	[1, 3, 8, 7, 0, 10]
3	Key = 7, compare 7 with 8, and move 8 to the right. Insert 7 at the correct position.	[1, 3, 7, 8, 0, 10]
4	Key = 0, compare 0 with 8, and move 8 to the right. Compare 0 with 7, and move 7 to the right. Compare 0 with 3, and move 3 to the right. Compare 0 with 1, and move 1 to the right. Insert 0 at the correct position.	[0, 1, 3, 7, 8, 10]
5	Key = 10, no elements to move, insert 10 at the correct position. Now the array is sorted.	[0, 1, 3, 7, 8, 10]

Table 6.5: Demonstrating an example of insertion sort

The following is a simple Python implementation of insertion sort:

```
def insertion_sort_iterative(a):
    n = len(a)
    for i in range(1, n):
        pointer = a[i]
        j = i - 1
        while j >= 0 and pointer < a[j]:
            a[j + 1] = a[j]
```

```

        j -= 1
        a[j + 1] = pointer
    return a

```

The core of the insertion sort algorithm lies in its inner `while` loop (while `j >= 0` and `key < a[j]`). The condition in the `while` loop ensures that if the data is pre-sorted (or nearly sorted), the algorithm performs linearly in terms of time complexity. Let's discuss the algorithm's correctness and its complexity in detail.

Proof of correctness

The loop invariant for insertion sort states that at the beginning of each iteration of the outer loop (for `i` in range(1, `n`)), the elements in `a[0 : i]` consist of the same elements that can be found in `a[0 : i]`, but now in sorted order. To establish the correctness of the algorithm, we need to assess the following three conditions:

- **Initialization:** Before the first iteration (`i = 1`), the sub-array, `a[0 : 1]`, is just the first element of the array, which is trivially in the correct position (sorted).
- **Maintenance:** Let's assume the loop invariant holds before the i^{th} iteration. The algorithm inserts `a[i]` into the sorted sub-array, `a[0 : i]`, by moving elements that are greater than `a[i]` one position to the right. This insertion ensures that `a[0 : i + 1]` is sorted. Thus, the invariant is maintained.
- **Termination:** The algorithm terminates after n iterations of the outer loop, where n is the length of the array. At this point, the invariant guarantees that the entire array, `a[0 : n]`, is sorted, as all elements have been processed.

Complexity analysis

To understand the time complexity, we can analyze it in the following scenarios:

- **Best case:** $O(n)$ – As previously demonstrated, the condition in the `while` loop ensures that if the input is already sorted, the `while` loop does not execute, guaranteeing a linear time complexity.
- **Average case:** $O(n^2)$ – The algorithm performs $\frac{n(n-1)}{4}$ comparisons and $\frac{n(n-1)}{4}$ swaps on average.
- **Worst case:** $O(n^2)$ – This occurs when the input array is sorted in reverse order. In this scenario, the condition of the `while` loop causes it to execute $n - 1$ times in each iteration. Given that the outer loop runs n times, the maximum number of comparisons and swaps will be $\frac{n(n-1)}{2}$.

To assess the space complexity, we need to evaluate the auxiliary or temporary space required to run insertion sort. Given that insertion sort is an in-place algorithm, it only requires a constant amount of extra space for element swapping. Consequently, its space complexity is $O(1)$.

Having covered the major iterative sorting algorithms, we will now shift our focus to discussing recursive sorting methods.

Recursive sorting algorithms

The iterative, non-recursive sorting algorithms, while typically space efficient, often fall short in terms of time complexity, particularly when handling large datasets. This limitation becomes critical in scenarios involving substantial amounts of data, where more efficient sorting mechanisms are necessary. In *Table 6.1*, we discussed the general characteristics of recursive sorting algorithms, highlighting their potential to overcome the time complexity issues associated with iterative methods.

In this section, we analyze two major recursive sorting algorithms: merge sort and quick sort. Both algorithms leverage the divide-and-conquer strategy to achieve superior performance compared to their iterative counterparts. Merge sort, with its consistent $O(n \log n)$ time complexity, provides stable and predictable sorting efficiency. Quick sort is known for its average-case $O(n \log n)$ performance and combines simplicity with efficiency.

It's also worth mentioning that another significant recursive sorting algorithm, heap sort, will be introduced in *Chapter 13* when we discuss heap structures. Heap sort, like merge sort and quick sort, utilizes the divide-and-conquer approach but does so through the use of a binary heap data structure, providing an efficient and space-conscious sorting solution.

Merge sort

Merge sort is a comparison-based sorting algorithm that utilizes the divide-and-conquer strategy. It operates by recursively splitting the unsorted array into two roughly equal partitions until each sub-array consists of a single element (which is naturally sorted). The algorithm then merges these sub-arrays to create new sorted sub-arrays, continuing this process until only one sorted array remains, which represents the original array in sorted order. At the merge step, the smallest elements of each sub-array are compared and then two sub-arrays are combined.

Example 6.5

Table 6.6 illustrates the step-by-step process of merge sort for the array [38, 27, 43, 3, 9, 82]:

Split/merge	Content of the (sub)array
First split	Split [38, 27, 43, 3, 9, 82] into [38, 27, 43] and [3, 9, 82].
Second split (left half)	Split [38, 27, 43] into [38] and [27, 43]. Split [27, 43] into [27] and [43].
Merge (left half)	Merge [27] and [43] to get [27, 43]. Merge [38] and [27, 43] to get [27, 38, 43].
Second split (right half)	Split [3, 9, 82] into [3] and [9, 82]. Split [9, 82] into [9] and [82].

Split/merge	Content of the (sub)array
Merge	Merge [9] and [82] to get [9, 82].
(Right half)	Merge [3] and [9, 82] to get [3, 9, 82].
Final merge	Merge [27, 38, 43] and [3, 9, 82] to get [3, 9, 27, 38, 43, 82]. Now the array is sorted: [3, 9, 27, 38, 43, 82].

Table 6.6: Demonstrating an example of merge sort

Here is the Python implementation of the merge sort algorithm:

```
import numpy as np
def merge(A,p,q,r):
    n1=q-p+1
    n2=r-q
    n11=n1+1
    n22=n2+1
    left = [0 for i in range(n11)]
    right = [0 for i in range(n22)]
    for i in range(n1):
        left[i]=A[p+i-1]
    for j in range(n2):
        right[j]=A[q+j]

    left[n11-1]=1000 #very large number
    right[n22-1]=1000 #very large number
    i=0
    j=0
    for k in range(p-1,r):
        if left[i]<=right[j]:
            A[k]=left[i]
            i=i+1
        else:
            A[k]=right[j]
            j=j+1
    return(A)
def mergeSort(A,p,r):
    if p<r:
        q=int(np.floor((p+r)/2))
        mergeSort(A,p,q)
```

```

mergeSort (A, q+1, r)
merge (A, p, q, r)
return (A)

```

While there are numerous implementations of merge sort, the version we presented is quite straightforward. The key component of the algorithm is a function called `merge` used to merge the two partitions that have been recursively sorted.

Proof of correctness

The loop invariant definition in merge sort states that at the start of each iteration of the merging process, the `left_partition` and `right_partition` sub-arrays are sorted. We need to evaluate three conditions for the algorithm's correctness:

- **Initialization:** Before the first merge operation, the sub-arrays contain single elements, which are inherently sorted.
- **Maintenance:** Assume the invariant holds before merging two sub-arrays. During the merge process, the smallest remaining element from either sub-array is selected and added to the merged array. This maintains the sorted order of the merged array. The invariant holds because each selection step ensures the merged array remains sorted.
- **Termination:** The algorithm terminates after all sub-arrays have been merged into a single sorted array. At this point, the invariant guarantees that the entire array is sorted.

Complexity analysis

Merge sort is a recursive sorting algorithm that operates according to the recurrence function $T(n) = 2T(\frac{n}{2}) + O(n)$. At each recursive step, merge sort divides the problem into two nearly equal sub-problems, reflecting the general divide-and-conquer relation $T(n) = aT(\frac{n}{b}) + f(n)$ with $a = 2$ and $b = 2$. The division of the array and the merging of the sorted sub-arrays are both linear time operations, hence $f(n) = O(n)$.

To evaluate the running time complexity of merge sort, we need to solve this recurrence function using methods such as the master theorem. The master theorem provides a straightforward way to solve recurrences of the form $T(n) = aT(\frac{n}{b}) + f(n)$ by comparing $f(n)$ to $n^{\log_b a}$.

In the case of merge sort, $a = 2$, $b = 2$, and $f(n) = O(n)$.

We calculate $\log_b a = \log_2 2 = 1$. The following apply according to the master theorem:

- If $f(n) = O(n^c)$ where $c < \log_b a$, then $T(n) = \Theta(n^{\log_b a})$
- If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$
- If $f(n) = \Omega(n^c)$ where $c > \log_b a$, and if $af(\frac{n}{b}) \leq cf(n)$ for some $c < 1$ and sufficiently large n , then $T(n) = \Theta(f(n))$

In our case, $f(n) = O(n)$ matches the second case of the master theorem because $f(n)$ is $\Theta(n^{\log_2 2}) = \Theta(n)$. Thus, case 2 applies, leading to the following conclusion:

$$T(n) = \Theta(n \log n).$$

To assess the time complexity of merge sort, we can analyze it in the following scenarios:

- **Best case:** $O(n \log n)$ – Merge sort consistently divides the array and merges sub-arrays in logarithmic steps
- **Average case:** $O(n \log n)$ – The algorithm performs consistently in all cases due to its divide-and-conquer approach
- **Worst case:** $O(n \log n)$ – Merge sort is not adaptive, meaning its performance remains consistent regardless of the initial order of the elements

To determine the space complexity, we need to consider the auxiliary or temporary space required to execute merge sort. In the merge sort algorithm, two arrays, `left_partition` and `right_partition`, are used as temporary storage during the merge operation. The combined size of these two arrays is equal to the input size, resulting in a space complexity of $O(n)$.

In summary, merge sort is a highly efficient and stable sorting algorithm with a time complexity of $O(n \log n)$ in the best, average, and worst cases. It employs a divide-and-conquer approach, dividing the array into smaller sub-arrays, sorting them recursively, and then merging them in sequence. Although it necessitates $O(n)$ additional space, its consistent performance and stability make it an excellent choice for sorting large datasets, especially when stability is essential. The recursive nature and dependable performance of merge sort make it a foundational algorithm in computer science.

Quick sort

Quick sort is a highly efficient divide-and-conquer sorting algorithm that, like merge sort, relies on comparisons. It recursively selects a pivot element from the array and partitions the remaining elements into two sub-arrays based on whether they are smaller or larger than the pivot. These sub-arrays are then sorted recursively. This process repeats until the base case of an empty or single-element sub-array, which is inherently sorted, is reached. The effectiveness of quick sort depends largely on the choice of pivot and the method used for partitioning.

The quick sort algorithm is comprised of three key components:

- **Pivot selection:** This step involves choosing an element as the pivot from the array. Common choices include the first, last, middle, or a randomly selected element. The specific pivot selection strategy can influence the algorithm's overall performance. In the probabilistic quick sort, a pivot is randomly selected (see *Chapter 9*).

- **Partitioning:** During this step, the array is divided into two smaller sub-arrays. All elements smaller than the pivot are placed in one sub-array, while those greater than the pivot are placed in the other. After this step, the pivot is in its final sorted position within the array.
- **Recursion:** The partitioning step is repeated recursively until all sub-arrays are either empty or contain a single element, ultimately resulting in a fully sorted array.

Example 6.6

Let's illustrate the operation with an array of six random numbers—[35, 12, 99, 42, 5, 8]:

1. Choose a pivot (e.g., 8) and rearrange the array so that elements less than the pivot come before it and elements greater come after it.
2. Recursively sort left sub-array [5]:
 [5] is already sorted.
3. Recursively sort right sub-array [99, 42, 35, 12]:
 Choose a pivot (e.g., 35) and rearrange: [5, 8, 12, 35, 42, 99].
4. Recursively sort sub-arrays [12] and [42, 99]:
 - [12] is already sorted.
 - Choose a pivot (e.g., 42) and rearrange [42, 99]: [5, 8, 12, 35, 42, 99].
5. Now the array is sorted: [5, 8, 12, 35, 42, 99].

Here is the Python implementation of the quick sort algorithm:

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    else:  
        pivot = arr[len(arr) // 2]  
        left = [x for x in arr if x < pivot]  
        middle = [x for x in arr if x == pivot]  
        right = [x for x in arr if x > pivot]  
        return quick_sort(left) + middle + quick_sort(right)
```

As demonstrated, the core concept in the quick sort algorithm is selecting a pivot element for partitioning. This process is then repeated recursively.

Proof of correctness

The loop invariant definition in merge sort guarantees that at the beginning of each iteration of the partitioning process, all elements in `left_partition` are not greater than the `pivot`, and all elements in `right_partition` are not less than the `pivot`. We need to evaluate three conditions for the algorithm's correctness:

- **Initialization:** Before the first partitioning operation, the sub-array is the entire array, and the loop invariant trivially holds since no elements have been processed yet.
- **Maintenance:** During the partitioning process, elements are compared to the `pivot` and swapped as necessary. This ensures that the two sub-arrays, `left_partition` and `right_partition`, are maintained according to the definition; `left_partition` contains elements smaller than the `pivot`, and `right_partition` contains elements greater than the `pivot`.
- **Termination:** The algorithm terminates when sub-arrays of length 0 or 1 are reached, which are inherently sorted. The loop invariant guarantees that the elements are correctly partitioned at each step, ensuring the entire array is sorted upon completion.

Complexity analysis

Quick sort is characterized by the following recurrence function:

$$T(n) = T(k) + T(n - k - 1) + O(n)$$

Here, k is the number of elements smaller than the `pivot`. To simplify the analysis, we assume an ideal scenario where the `pivot` always splits the list into two equal partitions, leading to the following recurrence function:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

This can be expressed in the general form of a divide-and-conquer function:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \text{ with } a = 2, b = 2, \text{ and } f(n) = O(n)$$

To determine the time complexity, we apply the master theorem. The master theorem helps solve recurrences of the form $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ by comparing $f(n)$ to $n^{\log_b a}$. In the case of quick sort, $a = 2$, $b = 2$, and $f(n) = O(n)$.

In our case, $f(n) = O(n)$ matches the second case of the master theorem because $f(n)$ is $\Theta(n^{\log_2 2}) = \Theta(n)$. Thus, case 2 applies, leading to the following conclusion:

$$T(n) = \Theta(n \log n)$$

This indicates that the average-case time complexity of quick sort is $\Theta(n \log n)$. Due to this efficiency, quick sort is highly effective for large datasets. However, in the worst-case scenario, which occurs with poor pivot selection, resulting in consistently unbalanced partitions (e.g., always choosing the smallest or largest element as the pivot), the recurrence relation becomes the following:

$$T(n) = T(n - 1) + T(0) + O(n)$$

This simplifies to the following:

$$T(n) = T(n - 1) + O(n)$$

Solving this recurrence relation yields the following:

$$T(n) = \Theta(n^2)$$

Therefore, the worst-case time complexity of quick sort is $O(n^2)$. However, strategies such as random pivot selection or the median-of-three method can greatly reduce the chances of encountering this worst-case scenario, helping to preserve the algorithm's average-case performance. The following is a summary of quick sort's time complexity:

- **Best case:** $O(n \log n)$ – This occurs when the pivot consistently splits the array into two nearly equal halves
- **Average case:** $O(n \log n)$ – The algorithm performs well on average with random pivot selection
- **Worst case:** $O(n^2)$ – Occurs when the pivot selection consistently results in the most unbalanced partitions (e.g., the smallest or largest element)

Quick sort is a recursive algorithm characterized by the recurrence relation $T(n) = 2T(\frac{n}{2}) + O(n)$. In each recursion step, quick sort divides the problem into two nearly equal sub-problems, leading to the relation $T(n) = aT(\frac{n}{b}) + f(n)$, with $a = b = 2$. The dividing and merging operations occur in linear time, represented by $f(n) = O(n)$. To understand the time complexity, we can analyze it in the following scenarios.

For the space complexity, we need to consider the auxiliary or temporary space required to execute quick sort. Quick sort requires additional space for the recursion stack. In the average case, the stack depth is $O(\log n)$. However, in the worst case (e.g., unbalanced partitions), the stack depth can be $O(n)$.

In summary, the performance of quick sort is significantly influenced by the choice of pivot and partitioning method. Poor pivot selection can result in a worst-case time complexity of $O(n^2)$, while the average-case performance is $O(n \log n)$. Techniques such as random pivot selection or the median-of-three method can help mitigate the risk of poor pivot selection. Although quick sort is not a stable sorting algorithm, it is in-place and requires $O(\log n)$ auxiliary space for the recursion stack in the average case. Its efficiency and simplicity make quick sort a popular choice for sorting large datasets in various applications.

All the sorting algorithms we've discussed so far, whether recursive or non-recursive, have been comparison-based. Now, we turn to another significant topic in sorting algorithms – non-comparison-based methods, which can achieve sorting in linear time.

Non-comparison-based sorting

We have noted that all comparison-based sorting algorithms have a lower-bound time complexity of $\Omega(n \log n)$, which means no comparison-based algorithm can achieve better performance than $n \log n$ in the general case. However, there are sorting algorithms that do not depend on comparisons. These non-comparison-based algorithms leverage certain assumptions or information about the data to achieve better performance.

Unlike comparison-based algorithms, non-comparison-based algorithms can achieve a lower bound as efficiently as linear time. This remarkable efficiency is why they are sometimes referred to as linear-time sorting algorithms. By making use of specific properties of the data, such as a limited range of integer values or the distribution of digits, these algorithms can bypass the comparison limit and sort much faster under the right conditions. In this section, we will explore three well-known non-comparison-based sorting algorithms: counting sort, Radix sort, and bucket sort.

Counting sort

Counting sort is the first non-comparison-based sorting algorithm we will discuss in this section. It sorts a collection of elements in an array by counting the frequency of each distinct element in the input array. This frequency count is then used to determine the correct position of each element in the sorted output. As a result, no comparison operations are involved in this sorting process. Counting sort is particularly effective for sorting integers within a specific range and is known for its linear time complexity, making it highly efficient under the right conditions. Unlike other sorting algorithms, counting sort does not perform comparisons between elements, thus bypassing the $\Omega(n \log n)$ lower bound for comparison-based sorts. Let's demonstrate the counting sort algorithm step by step with an example.

Counting sort is an efficient algorithm with a time complexity of $O(n + k)$. It is also a stable sorting method with minimal additional memory usage. Although it requires extra space proportional to the range of input values, it maintains linear space complexity. Due to its linear time complexity, counting sort is widely used for sorting large datasets with a limited range of elements. Its simplicity and efficiency under the right conditions make it a valuable tool for algorithm designers. Let's explain the counting sort algorithm in the following example.

Example 6.7

Let's illustrate the operation of counting sort with an array of six random numbers—[4, 2, 2, 8, 3, 3]:

Determine the range:

- Find the maximum element (8) and the minimum element (2)
- Create a count array with size $max - min + 1 = 8 - 2 + 1 = 7$, initialized to 0

Count occurrences:

- Count each element's occurrences and store them in the count array
- Count array: [2, 2, 1, 0, 0, 0, 1]
- This corresponds to elements 2, 3, 4, 5, 6, 7, 8

Cumulative count:

- Modify the count array by adding the count of the previous element to each element. This helps in determining the positions of elements.
- Cumulative count array: [2, 4, 5, 5, 5, 5, 6].

Build the output array:

- Place each element in its correct position in the output array based on the cumulative count. We start from the last element down to the first element.

Table 6.7 illustrates the step-by-step process of counting sort for the array [4, 2, 2, 8, 3, 3]:

Element in array	Cumulative count array	Output array
3	2, 3, 5, 5, 5, 5, 6	0, 0, 0, 3, 0, 0
3	2, 2, 5, 5, 5, 5, 6	0, 0, 3, 3, 0, 0
8	2, 2, 5, 5, 5, 5, 5	0, 0, 3, 3, 0, 8
2	1, 2, 5, 5, 5, 5, 5	0, 2, 3, 3, 0, 8
2	0, 2, 5, 5, 5, 5, 5	2, 2, 3, 3, 0, 8
4	0, 2, 4, 5, 5, 5, 5	2, 2, 3, 3, 4, 8

Table 6.7: Demonstrating an example of counting sort in operation

The following is a simple Python implementation of the counting sort algorithm:

```
def counting_sort(arr):
    if not arr:
        return arr
    max_val = max(arr)
    min_val = min(arr)
    range_of_elements = max_val - min_val + 1
    # Create a count array to store count of individual elements and
    # initialize it to 0
    count = [0] * range_of_elements
    output = [0] * len(arr)
    # Store the count of each element
    for num in arr:
        count[num - min_val] += 1
    # Change count[i] so that count[i] contains the actual position of
    # this element in the output array
    for i in range(1, len(count)):
        count[i] += count[i - 1]
    # Build the output array
    for num in reversed(arr):
        output[count[num - min_val] - 1] = num
        count[num - min_val] -= 1
    # Copy the output array to arr, so that arr contains sorted
    # numbers
    for i in range(len(arr)):
        arr[i] = output[i]
    return arr
```

The `counting_sort` algorithm is quite simple. The first two loops (`for i in range(1, len(count)):` and `for i in range(1, len(count)):`) build a cumulative count array. This array is then used in the third loop (`for k in reversed(a):`) to construct the sorted array.

Proof of correctness

The loop invariant in counting sort states that after processing each element, the cumulative count array correctly reflects the number of elements less than or equal to each value in the array, and the output array contains elements in sorted order up to the current index. We need to evaluate three conditions for the algorithm's correctness:

- **Initialization:** Before any elements are processed, the count array is initialized to 0, and the output array is empty. The invariant trivially holds.

- **Maintenance:** As each element is processed, the cumulative `count` array is updated to reflect the correct count of elements. During the construction of the output array, elements are placed in their correct positions based on the cumulative counts, ensuring that the sorted order is maintained.
- **Termination:** The algorithm terminates after all elements have been processed and placed in the output array. At this point, the invariant guarantees that the output array is sorted, and the cumulative `count` array accurately reflects the positions of elements.

Complexity analysis

Counting sort has a linear time complexity that is influenced by both the size of the array (n) and the range of the input values k (where $k = \text{max} - \text{min} + 1$). Let's analyze the time complexity of counting sort in three scenarios:

- **Best case:** In the best-case scenario, counting sort iterates through the input array to compute the frequency of each element (`count`), then iterates through the `count` array to calculate cumulative counts, and finally iterates through the input array again to place elements in their correct positions in the output array. This process involves $O(n)$ time to count elements, $O(k)$ time to compute cumulative counts, and $O(n)$ time to build the output array, resulting in a total time complexity of $O(n + k)$.
- **Average case:** The average-case time complexity of counting sort remains $O(n + k)$ because the steps involved (counting elements, computing cumulative counts, and building the output array) are consistent regardless of the initial order or distribution of elements within the range. Each element in the input array is processed a constant number of times.
- **Worst case:** In the worst-case scenario, counting sort still performs with $O(n + k)$ time complexity. The worst case occurs when the range of input values is large compared to the size of the input array. Despite this, the operations of counting, computing cumulative counts, and placing elements in the output array are performed in linear time relative to the number of elements and the range of values.

Now, we need to estimate the space complexity of counting sort. Counting sort requires additional space to store both the `count` array and the output array (`temp`), leading to an auxiliary space complexity of $O(n + k)$. Let's look at the auxiliary space we need in counting sort. First is the `count` array, which is used to store the frequency of each element within the range $[\text{min}, \text{max}]$. The size of the `count` array is proportional to the range of input values, which is k . The second auxiliary memory space is the output array. An output array (`temp`) of the same size as the input array is used to store the sorted elements. This requires $O(n)$ additional space. Then, the total auxiliary space required by counting sort is the sum of the space for the `count` array and the output array, resulting in $O(n + k)$.

Counting sort is an efficient linear time sorting algorithm, but it has a major limitation: its efficiency is highly dependent on the range of elements in the array to be sorted. Additionally, counting sort is limited to integer numerical data. In the next sub-section, we will discuss Radix sort, which can address some of the limitations inherent in counting sort.

Radix sort

Radix sort, another non-comparative sorting technique, sorts elements by sequentially processing individual digits or characters. It leverages the concept that sorting can be achieved one position at a time, beginning with the least significant and progressing toward the most significant. By employing a stable sorting algorithm such as counting sort for each position, Radix sort maintains the original order of elements, sharing the same digit or character. This approach renders Radix sort highly efficient for datasets comprised of numbers or strings with a fixed or limited length.

Radix sort is particularly effective for sorting large numbers of integers or strings with a fixed number of digits or characters. The algorithm leverages counting sort to maintain stability and achieve linear time complexity for each digit. While Radix sort requires additional space for the output and count arrays, its overall efficiency makes it an excellent choice for specific types of data where the number of digits or characters is known and limited. Understanding Radix sort and its mechanisms can provide significant performance advantages in appropriate scenarios.

Besides its linear time complexity and stability, the Radix sort algorithm offers several other advantages, including the following:

- **Suitable for large data:** Radix sort is especially effective for sorting large datasets where the range of input values is not significantly larger than the number of elements. It handles large amounts of data well, particularly when the keys are integers or strings of a fixed length.
- **Predictable performance:** Radix sort consistently performs well, with no worst-case degradation, as seen in quick sort. Its time complexity is predictable and does not depend on the input data's initial order.
- **Extensibility:** Radix sort can be easily adapted to sort data types other than integers, such as strings or other sequences, by using different bases or treating each character as a “digit.”

Let's demonstrate the Radix sort algorithm step by step with an example.

Example 6.8

Let's illustrate the operation of Radix sort with an array of six random numbers—[170, 45, 75, 90, 802, 24]:

1. Sort the original array by the least significant digit (1's place):

Apply counting sort to the least significant digit: [170, 90, 802, 24, 45, 75]

2. Sort by second least significant digit (10's place):

Apply counting sort to the second least significant digit: [802, 24, 45, 75, 170, 90]

3. Sort by most significant digit (100's place):

Apply counting sort to the most significant digit: [24, 45, 75, 90, 170, 802]

Now the array is sorted: [24, 45, 75, 90, 170, 802].

Here is the Python implementation of the Radix sort algorithm:

```
def count_sort(a, e):
    size = len(a)
    result = [0] * size
    count = [0] * 10 # For digits 0-9
    for i in range(size):
        digit = (a[i] // e) % 10
        count[digit] += 1
    for i in range(1, 10):
        count[i] += count[i - 1]
    for i in range(size - 1, -1, -1):
        digit = (a[i] // e) % 10
        result[count[digit] - 1] = a[i]
        count[digit] -= 1
    for i in range(size):
        a[i] = result[i]
def radix_sort(a):
    maxv = max(a)
    e = 1
    while maxv // e > 0:
        count_sort(a, e)
        e *= 10
```

In the preceding Radix sort algorithm implementation, the `radix_sort(a)` function takes the input array (`a`) and applies counting sort (`count_sort(a, e)`), starting from the least significant digit. It sorts the data by each digit in the appropriate place (`e`).

Proof of correctness

The loop invariant is defined in Radix sort as after sorting by each digit (starting from the least significant to the most significant), the array is partially sorted with respect to that digit, while maintaining the relative order of elements with the same digit. We need to evaluate three conditions for the algorithm's correctness:

- **Initialization:** Before any digits are processed, the array is unsorted. The invariant trivially holds as there is no partial sorting.

- **Maintenance:** During each iteration, counting sort is applied to sort elements based on the current digit. Since counting sort is stable, it maintains the relative order of elements with the same digit. This ensures that after each pass, the array is partially sorted with respect to that digit.
- **Termination:** The algorithm terminates after sorting by the most significant digit. At this point, the invariant guarantees that the array is fully sorted because all digits have been processed in order of significance.

Complexity analysis

Radix sort has a time complexity that depends on the number of digits in the largest number, d , the number of elements, n , and the range of digit values, k . Let's analyze the time complexity of Radix sort in three scenarios:

- **Best case:** In the best case, Radix sort processes each digit of the numbers in the array. For each digit, it uses counting sort, which operates in $O(n + k)$ time. Since there are d digits, the overall time complexity is $O(d \cdot (n + k))$.
- **Average case:** The average-case time complexity of Radix sort remains $O(d \cdot (n + k))$ because the steps involved (sorting by each digit using counting sort) are consistently executed for each digit, regardless of the distribution of the input values.
- **Worst case:** In the worst case, Radix sort still performs with $O(d \cdot (n + k))$ time complexity. This is because each digit is processed in linear time by counting sort, and the process is repeated d times for d digits.

Speaking about the space complexity of Radix sort, it requires additional space to store the count array and the output array used by counting sort, resulting in an auxiliary space complexity of $O(n + k)$.

Radix sort is more efficient when the elements of the array are spread out over a wide range, meaning the values are sparse. If the elements are densely packed within the range, an alternative approach is bucket sort, which will be discussed in the next sub-section.

Bucket sort

The final non-comparison-based sorting algorithm we will examine is bucket sort. This approach involves distributing the elements of an array into multiple containers, known as "buckets," each associated with a specific range of values. The elements within each bucket are then sorted independently, often using another sorting algorithm, and subsequently combined to produce the fully sorted array. Bucket sort demonstrates exceptional efficiency when the input data is evenly spread across a known range. It capitalizes on the divide-and-conquer strategy, simplifying the sorting task by breaking it down into smaller, more manageable sub-problems.

Bucket sort is particularly effective for sorting large datasets with uniformly distributed values, providing significant performance advantages over comparison-based sorting algorithms in these scenarios. Understanding Bucket sort and its mechanisms can help optimize sorting performance in suitable applications. Let's demonstrate the bucket sort algorithm step by step with an example.

Example 6.9

Let's illustrate the operation of bucket sort with an array of six random numbers— [0.78, 0.17, 0.39, 0.26, 0.72, 0.94]:

1. Distribute elements into buckets:

Create an empty list of buckets.

Distribute each element into the appropriate bucket based on its value.

Buckets: [[0.17, 0.26], [0.39], [0.72, 0.78], [0.94]].

2. Sort each bucket:

Sort the elements within each bucket. This can be done using another sorting algorithm such as insertion sort.

Sorted buckets: [[0.17, 0.26], [0.39], [0.72, 0.78], [0.94]].

3. Concatenate sorted buckets:

Concatenate all the sorted buckets to form the final sorted array:

[0.17, 0.26, 0.39, 0.72, 0.78, 0.94]

The array is now sorted.

The following is a simple Python implementation of the bucket sort algorithm:

```
def bucket_sort(a):
    number_of_buckets = len(a)
    buckets = [[] for _ in range(number_of_buckets)]
    for i in a:
        idx = int(i * number_of_buckets)
        buckets[idx].append(i)
    sorted_a = []
    for b in buckets:
        sorted_a.extend(insertion_sort(b))
    return sorted_a
```

The algorithm is straightforward. Initially, the buckets are set up, and the elements of the array, `a`, are distributed into these buckets (`for i in a:`). Each bucket is then sorted using insertion sort and concatenated (`for b in buckets:`). Finally, the sorted buckets are concatenated together.

Proof of correctness

The loop invariant in bucket sort is defined as at the start of each iteration, the elements in each bucket are partially sorted with respect to their position within the bucket. We need to evaluate three conditions for the algorithm's correctness:

- **Initialization:** Before any elements are processed, the buckets are empty. The invariant trivially holds as there is no partial sorting.
- **Maintenance:** During each iteration, elements are distributed into buckets based on their values. Sorting each bucket individually ensures that the elements within each bucket are sorted. This maintains the invariant.
- **Termination:** The algorithm terminates after all buckets have been sorted and concatenated. At this point, the invariant guarantees that the entire array is sorted because each bucket is sorted, and the buckets are concatenated in order.

Complexity analysis

Bucket sort has a time complexity that depends on both the number of elements, n , and the number of buckets, k . The distribution of elements among the buckets also plays a crucial role in determining the overall time complexity. Let's examine the time complexity of Radix sort in the following three scenarios:

- **Best case:** In the best-case scenario, the elements are uniformly distributed across the buckets, and each bucket contains a roughly equal number of elements. Sorting each bucket individually takes constant time if an efficient sorting algorithm such as insertion sort is used within the buckets. As a result, the overall time complexity is $O(n + k)$.
- **Average case:** On average, the time complexity of bucket sort remains $O(n + k)$ as long as the elements are evenly distributed, and the number of buckets, k , is proportional to the number of elements, n . Each element is placed into a bucket in $O(1)$ time, and each bucket is sorted individually in linear time relative to the number of elements it contains.
- **Worst case:** In the worst-case scenario, if all elements are placed into a single bucket, the algorithm's time complexity degrades to $O(n^2)$. This is because the entire array would need to be sorted within one bucket, resulting in quadratic time complexity when using a sorting algorithm such as insertion sort within that bucket.

For the space complexity of bucket sort, we know that it requires additional space to store the buckets and the output array, leading to an auxiliary space complexity of $O(n + k)$. The auxiliary space has two components: the first is the buckets. The buckets are used to store elements within specific value ranges. The number of buckets, k , is typically chosen based on the range and distribution of input values. The second auxiliary space is the output array. An output array of the same size as the input array is used to store the sorted elements temporarily. This requires $O(n)$ additional space. Then, the total auxiliary space required by bucket sort is the sum of the space needed for the buckets and the output array, resulting in $O(n + k)$.

This concludes our discussion on non-comparison sorting algorithms. These algorithms provide better time complexities compared to comparison-based sorts but come with certain limitations. They are restricted to specific data types and rely on assumptions about the data, which can narrow their range of applications.

Summary

This chapter provided a comprehensive exploration of various sorting algorithms, highlighting their underlying principles, efficiencies, and practical implementations. The chapter began by discussing the fundamental properties that differentiate sorting algorithms, such as comparison versus non-comparison-based methods, stability, adaptability, and memory usage. These properties were crucial for understanding why certain algorithms are more suitable for specific types of data and applications. The chapter emphasized the importance of time complexity, particularly noting that comparison-based algorithms have a lower-bound complexity of $\Omega(n \log n)$, while non-comparison-based algorithms can achieve linear time complexity under the right conditions.

The chapter then discussed specific sorting algorithms, starting with the iterative methods: bubble sort, selection sort, and insertion sort. These algorithms, despite their simplicity and ease of implementation, were shown to be inefficient for large datasets due to their $O(n^2)$ time complexity. The explanations included detailed step-by-step operations, code implementations, and proofs of correctness using loop invariants. Each algorithm's unique characteristics, such as stability and space efficiency, were discussed, providing a clear understanding of their strengths and limitations.

Finally, the chapter explored the more advanced recursive sorting algorithms, including merge sort and quick sort. These algorithms leveraged the divide-and-conquer strategy to achieve more efficient sorting with an average time complexity of $O(n \log n)$. Merge sort's consistent performance and stability were contrasted with quick sort's potential for $O(n^2)$ worst-case time complexity, which could be mitigated by good pivot selection strategies. Additionally, the chapter touched on non-comparison-based algorithms such as counting sort, Radix sort, and bucket sort, explaining how they achieve linear time complexity by utilizing specific data properties. The chapter concluded by reinforcing the importance of choosing the right sorting algorithm based on the dataset characteristics and the specific requirements of the application.

In the next chapter, we focus on another core problem in algorithm design—searching. Then, *Chapter 8* examines the relationship between sorting and searching, uncovering fascinating patterns and connections.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022.
 - *Chapter 2, Getting Started*
 - *Chapter 4, Divide-and-Conquer (includes merge sort)*
 - *Chapter 7, Quicksort*
 - *Chapter 8, Sorting in Linear Time*
 - *Chapter 9, Medians and Order Statistics*
- *Algorithms*. By R. Sedgewick and K. Wayne. Fourth Edition. Addison-Wesley. 2011.
 - *Chapter 2: Elementary Sorts*
- *The Art of Computer Programming, Volume 3: Sorting and Searching*. By Donald E. Knuth.
 - *Chapter 5, Sorting*
- *Data Structures and Algorithm Analysis in C++*. By Mark Allen Weiss.
 - *Chapter 7, Sorting*
- *Algorithm Design Manual*. By Steven S. Skiena.
 - *Chapter 4, Sorting and Searching*

Search Algorithms

In the data and information processing domain, search and information retrieval play a very crucial role. The efficiency and accuracy of search algorithms directly impact the effectiveness of various applications, from database management systems to search engines. This chapter discusses the critical importance of search algorithms, clarifying their fundamental properties through a series of examples.

Search algorithms are designed to optimize the retrieval process, making it faster and more efficient. A crucial aspect of this optimization is sorting, which organizes data in a manner that facilitates rapid search operations. The interplay between sorting and searching is evident in numerous applications where sorted data allows for more sophisticated and quicker search techniques. We will explore this essential relationship between sorting and searching in the next chapter. However, when we separate the concept of search from sorting, we are often left with the basic sequential search. Sequential search operates in linear time, scanning each element one by one until the desired result is found. This method, while straightforward, is not always the most efficient, especially for large datasets.

To address the limitations of sequential search, we can leverage data assumptions to devise more advanced techniques such as hashing. Hashing transforms data into a fixed-size value or hash code (hash value), allowing for constant time search operations under ideal conditions. This chapter explores these advanced techniques, illustrating how assumptions about data can significantly enhance search performance. By employing hashing functions, we can achieve constant time complexity, drastically improving the efficiency of search operations in many applications.

In this chapter, search algorithms are categorized into three primary groups: linear-time search algorithms, sub-linear (e.g., logarithmic time) search algorithms, and constant-time search algorithms that utilize hashing. Each category is examined in detail, with a focus on its properties, use cases, and performance characteristics. The chapter is structured as follows:

- Properties of search algorithms
- Linear-time and logarithmic search algorithms
- Hashing

Properties of search algorithms

Before diving into search algorithms and their properties, it's important to differentiate between two types of search in computer science: **algorithmic** search and search in **artificial intelligence (AI)**. While these two types of search share some similarities, they have distinct differences and objectives.

The concept of *search* in algorithms and *search* in AI can differ significantly in terms of their goals, methodologies, and applications. Here's a comparison highlighting the key differences.

Algorithmic search refers to the process of finding a specific element or set of elements within a data structure, such as an array, list, or tree. The primary goal is to locate the desired element(s) as quickly as possible, often measured by time complexity (e.g., $O(n)$, $O(\log n)$). On the other hand, the correctness of algorithmic search is to ensure the algorithm correctly identifies the presence or absence of the target element.

We implement algorithmic search using one of the following methods:

- **Linear search:** Iterates through each element until the target is found or the end of the structure is reached
- **Binary search:** Efficiently locates elements in sorted arrays by repeatedly dividing the search interval in half
- **Hashing:** Uses hash functions to map elements to specific positions for quick access
- **Tree traversal:** Searches within tree structures, such as binary search trees, AVL trees, and red-black trees

The major application of algorithmic search is in data retrieval, which is used in databases, filesystems, and general data processing tasks.

On the other hand, we have search algorithms in AI, or simply AI search. AI search involves finding a sequence of actions or a path from an initial state to a goal state within a problem space. It often deals with more complex and dynamic environments and has different objectives from algorithmic search. The primary objective of AI search is problem-solving – finding a solution to a complex problem that may involve navigating through a large state space. Additionally, AI search aims to find the best or most efficient solution among many possible options and must be capable of handling dynamic and uncertain environments where conditions and goals may change.

AI search algorithms are implemented with different strategies depending on the problem type and our knowledge of the goal state. Although these methods can be categorized in various ways, the most well-known AI search families are uninformed search and informed search. Uninformed search techniques, such as **breadth-first search (BFS)** and **depth-first search (DFS)**, explore the search space without specific knowledge about the goal. In contrast, informed search techniques, such as **A* search** and **greedy search**, use heuristics to guide the search process toward the goal more efficiently. Heuristics provide an estimated cost to reach the goal, helping to prioritize paths that appear more promising.

AI search has many applications. For example, in robotics, search helps agents navigate and perform tasks in dynamic environments. In game playing, AI agents search for optimal moves in games such as chess, Go, and video games.

In conclusion, algorithmic search focuses on efficiently finding specific elements within structured data using well-defined procedures and data structures. In contrast, AI search involves exploring large and often unstructured problem spaces to find optimal or feasible solutions to complex problems, using both uninformed and informed techniques, often incorporating heuristics and learning methods.

In essence, while both types of search aim to find solutions, algorithmic search is typically more concerned with data retrieval and manipulation within well-defined constraints, whereas AI search addresses broader and more complex problem-solving scenarios, often requiring adaptability and learning.

In this section, we introduce several properties to evaluate and compare different algorithmic search algorithms (in this chapter, we will use *search* to refer to algorithmic search). These properties, in addition to time and space complexities, provide a baseline to understand the behavior and efficiency of search algorithms. By considering these properties, we can choose the most suitable algorithm for a given scenario:

- **Data structure requirement:** Different search algorithms may require specific data structures to operate efficiently. For example, binary search necessitates a sorted array to function correctly, whereas sequential search can work with any linear data structure, such as arrays or linked lists.
- **Adaptability:** Some search algorithms have the ability to adapt to the characteristics of the input data, thereby improving their performance. For example, interpolation search adapts to the distribution of data and can perform significantly better than binary search on uniformly distributed datasets.
- **Implementation complexity:** The complexity of implementing an algorithm is a practical consideration, especially in time-constrained situations. Simpler algorithms such as sequential search are easy to implement and understand, while more complex algorithms such as balanced search trees (see *Chapter 13*) or hashing require a deeper understanding of data structures and algorithm design.
- **Preprocessing requirements:** Certain search algorithms require preprocessing of the data before they can be applied. For example, binary search requires the data to be sorted, which adds to the overall time complexity. Preprocessing steps can sometimes negate the benefits of a faster search time, especially if the data changes frequently and requires constant re-sorting.
- **Optimality:** Some algorithms are considered optimal for specific scenarios based on their time complexity and performance characteristics. Binary search, for instance, is optimal for searching within a sorted array due to its logarithmic time complexity. However, optimality can vary based on the context and specific requirements of the application. An algorithm that is optimal in one scenario may not be the best choice in another if the underlying assumptions or conditions change.

By examining these key characteristics – data structure requirements, adaptability, implementation complexity, preprocessing requirements, and optimality – we can make informed decisions about which search algorithms to use in different contexts. This comprehensive understanding ensures that we select the most efficient and effective search techniques for our data processing needs.

Among the properties of search algorithms, time complexity stands out as the most critical factor. The efficiency of a search algorithm is predominantly determined by its time complexity, which dictates how quickly an algorithm can locate an element within a dataset. As a result, search algorithms are often classified according to their time complexities to facilitate a clear understanding of their performance under different conditions.

In the upcoming sections, we will explore search algorithms categorized by their time complexities. These categories provide a structured way to analyze and compare various search techniques:

- **Linear-time search algorithms:** These algorithms, such as the sequential search, operate in $O(n)$ time, meaning the time required to find an element grows linearly with the size of the dataset. We will explore scenarios where linear-time algorithms are suitable and their implementation details.
- **Sub-linear time search algorithms:** This category includes algorithms such as binary search, which operate in $O(\log n)$ time. Binary search is particularly efficient for sorted datasets and leverages the divide-and-conquer strategy to quickly narrow down the search space. In *Chapter 13*, we will discuss a search algorithm based on a specific data structure called **binary search trees (BSTs)**. BSTs maintain a sorted order of elements, allowing for efficient search, insertion, and deletion operations.
- **Constant-time search algorithms:** These algorithms aim to achieve $O(1)$ time complexity, where the time required to find an element remains constant regardless of the dataset size. Hashing is a primary technique used to achieve constant-time search operations. We will examine how hashing functions work, their implementation, and the conditions under which they provide optimal performance.

By understanding and categorizing search algorithms based on their time complexities, we can better appreciate their strengths and limitations. This structured approach allows us to select the most appropriate algorithm for a given application, ensuring efficient and effective data retrieval.

Linear-time and logarithmic search algorithms

In the study of search algorithms, understanding both linear and sub-linear search methods is crucial for selecting the most efficient approach for a given problem. Linear search, the most straightforward method, involves checking each element in a dataset sequentially until the target is found or the end of the dataset is reached. While simple and effective for small or unsorted datasets, its $O(n)$ time complexity makes it impractical for large datasets. In contrast, sub-linear search algorithms, such as binary search and jump search, offer more efficient solutions with time complexities better than

$O(n)$, typically exploiting the properties of sorted data to significantly reduce the number of comparisons needed. By comparing these two classes of algorithms, we can appreciate the advancements in search techniques and their applications in optimizing data retrieval processes.

Linear or sequential search

A general search algorithm, regardless of any assumptions about data (whether sorted or unsorted), has an asymptotic upper bound of $O(n)$. This is because, in the worst case, we may need to visit and evaluate every single element in the dataset to determine whether the target element is present. Therefore, the time complexity of a linear search algorithm is $O(n)$. Let's examine this in both recursive and iterative (non-recursive) implementations of linear search algorithms.

The iterative (non-recursive) implementation of linear search involves incrementing through each element of the array and checking whether it matches the target element. The following is a simple Python implementation of linear search:

```
def iterative_linear_search(a, target):
    for index in range(len(a)):
        if a[index] == target:
            return index
    return -1
```

Estimating the time complexity of iterative linear search is straightforward. The algorithm consists of a loop that executes all commands inside it n times, where n is the number of elements in the array. Additionally, the final instruction after the loop runs once. This results in an upper bound for the running time of $O(n)$.

The recursive implementation of linear search involves checking the current element and making a recursive call to check the next element if the target is not found. The following is a Python code to implement recursive sequential search:

```
def recursive_linear_search(a, target, index=0):
    if index >= len(a):
        return -1
    if a[index] == target:
        return index
    return recursive_linear_search(a, target, index + 1)
```

The `recursive_linear_search` function takes three parameters: `a` (the array to search), `target` (the element to search for), and `index` (the current index in the array, defaulting to 0). If `index` is greater than or equal to the length of the array, it means we have reached the end of the array and the target is not found. The function returns -1. If the current element at `a[index]` matches `target`, the function returns the current `index`. If the current element does not match the target,

the function calls itself recursively, moving to the next index (`index + 1`). The algorithm can be described using a subtractive recurrence function, specifically the following:

$$T(n) = T(n - 1) + O(1).$$

This recurrence relation indicates that with each recursion, the size of the problem is reduced by one (i.e., the amount of data to be evaluated against the search criteria decreases by one). This fits the following general form:

$$T(n) = aT(n - b) + \Theta(n^k).$$

Using the Master Theorem for subtractive recurrences, we can identify which case applies:

- **Case 1:** If $a < 1$, then $T(n) = \Theta(n^k)$
- **Case 2:** If $a = 1$, then $T(n) = \Theta(n^{k+1})$
- **Case 3:** If $a > 1$, then $T(n) = \Theta(a^{n/b} \cdot f(n))$

With the $a = 1$, $b = 1$, and $k = 0$ parameters in the subtractive recurrence function, *Case 1* applies. Thus, the time complexity $T(n)$ is $\Theta(n)$, confirming that the running time of the algorithm is linear.

Both implementations demonstrate $O(n)$ time complexity, meaning that in the worst case, every element in the array must be checked to find the target element. However, both implementations have a $O(1)$ or constant space complexity, which is a significant advantage. Linear search is simple and easy to understand, requiring no specific data format, preprocessing, or prior sorting.

Linear search is used in scenarios where simplicity and ease of implementation are priorities, and when the dataset is relatively small or unsorted. It is particularly useful in the following situations:

- **Unsorted or unstructured data:** When the data is not sorted or stored in a specific structure that facilitates faster search methods, linear search is a straightforward and viable option
- **Small datasets:** For small datasets, the overhead of more complex search algorithms may not be justified, making linear search an efficient choice
- **First occurrence search:** When you need to find the first occurrence of an element in an array or list, linear search is appropriate
- **Single or few searches:** If you only need to perform a single search or a few searches, the simplicity of linear search can outweigh the benefits of more complex algorithms that require preprocessing (such as sorting)

Linear search is used in various applications where the preceding conditions apply. Some common applications include the following:

- **Scanners and parsers:** Linear search is often used in lexical scanners and parsers to find tokens or specific patterns within a sequence of characters or data

- **Lookup operations in unsorted lists:** When working with unsorted lists or arrays, linear search is used to find specific elements or values
- **Validation and verification:** Linear search is used to validate input or verify the presence of an element in a list, such as checking whether a user-entered value exists in a database or list
- **Real-time systems:** In real-time systems where data is constantly changing and sorting is not feasible, linear search provides a quick way to find elements without requiring preprocessing
- **Embedded systems:** In embedded systems with limited resources, the constant space complexity of linear search makes it a suitable choice for searching operations

While linear search may not be the most efficient algorithm for large or sorted datasets, its simplicity, constant space complexity, and flexibility make it a valuable tool in various applications, particularly when working with unsorted or small datasets.

Sub-linear search

In algorithm design, sub-linear search algorithms represent a class of techniques that offer search solutions more efficiently than linear time. These algorithms are particularly powerful because they can locate an element without needing to examine every item in the dataset. By leveraging the properties of sorted data and advanced partitioning strategies, sub-linear search algorithms reduce the number of comparisons required, thereby accelerating the search process. This efficiency makes them invaluable in handling large datasets where linear search methods would be impractically slow.

Sub-linear search algorithms, such as binary search, jump search, and Fibonacci search, exploit different strategies to minimize the search space quickly. Binary search, for instance, repeatedly divides the array into two halves, while jump search segments the data into blocks and performs linear searches within these smaller sections. Fibonacci search utilizes the Fibonacci sequence to determine the range of the search space, optimizing for data that fits well into memory blocks. Each of these algorithms offers unique advantages and is suited to specific types of problems, highlighting the versatility and power of sub-linear search methods in computational efficiency. Let's deep dive into the most well-known sub-linear search algorithms.

Binary search

Binary search is an effective method for locating an element in a sorted array. The algorithm operates by continuously halving the search space. If the middle element matches the target value, the search is complete. If not, the search proceeds in the half of the array where the target value is likely to be found. Binary search is often implemented recursively. The following is a recursive implementation of binary search:

```
def recursive_binary_search(a, target, left, right):  
    if right >= left:  
        mid = left + (right - left) // 2
```

```

    if a[mid] == target:
        return mid
    elif a[mid] > target:
        return recursive_binary_search(a, target, left, mid - 1)
    else:
        return recursive_binary_search(a, target, mid + 1, right)
return -1

```

Binary search works by repeatedly dividing the search interval in half. The time complexity of binary search is $O(\log n)$. Let's analyze and prove this time complexity in detail.

Given a sorted array, `a`, and a target value, `target`, binary search follows these steps:

1. **Initialization:** Set two pointers, `left` and `right`, at the beginning and end of the array, respectively.
2. **Middle element:** Calculate the middle index, `mid = left + (right - left) // 2`.
3. **Comparison:**
 - If `a[mid] == target`, the target is found, and the `mid` index is returned
 - If `a[mid] < target`, update `left` to `mid + 1` and repeat the process
 - If `a[mid] > target`, update `right` to `mid - 1` and repeat the process
4. **Termination:** The process continues until `left > right`. If the target is not found, return `-1`.

The initial search space is the entire array of size n . At each step, the algorithm compares the target with the middle element. Based on the comparison, the search space is halved; either the left half or the right half is discarded:

- After the first step, the search space is $\frac{n}{2}$
- After the second step, the search space is $\frac{n}{4}$
- After the k^{th} step, the search space is $\frac{n}{2^k}$

The algorithm stops when the search space is reduced to 1 element, i.e., $\frac{n}{2^k} = 1$. Now we solve it for k : $\frac{n}{2^k} = 1 \implies n = 2^k \implies k = \log n$. Therefore, the binary search algorithm performs at most $\log n$ comparisons.

We can also prove the time complexity using a recurrence function. Let's define $T(n)$ as the time complexity of binary search on an array of size n . If the array size is 1 ($n = 1$), the time complexity is $O(1)$ because it involves a single comparison: $T(1) = O(1)$.

For an array of size n , we perform one comparison to check the middle element, then we recursively search in the left or right half, each of size $\frac{n}{2}$. Then, the recurrence function of recursive binary search is as follows:

$$T(n) = T\left(\frac{n}{2}\right) + O(1).$$

Using the Master Theorem for divide-and-conquer recurrence relations, *Case 2* applies. Therefore, the following applies:

$$T(n) = \Theta(\log n).$$

Unlike linear search, binary search is highly efficient, making it suitable for large datasets. The algorithm is straightforward to implement and understand. The iterative version uses constant space, $O(1)$, and even the recursive version has relatively low space overhead, $O(\log n)$. The primary use case for binary search is finding an element in a sorted array or list. Binary search is also used in dictionary operations such as finding words in a sorted list of entries.

On the other hand, binary search only works on sorted arrays. If the data is unsorted, it must first be sorted, which adds additional overhead. Binary search is best suited for static arrays where the data does not change frequently. Frequent insertions and deletions would require re-sorting the array, increasing the sorting overhead. Additionally, binary search is less efficient with linked lists or other non-contiguous memory structures because it relies on efficient random access provided by arrays.

The concept of dividing a dataset at the midpoint can be extended to partitioning the dataset into more than two segments. Ternary search is an example of this approach. It is a divide-and-conquer search algorithm that operates on a sorted array by dividing the array into three parts and determining in which part the target element lies, thereby reducing the search space to one-third. This process is repeated until the target element is found or the search space is exhausted. Consequently, the time complexity of ternary search is $O(\log_3 n)$.

Interpolation search

Interpolation search is an improvement over binary search for searching a target value within a sorted array. While binary search always probes the middle element, interpolation search makes an educated guess about where the target value might be located based on the values at the array's boundaries and the target value itself.

The following is a recursive implementation of interpolation search:

```
def recursive_interpolation_search(a, target, low, high):
    if low <= high and target >= a[low] and target <= a[high]:
        pos = low + ((high - low) // (a[high] - a[low]) * (target -
a[low]))
        if a[pos] == target:
            return pos
        if a[pos] < target:
```

```

        return recursive_interpolation_search(a, target, pos + 1,
high)
        return recursive_interpolation_search(a, target, low, pos - 1)
    return -1 # Target not found

```

When comparing the code for interpolation search and binary search, the main difference lies in how the midpoint (or position) is calculated in each algorithm. This difference reflects the approach each algorithm takes to locate the target element. In binary search, the midpoint is calculated as the average of the left and right indices:

```
mid = left + (right - left) // 2
```

Binary search assumes a uniform distribution of elements and divides the search interval in half, regardless of the actual values of the elements. In interpolation search, the midpoint (represented by `pos`) is estimated based on the value of the target element relative to the values at the current low and high indices:

```
pos = low + ((high - low) // (arr[high] - a[low]) * (target - a[low]))
```

Interpolation search attempts to improve the midpoint estimation by considering the distribution of the elements. This makes it more efficient for uniformly distributed datasets, as it can potentially reduce the number of comparisons by jumping closer to the target element.

Let's explore the intuition behind interpolation search. Imagine the array as a number line. Interpolation search estimates the position of the target value based on its relative position between the minimum and maximum values in the array. When the values are uniformly distributed, this estimation tends to be quite accurate, resulting in faster convergence compared to binary search.

We know that the time complexity of binary search is $O(\log n)$. Now the question is: what is the time complexity of interpolation search? First, we need to find the recurrence function describing the behavior of the interpolation search algorithm. Interpolation search operates by estimating the position of the target value within the array and then recursively or iteratively refining this estimate. The efficiency of this process depends significantly on the distribution of the values within the array. For uniformly distributed data, the estimated position is close to the actual position of the target, leading to fewer comparisons. The position estimate is given by the following:

$$pos = low + \left(\frac{(high - low)}{(a[high] - a[low])} \right) \times (target - a[low])$$

In each iteration, the size of the search interval is reduced proportionally based on the estimated position. On average, interpolation search reduces the search space more significantly than binary search. However, the estimated position heavily depends on the distribution of the data. In the worst-case scenario, when the distribution is highly skewed, the time complexity is $T(n) = O(n)$, making interpolation search no better than linear search.

In a more realistic average case, where the data distribution is closer to uniform, interpolation search performs better. To analyze this, we need to determine the recurrence function for this scenario. For uniformly distributed data, we assume that the problem size is reduced by the square root of the original problem size in each recursion. This suggests the following recurrence relation for interpolation search in the average-case scenario:

$$T(n) = T(\sqrt{n}) + O(1)$$

Since the recurrence function does not fit the standard format of the Master Theorem, we need to use alternative approaches, such as the substitution method, to solve it.

Let's perform a variable change: $n = 2^m$. We can then rewrite the recurrence function as follows:

$$T(2^m) = T(2^{\frac{m}{2}}) + O(1)$$

Next, make another variable change: $S(m) = T(2^m)$. The recurrence relation now becomes the following:

$$S(m) = S\left(\frac{m}{2}\right) + O(1)$$

This is a familiar recurrence function (similar to binary search), and we can conclude the following:

$$S(m) = O(\log m)$$

Replacing m back with $\log n$, we get this:

$$S(m) = T(2^m) = T(2^{\log n}) = T(n) = O(\log \log n)$$

In the worst case, if the distribution of elements is highly skewed or not uniform, the position estimates can be inaccurate, leading to a linear search behavior. This results in a worst-case time complexity of $O(n)$. On the other hand, the space complexity is $O(1)$ because interpolation search only needs to keep the `pos` variable in memory, which does not depend on the size of the data.

Interpolation search is highly efficient for uniformly distributed data but can degrade to linear search performance for non-uniform distributions. This analysis shows that while interpolation search has potential advantages, its efficiency is highly dependent on the data distribution.

Exponential search

Exponential search, also known as **galloping search** or **doubling search**, is an algorithm for finding a range in a sorted array where a target value might be located. It works by initially checking the first element and then repeatedly doubling the interval size until it finds a range that may contain the target. Once this range is found, binary search is used within the range to locate the target value.

The time complexity of exponential search is $O(\log n)$ in both the worst-case and average-case scenarios. This efficiency is due to the algorithm's approach of rapidly increasing the interval size and then using a binary search within the identified range.

Exponential search first finds a range where the target element might be located by repeatedly doubling the index. This step takes $O(\log n)$ time because the doubling process essentially performs a binary search on the indices. Once the range is found, a binary search within that range takes another $O(\log n)$ time. However, since the range-finding step already reduces the problem size significantly, the overall time complexity remains $O(\log n)$.

The space complexity of exponential search depends on the implementation method. In the iterative approach, the space complexity is $O(1)$. The following is a simple iterative implementation of exponential search:

```
def iterative_exponential_search(a, target):
    if a[0] == target:
        return 0
    n = len(a)
    i = 1
    while i < n and a[i] <= target:
        i = i * 2
    return binary_search(a, i // 2, min(i, n - 1), target)
```

Let's briefly explain the code. The key part of the code is the line `while i < n and a[i] <= target:`, where the exponential growth occurs: `i = i * 2`. In this step, the index doubles (1, 2, 4, 8, and so on) until a value larger than or equal to the target is found. Once this range is identified, `binary_search` is performed within that range to locate the target element.

The iterative implementation uses a constant amount of extra space regardless of the size of the input array. The only additional memory required is for a few variables to keep track of the indices and the target element.

Unlike the iterative approach, in the recursive method, the space complexity is $O(\log n)$. Let's have a look at the recursive implementation:

```
def recursive_exponential_search(a, target, i=1):
    n = len(a)
    if a[0] == target:
        return 0
    if i < n and a[i] <= target:
        return recursive_exponential_search(a, target, i * 2)
    return binary_search(a, i // 2, min(i, n - 1), target)
```

The recursive implementation uses additional space on the call stack for each recursive call. Since the depth of the recursion is proportional to $\log n$, the space complexity is $O(\log n)$.

Exponential search is an effective algorithm for quickly narrowing down the search range in large, sorted arrays. By combining the strengths of exponential and binary search, it offers both efficiency

and flexibility. Its main advantage lies in its ability to handle large datasets efficiently, although it requires the data to be sorted and introduces some complexity with its dual-phase search approach.

Jump search

Jump search is an algorithm for finding an element in a sorted array. It works by dividing the array into blocks of a fixed size, jumping ahead by that block size, and then performing a linear search within the block where the target element might be located. The optimal step size for jumping is typically $\sqrt{n}$, where n is the number of elements in the array. This method aims to reduce the number of comparisons by skipping over large sections of the array initially. We will prove that the time complexity of jump search is $O(\sqrt{n})$. The iterative implementation of jump search is as follows:

```
import math
def jump_search(a, target):
    n = len(a)
    step = int(math.sqrt(n))
    prev = 0
    while a[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))
        if prev >= n:
            return -1
    while a[prev] < target:
        prev += 1
        if prev == min(step, n):
            return -1
    if a[prev] == target:
        return prev
    return -1
```

Recursive implementation of jump search is less common due to the nature of the algorithm. However, it can be implemented as follows:

```
import math
def recursive_jump_search(a, target, prev=0, step=None):
    n = len(a)
    if step is None:
        step = int(math.sqrt(n)) # Block size to jump
    if prev >= n:
        return -1
    if a[min(step, n) - 1] < target:
        return recursive_jump_search(a, target, step, step + int(math.
sqrt(n)))
    while prev < min(step, n) and a[prev] < target:
        prev += 1
```

```
if prev < n and a[prev] == target:
    return prev
return -1
```

Let's analyze the algorithm and then estimate the time complexity of jump search. The algorithm is implemented in three steps:

1. **Initialization:**
 - Set the block size to $\sqrt{n}$.
 - Initialize `prev` to 0 and `step` to $\sqrt{n}$.
2. **Jumping phase:** Jump ahead by the block size until the value at the current position is greater than or equal to the target or the end of the array is reached.
3. **Linear search phase:** Perform a linear search within the identified block.

Now, let's analyze the time complexity. We know that the size of each block is $\sqrt{n}$. In the worst case, we may need to jump through the entire array to find the block containing the target element. The number of jumps required to reach the target block is $(\lceil \frac{n}{\sqrt{n}} \rceil = \lceil \sqrt{n} \rceil)$. Each jump involves a comparison, and thus the number of comparisons in the jumping phase is $\sqrt{n}$.

At this stage, we are ready to start the linear search phase. After identifying the potential block, a linear search is performed within this block. The maximum number of elements to search within a block is $\sqrt{n}$. Thus, the number of comparisons in the linear search phase is, at most, $\sqrt{n}$.

We had two following phases:

- Jumping phase comparisons: $O(\sqrt{n})$
- Linear search phase comparisons: $O(\sqrt{n})$

We add two phases together; the total number of comparisons is as follows:

$$O(\sqrt{n}) + O(\sqrt{n}) = O(\sqrt{n})$$

The total time complexity of the jump search algorithm is $O(\sqrt{n})$. This is because the algorithm performs up to $\sqrt{n}$ comparisons during the jumping phase and up to $\sqrt{n}$ comparisons during the linear search phase within the block. The combination of these phases results in an overall time complexity of $O(\sqrt{n})$. This analysis shows that jump search is less efficient than binary search, $O(\log n)$, but can still be useful in scenarios where the simplicity of implementation and constant space complexity $O(1)$ are advantageous.

Jump search is more efficient than linear search for large arrays because it reduces the number of comparisons by skipping over blocks of elements. The jump search algorithm is relatively simple to implement and understand and, unlike binary search, jump search does not require any preprocessing of the data other than ensuring it is sorted.

However, jump search comes with some limitations. Like binary search, jump search only works on sorted arrays. Also, the efficiency of jump search depends on choosing an optimal block size, which is generally $\sqrt{n}$, but this might not always be the most efficient in practice. For very small arrays, the overhead of calculating the block size and performing the jumps might make it less efficient than simpler algorithms such as linear search.

Jump search is useful when searching in large sorted arrays where binary search might be less intuitive or when the data is accessed sequentially. In databases, jump search can be used for indexing and querying sorted data efficiently. The other use case for jump search is in memory-constrained environments. Given its $O(1)$ space complexity, jump search is suitable for memory-constrained environments where additional space for data structures such as binary search trees or hash tables is not available.

In summary, jump search is an effective algorithm for searching in sorted arrays, striking a balance between the simplicity of linear search and the efficiency of binary search. Its primary advantage lies in its $O(\sqrt{n})$ time complexity, making it suitable for certain applications where sorted data needs to be searched quickly.

Recap

All sub-linear search algorithms rely on sorted data, which means they must first sort the data if it is not already sorted. This poses a significant limitation when the data is dynamically changing. We will discuss this in detail in the next chapter.

Most sub-linear search algorithms are improvements or extensions of binary search, except for jump search, which is based on segmenting the data and then searching within each segment. The main idea behind binary search and its variants is to find the most optimal estimation of the mid position in the array. This differentiates binary search from interpolation and exponential search. On the other hand, the goal of every search algorithm is to reduce the search space or scope we are looking in for the target and to minimize the risk of missing it. In all variants of binary search, this is done by one or more midpoints, while in jump search, this is achieved by evenly splitting the search space.

If we split the search space (dataset) into many partitions, for example, $\sqrt{n}$, we are referring to jump search. While it may seem similar to binary search, which has a single midpoint, jump search divides the dataset into multiple partitions and uses multiple midpoints (*number of partitions = number of midpoints + 1*). This distinction leads to a different time complexity pattern for jump search, which is $O(\sqrt{n})$, compared to the $O(\log n)$ complexity of binary search and its variations.

An interesting approach is to use the Fibonacci sequence to partition the search space. This leads us to the Fibonacci search. Fibonacci search is an efficient search algorithm that works on sorted arrays. It leverages the properties of Fibonacci numbers to divide the array into smaller sections, making it similar to both binary and jump searches. The main advantage of Fibonacci search is its efficiency when dealing with arrays that fit well into memory blocks.

Hashing

In the previous two sections, we explored two groups of search algorithms: those with linear time complexity and those with more efficient, sub-linear time complexity. Linear search algorithms, such as the simple sequential search, operate with $O(n)$ time complexity, making them straightforward but less efficient for large datasets. On the other hand, sub-linear search algorithms, such as binary search and jump search, offer significantly better time complexities, often $O(\log n)$ or $O(\sqrt{n})$, by leveraging the properties of sorted data to minimize the number of comparisons needed.

However, achieving this improved time complexity comes with a cost: the time required to sort the data. Sorting is a prerequisite for the efficiency of sub-linear search algorithms. Without sorted data, the theoretical benefits of sub-linear time complexity cannot be realized. The process of sorting itself can be time-consuming, typically $O(n \log n)$ for efficient algorithms such as quick sort or merge sort. Therefore, while sub-linear search algorithms provide faster search times, they are only practical when the data can be sorted efficiently or when the data remains relatively static, allowing the sorting step to be amortized over multiple searches. This trade-off between sorting time and search efficiency is a critical consideration in the design and application of search algorithms.

The question now arises: can we achieve even better performance than sub-linear search algorithms? Specifically, is it possible to perform a search in constant time, or $O(1)$? To answer this question, we need to review the objective of the search operation. The objective of any search algorithm is to efficiently find the index or address of a key within a data structure. One approach to achieving constant time search is through a technique known as hashing.

In hashing, we use a hash function that takes the target key as input and computes a value that directly corresponds to the location of the key in the data structure. This value, known as the hash value (or hash code), is then used to index into a hash table, allowing for constant time access to the data.

Before diving into the subject of hashing, let's define some basic terms:

- **Key:** In the context of search algorithms and data structures, a key is a unique identifier used to search, access, or manage elements within a collection, such as an array, list, or database. Keys are essential for efficient data retrieval and manipulation. For example, in a dictionary, the key might be a word, and the associated value could be the definition of that word. Keys are used in various data structures, such as hash tables, where they are input into a hash function to generate an index.
- **Index:** An index is a numerical representation of a position within a data structure, such as an array or list. It indicates where a particular element is stored within that structure. For example, in an array of $[10, 20, 30, 40]$, the index of the 30 element is 2. Indexes are crucial for accessing elements directly in data structures that support random access, such as arrays and lists.

- **Address:** An address refers to a specific location in memory where a data element is stored. In the context of search and data structures, the address is often the actual memory location corresponding to a particular index or key. In low-level programming, such as C or C++, an address might be something such as `0x7fee44b8b60`, indicating the exact memory location of a variable. Addresses are used to directly access and manipulate data stored in memory. In higher-level programming, addresses are often abstracted away, but understanding them is crucial for optimizing performance and understanding memory management.

In hash tables, a key is passed through a hash function to generate an index. This index is then used to locate the corresponding data in the hash table. On the other hand, in arrays, an index directly corresponds to an address in memory where the data element is stored.

Hash functions

A **hash function** is a mathematical function that transforms an input key into a numerical value, called a **hash value**. The hash value is then mapped to an index in a hash table. A good hash function distributes keys uniformly across the hash table to minimize collisions, where two or more keys hash to the same index. An effective hash function has several key properties:

- **Deterministic:** A hash function must consistently produce the same output (hash value) for the same input. This ensures predictability and reliability in applications such as data retrieval and verification.
- **Fixed output size:** Regardless of the size of the input data, the output hash value should have a fixed length. This makes hash values easy to store and compare, enhancing efficiency in various algorithms.
- **Efficiency:** Hash functions should be computationally fast, allowing for quick generation of hash values even for large inputs. This is essential for real-time applications and algorithms that rely on hashing for performance.
- **Uniformity:** A good hash function distributes its output values uniformly across the output space. Even a small change in the input should result in a significantly different hash value, preventing patterns and making it difficult to reverse-engineer the input.
- **Collision resistance:** It should be computationally infeasible to find two different inputs that produce the same hash value (a collision). Collision resistance is critical for security applications such as password storage and digital signatures.

There are additional properties that are particularly important for cryptographic hash functions (beyond the scope of this book):

- **Pre-image resistance:** Given a hash value, it should be difficult to find the original input that produced it. This property protects against attacks that attempt to recover the original data from its hash.
- **Second pre-image resistance:** Given one input and its hash value, it should be difficult to find a second input that produces the same hash.
- **Non-correlation:** There should be no correlation between different parts of the input and the resulting hash value.

By applying a hash function to the input data, we construct a hash table. A hash table is a data structure that stores key-value pairs. Each key is processed by the hash function to generate a hash value, which determines the index where the corresponding value is stored. The primary advantage of a hash table is that it enables search, insert, and delete operations to be performed in average-case $O(1)$ time complexity.

The efficiency of a hash table depends on how well the hash function distributes keys across the table. Ideally, a good hash function will minimize the number of empty cells (making the table less sparse) and reduce the number of collisions (where multiple keys hash to the same index). The art of designing a good hash function lies in achieving a balance between these factors to ensure optimal performance.

Meanwhile, it is crucial to note that when selecting a hash function for a specific algorithm, it is important to consider the application. Different applications may require different properties. For example, cryptographic applications demand strong collision resistance, while data structures might prioritize speed.

Constant time search using hashing

The primary goal of hashing is to achieve constant time search by directly mapping keys to their locations in a hash table using a hash function. The most straightforward hash function is $h(key) = key$, where h is the hash function. This method is known as direct addressing, and the resulting hash table is called a direct-address table.

While direct addressing is simple and easy to understand, it comes with several significant limitations:

- **Sparse hash table:** Direct addressing often creates a very sparse hash table, meaning that the table size must be as large as the range of possible input keys. For example, if the input keys range from 1 to 1,000,000, the hash table must have 1,000,000 slots, even if only a few keys are actually used. This results in inefficient use of memory.
- **High likelihood of collisions:** In direct addressing, if two different keys map to the same location (collision), it can cause issues with data retrieval and insertion. Although direct addressing assumes that each key is unique, in practice, collisions are often unavoidable.

- **Limited to numerical data:** Direct addressing is only effective for numerical, integer data. It does not work well with other data types, such as strings or composite objects, limiting its applicability in many real-world scenarios.

Implementing constant time search using hashing involves two key steps. First, it is essential to design an efficient hash function that satisfies as many of the desirable properties as possible. Despite efforts to create a good hash function, collisions are inevitable. Therefore, the second step is to handle collisions effectively during data structure operations, including search and retrieval.

Types of hash function used in search

Hash functions are crucial in determining the efficiency of hash-based search algorithms. Let's explore some commonly employed hash functions, along with explanations and examples.

Division-remainder (modulo) method

The **division-remainder (modulo) method** is a straightforward and commonly used technique for generating hash values. In this method, the hash value is obtained by taking the remainder of the division of the key by the size of the hash table. The formula used is as follows:

$$h(key) = key \bmod m$$

Here, h is the hash function, key is the input data, and m is the size of the hash table. Let's consider an example to illustrate how this method works.

Example 7.1

Using the division-remainder hash method, we determine the hash value for the example key, 987654321, with a hash table size of 100.

Let's determine the hash value step by step:

1. **Apply the modulo operation:** Calculate the hash value by taking the key modulo the size of the hash table:

$$h(987654321) = 987654321 \bmod 100$$

2. **Calculate the remainder:** Perform the division and find the remainder:

$$987654321 \bmod 100 = 21$$

Therefore, the hash value for the key 987654321 with a hash table size of 100 is 21.

The modulo operation is straightforward and easy to implement, making this method simple and efficient. It is also computationally efficient, as the modulo operation is relatively fast. Moreover, the same input key will always produce the same hash value, ensuring consistency in the hash table.

However, the modulo method has limitations. If the size of the hash table is poorly chosen (e.g., a power of 2), the hash values may not be uniformly distributed, leading to clustering. To mitigate this issue, prime numbers are typically recommended for the table size. Additionally, if the input keys have patterns or common factors, this method may lead to collisions and clustering, reducing the efficiency of the hash table. Like all hash functions, the division-remainder method can result in collisions. Effective collision handling strategies, such as chaining or open addressing, are necessary to maintain performance.

The division-remainder (modulo) method is a simple and efficient hash function commonly used in various applications. It generates hash values by taking the remainder of the division of the key by the hash table size. While it offers simplicity and computational efficiency, it is important to choose an appropriate table size (preferably a prime number) to ensure a uniform distribution of hash values. Additionally, collision-handling mechanisms are essential to address the inherent limitations of this method.

Multiplication method

The **multiplication hash method** is a technique used to generate hash values by multiplying the key by a constant fraction and then extracting an appropriate portion of the result to produce the hash code. This method aims to distribute keys more uniformly across the hash table. Let's explain this method step by step:

1. Multiply the key by a constant, A , where $0 < A < 1$.
2. Extract the fractional part of the product.
3. Multiply the fractional part by the hash table size m .
4. Take the floor of the result to get the hash value.

The formula for the hash function is as follows:

$$h(key) = \lfloor m (key \cdot A \bmod 1) \rfloor$$

Here, $A = 0.618033$ (an approximation of the golden ratio). Let's see this method in an example.

Example 7.2

Using the multiplication hash method, we determine the hash value for the example key, *123456*, and a hash table size of *100*.

Let's determine the hash value step by step:

1. Multiply the key by A : $123456 \times 0.618033 = 76293.192648$
2. Extract the fractional part: 0.192648
3. Multiply by table size: $0.192648 \times 100 = 19.2648$
4. Take the floor: $\lfloor 19.2648 \rfloor = 19$

Therefore, the hash value for the key *123456* with a hash table size of *100* is *19*.

The multiplication method tends to distribute keys more uniformly across the hash table, thereby reducing clustering. Additionally, unlike the division method, the effectiveness of the multiplication method is less dependent on the table size being a prime number. Moreover, the multiplication and modulo operations are computationally efficient.

However, the multiplication method has some limitations. Its effectiveness heavily depends on the choice of the constant *A*. While the golden ratio is commonly used, other values may need to be tested for optimal performance. For very large keys or highly precise values of *A*, precision issues in floating-point arithmetic could affect the hash values. Lastly, the need to handle floating-point arithmetic can make the implementation slightly more complex compared to the division method.

Mid-square method

The mid-square hash function is a technique used to generate hash values by squaring the key and then extracting an appropriate number of middle digits or bits from the result. This method aims to distribute keys more uniformly across the hash table, leveraging the properties of squaring to spread out the values. Let's consider an example to illustrate the mid-square hash function.

Example 7.3

Using the mid-square hash method, we determine the hash value for the example key *456* with a hash table size of *100*:

1. **Square the key:** Square the key to obtain a large number: $456^2 = 207936$.
2. **Extract the middle digits:** Extract an appropriate number of middle digits from the squared value. The number of digits to extract can depend on the size of the hash table. For simplicity, let's extract two middle digits:
 - Squared value: *207936*
 - Middle digits: *07* (from the middle of the squared number)
3. **Use the middle digits as the hash value:** Use these middle digits to determine the index in the hash table. Therefore, the key *456* is mapped to index *07* in the hash table.

Key properties of the mid-square hash function are as follows:

- **Uniform distribution:** By squaring the key and extracting the middle digits, this method tends to produce a more uniform distribution of keys, as squaring helps to spread out the values.
- **Simplicity:** The mid-square method is straightforward to implement. It involves squaring the key and then extracting the middle portion of the result.
- **Independence from key size:** This method is relatively independent of the size of the key, making it suitable for various key lengths.

On the other hand, the mid-square method has some limitations:

- **Dependent on middle digits:** The efficiency of this method relies on the middle digits of the squared value. If the middle digits are not well distributed, it can lead to clustering.
- **Selection of digits:** Deciding the number of middle digits to extract can be challenging and may require experimentation to optimize for specific applications.
- **Limited key range:** For very small keys, the squared value might not provide enough digits to extract, reducing the effectiveness of the method.
- **Computational cost:** Squaring the key can be computationally expensive for very large keys, especially in environments where processing power is limited.

The mid-square hash function is an effective method for generating hash codes by squaring the key and extracting the middle digits. This technique leverages the properties of squaring to spread out the values and achieve a more uniform distribution of keys across the hash table. Despite its simplicity and independence from key size, the method's efficiency depends on the distribution of the middle digits and may involve some computational cost for large keys. Overall, the mid-square hash function remains a useful tool in the design of hash functions for various applications.

Folding method

The folding hash function is a technique used to generate hash codes by splitting the key into several parts, adding these parts together, and then taking the modulus with the table size to produce the hash code. This method is particularly useful for large keys, such as phone numbers or identification numbers, and aims to distribute keys more uniformly across the hash table. Let's consider a practical example to illustrate the folding hash function.

Example 7.4

Using the folding hash method, we determine the hash value for the example key 987654321 with a hash table size of 100:

1. **Split the key:** Divide the key into equal parts. For simplicity, let's split it into parts of 3 digits each: 987, 654, 321
2. **Add the parts:** Sum the parts together: $987 + 654 + 321 = 1962$
3. **Take the modulus:** Take the modulus of the sum with the table size to get the hash code: $1962 \bmod 100 = 62$

Therefore, the key 987654321 is mapped to index 62 in the hash table.

The folding hash method has the following properties:

- **Uniform distribution:** The folding method aims to produce a more uniform distribution of keys by ensuring that all parts of the key contribute to the hash value. This helps to reduce clustering and improve the overall performance of the hash table.
- **Simplicity:** The algorithm is straightforward to implement and understand. It simply involves splitting, summing, and taking the modulus.
- **Flexibility:** It can handle keys of various sizes by adjusting the way the key is split into parts.

On the other side, folding hash comes with its limitations:

- **Dependent on key structure:** The efficiency of the folding hash function depends on the structure of the keys. If the parts of the key have similar patterns or values, it may not distribute the keys uniformly.
- **Not ideal for small keys:** For small keys, the overhead of splitting and summing may not provide significant benefits over simpler hash functions such as division-remainder.
- **Handling different lengths:** If keys have varying lengths, it might be challenging to decide how to split them uniformly, potentially leading to uneven distribution.
- **Sum overflow:** For very large keys, the sum of the parts might exceed the typical integer range, causing overflow issues. However, this can be mitigated by using modular arithmetic at each step.

The folding hash function is an effective and straightforward method for hashing large keys. By splitting the key into parts, summing these parts, and then taking the modulus, it can produce a relatively uniform distribution of hash values, improving the performance of hash tables. However, its efficiency can be affected by the structure of the keys, the need to handle different key lengths, and potential overflow issues. Despite these limitations, the folding method remains a valuable tool in the design of hash functions for various applications.

Universal hashing

The universal hash method is a technique designed to minimize the probability of collisions in a hash table. It involves using a family of hash functions and randomly selecting one of these functions for hashing. This method provides a probabilistic guarantee that the number of collisions will be low, making it particularly useful in applications where security and performance are critical. The following is the step-by-step implementation of the universal method:

- Define a family of hash functions H from which a specific function $h \in H$ is chosen. Each hash function h in the family should be able to map keys uniformly across the hash table.
- Randomly select a hash function h from the family H to use for hashing the keys.
- Use the selected hash function h to compute the hash values of the keys.

Example 7.5

Using the folding hash method, let's determine the hash value for the example key *123456* with a hash table size of *100*.

Let's calculate $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$.

Here, p is a prime number larger than any possible key, a and b are randomly chosen integers such that $1 \leq a < p$ and $0 \leq b < p$, and m is the size of the hash table.

Now, we use the following parameters in this example:

- Key: *123456*
- Table size m : *100*
- Prime number p : *101*
- Randomly chosen a and b : Let's assume $a = 34$ and $b = 7$
- We compute the hash value:

$$h_{34,7}(123456) = ((34 \times 123456 + 7) \bmod 101) \bmod 100$$

- First, we calculate the intermediate values:

$$34 \times 123456 + 7 = 4197503$$

$$4197503 \bmod 101 = 90$$

- Then, we calculate the final hash value: $90 \bmod 100 = 90$

Therefore, the hash value for the key *123456* with the selected parameters is *90*.

Universal hashing significantly reduces the probability of collisions, offering a strong probabilistic guarantee that different keys will hash to different values. By randomly selecting a hash function from a family, it also provides resistance against adversarial attacks, making it useful in cryptographic applications. Furthermore, this method can be adapted to various key distributions by choosing an appropriate family of hash functions.

However, universal hashing has some limitations. Defining and implementing a family of hash functions is more complex compared to simpler methods such as division-remainder or multiplication. Additionally, the need to randomly select a hash function and compute potentially more complex hash values can introduce additional computational overhead. Lastly, the effectiveness of universal hashing relies on the true randomness of the chosen hash function, which may be difficult to achieve in some environments.

The next two methods are tailored for hashing string data, where characters in the string contribute to the hash code.

Polynomial hashing for strings

Polynomial hashing (also known as **Rabin-Karp rolling hash**) is a technique to efficiently calculate hash values for substrings within a larger string. It treats the characters of the string as coefficients of a polynomial, where each character's ASCII (or Unicode) value is multiplied by a prime number raised to a power corresponding to its position in the string. In the polynomial hashing method, each character in the string is treated as a coefficient in a polynomial. The hash code is the polynomial evaluated at a certain value. For the string *abcd* and a chosen base of 31, the hash code can be calculated as the following:

$$h(abcd) = (a \times 31^3 + b \times 31^2 + c \times 31 + d) \bmod m$$

Polynomial hashing is implemented in the following steps:

1. **Initialization:** Choose a prime number p (commonly 11 or 31) and a modulus m (often a large prime number to minimize collisions that corresponds to the size of the hash table).
2. **Hash calculation:**
 - I. Initialize the hash value to 0.
 - II. Iterate over each character in the string.
 - III. For each character, do the following:
 - Multiply the current hash value by p .
 - Add the ASCII value of the character.
 - Apply the modulo m operation to the result to ensure the hash value remains within a manageable range.
3. **Rolling hash:** To compute the hash of a substring, we can subtract the hash of the characters that are no longer part of the substring and add the hash of the newly included characters. This “rolling” update is efficient and allows for fast comparisons of different substrings.

The following is a Python example of polynomial hashing:

```
def polynomial_hash(string, p=11, m=2**31):
    hash_value = 0
    for char in string:
        hash_value = (hash_value * p + ord(char)) % m
    return hash_value
# Example usage
string = "Hello"
hash_value = polynomial_hash(string)
print(f"The polynomial hash value of '{string}' is: {hash_value}")
```

Let's explain the example Python code:

- The `polynomial_hash` function takes a string as input along with optional parameters, `p` (prime number) and `m` (modulus)
- It initializes `hash_value` to 0
- It iterates over each character (`char`) in the string
- For each character, it performs the hash update calculation:
 - `hash_value * p` effectively shifts the previous characters one position to the left in the polynomial
 - `ord(char)` gets the ASCII value of the character
 - The result is taken modulo `m` to prevent overflow and ensure a consistent range for the hash values
 - Finally, it returns the computed hash value
 - The polynomial hash value of "Hello" is 99162322

Polynomial hashing is particularly efficient for comparing substrings within a larger string. It avoids recalculating the entire hash for each substring, making it suitable for applications such as plagiarism detection or pattern matching. Collisions are still possible, especially if the modulus m is not chosen carefully. Using a large prime number for m can help reduce collision frequency. The choice of prime number p and modulus m can affect performance and collision probability. Experimentation might be needed to find optimal values for specific applications.

DJB2 hash function for strings

The **DJB2 hash function** is a simple and effective algorithm for generating hash values (numeric representations) from strings. It is known for its speed and decent distribution of hash values, making it suitable for various applications such as hash tables. The following are the four steps of this hash function:

1. **Initialization:** The hash value is initialized to a starting value of 5381. This initial value was chosen somewhat arbitrarily but has proven to work well in practice.
2. **Iteration:** The function iterates over each character in the input string.
3. **Hash update:** For each character, the current hash value is multiplied by 33 (left-shifted by 5 bits and then added to itself). The ASCII value of the character is added to the hash value.
4. **Finalization:** After processing all characters, the hash value is typically masked to ensure it fits within a 32-bit unsigned integer range.

The following is a Python example of this hash function:

```
def djb2(string):  
    hash = 5381  
    for char in string:  
        hash = ((hash << 5) + hash) + ord(char)  
    return hash & 0xFFFFFFFF
```

The `djb2` function takes a string as input. It initializes the hash variable to 5381. It iterates over each character (`char`) in the string. For each character, it performs the hash update calculation: `(hash << 5) + hash` is equivalent to multiplying hash by 33.

`ord(char)` gets the ASCII value of the character. Finally, it returns the hash value after masking it to a 32-bit unsigned integer. For example, if `string = "Hello"`, the DJB2 hash function generates 99162322 as output.

DJB2 is relatively easy to understand and implement. It's computationally fast, which is advantageous in performance-critical scenarios. While DJB2 generally performs well, it's not perfect and collisions can occur, especially with very large datasets.

Collision handling

One of the primary challenges in using hash functions is dealing with collisions. Collisions occur when two different keys produce the same hash value, leading to potential conflicts in data retrieval and storage. Efficiently handling collisions is crucial for maintaining the performance and reliability of hash tables. Various strategies, such as chaining and open addressing, are employed to address this problem, but each comes with its own set of trade-offs and complexities. Understanding and mitigating the collision problem is essential for designing robust and efficient hash-based data structures. Here are the main types of collision handling techniques, each explained with examples, limitations, and properties.

Chaining

In chaining, each position in the hash table points to a linked list (or chain) of elements that hash to the same index. When a collision occurs, the new element is simply added to the end of the list.

Example 7.6

Here, we are handling collision using chaining for a hash table of size 10 and keys 12, 22, and 32 (all produce a hash value of 2 with $h(k) = k \bmod 10$).

Index	Hash table
0	
1	
2	12 → 22 → 32
3	
4	
5	
6	
7	
8	
9	

Table 7.1 – An example of handling collision using chaining

Chaining is straightforward to implement and supports dynamic sizing, allowing the lists to grow as more elements are added. However, this approach requires additional memory for the linked lists. Moreover, if many collisions occur, chains can become long, leading to $O(n)$ time complexity in the worst case. In the average case, the running time for insert, delete, and search operations is $O(1)$ if the chains are short. For a more precise analysis, it is essential to introduce the load factor.

The load factor, often denoted as λ , is a crucial concept in the context of hash tables, including those using chaining for collision handling. It measures how full the hash table is and is defined as the ratio of the number of elements n in the hash table to the number of slots m in the table:

$$\lambda = \frac{n}{m}$$

The load factor helps in understanding the performance of the hash table. A low load factor indicates that the table has many empty slots, leading to shorter chains and faster average search times. A high load factor, on the other hand, indicates longer chains and potentially slower search times.

Revisiting the running time of basic operations (insert, delete, and search) with chaining for collision handling, the average running time is $O(1 + \lambda)$. This includes the constant time to access a slot in the hash table and λ to scan the chain, which is, on average, of size λ , assuming that data is uniformly distributed across slots to create evenly sized chains.

In many implementations, the load factor is used as a threshold to decide when to resize the hash table. When the load factor exceeds a certain value, the hash table is resized (typically doubled in size) to maintain efficient operations.

Example 7.7

Consider a hash table with 10 slots (size $m = 10$) and the following keys: 12, 22, 32, 42, and 52. Let's use a simple hash function:

$$h(\text{key}) = \text{key} \bmod 10:$$

The hash values are 2, 2, 2, 2, and 2. All keys are stored in the chain at index 2:

Index	Hash table
0	
1	
2	12 → 22 → 32 → 42 → 52
3	
4	
5	
6	
7	
8	
9	

Table 7.2 – An example of handling collision using simple chaining

- Number of elements: $n = 5$
- Number of slots: $m = 10$
- Load factor: $\lambda = \frac{5}{10} = 0.5$

The properties of the load factor are as follows:

- A lower load factor generally means higher efficiency, as chains are shorter and search times are quicker
- A higher load factor indicates better memory utilization, as more slots are being used, but it can also lead to decreased performance due to longer chains
- Maintaining an optimal load factor (typically below 0.75) often involves dynamic resizing of the hash table to balance between performance and memory utilization

However, the load factor has limitations:

- As the load factor increases, the chains at each index grow longer, leading to increased search, insertion, and deletion times
- A higher load factor increases the likelihood of collisions, which can degrade the performance of the hash table
- Frequent resizing to maintain an optimal load factor can introduce overhead and impact performance during the resize operation

Many hash table implementations set a threshold load factor (e.g., 0.75). When the load factor exceeds this threshold, the table is resized. Upon resizing, a common strategy is to double the size of the hash table and rehash all existing elements to the new table.

Open addressing

Open addressing handles collisions directly within the hash table by using a systematic sequence of probes to find an empty slot for each key. When a collision occurs, the algorithm probes the table according to a specific strategy until it finds an empty slot. Here, we introduce three common types of open addressing – linear probing, quadratic probing, and double hashing:

- **Linear probing:** When a collision occurs, linear probing checks the next slot in the table, continuing this process until an empty slot is found.

Example 7.8

To handle collisions using linear probing for a hash table of size 10 and keys 12, 13, 22, and 32, we use the hash function $h(k) = k \bmod 10$. The hash values for these keys are 2, 3, 2, and 2, respectively.

1. Calculate hash values:
 - For key 12: $12 \bmod 10 = 2$
 - For key 13: $13 \bmod 10 = 3$

- For key 22: $22 \bmod 10 = 2$
 - For key 32: $32 \bmod 10 = 2$
2. Handle collisions using linear probing:
- Key 12 hashes to index 2, so it is placed at index 2.
 - Key 13 hashes to index 3, so it is placed at index 3.
 - Key 22 hashes to index 2, which is already occupied by 12. Using linear probing, we check the next slot (index 3), which is occupied by 13. We then check the next slot (index 4), which is empty, so 22 is placed at index 4.
 - Key 32 hashes to index 2, which is occupied by 12. Using linear probing, we check the next slots (index 3, occupied by 13, and index 4, occupied by 22). The next empty slot is at index 5, so 32 is placed at index 5.
3. Here is the resulting hash table:

Index	Hash table
0	
1	
2	12
3	13
4	22
5	32
6	
7	
8	
9	

Table 7.3 – An example of handling collision using linear probing

The example demonstrates how linear probing resolves collisions by sequentially checking the next available slot in the hash table.

Linear probing is straightforward and easy to implement. It accesses contiguous memory locations, making it cache-friendly. However, it can lead to primary clustering, where groups of filled slots form, resulting in longer probe sequences. In the worst-case scenario, its running time degrades to $O(n)$ as the table fills up.

- **Quadratic probing:** This is similar to linear probing but uses a quadratic function to determine the next slot. The probe sequence is $h(k) = (h(k) + i^2) \bmod m$, where i is the probe number.

Example 7.9

To handle collisions using quadratic probing for a hash table of size 10 and keys 12, 13, 22, and 32, we use the hash function $h(k) = k \bmod 10$. The hash values for these keys are 2, 3, 2, and 2, respectively:

1. Calculate hash values:
 - For key 12: $12 \bmod 10 = 2$
 - For key 13: $13 \bmod 10 = 3$
 - For key 22: $22 \bmod 10 = 2$
 - For key 32: $32 \bmod 10 = 2$
2. Handle collisions using quadratic probing:
 - Key 12 hashes to index 2, so it is placed at index 2.
 - Key 13 hashes to index 3, so it is placed at index 3.
 - Key 22 hashes to index 2, which is already occupied by 12. Using quadratic probing, we check the next slots as follows:
$$h(22,1) = (2 + 1^2) \bmod 10 = 3, \text{ which is occupied by 13}$$
$$h(22,2) = (2 + 2^2) \bmod 10 = 6, \text{ which is empty, so 22 is placed at index 6}$$
 - Key 32 hashes to index 2, which is occupied by 12. Using quadratic probing, we check the next slots as follows:
$$h(32,1) = (2 + 1^2) \bmod 10 = 3, \text{ which is occupied by 13}$$
$$h(32,2) = (2 + 2^2) \bmod 10 = 6, \text{ which is occupied by 22}$$
$$h(32,3) = (2 + 3^2) \bmod 10 = 1, \text{ which is empty, so 32 is placed at index 1}$$

3. Here is the resulting hash table:

Index	Hash table
0	
1	32
2	12
3	13
4	
5	
6	22
7	
8	
9	

Table 7.4 – An example of handling collision using quadratic probing

This example demonstrates how quadratic probing resolves collisions by checking progressively further slots, using a quadratic function to determine the probe sequence.

In quadratic probing, the likelihood of primary clustering is reduced compared to linear probing. However, its access pattern is less cache-friendly. While quadratic probing mitigates primary clustering, it can still experience secondary clustering, where elements hashing to the same initial index follow the same probe sequence. Additionally, quadratic probing is more complex to implement compared to linear probing.

- **Double hashing:** This uses a secondary hash function to determine the probe sequence, reducing clustering further. The probe sequence is as follows:

$$h(\text{key}, i) = (h_1(\text{key}) + i \cdot h_2(\text{key})) \bmod m.$$

Example 7.10

To handle collisions using double hashing for a hash table of size 10 and keys 12, 13, 22, and 32, we use the primary hash function $h_1(\text{key}) = \text{key} \bmod 10$. The hash values for these keys are 2, 3, 2, and 2, respectively. We also use a secondary hash function, $h_2(\text{key}) = 1 + (\text{key} \bmod 9)$, to determine the probe sequence:

1. Calculate the primary hash values:

- For key 12: $12 \bmod 10 = 2$
- For key 13: $13 \bmod 10 = 3$
- For key 22: $22 \bmod 10 = 2$
- For key 32: $32 \bmod 10 = 2$

2. Handle collisions using double hashing:

- Key 12 hashes to index 2, so it is placed at index 2.
- Key 13 hashes to index 3, so it is placed at index 3.
- Key 22 hashes to index 2, which is already occupied by 12. Using the secondary hash function, we calculate the probe sequence:

Secondary hash for 22:

$$h_2(22) = 1 + (22 \bmod 9) = 1 + 4 = 5$$

Probe sequence:

$$h(22, i) = (h_1(22) + i \cdot h_2(22)) \bmod 10$$

First probe: $(2 + 1 \times 5) \bmod 10 = 7$, which is empty, so 22 is placed at index 7

- Key 32 hashes to index 2, which is occupied by 12. Using the secondary hash function, we calculate the probe sequence:

Secondary hash for 32:

$$h_2(32) = 1 + (32 \bmod 9) = 1 + 5 = 6$$

Probe sequence:

$$h(32, i) = (h_1(32) + i \cdot h_2(32)) \bmod 10$$

First probe: $(2 + 1 \times 6) \bmod 10 = 8$, which is empty, so 32 is placed at index 8

3. Here is the resulting hash table:

Index	Hash table
0	
1	
2	12
3	13
4	
5	
6	
7	22
8	32
9	

Table 7.5 – An example of handling collision using double hashing

The example demonstrates how double hashing resolves collisions by using a secondary hash function to determine the probe sequence, reducing the likelihood of clustering.

The major advantage of double hashing is that it significantly reduces both primary and secondary clustering. It maintains $O(1)$ average-case performance for insert, delete, and search operations. However, double hashing is more complex to implement due to the need for two hash functions. Additionally, it may be less cache-efficient because of non-contiguous memory accesses.

Cuckoo hashing – the bird-inspired collision resolution

Cuckoo hashing is a unique open addressing scheme for handling collisions in hash tables, inspired by the behavior of cuckoo birds. Just like a cuckoo chick pushes other eggs out of a nest, cuckoo hashing allows a new key to “kick out” an existing key to make space for itself. Let’s explain how it works step by step:

1. **Two hash functions:** Cuckoo hashing employs two independent hash functions (h_1 and h_2) that map keys to slots in the hash table. This gives each key two potential locations where it can be stored.

2. **Insertion:** When inserting a key, the algorithm first tries to place it in the slot determined by $h_1(key)$:
 - If the slot is empty, the key is inserted
 - If the slot is occupied, the existing key is “kicked out” and the new key takes its placeThe kicked-out key is then attempted to be inserted into its alternate location based on $h_2(key)$. This process continues, potentially displacing more keys until an empty slot is found or a maximum number of displacements is reached.
3. **Lookup:** To find a key, the algorithm checks both possible locations ($h_1(key)$ and $h_2(key)$). If the key is found in either location, the lookup is successful.
4. **Deletion:** Deleting a key is straightforward; simply remove it from the slot where it's found.

Let's illustrate this hashing method in the following example.

Example 7.11

Let's handle collisions using cuckoo hashing for a hash table of size 10 and keys 12, 13, 22, and 32:

1. We use the following hash functions:
 - $h_1(key) = key \bmod 10$
 - $h_2(key) = (key/10) \bmod 10$ ('/' is integer division)
2. Insertion steps:
 - Key 12:
Hash values: $h_1(12) = 12 \bmod 10 = 2$
Place 12 in Table 1 at index 2.
 - Key 13:
Hash values: $h_1(13) = 13 \bmod 10 = 3$
Place 13 in Table 1 at index 3.
 - Key 22:
Hash values: $h_1(22) = 22 \bmod 10 = 2$.
Index 2 in Table 1 is occupied by 12. Kick out 12 and place 22 in Table 1
 - index 2.
Reinsert 12 using h_2 : $h_2(12) = (12/10) \bmod 10 = 1$.
Place 12 in Table 2 at index 1.

- Key 32:

Hash values: $h_1(32) = 32 \bmod 10 = 2$.

Index 2 in Table 1 is occupied by 22. Kick out 22 and place 32 in Table 1 at

- index 2.

Reinsert 22 using h_2 : $h_2(22) = (22/10) \bmod 10 = 2$.

Index 2 in Table 2 is empty, so place 22 in Table 2 at index 2.

3. Here are the resulting hash tables:

Index	Hash table (1)		Index	Hash table (2)
0			0	
1			1	12
2	32		2	22
3	13		3	
4			4	
5			5	
6			6	
7			7	
8			8	
9			9	

Table 7.6 – An example of handling collision using cuckoo hashing

Cuckoo hashing offers constant $O(1)$ worst-case lookup time, which is a significant advantage over other collision resolution techniques such as linear probing or chaining. It also eliminates clustering by ensuring that each key can reside in one of two possible locations. However, cuckoo hashing requires a relatively low load factor to function effectively, meaning the table might need to be larger compared to other methods. Additionally, if the table becomes too full or cycles are detected, a rehashing operation is necessary, which can be costly.

Cuckoo hashing is an excellent choice when extremely fast lookups and deletions are required, even under heavy load. However, it is important to be aware of the limitations regarding load factor and the potential overhead of rehashing.

Recap

Hashing offers several advantages, most notably its speed. Hashing provides average-case $O(1)$ time complexity for search, insert, and delete operations, making it extremely fast. Hash tables can handle a large number of entries efficiently, provided the load factor (the ratio of entries to table size) is managed well. Additionally, hash tables are versatile and can be used in various applications, such as implementing dictionaries, caches, and sets.

However, hashing also has its limitations. Collisions, while minimized, can still occur and must be managed efficiently to maintain performance. Hash tables may require more memory than other data structures, particularly if the table is sparsely populated. In the worst-case scenario, where many collisions occur, the time complexity can degrade to $O(n)$.

Despite these challenges, hashing remains a powerful method for achieving constant time search operations. Hash tables are widely used in applications such as database indexing (where they enable fast record retrieval) and caching, where they help quickly locate cached data. They are also essential in compilers and interpreters for storing information about identifiers and in many programming languages for implementing sets and associative arrays (maps). The efficiency and versatility of hashing make it an indispensable technique in computer science for fast and efficient data retrieval.

Summary

In this chapter, we explored the fundamental concepts of search and search algorithms, beginning with an overview of linear and sub-linear search methods. We examined how linear search, despite its simplicity and ease of implementation, has limitations in efficiency, especially for large datasets. We then discussed sub-linear search algorithms, such as binary search, jump search, and interpolation search, highlighting their improved time complexities and discussing the conditions under which they performed best.

Lastly, we introduced the concept of hashing and its critical role in achieving constant time complexity for search, insert, and delete operations. We covered different hashing methods, including division-remainder, multiplication, mid-square, and universal hashing, explaining how each method worked and their respective strengths and weaknesses.

We also discussed various techniques for handling collisions, such as linear probing, quadratic probing, double hashing, and cuckoo hashing. Each method's advantages and limitations were analyzed, showing how they addressed the challenges of search operations in different scenarios. These techniques demonstrated how sophisticated strategies could optimize search operations by efficiently managing the search space and minimizing collisions.

The chapter concluded by emphasizing the importance of hashing in various applications, such as database indexing, caching, symbol tables, and implementing sets and maps in programming languages. Despite challenges such as collision handling and memory overhead, hashing was shown to be a versatile and efficient technique in computer science for fast data retrieval.

In the next chapter, we will explore the relationship between sorting and searching in computational systems and algorithm design. We will examine how to strike a balance between these two processes to minimize computational costs.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022:
 - *Chapter 11, Hash Tables*
- *The Art of Computer Programming*. By D. E. Knuth. Volume 3: Sorting and Searching (Second ed.). Addison-Wesley. 1998:
 - *Section 6.1, Searching*
 - *Section 6.2, Binary Search*
 - *Section 6.4, Hashing*
- *Data Structures and Algorithm Analysis in C++*. By M. A. Weiss. (Fourth ed.). Pearson. 2012:
 - *Chapter 5, Hashing*
 - *Chapter 7, Search Trees*
- *Algorithms*. By R. Sedgewick, K. Wayne. Fourth Edition. Addison-Wesley. 2011:
 - *Section 3.4, Hash Tables*

Symbiotic Relationship between Sort and Search

With some estimates indicating that sorting consumes a significant portion of global computing power, this chapter examines the symbiotic relationship between sorting and searching. We will provide an overview of these fundamental operations and explore how they interact and complement each other. The discussion will emphasize real-world examples to illustrate how to balance these crucial tasks in data processing.

This chapter serves as a practical guide for applying analytical techniques to address complex real-world problems. You will compare the complexities of sorting and searching to understand how to achieve equilibrium in their application. By examining real-world scenarios, you will gain insights into how algorithm analysis can be effectively utilized in decision-making. Through these examples, the chapter aims to highlight the importance of choosing the right approach for specific situations, optimizing both sorting and searching processes to enhance overall efficiency and performance.

We will explore the following topics in this chapter:

- Striking the right balance between sorting and searching
- The efficiency dilemma – to organize or not?

Striking the right balance between sorting and searching

Let's take a moment to reflect on our interactions with various information systems, ranging from local databases to search engines and libraries. How many of these interactions involve search-related tasks? Consider decision-making processes. Can we assume that, in most cases, making a decision involves searching for the best possible option? Similarly, can we infer that most problem-solving tasks involve some form of search?

In the previous chapter, we explored the differences between algorithmic search and AI-driven search. While this chapter focuses on algorithmic search, some of the questions we pose will be relevant to AI search as well.

The answer to the questions we posed earlier is, yes, most of our interactions with computers involve direct search tasks or can be framed as search problems, such as *optimization* and *constraint-satisfaction problems*. The concept of search extends beyond just computer systems. In our everyday lives, we frequently solve search problems. Whether it's searching for the right shirt in a cluttered closet on a cold winter day or finding the electric bill amid a stack of important documents, flyers, and miscellaneous papers, we are constantly engaging in search activities. This demonstrates how fundamental and pervasive search operations are, both in computing and in our daily routines.

This highlights the substantial computational and human effort invested in search tasks. In algorithm design and analysis, this effort is often measured in time and space complexities. While there are additional cost factors involved, these are beyond the scope of our current discussion.

Symbiotic link between sorting and searching

The primary objective of this section is to analyze the cost of search operations, focusing on their time and space complexity from a practical perspective. Understanding these costs is crucial for optimizing search algorithms and improving overall system efficiency.

In the previous chapter, we demonstrated that the time complexity of a linear search without any pre-processing, such as sorting, is $O(n)$ in the worst case. This means that we may need to make up to n comparisons to locate the target key. Additionally, if we search for all items with the same key, the number of comparisons remains n and cannot be reduced.

However, we can achieve better performance than $O(n)$ by investing some computational resources to sort the data before performing any search operations. As discussed in *Chapter 6, Merge Sort*, is one of the most efficient sorting algorithms, with a running time of $O(n \log n)$. Once the data is sorted, we can utilize more efficient search algorithms, such as those in the binary search family. For instance, a standard *binary search* has a running time of $O(\log n)$. Furthermore, for data that is uniformly distributed, we could employ even more efficient algorithms such as *interpolation search*, which has an average-case running time of $O(\log \log n)$.

For the purposes of this discussion, let's assume we opt for the standard binary search with its $O(\log n)$ running time. By combining sorting with efficient search algorithms, we can significantly reduce the overall computational cost compared to using linear search on unsorted data. As we can see, sorting and searching exhibit a symbiotic relationship.

We borrowed the term “symbiotic” from biological systems. In biology, a symbiotic relationship refers to a mutually beneficial interaction between two different organisms, species, or entities. In such relationships, both parties typically derive some form of benefit, which can vary in nature. There are three main types of symbiosis:

- **Mutualism:** Both organisms benefit from the relationship. For example, bees and flowers have a mutualistic relationship; bees get nectar from the flowers, and flowers get pollinated by the bees.

- **Commensalism:** One organism benefits while the other is neither helped nor harmed. For example, barnacles attach to a whale and benefit from being transported to different feeding grounds without affecting the whale.
- **Parasitism:** One organism benefits at the expense of the other. For example, ticks feed on the blood of mammals, benefiting the tick but potentially harming the host.

In a broader sense, “symbiotic” can describe relationships in various contexts where cooperation and mutual benefit are key. However, sorting and searching can be considered to have a unique form of a symbiotic relationship, akin to parasitism. In this relationship, sorting benefits searching because sorted data can significantly enhance the efficiency of search operations. For instance, binary search requires sorted data to function and can quickly locate an element in logarithmic time, which is much faster than a linear search in unsorted data. Additionally, certain search algorithms, such as those used in databases or for specific applications, inherently involve sorting as a pre-processing step to optimize the search process.

However, the reverse is not true. Efficient search algorithms do not improve the efficiency of sorting algorithms. In comparison-based sorting algorithms, the performance of sorting algorithms remains unaffected by the efficiency of search algorithms. Sorting has its inherent complexity, and the costs associated with it do not diminish through advancements in search techniques. Thus, while sorting provides a significant advantage to searching, the benefit is one-sided. The symbiotic relationship here is more parasitic, with searching reaping the benefits of sorted data without reciprocating any advantage to the sorting process.

Here is how searching and sorting interact:

- **Efficiency of search operations:** The binary search algorithm requires data to be sorted to work correctly. When data is sorted, binary search can quickly locate an element in $O(\log n)$ time, which is much faster than the $O(n)$ time complexity of a linear search in unsorted data.
- **Sorting as a pre-processing step:** Many search-related problems benefit from sorting as a preliminary step. For instance, when dealing with range queries or searching for multiple elements, sorting the data first can lead to more efficient algorithms.
- **Complex algorithms:** Some complex algorithms combine sorting and searching to solve problems more efficiently. For example, algorithms for finding the median, mode, or other statistics often start with sorting data.
- **Data structures:** Data structures such as balanced binary search trees (e.g., AVL trees and red-black trees) inherently maintain a sorted order, allowing efficient search, insertion, and deletion operations. Similarly, B-trees used in databases maintain sorted data to optimize search operations.

Sorting and searching are not the only concepts in computer science with symbiotic ties. There are other pairs of concepts that exhibit similar mutually beneficial relationships. Let's take a look at some of them:

- **Exploration and exploitation in AI search:**
 - **Exploration:** Implemented by algorithms such as **Breadth-First Search (BFS)**, exploration involves examining a wide area of the search space. The more we invest in exploration, the less we need to rely on exploitation, which can be risky.
 - **Exploitation:** Implemented by algorithms such as **Depth-First Search (DFS)**, exploitation focuses on exploring one path deeply before trying others. Balancing exploration and exploitation is crucial to avoid the risk of failed searches and ensure efficient problem-solving.
- **Cost versus heuristics in AI search strategies:**
 - In AI search strategies, there exists a symbiotic relationship between the cost of search and the use of heuristics. Early in the search process, investing in well-chosen heuristics can significantly lower the overall search cost by steering the search in more productive directions. This balance leads to more efficient problem-solving. A prime example is the A* search algorithm, which optimally combines both heuristics and path cost to find the most efficient solution.

These examples demonstrate that the symbiotic relationship between different concepts is a common theme in computer science. Balancing and optimizing these relationships are key to developing efficient algorithms and solving complex computational problems.

Let's conclude this section with an illustrative example. Imagine that there is a hypothetical currency used to pay for the computational complexity of tasks, and this currency is called "comparison." When searching and retrieving an item from an unsorted database, we pay a cost of n comparisons, where n is the size of the dataset. However, if we invest in sorting the data beforehand using an efficient sorting algorithm, the cost of searching can be reduced to $\log n$ comparisons.

But this reduction in search cost comes with an upfront cost. Sorting the data efficiently requires approximately $n \cdot \log n$ comparisons. Therefore, the total cost of first sorting and then searching is $n \cdot \log n$ (for sorting) plus $\log n$ (for searching), which simplifies to approximately $n \cdot \log n$ comparisons, since $\log n$ is relatively small compared to $n \cdot \log n$.

Given this, you might wonder about the benefit of investing in sorting. The benefit of sorting becomes evident when multiple search operations are needed. If we sort the data once and then perform many searches, the initial sorting cost is amortized over numerous searches, making each individual search operation significantly faster and more efficient. Thus, the upfront investment in sorting pays off in scenarios where frequent searches are required, highlighting the symbiotic relationship where sorting optimizes the overall efficiency of search tasks.

However, there is a strong underlying assumption in this scenario that may not hold true in many real-world applications – the assumption that data is static and does not change. In reality, datasets are often dynamic, requiring not only search operations but also frequent insertions and deletions.

When dealing with unsorted data, inserting a new item is straightforward and incurs no comparison cost, as the new item is simply added to the end of the collection. Deleting an item, however, requires searching for it, which can take up to n comparisons, followed by its removal.

In contrast, with sorted data, inserting a new item necessitates maintaining the order, which often means re-sorting the data or finding the appropriate position for the new item, incurring additional comparison costs. Deleting an item from a sorted dataset involves searching for it in logarithmic time, thanks to the order, but it does not require re-sorting after removal.

In conclusion, while sorting data upfront can significantly reduce the cost of search operations when searches are frequent, this benefit diminishes in dynamic datasets. Every insertion may require additional sorting, increasing the overall cost. Therefore, in environments where data is frequently modified, the advantage of pre-sorting for efficient searching must be weighed against the ongoing costs of maintaining the sorted order. This consideration challenges the impact of sorting on improving search efficiency in dynamic scenarios.

In the next section, we will demonstrate how achieving a balance between sorting and searching can lead to the design of efficient algorithms. We will explore this balance through real-world examples, both within and outside of information systems, to help make more effective and efficient decisions.

The efficiency dilemma – to organize or not?

We have observed numerous professionals and university professors maintaining cluttered offices with papers, books, and bills scattered everywhere. They often struggle to locate the items they need, spending considerable amounts of time in the process. Fearing the potential loss of something important, they tend to hold onto almost everything. In contrast, some individuals place great emphasis on organization (i.e., sorting). They invest substantial amounts of time in arranging and categorizing their belongings. However, the time they dedicate to organizing is rewarded when it comes to retrieving the things they need.

A fundamental question is, which approach is more efficient – being organized or disorganized? In this section, I aim to explore the notion that the value placed on organization may be overestimated, drawing insights from the symbiotic relationship between sorting and searching.

Professionals with cluttered offices often operate in an environment where immediate retrieval of specific items is challenging. This disorganized approach can lead to inefficiencies, as significant time is spent searching for necessary documents or tools. Despite the apparent chaos, there is often an underlying system that makes sense to the individual, although it may not be immediately apparent to others.

Conversely, individuals who prioritize organization invest time upfront to categorize and sort their items. This methodical approach can initially seem time-consuming, but it pays off when they need to retrieve something quickly. The efficiency gained in retrieval often justifies the time spent organizing. By having a well-structured system, they minimize the search time, making their workflow smoother and more predictable.

The core question of whether organization or disorganization is more efficient depends on the context and the specific needs of the individual. For some, a cluttered workspace may stimulate creativity and flexibility, while for others, an organized environment provides clarity and efficiency.

Drawing parallels from sorting and search algorithms, we can see that the effectiveness of each approach can vary. In computational terms, an organized system (such as a sorted array) allows for faster search times (using algorithms such as binary search), whereas a disorganized system (such as an unsorted array) might require more exhaustive search methods (such as linear search). However, the initial sorting process itself has a cost, and the decision to sort depends on the frequency and urgency of search operations.

In conclusion, the value of organization versus disorganization can be seen as analogous to the trade-offs between sorting and searching in algorithms. Both approaches have their merits and drawbacks, and the optimal strategy often involves finding a balance that suits the specific requirements and constraints of the situation. By understanding the symbiotic relationship between sorting and searching, we can make more informed decisions about how to manage our resources and time effectively.

Think like a computer scientist

Let's consider the following scenario – Janet is a professor of mathematics who receives a pile of scientific papers; let's assume, on average, she receives p papers each day. She needs to retrieve a couple of papers daily for her research. The number of papers she picks varies, but on average, she retrieves s papers per day. A few papers are her favorites, which she references more often, while the majority are rarely accessed.

Applying our understanding of searching, sorting, and their computational costs, we aim to assist Janet in making efficient decisions. First, Janet needs to decide whether she should sort the collection of papers on her desk and, if so, how often. This decision will depend on balancing the cost of sorting against the efficiency gained in retrieving papers. Second, she wants to calculate the total running time of her operations over k days, considering both sorting and retrieving the times.

The following are our assumptions:

- Average number of new papers received per day: p
- Average number of scientific papers accessed per day: s
- Total number of days: k

If the papers are not sorted, accessing each paper would take $O(n)$ time, where n is the total number of papers on her desk. After d days, the total number of papers is approximately $p \cdot d$. If the papers are sorted, accessing each paper would take $O(\log n)$ time due to binary search. Sorting n papers takes $O(n \log n)$ time.

Let's estimate the total running time without sorting. The total access time for one day is $s \cdot O(n)$. Over k days, the running time is:

$$\sum_{d=1}^k s \cdot O(pd) \approx O(sp k^2)$$

If Janet sorts her papers every day, then sorting time for one day is $O((pd) \log(pd))$. Access time for one day can be estimated as:

$s \cdot O(\log(pd))$, and over k days:

$$\sum_{d=1}^k (O(pd \log(pd)) + sO(\log(pd))) \approx O(pk^2 \log(pk^2))$$

Sorting papers frequently can be beneficial if the number of papers that Janet accesses is large compared to the number she receives. More precisely, if $s > \log(n)$, where n is the total number of papers, then sorting is advantageous. For instance, if Janet has 10,000 papers in her archive and retrieves more than 100 papers in one day (since $\log(10,000) = 100$), sorting becomes beneficial. However, this number will increase daily, which is not very realistic.

Regardless of this, we have made several implicit assumptions about Janet's situation:

- **Sorting capability:** We assumed that Janet could sort her papers using an efficient algorithm such as merge sort. However, humans typically sort items no better than simple algorithms such as bubble sort, which is much less efficient.
- **Equal access probability:** We assumed that all papers have the same likelihood of being accessed. In reality, some papers are referenced frequently while others are rarely or never retrieved, possibly following a power law distribution.
- **Returning papers to the top:** In the unsorted scenario, we assumed Janet usually returns a used paper to the top of the pile. This means she wouldn't have to search through the entire pile to find the paper again. Additionally, while humans are poor at systematic sorting, they often have a strong ability to locate items among a random assortment, similar to a mental hashing mechanism.

Considering these points, the initial analysis that favors frequent sorting may not hold. The natural human ability to locate frequently used items, and the impracticality of maintaining a sorted system in a dynamic environment, suggest that sorting may not always be the most efficient approach for Janet.

It is important to note that the example provided is a computational analysis that can certainly inform decision-making, but it is not the sole factor to consider. In other words, computational efficiency does not necessarily translate to productivity when a human is involved or when a human is part of

the process. Having a neat and organized environment can psychologically improve our mood, help us concentrate, and foster deep work.

It is not surprising that, even in hypothetical scenarios involving robot uprisings, the robots themselves may exhibit disorder and lack of organization!

While our analysis suggests that sorting may not always be the most efficient approach from a purely computational standpoint, human factors such as the need for a tidy workspace and the psychological benefits of organization should not be overlooked. These factors can significantly enhance overall productivity and well-being, making the decision to sort and organize papers more complex than simple computational efficiency would suggest.

Summary

In this chapter, we examined the symbiotic relationship between sorting and searching in detail. It was demonstrated that while sorting can significantly reduce the time complexity of search operations, the overall efficiency gained from sorting must be carefully weighed against the costs of maintaining a sorted dataset, especially in dynamic environments where data is frequently updated. Through various scenarios and examples, we highlighted the trade-offs involved in deciding whether and how often to sort data. We also pointed out that the computational benefits of sorting do not necessarily translate to practical efficiency when human factors and psychological impacts are considered. Our analysis revealed that humans' natural ability to locate items and the psychological advantages of an organized workspace add layers of complexity to the decision-making process.

The chapter concluded with a recognition that while computational analysis provides valuable insights, it is not the sole determinant of productivity, especially when humans are involved. The hypothetical benefits of sorting must be balanced with practical considerations and the inherent strengths and preferences of individuals. As we move forward, the next chapter will introduce randomized algorithms, offering a new perspective on handling computational problems. These algorithms leverage randomness to achieve efficient solutions and provide an exciting contrast to the deterministic approaches discussed thus far.

References and further reading

- *Algorithms to Live By: The Computer Science of Human Decisions*. By Brian Christian and Tom Griffiths. Henry Holt and Co. 2016
- *Deep Work: Rules for Focused Success in a Distracted World*. By Cal Newport, Grand Central Publishing, 2016

Randomized Algorithms

Building on the deterministic algorithms discussed in previous chapters, we now turn our attention to situations where uncertainty and randomness are significant factors. In this chapter, we will explore how to design algorithms that make the best possible decisions despite the presence of unpredictable elements. Randomized algorithms introduce elements of chance into their logic, providing innovative ways to tackle problems that may be challenging or inefficient to solve using purely deterministic methods. These algorithms can often simplify solutions, improve performance, and offer new perspectives on problem-solving. Throughout this chapter, we will examine various strategies and techniques that leverage randomness to achieve desired outcomes. By understanding the principles behind randomized algorithms, we can develop robust solutions that excel in uncertain environments. Our discussion will cover theoretical foundations, practical applications, and examples that illustrate the power and versatility of incorporating randomness into algorithm design.

Please note that although we use some preliminary topics in probability theory in this chapter, a sufficient amount of familiarity with probability theory is necessary to understand this chapter holistically. If you are not well versed in probability theory, we recommend refreshing your knowledge using reputable reference books and textbooks on the subject before proceeding.

In this chapter, we will cover the following topics:

- A review of probabilistic algorithms
- Analysis of randomized algorithms
- Case studies

A review of probabilistic algorithms

To begin, let's explore whether we can apply the concepts covered so far to solve the following hypothetical problems:

- A new online dating app, *Matcher*, has been developed to help users find potential partners. The app functions like a game, aiming to match users with their best possible dates:
 - There are N potential matches available on the app (the total number of matches is unknown to the users). Let's consider a user named Tom.
 - When Tom opens the *Matcher* app, he is presented with one potential match at a time, selected randomly. Tom has the option to either like (swipe right) or dislike (swipe left) each profile.
 - Tom's decisions are irreversible; once he swipes right or left on a profile, he cannot change his mind. All decisions are final.
 - Tom can like a maximum of n profiles (where n is much smaller than N). Once Tom has liked n profiles, the app will no longer show him any more profiles.
 - The interaction ends either when Tom has liked n profiles or when there are no more profiles left to show.
 - It is important to note that liking a profile does not guarantee a match; it still requires the other person to like Tom's profile back.
 - The challenge for Tom is to make the best possible use of his n likes to choose the most suitable matches. He needs to decide when to stop browsing and start liking profiles to maximize the chances of selecting the best available options.
- Fang has been invited to her friend's house, which is located in the middle of a long, one-way street. Parking is allowed only on one side of the street. The street is busy, and finding a parking spot is challenging. Based on experience, about 10% of the n parking spots are typically available at any given time. Fang can only see one spot at a time as she passes by and cannot see the spots ahead. The objective is to determine the optimal moment for her to decide to take an available parking spot.

Both problems mentioned share a fundamental similarity: they are all search problems. In each scenario, the objective is to identify the best option from a sequence of possibilities:

- Tom is searching for the best match on a dating app
- Fang is trying to find the closest parking spot to her friend's house

However, these problems differ from traditional search settings. Unlike classical search problems where all data items are available for comparison, here, data items appear sequentially and unpredictably. This sequential arrival of data means we cannot predict the next item, rendering standard comparison-based search algorithms ineffective.

In these scenarios, the challenge is not about identifying the best data item through comparison but deciding when to stop searching and make a selection. The decision-making process involves evaluating each option as it arrives and determining whether to accept it or continue searching for a potentially better option. The following are key characteristics of these problems:

- **Sequential data arrival:** Data items arrive one at a time, and decisions must be made immediately without knowing future data.
- **Irreversible decisions:** Once a decision is made to accept or reject an option, it cannot be reversed. This adds a layer of complexity as you cannot revisit previous choices.
- **Optimal stopping rule:** The core of these problems is finding the optimal stopping point. This involves deciding at what point to stop searching and accept the current option as the best available.

Optimal stopping theory provides a framework for solving these types of problems. The theory helps in determining the point at which the probability of finding a better option is outweighed by the cost of continuing the search. For example, let's consider the following:

- Tom can use a strategy where he initially reviews a set number of profiles without making any decisions (to gather information about the quality of matches) and then selects the next profile that is better than those he has reviewed so far
- Fang can drive past the first few parking spots (to gauge availability and proximity) and then take the first spot that is closer than those she has already seen

These problems exemplify the practical application of optimal stopping theory in real-life scenarios where data arrives sequentially and decisions must be made in real time. By understanding and applying this theory, one can make informed decisions on when to stop searching and accept an option, even in the face of uncertainty and incomplete information. This approach shifts the focus from finding the best data item to determining the optimal moment to make a decision.

The optimal stopping problem is a prime example of a randomized algorithm. Before we explore randomized algorithms in detail, let's revisit the fundamental concept of an algorithm. An algorithm is a system that transforms a set of input data into a set of output data. *Figure 9.1* provides a simple block diagram to illustrate this concept.

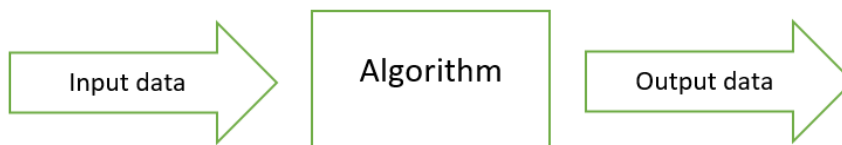


Figure 9.1: A block diagram showing the mapping of input data to output data through an algorithm

Let's revisit the concept of an algorithm through the lens of mapping input to output. Assume we have an input dataset represented by $A = \langle a_1, a_2, \dots, a_n \rangle$. An algorithm, which consists of a set of procedural instructions, transforms A into an output $B = \langle b_1, b_2, \dots, b_m \rangle$.

In the context of deterministic algorithms, this transformation process is consistent and repeatable. Given the same input A , a deterministic algorithm will always produce the same output B . This predictability is a hallmark of deterministic algorithms, ensuring that their behavior is fully determined by the input they receive.

For instance, consider the sorting and searching algorithms we have introduced in previous chapters. These algorithms are deterministic because they guarantee the same result each time they are run with the same input.

Deterministic algorithms offer predictability and reliability by ensuring that the same input will always produce the same output. This characteristic is fundamental to many algorithms used in computer science, especially in tasks that require consistent and repeatable results, such as sorting and searching. Understanding this concept lays the groundwork for exploring randomized algorithms, which introduce elements of chance to achieve different goals and handle uncertainty in unique ways.

Non-deterministic algorithms

Non-deterministic algorithms are theoretical constructs used primarily in the study of computational complexity. They assume the existence of a “non-deterministic” machine, such as a non-deterministic Turing machine, which can make arbitrary choices to explore different computational paths simultaneously. Non-deterministic algorithms are often used to define classes of problems, such as **nondeterministic polynomial time (NP)**, which includes problems for which a solution can be verified in polynomial time by a deterministic algorithm. Non-deterministic algorithms are not practically implementable on standard deterministic machines. They serve as a way to understand the potential power of parallel computation and to classify problem complexity.

Note

Non-deterministic algorithms are not practically implementable on standard deterministic machines. Randomized algorithms, on the other hand, are practical algorithms that use random numbers to influence their decision-making process. These algorithms can be implemented on standard computers and have applications in many areas of computer science and engineering.

Figure 9.2 provides a simple block diagram to illustrate the concept of randomized algorithms. Randomized algorithms include an element of chance that acts as an additional hidden input, which we cannot control. This element of randomness introduces uncertainty, effectively serving as a second input to the algorithm. It is this inherent randomness that causes the algorithm to have different behavior and/or performance even when the explicit input remains the same.

Consider the **randomized quick sort** algorithm. Unlike the traditional quick sort, which typically selects a fixed pivot (such as the first, last, or middle element), randomized quick sort selects a pivot at random. This random selection can lead to different sequences of comparisons and swaps each time the algorithm is run, even on the same input sequence. For instance, given the input sequence $A = [3, 1, 4, 1, 5, 9, 2, 6, 5]$, a traditional quick sort algorithm might always choose the middle element as the pivot. In contrast, randomized quick sort might choose any element as the pivot, resulting in a different sequence of steps and partitioning in each run. Consequently, the intermediate steps can vary, potentially leading to different overall running times even with the same data.

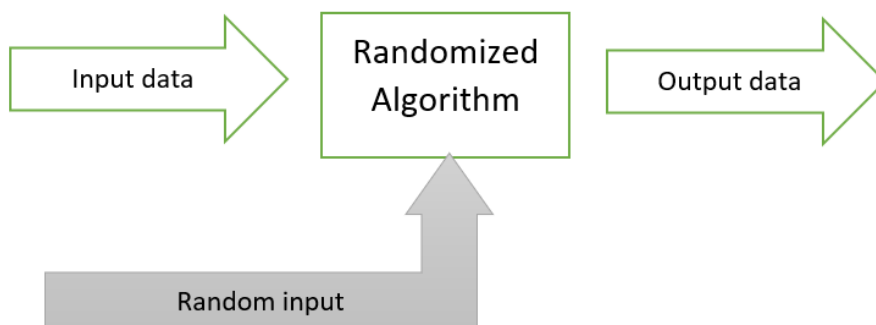


Figure 9.2: A block diagram showing the mapping of input data to output data through a non-deterministic algorithm

Any random input to the algorithm introduces variability in the algorithm's performance and behavior. While the average case time complexity of randomized quick sort remains $O(n \log n)$, the actual runtime for a specific instance can differ due to the random pivot selection. In the worst case, if the pivot selection is particularly poor repeatedly, the performance can degrade, although such scenarios are statistically unlikely.

By introducing randomness, these algorithms can avoid pathological cases that deterministic algorithms might encounter. For instance, randomized quick sort avoids the worst-case scenario of $O(n^2)$ that occurs in traditional quick sort with certain inputs. Randomized algorithms are often more robust and adaptable. They perform well on average across a wide range of inputs, making them versatile tools in practice. In the next section, we explore the framework to analyze randomized algorithms.

To effectively study randomized algorithms, we need to explore several key topics:

- **Analysis of randomized algorithms:** Understanding the behavior of algorithms in terms of probability and expected outcomes is crucial. This involves analyzing the average-case performance rather than focusing solely on worst-case scenarios. This topic will be covered in detail in the upcoming section titled *Analysis of randomized algorithms*.

- **Randomized data structures:** Designing data structures that incorporate randomness can lead to more efficient operations. Examples include skip lists and hash tables. Skip lists, in particular, will be explored in *Chapter 11*.
- **Case studies:** To apply the concepts learned, we will analyze specific problems under uncertainty, such as those mentioned at the beginning of this section. Detailed solutions and discussions will be presented in the section titled *Case studies*.

By exploring these topics, we will gain a comprehensive understanding of how randomized algorithms operate and how to leverage them effectively in various applications.

Analysis of randomized algorithms

To analyze randomized algorithms, we often use **probabilistic analysis**. Instead of focusing on the worst-case scenario, we examine the *expected* performance of the algorithm over all possible random choices. This expected value provides a more realistic picture of the algorithm's typical behavior. Here are the key principles:

- We calculate the expected value of key performance metrics, such as running time or space usage. This involves averaging the performance over all possible inputs, weighted by their probability.
- We study the distribution and variance of the algorithm's performance metrics to understand how much they deviate from the expected value. This helps in assessing the reliability and consistency of the algorithm.
- The focus is on analyzing the algorithm's behavior on typical or randomly chosen inputs rather than worst-case inputs. This provides a more realistic measure of the algorithm's practical performance.
- We use realistic input models that reflect the expected usage scenarios of the algorithm. For example, in sorting, assume inputs are randomly permuted rather than always sorted or reverse sorted.
- We establish performance bounds that hold with high probability. For example, an algorithm might run in $O(n \log n)$ time with high probability, even if it occasionally runs slower.
- Random variables are employed to model the internal randomness of the algorithm. We analyze how these variables influence the algorithm's behavior and performance. We also consider the independence or correlation between random variables to simplify the analysis or derive more accurate performance estimates.

In addition to the principles discussed earlier, the analysis of randomized algorithms often involves specific techniques and models. Random sampling is a fundamental technique used to make decisions or estimate properties of larger datasets efficiently. Monte Carlo and Las Vegas algorithms leverage randomness in different ways. Monte Carlo algorithms provide probabilistic guarantees and run in fixed time with some probability of error, while Las Vegas algorithms guarantee correctness but have

variable running times. Markov chains and random walks are essential models for analyzing more advanced randomized algorithms. Markov chains help understand the behavior of systems with state transitions, while random walks are used in various network algorithms and optimization problems.

The approach to analysis can be either adaptive or adversarial. Adaptive analysis studies how algorithms respond to varying input conditions or environmental changes. Adversarial analysis, on the other hand, considers scenarios where inputs are designed to challenge the algorithm's performance to its limits. It is important to note that while these principles and techniques are crucial for a comprehensive understanding of randomized algorithms, they extend beyond the scope of this book. Therefore, we will not cover all of these principles in detail here.

It is often most effective to illustrate the analysis of randomized algorithms through a few examples, demonstrating the step-by-step process in detail. The first example is the well-known *Monty Hall problem*. The second example is the *birthday paradox* and finally, we discuss the well-known *hiring secretary problem*.

Monty Hall problem

The **Monty Hall problem** is a classic brain teaser that illustrates the power and sometimes counterintuitive nature of probabilistic reasoning, which is a core concept in the analysis of randomized algorithms.

The problem is as follows: You are presented with three doors (labeled as A, B, and C). Behind one door is a car, and behind the other two are goats. You pick a door (let's say door A). The host, who knows what is behind each door, opens another door (let's say door B) to reveal a goat. The host then offers you the choice to switch to the remaining closed door (door C). Should you stick with your initial choice or switch to maximize your chances of winning the car?

The counterintuitive answer is yes, you should absolutely switch. Switching doors doubles your chances of winning the car. At first glance, it might seem that after one door is revealed, the odds are 50/50 between the remaining two doors. However, this is a common misconception. The correct strategy is to always switch doors, and here is why:

- The probability that the car is behind door A (your initial choice): $\frac{1}{3}$.
- The probability that the car is behind door B or C: $\frac{2}{3}$.
- The host's action of opening door B (which always reveals a goat) does not change the initial probabilities. Instead, it provides additional information that affects the distribution of these probabilities.

Let's break down the probabilities and consider the two possible scenarios for the location of the car:

- The probability of the car being behind your initial choice (door A): $\frac{1}{3}$:
 - If you stick with door A, you win the car with a probability of $\frac{1}{3}$
 - If you switch to door C, you lose with a probability of $\frac{1}{3}$

- The probability of the car being behind one of the other doors (door B or door C): $\frac{2}{3}$:
 - Since the host reveals a goat behind door B, the car must be behind door C if it is not behind door A
 - If you switch to door C, you win the car with a probability of $\frac{2}{3}$

Then, by switching doors, you effectively bet on the probability that your initial choice was wrong, which is $\frac{2}{3}$. Therefore, switching doors increases your chances of winning the car from $\frac{1}{3}$ to $\frac{2}{3}$. This counterintuitive result is a classic example of how human intuition can often be misleading in probabilistic scenarios. The Monty Hall problem highlights the importance of understanding probability and making decisions based on mathematical analysis rather than gut feelings.

Birthday paradox

The **birthday paradox**, also known as the **birthday problem**, is a famous probability problem that demonstrates a counterintuitive result. It deals with the likelihood that, in a group of people, at least two individuals will share the same birthday. The surprising aspect is how small the group needs to be for this probability to be relatively high.

The problem can be stated as follows: What is the probability that, in a group of n people, at least two people share the same birthday? Assume there are 365 days in a year, and each person's birthday is equally likely to fall on any of these days.

To understand the birthday paradox, it is easier to calculate the complementary probability—the probability that no two people in the group share the same birthday—and then subtract this from 1. Let's break down the calculation of the complementary probability of the birthday problem as follows:

- The probability that the first person has a unique birthday is $\frac{365}{365}$ (since no one else has been chosen yet)
- The probability that the second person has a different birthday from the first is $\frac{364}{365}$
- The probability that the third person has a different birthday from the first two is $\frac{363}{365}$
-
- For n people, the probability $P(\text{unique birthday})$ that all birthdays are unique is as follows:

$$P(\text{unique birthday}) = \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \dots \times \frac{365 - (n - 1)}{365}$$

Or, it is as follows:

$$P(\text{unique birthday}) = \prod_{i=0}^{n-1} \left(1 - \frac{i}{365}\right)$$

Then, the probability $P(\text{shared birthday})$ that at least two people share the same birthday is as follows:

$$P(\text{shared birthday}) = 1 - P(\text{unique birthday})$$

The results show that the probability of shared birthdays increases rapidly with the size of the group. The following is a simple Python core to simulate the birthday paradox:

```
import random
import matplotlib.pyplot as plt
def simulate_birthday_paradox(trials, n):
    shared_birthday_count = 0
    for _ in range(trials):
        birthdays = []
        for person in range(n):
            birthday = random.randint(1, 365)
            if birthday in birthdays:
                shared_birthday_count += 1
                break
        birthdays.append(birthday)
    return shared_birthday_count / trials
def main():
    trials = 10000
    results = []
    group_sizes = range(2, 367)
    for n in group_sizes:
        probability = simulate_birthday_paradox(trials, n)
        results.append(probability)
        print(f"Group size: {n}, Probability of shared birthday:
{probability:.4f}")
    plt.figure(figsize=(10, 6))
    plt.plot(group_sizes, results, marker='o')
    plt.title('Birthday Paradox Simulation')
    plt.xlabel('Group Size')
    plt.ylabel('Probability of Shared Birthday')
    plt.grid(True)
    plt.show()
if __name__ == "__main__":
    main()
```

This function runs the birthday paradox simulation for a given group size n and a number of trials, which is 10,000. It counts the number of trials where at least two people share the same birthday. For each trial, it generates random birthdays for n people and checks for duplicates. *Figure 9.3* illustrates the results of the simulation.

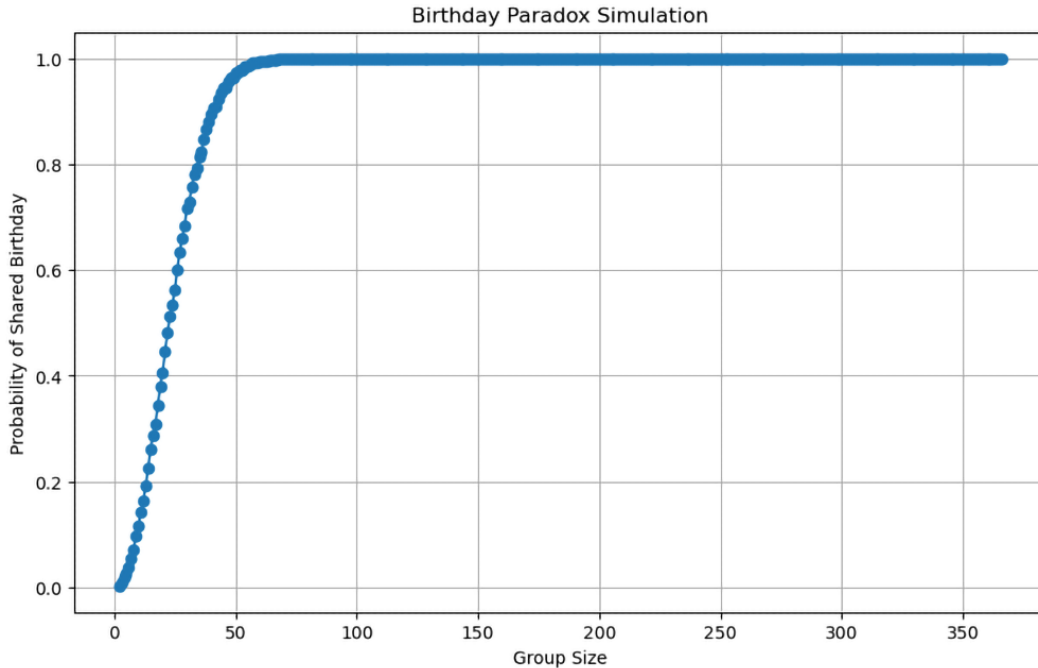


Figure 9.3: The simulation demonstrates how quickly the probability of sharing a birthday increases with the size of the group

The rapid increase in the probability of shared birthdays is counterintuitive because our instinct might suggest that a much larger group would be needed for a high probability of shared birthdays. This can be observed in *Figure 9.3*.

The birthday paradox illustrates how human intuition can often misjudge probabilities in scenarios involving combinations and large numbers. It serves as a valuable lesson in probability theory and demonstrates the importance of mathematical calculation in understanding seemingly simple problems. The paradox also has practical applications, such as in cryptography, where the concept of hash collisions is analogous to the birthday problem.

Hiring secretary problem

The **hiring secretary problem**, also known as the **secretary problem** or the **best choice problem**, is a famous problem in optimal stopping theory. It describes a scenario where an employer wants to hire the best secretary out of n applicants who are interviewed sequentially. The employer must decide whether to hire a candidate immediately after the interview, without the possibility of returning to previous candidates. The goal is to maximize the probability of selecting the best candidate.

The following are given in this problem:

- **Sequential interviews:** n candidates are interviewed one by one in random order.
- **Immediate decision:** After each interview, the employer must decide whether to hire that candidate. If rejected, the candidate cannot be reconsidered.
- **Objective:** The goal is to maximize the probability of selecting the best candidate.

The optimal strategy for this problem is surprisingly elegant and counterintuitive. It involves two phases:

- **Observation phase:** Reject the first k candidates outright. This phase is purely for observation to get a sense of the quality of candidates. The first k candidate is also called a training sample.
- **Selection phase:** From the $(k + 1)^{th}$ candidate onward, hire the first candidate who is better than all the previously interviewed candidates.

One question that might arise is this: What if none of the candidates after the first k are better than those initially rejected? In this unfortunate scenario, the only option is to hire the last candidate. This outcome increases the overall cost of interviewing and heightens the risk of hiring an unsuitable candidate.

The value of k that maximizes the probability of selecting the best candidate can be approximated by $\frac{n}{e}$, where e is the base of the natural logarithm (approximately 2.718). For large n , this simplifies to roughly 37% of the total number of candidates.

The probability $P_n(k)$ of selecting the best candidate using the optimal stopping rule is given by the following:

$$P_n(k) = \frac{k-1}{n} \sum_{j=k}^n \frac{1}{j-1}$$

We know that $P_n(k = 1) = \frac{1}{n}$.

The following Python code estimates P_n for various values of k and also estimates the ratio of k/n that maximizes the probability P_n :

```
import numpy as np
import matplotlib.pyplot as plt
def calculate_p_n(n, k):
    if k == 1:
        return 1 / n
```



```

    sum_term = sum(1 / (j - 1) for j in range(k, n + 1))
    return (k - 1) / n * sum_term
def find_optimal_k(n):
    probabilities = [calculate_p_n(n, k) for k in range(1, n + 1)]
    optimal_k = np.argmax(probabilities) + 1
    return optimal_k, probabilities
n_values = np.arange(10, 501, 1) # Smoother plot with more points
optimal_k_ratios = []
for n in n_values:
    optimal_k, probabilities = find_optimal_k(n)
    optimal_k_ratios.append(optimal_k / n)
plt.figure(figsize=(10, 6))
plt.plot(n_values, optimal_k_ratios, marker='o', linestyle='-',
markersize=4, label='Optimal k/n Ratio')
plt.axhline(1/np.e, color='r', linestyle='--', label='1/e
(approximately 0.3679)')
plt.title('Optimal k/n Ratio for Different Values of n')
plt.xlabel('n')
plt.ylabel('Optimal k/n Ratio')
plt.legend()
plt.grid(True)
plt.show()

```

Let's briefly explain the code. The `calculate_p_n` function calculates the probability $P_n(k)$ for a given n and k using the formula provided. The `find_optimal_k` function computes $P_n(k)$ for all k from 1 to n and finds the value of k that maximizes P_n . The parameters are as follows:

- `n_values`: Different values of n to analyze.
- `optimal_k_ratios`: This stores the ratio k/n that maximizes P_n for each n .
- The first plot (Figure 9.4) shows $P_n(k)$ versus k for different values of n . The second plot (Figure 9.5) shows the ratio of k/n that maximizes P_n for different values of n . It also includes a horizontal line at $1/e$ for reference.

Figure 9.4 shows how the probability $P_n(k)$ varies with k for different values of n and Figure 9.5 demonstrates that the ratio k/n that maximizes this probability tends to approximate $1/e = 0.37$ or 37% of the total number of candidates, validating the theoretical approximation.

The logic behind this strategy is based on the trade-off between gathering enough information to make an informed decision and the risk of missing out on the best candidate by waiting too long. By rejecting the first k candidates, the employer sets a benchmark for evaluating the subsequent candidates. The probability of selecting the best candidate using this strategy is approximately $1/e$, or about 37% (see Figure 9.4). This means that, on average, the employer has a 37% chance of selecting the best candidate by following the optimal strategy.

Example 9.1

Using the optimal stopping theory, suggest a strategy to hire an individual among n candidates.

Consider $n = 10$ candidates. Using the optimal stopping rule:

- **Observation phase:** Reject the first $10/e \approx 3.7 \approx 4$ candidates.
- **Selection phase:** From the fifth candidate onward, hire the first one who is better than all previous candidates.

If the best candidate is among the first four, they will be missed. If the best candidate is in the fifth position or later, there is a good chance they will be selected because they are likely to be better than those in the observation phase.

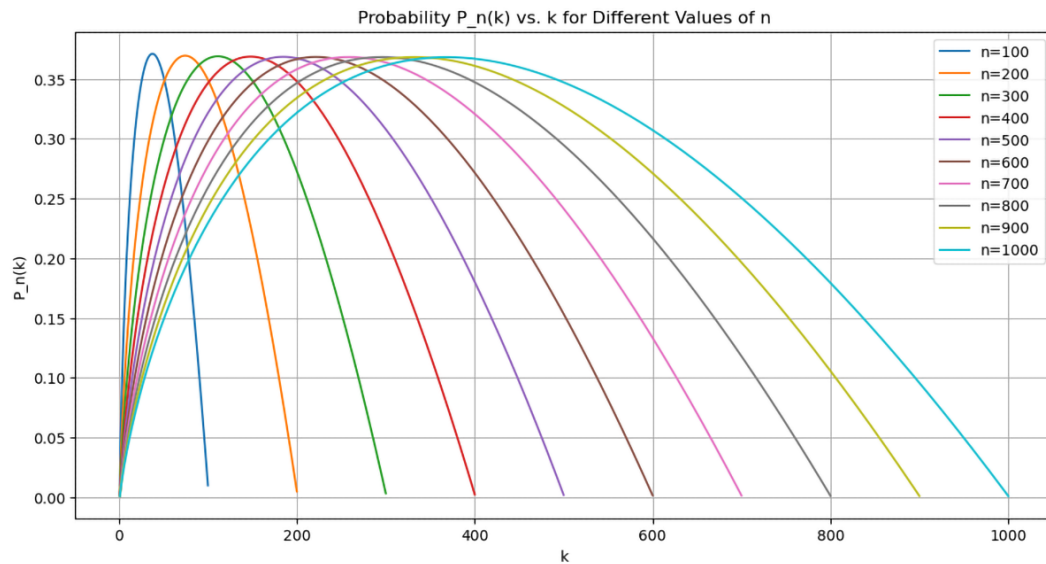


Figure 9.4: The probability $P_n(k)$ versus k for a various number of candidates (n)

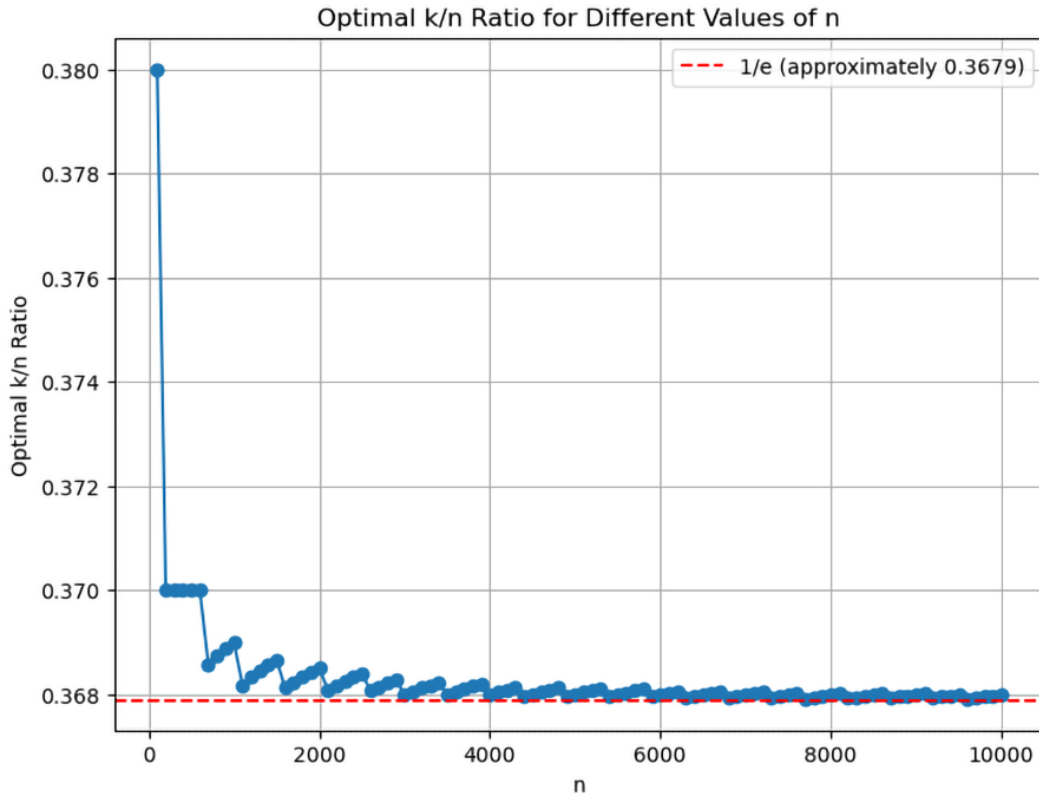


Figure 9.5: Optimal k/n ratio for various number of candidates (n). As n increases, the k/n ratio converges the golden ratio $1/e = 0.37$ or 37% of total number of candidates.

The preceding hiring problem is a classic example of optimal stopping theory, demonstrating the application of probabilistic decision-making in real-time scenarios. It has several interesting implications and extensions:

- **Unknown number of candidates:** If the number of candidates n is unknown, adaptive strategies can be developed
- **Multiple selections:** Variants where more than one candidate can be selected, adjusting the strategy accordingly

This theory has many practical applications. While the model is simplified, it provides insights into decision-making processes in hiring and other sequential selection scenarios (see next section). Similar strategies can be applied in online auctions, where bidders must decide when to stop bidding based on observed prices. The theory has some behavioral insights too. The problem highlights how optimal strategies often defy intuition, requiring a rigorous mathematical approach to identify the best course of action.

The hiring secretary problem elegantly combines elements of probability, decision theory, and optimal stopping. It provides a clear example of how structured strategies can significantly improve decision-making in scenarios characterized by uncertainty and sequential choices. By understanding the principles behind this problem, one gains valuable insights into the broader field of optimal stopping theory and its applications.

Case studies

At the beginning of this chapter, we introduced three problems faced by Tom and Fang. These problems involve randomized algorithms and probabilistic reasoning and can be solved using the optimal stopping theory. Essentially, these problems focus on determining when to stop searching rather than what to search for. As case studies, we will analyze and solve these problems in detail, applying the concepts we have learned in this chapter.

Optimal selection in an online dating app

A new online dating app, *Matcher*, has been designed to help users find their best possible partners. The app operates similarly to a game, where users are presented with one potential match at a time, selected randomly from a pool of N potential matches (the total number of matches is unknown to the users). Tom, our user, has a maximum of n likes available (where $n \ll N$). His goal is to maximize his chances of finding the best possible matches using his limited likes.

When Tom opens the *Matcher* app, he can either *like* (swipe right) or *dislike* (swipe left) each profile. Once a decision is made, it is final, and he cannot change his mind. Tom needs to carefully decide when to stop browsing and start using his likes to maximize his chances of selecting the best available matches. We should note the following:

- Tom must decide in real time whether to *like* or *dislike* each profile as they are presented sequentially
- Once Tom has *liked* n profiles, the app will no longer show him any more profiles
- The objective is to maximize the probability of selecting the best matches within the constraints of his n likes

There are some constraints:

- Tom cannot revisit previous profiles
- *Liking* a profile does not guarantee a match; it also requires the other person to like Tom's profile back

The optimal stopping theory provides a strategy to maximize the chances of selecting the best option from a sequence of choices. The strategy involves two phases: an observation phase and a selection phase:

- **Observation phase:** Tom should observe and reject the first k profiles to gather information about the pool of candidates. The optimal choice of k in classical secretary problems is approximately $\frac{n}{e}$. However, since N is unknown, Tom can use an adaptive strategy to estimate k .
- **Selection phase:** After the observation phase, Tom should start liking the next profile that is better than all the profiles he has seen so far during the observation phase. If Tom does not find a better profile, he will like the last few profiles he encounters, ensuring he uses all n likes.

In the absence of knowledge about the total number of profiles (N), one practical approach is to assume N as a large number, such as 1000. Tom can then use a strategy based on the optimal stopping theory. However, this approach has significant drawbacks when implemented in a real-world scenario.

Tom assumes N is large, for example, 1000:

- **Observation phase:** Reject the first 37% of the profiles. For $N = 1000$, this means rejecting the first 370 profiles.
- **Selection phase:** Start liking profiles from the 371st profile onward, only if they are better than all the profiles seen in the observation phase.

These are the drawbacks of the proposed strategy are twofold:

- **Human memory limitation:** It is impractical for Tom to remember and compare 370 profiles with every new profile he sees. This requires an immense cognitive load and is not feasible for most individuals.
- **Potential bias in ordered profiles:** If the profiles are ordered in any way (e.g., by the number of likes they have received), this strategy can fail. If the best profiles are at the beginning or end, the strategy won't work effectively because it relies on the randomness of profile order.

Given the impracticality of the preceding strategy, we can adopt a more practical approach that mitigates these issues. Instead of using a fixed 37% of a large assumed N , reduce the observation phase to a manageable number of profiles based on Tom's total number of likes (n). For example, use a smaller fraction such as $\frac{n}{2}$ for the observation phase. The other benefit of this practical approach is that it uses a relative comparison strategy where Tom only needs to remember the best profile seen so far rather than all profiles.

Let's explore the proposed practical approach. Tom can choose to observe the first k profiles where $k \approx \frac{n}{2}$. This is a heuristic that balances exploration and exploitation in the absence of information about N .

Let's assume Tom has 10 likes available. First, we choose k :

$$k = \frac{n}{2} = \frac{10}{2} = 5$$

Tom observes and rejects the first five profiles. From the sixth profile onward, Tom likes the next profile that is better than any of the first five profiles he observed. If none of the subsequent profiles are better, he uses his likes on the last profiles he sees.

By using this strategy, Tom maximizes his chances of finding high-quality matches within the constraints of his n likes. The heuristic approach balances the need to gather information and the need to make timely decisions without knowing the total number of candidates.

The revised optimal stopping strategy for Tom on the *Matcher* app involves using a fraction of his likes for the observation phase and then applying the gathered information to make decisions in the selection phase. This method provides a practical solution for sequential decision-making under uncertainty, ensuring Tom uses his likes effectively to maximize his chances of finding the best matches.

Finding the closest parking spot

Fang is driving to her friend's house on a busy one-way street with n parking spots, where parking is only allowed on one side of the street. Based on past experience, about 10% of the parking spots are typically available at any given time. Fang can only see one spot at a time as she passes by and cannot see the spots ahead. The goal is to determine the optimal moment for her to take an available parking spot using the optimal stopping theory. Figure 9.6 illustrates this problem. In the figure, the left side of the street shows where cars can park, with some available spots highlighted. The right side of the street is a no-parking zone.

The optimal stopping theory helps Fang decide when to stop searching and take an available parking spot by balancing the exploration of spots and the exploitation of the best spot found so far.

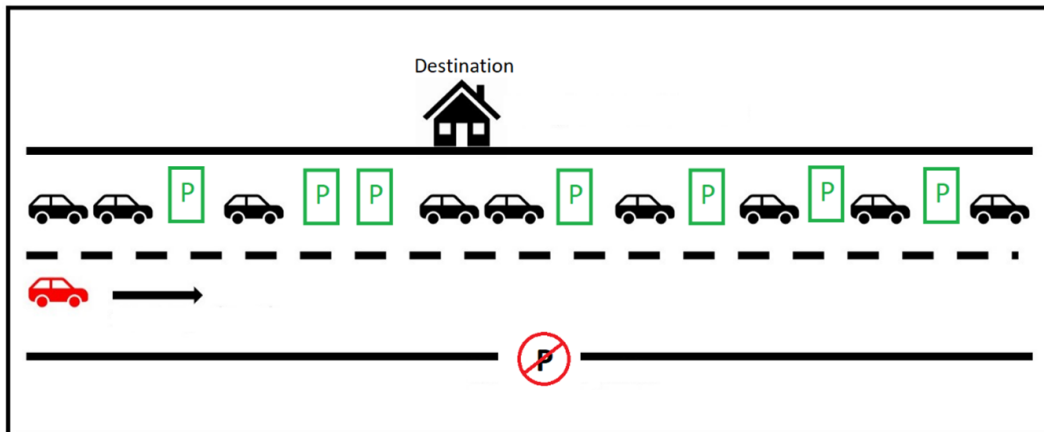


Figure 9.6: Illustration of Fang's parking problem

In both the hiring and dating app problems, there were implicit and subjective criteria to rank the candidates. In contrast, the parking problem has an explicit criterion: the distance from the parking spot to Fang's friend's house. Therefore, our strategy highly depends on the location of the destination. We can identify three scenarios:

- **Destination at the beginning of the street:** This is the easiest case because Fang should choose the first available parking spot she encounters. There is no need to apply the optimal stopping theory here, as this represents the best-case scenario.
- **Destination at the end of the street:** This is the worst-case scenario. Fang needs to find the closest available parking spot, so she should reject as many spots as possible to maximize her chances of finding a spot near the end. The optimal stopping theory is particularly useful in this situation.
- **Destination in the middle of the street:** This represents an average-case scenario, assuming the destination is exactly in the middle of the street. Here, the optimal stopping theory is also applicable to help Fang decide when to stop and take an available parking spot. We solve the problem for this case.

The average-case scenario can be approached in two parts:

- **Before reaching the destination:** This part is similar to the worst-case scenario. Fang will try to reject as many spots as possible using the optimal stopping theory. She should reject the first k spots and then select the $(k + 1)^{th}$ spot that is available.
- **After passing the destination:** If Fang cannot find any available spot before reaching her destination, the strategy changes. At this point, the problem turns into the best-case scenario. Fang should park at the very first available spot she encounters after passing the destination.

We can solve this problem for the average-case scenario step by step (hereafter, the destination is referred to as the midpoint):

- **Determine k :** For the optimal stopping strategy, we calculate $k \approx \frac{n}{e}$. However, given that only about 10% of all parking spots are typically available at any time, we need to adjust our calculations accordingly. Fang can estimate the total number of cars that can be parked on the street by considering the length of the street divided by the average length of a sedan car. Since her destination is in the middle of the street, we multiply the availability by 0.5. If her destination were, for example, in the first third of the street, this coefficient would be 0.33. Thus, we adjust k as follows:

$$k \approx \frac{5n}{100 \times e} = \frac{n}{54.3}$$

The following represents the number of spots Fang will pass before she starts considering parking seriously:

- **Before the midpoint:** Fang should drive past the first k spots without parking. After passing k spots, Fang will then park in the next available spot that is better than any of the spots she has seen so far.
- **After the midpoint:** If Fang has not found a suitable spot by the time she reaches the midpoint, she will park at the very first available spot she encounters.

This approach divides the problem into manageable parts and applies the optimal stopping theory effectively within each part. Before the midpoint, by rejecting the first k spots, Fang gathers enough information to make an informed decision based on the quality of the spots. This strategy maximizes her chances of finding a better spot close to her destination. However, after the midpoint, we switch to the best-case scenario approach, ensuring that Fang minimizes her walking distance to her friend's house if she has not found a spot before the midpoint.

Summary

In this chapter, we explored various problems that involved making optimal decisions under uncertainty using the optimal stopping theorem and randomized algorithms. We examined scenarios such as the hiring problem, the *Matcher* dating app, and Fang's parking problem, each requiring a strategic balance between gathering information and making timely decisions. Through these examples, we illustrated how the optimal stopping theorem provides a structured approach to maximize the chances of selecting the best option by setting appropriate observation phases and selection criteria. This chapter demonstrated the power of probabilistic reasoning and optimal stopping rules in practical decision-making scenarios. In the next chapter, we will explore dynamic programming, a powerful technique for solving complex problems by breaking them down into simpler subproblems.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022:
 - *Chapter 5, Probabilistic Analysis and Randomized Algorithms*
- *Algorithms to Live By: The Computer Science of Human Decisions*. By Brian Christian and Tom Griffiths. Henry Holt and Co. 2016.
- *Who Solved the Secretary Problem*. By T. S. Ferguson. *Statist. Sci.* 4 (3): 282–89.
- *Monty Hall Problem*. https://en.wikipedia.org/wiki/Monty_Hall_problem.

Dynamic Programming

Dynamic programming is a powerful technique that can dramatically lower the computational costs of many complex algorithms, though it comes with some trade-offs. This chapter introduces dynamic programming and includes a review of greedy algorithms.

We will begin by revisiting the principles of divide-and-conquer strategies to contrast them with dynamic programming. Dynamic programming stands out in algorithm design due to its ability to solve problems that involve overlapping subproblems and optimal substructure. By storing the results of these subproblems, dynamic programming prevents redundant calculations, leading to significant efficiency gains.

Through various examples, we will explore how dynamic programming can be applied to solve classical problems such as the knapsack problem, the longest common subsequence, and the Traveling Salesman Problem. Each example will illustrate the step-by-step approach to breaking down a problem, defining the state space, and formulating the recurrence relations necessary for the dynamic programming solution.

It is important to note that dynamic programming is an extensive topic that cannot be fully covered in just one chapter. In this chapter, we will focus on the key aspects of this method, including its main elements, applications, and illustrative examples. Additionally, we will discuss how to estimate the complexities of dynamic programming solutions, and we will explore the advantages and limitations of this approach.

Specifically, we'll cover all this in the following main sections:

- Dynamic programming versus divide-and-conquer
- Exploring dynamic programming
- Greedy algorithms – an introduction

Dynamic programming versus divide-and-conquer

In *Chapters 4* and *5*, we explored recursive algorithms and the methods to analyze their complexities. We also explored divide-and-conquer strategies. The fundamental idea behind a divide-and-conquer approach is to break down a problem into smaller subproblems, solve these subproblems optimally, and then combine their solutions to form the final solution. This process is typically carried out recursively, meaning that the problem is continuously divided into subproblems until we reach a point where the subproblem is so small that it can be solved intuitively or straightforwardly. This smallest, simplest problem is referred to as the **base case**.

Dynamic programming follows a similar strategy to divide-and-conquer. It breaks down a problem into subproblems with the explicit assumption that the optimal solution to any subproblem will contribute to the final optimal solution. This characteristic is known as the **optimal substructure**. While this property is also present in divide-and-conquer algorithms, it is typically assumed implicitly rather than explicitly stated.

However, dynamic programming extends beyond the divide-and-conquer approach in a key way that sets it apart. In dynamic programming, subproblems often share common sub-subproblems, meaning that there is overlap between the subproblems. This characteristic is known as **overlapping subproblems**. Not all problems exhibit this behavior. When a problem does not have overlapping subproblems, using a straightforward divide-and-conquer approach is more appropriate, as dynamic programming would not provide any additional benefit in such cases.

But why do overlapping subproblems matter? One of the primary goals in algorithm design and analysis, which we have consistently focused on, is reducing the computational cost or complexity of algorithms. The importance of overlapping subproblems lies in their potential to significantly lower this computational complexity. In problems where subproblems overlap, divide-and-conquer algorithms may redundantly solve these overlapping sub-subproblems multiple times. In contrast, dynamic programming stores the solutions to these sub-subproblems and reuses them whenever they are needed. This reuse of previously computed solutions can lead to a substantial reduction in computational complexity. However, this efficiency comes at a cost: the additional space required to store the solutions for these sub-subproblems.

Now that we understand the connection between divide-and-conquer and dynamic programming, let's discuss dynamic programming itself. First, it's crucial to clarify that the term dynamic programming does not pertain to writing code or computer programming. Instead, it refers to a method of mathematical optimization. Dynamic programming was originally proposed as a technique for solving optimization problems, which is why it is studied in both mathematical optimization and computer science. In this chapter, our focus will be on the application of dynamic programming within the field of computer science.

As we previously discussed, dynamic programming is built on two fundamental concepts: the assumption of optimal substructure and the presence of overlapping subproblems. Optimal substructure implies that the optimal solution to a problem can be formed from the optimal solutions of its subproblems. Overlapping subproblems occur when the same subproblems are solved multiple times during the recursive process. In the following subsections, we will explore these concepts in greater detail, examining how they serve as the foundation of dynamic programming and how they facilitate the development of efficient algorithms for tackling complex problems.

Optimal substructure

Optimal substructure is a cornerstone concept in algorithm design, signifying that an optimal solution to a given problem can be built efficiently using the optimal solutions of its smaller subproblems. This property suggests that the problem can be decomposed into simpler, overlapping subproblems, each of which can be solved independently. Once the subproblems are resolved optimally, their solutions can be combined to form the solution to the original, larger problem.

The presence of optimal substructure is crucial because it allows us to break down complex problems into more manageable components. This decomposition not only simplifies the problem-solving process but also makes it possible to apply algorithmic strategies such as dynamic programming and divide-and-conquer. In dynamic programming, the optimal substructure property ensures that once we have the optimal solutions for all subproblems, we can use them to systematically build the optimal solution for the entire problem. This is achieved by storing the results of subproblems in a table, preventing the need to recompute them and thus enhancing the algorithm's efficiency.

In divide-and-conquer algorithms, although in a different way, the optimal substructure is also vital. These algorithms divide the problem into non-overlapping subproblems, solve each subproblem independently, and then combine their solutions. The success of both dynamic programming and divide-and-conquer approaches hinges on the problem exhibiting optimal substructure, as it guarantees that solving smaller parts optimally will lead to the best possible solution for the overall problem.

Optimal substructure is not only a theoretical concept but also a practical tool in algorithm design. It helps us recognize when a problem can be efficiently tackled using dynamic programming or other strategies that rely on building up solutions from smaller components. Understanding and identifying optimal substructure in various problems is a key skill in developing efficient and effective algorithms. Next, we will explore this concept with some example problems.

The first example is the shortest path problem. In the shortest path problem on a graph, the goal is to find the shortest path from a source node (vertex) to a destination node. This problem exhibits optimal substructure because the shortest path between two nodes can be broken down into smaller shortest paths between intermediate nodes. Let's assume **S** is the source node and **G** is the destination. We aim to find the shortest path between these two nodes. Suppose **A** is a node on the optimal (shortest) path from **S** to **G**. The optimal substructure characteristic suggests that because **A** is part of the optimal path, the overall optimal path from **S** to **G** must consist of the optimal path from **S** to **A**, followed by the optimal path from **A** to **G**. Optimal substructure also implies that any subpath that is not optimal

cannot be part of the overall optimal solution. This means that if a subpath from **S** to **A** or from **A** to **G** is not the shortest, then the entire path from **S** to **G** cannot be the shortest (see *Figure 10.1*).

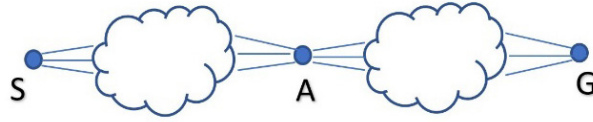


Fig 10.1: Illustrating optimal substructure in shortest path problem.

The second example is finding the **Longest Common Subsequence (LCS)**. The problem involves identifying the longest subsequence that can be found in a set of sequences. It is essential to understand that a subsequence differs from a substring. In a subsequence, unlike in a substring, the elements do not need to be in consecutive positions within the sequence. This distinction is crucial in many applications across computer science, such as in data compression, sequence alignment, and file comparison.

Optimal substructure in the LCS problem means that the solution to finding the LCS of two sequences can be constructed from the solutions to smaller subproblems. Specifically, the LCS of two sequences X and Y can be built by considering the LCS of prefixes of these sequences.

Let's assume we have two sequences X of length m and Y of length n . The LCS of these two sequences denoted as $\text{LCS}(X, Y)$, can be determined using the following approach: if either X or Y is an empty sequence, then the LCS is also an empty sequence. This is the base case for our recursion. If the last characters of X and Y match (say $X[m] = Y[n]$), then these characters must be part of the LCS. Therefore, the problem reduces to finding the LCS of the prefixes $X[1 \dots m-1]$ and $Y[1 \dots n-1]$, and then appending the matching character to this LCS.

However, if the last characters of X and Y do not match (i.e., $X[m] \neq Y[n]$), then the LCS is the longer of the LCS obtained by either of the following methods:

- Excluding the last character of X and considering the LCS of $X[1 \dots m-1]$ and $Y[1 \dots n]$
- Excluding the last character of Y and considering the LCS of $X[1 \dots m]$ and $Y[1 \dots n-1]$

Let $L(i, j)$ represent the LCS of the prefixes $X[1 \dots i]$ and $Y[1 \dots j]$. Then the optimal substructure property can be expressed as follows:

$$L(i, j) = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ L(i-1, j-1) + 1 & \text{if } X[i] = Y[j] \\ \max(L(i-1, j), L(i, j-1)) & \text{if } X[i] \neq Y[j] \end{cases}$$

Example 10.1:

Consider the following sequences:

$X = \text{abbbbsbbsh}$

$Y = \text{bbsdjhjs}$

If the last characters of both sequences are the same (here, both end in h), then h is part of the LCS, and the problem reduces to finding the LCS of $X = \text{abbbbsbbs}$ and $Y = \text{bbsdjhjs}$. If the last characters are different, we must consider the LCS of the smaller sequences $X = \text{abbbbsbbsh}$ and $Y = \text{bbsdjhjs}$, or $X = \text{abbbbsbbs}$ and $Y = \text{bbsdjhsh}$, and take the longer one.

The optimal substructure in the LCS problem ensures that the overall LCS can be constructed from the LCS of smaller subproblems. This property is crucial for dynamic programming solutions, where we systematically solve and store the results of these subproblems to build the final solution efficiently.

Not all problems are optimal substructure problems. One example is the longest path problem. The longest path problem, particularly in a graph where cycles may exist, is not considered to have the optimal substructure property. To understand why, let's first recall what optimal substructure means: a problem exhibits optimal substructure if the optimal solution to the problem can be constructed from the optimal solutions of its subproblems.

In the longest path problem, the goal is to find the longest simple path (a path that does not repeat nodes) between two nodes in a graph. This problem is particularly challenging when the graph contains cycles, as the presence of cycles can complicate the identification of the longest path. In the longest path problem, the optimal substructure property does not hold due to the following reasons:

- **Dependency on subsequent paths:** In many cases, a subpath that appears to be part of the longest path from one node to another might not lead to the overall longest path when extended to other nodes. This is because choosing a subpath that looks long at first might force you to take shorter paths later, which would reduce the overall path length.
- **Cycle involvement:** If the graph contains cycles, the longest path might involve traversing through parts of the graph in such a way that it doesn't neatly decompose into subproblems that independently contribute to the overall longest path. The decision to include or exclude certain edges can dramatically change the resulting path's length.
- **Non-additive nature:** In problems with optimal substructure, you can usually solve subproblems independently and then combine them to get the optimal solution. However, in the longest path problem, solving a subproblem optimally (i.e., finding the longest path from one node to another) does not guarantee that this subpath will be part of the overall longest path in the entire graph. For example, if you have found the longest path from vertex v_a to vertex v_b and then from vertex v_b to vertex v_c , the combination of these paths might not yield the longest path from v_a to v_c . The globally optimal solution might instead bypass v_b entirely if a longer alternative path is available.

Example 10.2:

Consider a weighted graph where the nodes are A , B , C , and D , and there are edges $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, and $A \rightarrow D$. The longest path from A to D could initially seem like it might go through B and C . However, if the direct edge $A \rightarrow D$ is longer than the sum of the paths $A \rightarrow B \rightarrow C \rightarrow D$, then the optimal solution doesn't involve the longest paths from A to B , B to C and C to D .

Obviously, the graph is directed and weighted. If the graph were undirected and unweighted, this scenario would be infeasible because it would violate the triangular inequality, which states that the direct distance between two points should be less than or equal to the sum of the distances through an intermediate point. However, even in a directed, unweighted graph like this, the result can be counterintuitive.

Let's assume that the nodes A , B , C , and D represent four cities: Toronto, Chicago, Denver, and Los Angeles, respectively. The edges represent the ticket prices of direct flights. For example, $A \rightarrow B$ represents the ticket price of a direct flight from Toronto to Chicago.

As depicted in *Figure 10.2*, the longest path from Toronto to Los Angeles is achieved through a direct flight (represented by the edge *Toronto* $\rightarrow$ *Los Angeles* with a weight of \$957) and does not include any stops in Chicago or Denver. This outcome might seem counterintuitive, as one might expect a longer path to involve more stops, but in this scenario, the direct flight is the most expensive and therefore forms the longest path in terms of cost.

It's also worth mentioning that in the absence of cycles in the graph, as in our simple example, one straightforward algorithm to solve the longest path problem is to negate all weights and then solve it as the shortest path problem. This transformation works because finding the shortest path in a graph with negated weights is equivalent to finding the longest path in the original graph.

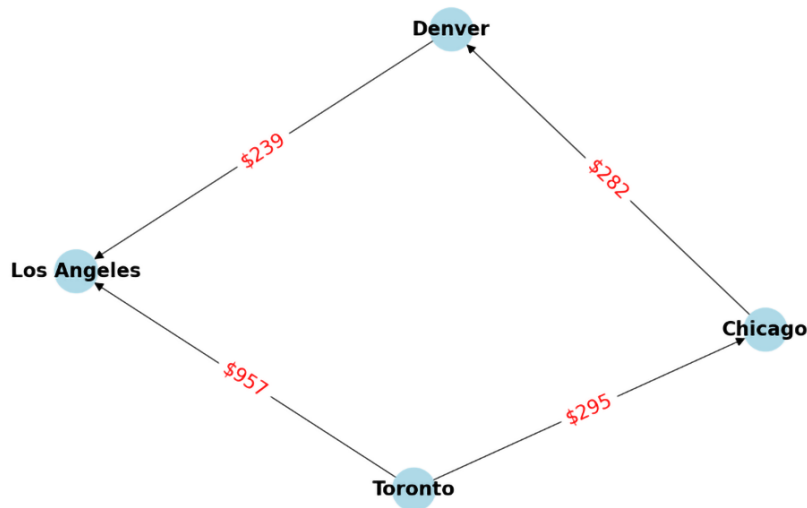


Figure 10.2: An example of the longest path problem

The lack of optimal substructure in the longest path problem means that dynamic programming and similar techniques that rely on breaking a problem down into subproblems do not work effectively. This is because the longest path problem requires global knowledge of the graph's structure and cannot simply be built up from smaller, independently solved subproblems. As a result, the longest path problem is typically more challenging to solve, especially in graphs that contain cycles and is not suited to approaches such as dynamic programming that rely on an optimal substructure.

Overlapping subproblems

The second element of dynamic programming that distinguishes it from divide-and-conquer algorithms is the concept of overlapping subproblems. In a well-designed divide-and-conquer strategy, each recursive step typically solves a new, distinct subproblem. A good example of this is merge sort, where, during each partitioning step, the partitions do not overlap, and each subproblem is independent of the others.

However, there are problems where certain subproblems overlap, meaning the same subproblems are solved multiple times in the recursive process. This is where dynamic programming excels by avoiding redundant computations. To illustrate this, let's consider the Fibonacci series as an example.

The Fibonacci series is defined as follows:

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2) \text{ for } n \geq 2$$

When calculating Fibonacci numbers using a divide-and-conquer approach, the same subproblems are solved repeatedly. For instance, to compute $F(5)$, the algorithm would need to calculate $F(4)$ and $F(3)$. But to compute $F(4)$, the algorithm again computes $F(3)$ and $F(2)$, meaning $F(3)$ is calculated multiple times. One way to avoid this redundancy is to store the results of previously computed subproblems in data structures with near-constant access time, such as a hash table.

This redundancy leads to an exponential time complexity, as the number of recursive calls grows exponentially with n . This is where dynamic programming provides a significant advantage. Instead of solving the same subproblems repeatedly, dynamic programming stores the results of these subproblems and reuses them when needed, drastically reducing the number of computations.

By leveraging overlapping subproblems, dynamic programming transforms the recursive solution for the Fibonacci series from an exponential time complexity $O(2^n)$ to a linear time complexity $O(n)$. This efficiency gain is why dynamic programming is preferred over divide-and-conquer approaches in problems where overlapping subproblems exist. In the next section, we will explore dynamic programming in more detail by examining several examples.

Exploring dynamic programming

One question that arises when we identify overlapping subproblems is: how can we store these solutions to avoid redundant computations? This is where dynamic programming introduces the concept of **memoization**. It involves storing the results of subproblems in a data structure, such as an array or a dictionary, so that when the same subproblem arises again, the stored result can be used immediately, eliminating the need for recalculation.

Before we begin, it's important to clarify that “memoization” is not a misspelling of “memorization.” These two terms have distinct meanings. Memoization comes from the Latin word “memo,” which means “to be remembered.” It refers to a technique in computer science where the results of expensive function calls are stored and reused to avoid redundant calculations. In contrast, memorization refers to the process of learning and committing information to memory, often through repetition.

Memoization is a technique used in dynamic programming to optimize recursive algorithms by storing the results of expensive function calls and reusing them when the same inputs occur again. Instead of recomputing the same result multiple times, memoization saves the result of each subproblem the first time it is solved and then returns the stored result whenever the same subproblem is encountered again. This significantly reduces the time complexity of algorithms that have overlapping subproblems.

Let's explore how memoization works. In a typical recursive algorithm, a function repeatedly calls itself with smaller inputs until it reaches a base case. Without memoization, the function might be called with the same inputs multiple times, resulting in redundant calculations. This inefficiency is common in straightforward recursive algorithms that use divide-and-conquer strategies. By storing the results of these repeated subproblems, memoization eliminates the need for these redundant calculations, making the algorithm more efficient.

With memoization, when a function is called, the algorithm first checks whether the result for the given input is already stored. If it is, the stored result is returned immediately, avoiding redundant computation. The results of subproblems are stored in a data structure, typically a dictionary or an array, which acts as a lookup table. This table is often referred to as a **memo**.

Example 10.3:

Let's revisit the Fibonacci series example to illustrate memoization and compare it to the approach without memoization.

The Fibonacci sequence is defined as follows:

$$F(0) = 0$$

$$F(1) = 1$$

$$F(n) = F(n - 1) + F(n - 2) \text{ for } n \geq 2$$

Without memoization

The following is a simple Python implementation of the Fibonacci sequence using a naive recursive approach:

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)
n = 10
print(f"Fibonacci number F({n}) is: {fib(n)}")
```

This approach has an exponential time complexity $O(2^n)$ because it recalculates the same Fibonacci numbers multiple times.

First, let's analyze the complexity of the recursive Fibonacci series algorithm. This algorithm divides the problem into two subproblems, where the sizes of the subproblems are reduced by one and two, respectively. The recursive part has two components, while the driving function, which computes the sum of the two previous Fibonacci numbers, operates in constant time ($f(n) = O(1) = c$).

The recurrence function for this algorithm can be expressed as follows:

$$T(n) = T(n-1) + T(n-2) + O(1)$$

This recurrence does not fit neatly into the standard forms for divide-and-conquer or subtracting recurrence functions. Therefore, we simplify it as follows:

$$T(n) = T(n-1) + T(n-2) + c \approx 2(T(n-1) + c)$$

With this simplification, the recurrence function resembles a form that can be analyzed using the Master Theorem for subtracting recurrences (see *Chapter 5*). According to the Master Theorem, since $a = 2$ and $b = 1$, the time complexity of the recursive Fibonacci sequence algorithm is this:

$$T(n) = \Theta(a^{n/b} \cdot f(n)) = \Theta(2^n)$$

This result shows that the recursive Fibonacci algorithm has exponential time complexity, specifically $\Theta(2^n)$, making it inefficient for large values of n . This inefficiency is why memoization or dynamic programming is often used to optimize the computation of Fibonacci numbers.

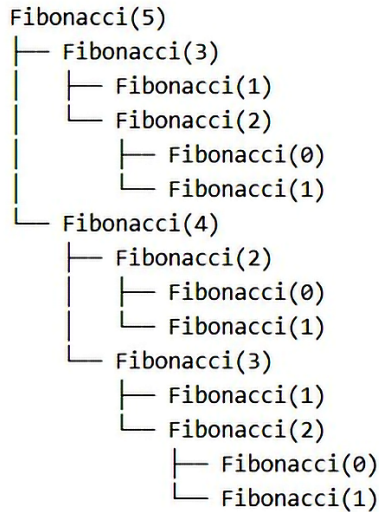


Figure 10.3: The tree demonstrating the non-memoized approach to calculating Fibonacci(5)

Figure 10.3 illustrates an example of the tree representing a divide-and-conquer (non-memoized) implementation to calculate Fibonacci(5). As shown, the leaf nodes repeatedly consist of Fibonacci(1) and Fibonacci(0). Additionally, many Fibonacci numbers are recalculated multiple times throughout the process.

With memoization

By applying memoization, the algorithm avoids redundant calculations. Here is a simple Python implementation of the Fibonacci sequence using dynamic programming, which uses memoization to optimize the calculation:

```
def dp_fib(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = dp_fib(n-1, memo) + dp_fib(n-2, memo)
    return memo[n]

n = 10
print(f"Fibonacci number F({n}) is: {dp_fib(n)}")
```

In this version, the time complexity is reduced to $O(n)$, because each Fibonacci number is computed only once and then stored for future reference.

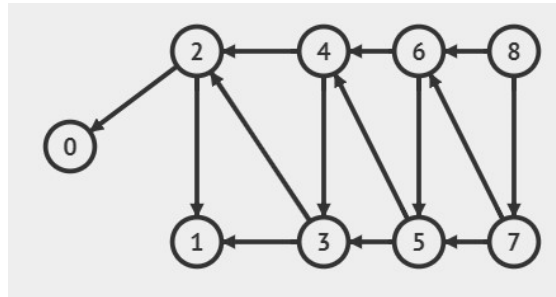


Figure 10.4: The graph demonstrating the memoized approach to calculating Fibonacci(8)

Figure 10.4 illustrates the graph representing the dynamic programming approach (using memoization) to calculate Fibonacci(8). In this approach, no Fibonacci number is redundantly recalculated. Instead, previously solved subproblems are reused, which is reflected in the edges extending from nodes. This reuse of subproblems is why we visualize divide-and-conquer algorithms with trees (Figure 10.3), while dynamic programming is better represented with graphs (Figure 10.4).

Memoization is crucial in dynamic programming, especially for problems with overlapping subproblems. By storing the results of these subproblems, memoization minimizes the number of calculations required to solve the overall problem, leading to significant improvements in time complexity. However, this benefit comes at the cost of additional space for storing these results. This tradeoff is usually favorable in scenarios where time efficiency is more critical. It's important to note that memoization is typically employed in a top-down dynamic programming approach, where the problem is solved recursively, and results are stored as they are computed. In the upcoming subsection, we briefly discuss top-down and bottom-up dynamic programming.

Top-down versus bottom-up approaches for dynamic programming

There are two main approaches to implementing dynamic programming: top-down and bottom-up. Both approaches aim to reduce computational complexity by avoiding redundant calculations, but they do so in different ways.

Top-down dynamic programming, also known as memoization, is a recursive approach. It starts solving the problem from the top, breaking it down into smaller subproblems. As the algorithm solves these subproblems, it stores its results in a data structure (usually a dictionary or an array) so that if the same subproblem needs to be solved again, the stored result can be used directly instead of recalculating it.

The top-down process begins with the original problem and recursively breaks it down into smaller subproblems. Each time a subproblem is solved, its result is stored (memoized). If a subproblem is encountered again, the algorithm retrieves the stored result instead of recomputing it.

The top-down approach is relatively straightforward to implement, particularly when adapting from a divide-and-conquer recursive solution. In top-down dynamic programming, only the subproblems essential to solving the original problem are addressed, which can be advantageous if certain subproblems aren't needed. However, this approach does have its drawbacks. It may require more memory due to the need to store recursive function calls on the call stack. Additionally, deep recursion can result in a stack overflow when dealing with very large problems. *Example 10.3* illustrates the memoized version of the Fibonacci sequence calculation, serving as a classic example of top-down dynamic programming.

The alternative to the top-down approach is the bottom-up implementation of dynamic programming. Bottom-up dynamic programming, also known as tabulation, is an iterative method. It begins by solving the smallest subproblems first and then uses those solutions to build up to the solution of the original problem. In this approach, a table (usually an array) is created, with each entry representing the solution to a subproblem. The process starts by addressing the base cases and solving the smallest subproblems first. The solutions are stored in the table and then iteratively used to tackle larger subproblems, ultimately leading to the final solution.

The bottom-up approach is generally more space-efficient than the top-down method because it avoids the overhead associated with recursive function calls. Additionally, it eliminates the risk of stack overflow by using an iterative process instead of recursion. This approach often provides a clearer view of how the solution is constructed, making it easier to manage and understand. However, the bottom-up approach has two significant drawbacks. First, it requires solving all subproblems, even those that may not be necessary for the final solution. Additionally, it can be more complex to implement initially, particularly when transitioning from a recursive approach.

The following is a Python implementation of bottom-up approach to the Fibonacci sequence:

```
def bottom_up_fib(n):
    if n <= 1:
        return n
    fib = [0] * (n+1)
    fib[1] = 1
    for i in range(2, n+1):
        fib[i] = fib[i-1] + fib[i-2]
    return fib[n]
n = 10
print(f"Fibonacci number F({n}) is: {bottom_up_fib(n)}")
```

Table 10.1 provides a summary of the characteristics of both the top-down and bottom-up approaches in dynamic programming.

Feature	Top-down (memoization)	Bottom-up (tabulation)
Approach	Recursive, solves the problem from the top (main problem) and breaks it down into subproblems.	Iterative, solves the problem from the bottom (base cases) and builds up to the main problem.
Space complexity	May use more memory due to the call stack, and memoization storage.	Typically uses less memory, as it mainly relies on a table or array.
Time complexity	Avoids re-computation by storing results of subproblems, generally $O(n)$.	Also avoids re-computation, generally $O(n)$.
Ease of implementation	Easier to implement if transitioning from a naive recursive solution.	It might be harder to implement initially, especially if converting from a recursive approach.
Risk of stack overflow	Higher risk due to deep recursion.	No risk, as it avoids recursion altogether.

Table 10.1: A comparison of top-down and bottom-up approaches

Both top-down and bottom-up approaches to dynamic programming achieve the same goal: optimizing the computation by avoiding redundant calculations. The choice between the two often depends on the specific problem, the programmer's comfort with recursion or iteration, and the memory and performance constraints of the task at hand. *Table 10.1* presents a comparison of the top-down and bottom-up approaches in dynamic programming.

Let's take a moment to discuss how to calculate the time complexity of dynamic programming implementations. As we've noted, dynamic programming algorithms are better represented by directed graphs rather than trees. In *Chapter 5*, we explored the use of recursion trees to estimate the time complexity of recursive algorithms. Here, instead of using trees, we'll utilize a graph to visualize a dynamic programming algorithm.

In a dynamic programming graph, nodes represent subproblems, and the edges emerging from these nodes represent the choices we encounter within each subproblem. We can interpret this from the perspective of the destination of the edges, given that the graph is directed. Each incoming edge to a node signifies a function call—this means the subproblem is being recalled rather than recalculated, as would happen in a typical divide-and-conquer approach.

Now, let's assume there are m subproblems (or functions) in total, and each subproblem is recalled r_i times, where $1 \leq i \leq m$. The time complexity of this dynamic programming algorithm can be expressed as follows:

$$T(n) = O\left(\sum_{i=1}^m i \cdot r_i\right), 1 \leq i \leq m$$

This approach typically results in a linear time complexity, particularly in cases where the number of subproblems m is proportional to the size of the input n . In the worst-case scenario, where $m = n$, each subproblem corresponds directly to a unique state of the input, and the algorithm must process each state exactly once.

When this occurs, the dynamic programming algorithm efficiently computes the solution in $O(n)$ time. This is because the algorithm leverages the overlapping subproblems by storing their results and reusing them as needed, rather than recalculating them. As a result, the total number of operations is linearly related to the size of the input, avoiding the exponential time complexity that would occur if each subproblem were recomputed multiple times, as in naive recursive approaches.

This linear time complexity is one of the key advantages of dynamic programming. It allows the algorithm to scale effectively with the size of the input, making it practical for solving large and complex problems that would otherwise be computationally infeasible using brute-force or divide-and-conquer strategies.

Let's apply the approach discussed earlier to analyze the time complexity of the dynamic programming implementation of the Fibonacci sequence, as shown in *Example 10.3*. By considering *Figure 10.4*, which illustrates the graph representing the problem of calculating the Fibonacci of number 8 ($F(8)$), we can derive the time complexity.

In the graph, each node corresponds to a subproblem—specifically, one of the Fibonacci numbers that needs to be computed. The graph for $F(8)$ includes 8 nodes, each representing an overlapping subproblem. Additionally, each node in this graph has two choices: one corresponding to the function call for the previous Fibonacci number ($F(n - 1)$) and the other for the Fibonacci number before that ($F(n - 2)$). These choices are depicted as outgoing edges from each node, or equivalently, two incoming edges to each node in the graph.

To calculate the time complexity, we observe that the graph structure results in every node (or subproblem) being processed once, with each node involving two operations: one for each of the outgoing edges. This results in a total of 8×2 operations for $F(8)$.

Generalizing this to $F(n)$, we observe that for any Fibonacci number n , the total number of operations can be expressed as follows:

$$T(n) = O(2 \times n) = O(n)$$

This formula shows that the dynamic programming approach to computing Fibonacci numbers of scales linearly with the input size n . The efficiency is achieved because dynamic programming avoids redundant calculations by storing and reusing the results of subproblems (memoization), leading to a significant reduction in time complexity compared to the naive recursive approach, which has an exponential time complexity.

Let's look at another classic example in the context of dynamic programming, which is the **knapsack problem**.

Solving the 0/1 knapsack problem using dynamic programming

The knapsack problem is a classic optimization problem where the goal is to determine the maximum value that can be obtained by selecting items to include in a knapsack, subject to a weight capacity constraint. Each item has a specific weight and value, and the challenge is to choose the optimal combination of items to maximize the total value without exceeding the knapsack's weight limit. Let's assume we have a set of n items, each with a weight w_i and a value v_i and a knapsack with a maximum weight capacity W . The objective is to maximize the total value V of the items in the knapsack such that the following holds:

$$V = \max \sum_{i=1}^n v_i \times x_i$$

subject to:

$$\sum_{i=1}^n w_i \times x_i \leq W$$

Depending on how we define x_i , we encounter the following types of knapsack problem:

- If $x_i \in \{0,1\}$, the problem is known as the 0/1 knapsack problem. This is the problem we will focus on solving in this chapter.
- If $x_i \in \{0,1,2,\dots,c\}$, where c is a constant, the problem is referred to as the **Bounded Knapsack Problem (BKP)**.
- If x_i can be any non-negative integer, the problem is called the **Unbounded Knapsack Problem (UKP)**.

First, let's examine the complexity of the 0/1 knapsack problem using a naïve approach. Given n items, a straightforward solution would involve generating all possible subsets of these n items, calculating the corresponding weight and value for each subset, and then identifying the optimal solution. This approach results in a time complexity of $O(2^n)$, making the problem non-polynomial and computationally infeasible for large values of n .

The goal is to solve the 0/1 knapsack problem using a dynamic programming approach. The knapsack problem exhibits both optimal substructure and overlapping subproblems, making it an ideal candidate for a dynamic programming solution.

Optimal substructure means that the optimal solution to a problem can consider a knapsack problem with n items and capacity W . Let's assume we have an optimal solution for this problem. If we remove the last item from this optimal solution, the remaining items must form an optimal solution for a knapsack problem with $n - 1$ items and capacity W minus the weight of the removed item.

In other words, the optimal solution to the larger problem (n items, capacity W) is composed of an optimal solution to a smaller subproblem ($n - 1$ items, reduced capacity) plus the decision of whether to include the n^{th} item or not. This property allows us to break down the problem into smaller subproblems and solve them recursively, which is the foundation of the dynamic programming approach.

For example, if we have an optimal solution for a knapsack problem with 5 items and a capacity of 12, and we remove the fifth item, the remaining 4 items must form an optimal solution for a knapsack problem with 4 items and a capacity reduced by the weight of the fifth item.

Overlapping subproblems in this problem refer to the situation where the same subproblems are solved multiple times during the process of finding the optimal solution. When solving the 0/1 knapsack problem using a recursive approach, we often find that the same subproblems are revisited. Specifically, when deciding whether to include or exclude an item from the knapsack, we are essentially solving the same problem but with reduced capacity or fewer items.

Consider a simple example with three items, and a knapsack capacity $W = 5$. Each item has a weight w_i and a value v_i :

- Item 1: $w_1 = 2, v_1 = 3$
- Item 2: $w_2 = 3, v_2 = 4$
- Item 3: $w_3 = 4, v_3 = 5$

When solving this problem recursively, we might face the following subproblems:

- Including Item 1: This leads to solving the subproblem with the remaining capacity $W = 3$ and the remaining items (Item 2 and Item 3)
- Excluding Item 1: This leads to solving the subproblem with the full capacity $W = 5$ and the remaining items (Item 2 and Item 3)

However, when we move on to consider Item 2, you will again encounter the same subproblems:

- Including Item 2: Solves the problem with a capacity of $W = 2$ (if Item 1 was included) or $W = 3$ (if Item 1 was excluded) and the remaining item (Item 3)
- Excluding Item 2: Solves the problem with the current capacity and just Item 3 remaining

As we can see, some subproblems (e.g., the subproblem with $W = 3$ and Item 3) are encountered multiple times. Without dynamic programming, we would solve these subproblems repeatedly, leading to unnecessary computations.

Now that we have a clear understanding of the two essential elements of dynamic programming in the 0/1 knapsack problem, let's walk through the step-by-step solution:

1. Create a two-dimensional table d with $n + 1$ rows and $W + 1$ columns, where n is the number of items and W is the total capacity of the knapsack. Let $d(i, w)$ represent the maximum value that can be obtained using the first i items with a knapsack capacity of w . For instance, $d(4, 5)$ would represent the maximum value achievable with the first 4 items, assuming the knapsack has a maximum capacity of 5, even though the actual capacity might be greater.

2. Initialize the first row ($i = 0$) and first column ($w = 0$), which means $d(i, 0) = 0$ for all i and $d(0, w) = 0$ for all w . Let's assume we have three items with the following weights and values: $w = \{1, 3, 4\}$ and $v = \{15, 20, 30\}$ and the knapsack capacity is 6. (see Table 10.2).

		w						
		0	1	2	3	4	5	6
i	0	0	0	0	0	0	0	0
	1	0						
	2	0						
	3	0						

Table 10.2: Initializing table **D**

1. Populate the table by the following rules:

For each item i and weight w_i :

- If the weight of a new item exceeds the current weight limit, we exclude the new item:
 $d(i, w) = d(i - 1, w)$ if $w_i > w$.
- Otherwise, we have two choices:
 - Include the current item: $d(i, w) = v_i + d(i - 1, w - w_i)$
 - Exclude the current item: $d(i, w) = d(i - 1, w)$

Choose the maximum of these two values:

$$d(i, w) = \max(v_i + d(i - 1, w - w_i), d(i - 1, w)) \text{ if } w_i \leq w$$

2. Generate the solution. The solution is located at $d(n, W)$, which represents the maximum value achievable with the entire set of items and the full knapsack capacity. Table 10.3 shows the completed table, **D**, with the final solution highlighted in the bottom-right corner. The maximum value obtainable is 45, indicating that only items 1 and 3 will be selected.

		w						
		0	1	2	3	4	5	6
i	0	0	0	0	0	0	0	0
	1	0	15	15	15	15	15	15
	2	0	15	15	20	30	30	30
	3	0	15	15	20	30	30	45

Table 10.3: Completed table **D**. The final result is highlighted

The following is a Python implementation of the 0/1 knapsack problem using dynamic programming:

```
def dp_knapsack(weights, values, W):
    n = len(weights)
    d = [[0 for _ in range(W + 1)] for _ in range(n + 1)]
    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                d[i][w] = 0
            elif weights[i - 1] <= w:
                dp[i][w] = max(values[i - 1] + d[i - 1][w - weights[i - 1]],
                                d[i - 1][w])
            else:
                d[i][w] = d[i - 1][w]
    return d[n][W]
weights = [2, 3, 4]
values = [3, 4, 5]
W = 5

result = knapsack(weights, values, W)
print("Maximum value:", result)
```

As we previously showed, a naïve divide-and-conquer approach for solving the 0/1 knapsack problem leads to exponential time complexity $O(2^n)$, as it explores every possible combination of items, making it impractical for large datasets. The dynamic programming approach, on the other hand, solves each subproblem only once and stores the results in a table, leading to a time complexity of $O(nW)$. The space complexity is also $O(nW)$ due to the storage of the **D** table.

Limitations of dynamic programming

In the previous sections, we demonstrated through two examples and a complexity analysis that dynamic programming offers numerous advantages. Here is a summary of these benefits:

- **Optimal solutions:** Dynamic programming guarantees finding the optimal solution for problems with overlapping subproblems and optimal substructure. By systematically solving and storing the solutions to subproblems, dynamic programming ensures that the final solution is the best possible.
- **Efficiency:** By avoiding the re-computation of overlapping subproblems, dynamic programming reduces the time complexity of many problems from exponential to polynomial, making it feasible to solve large-scale problems that would be intractable using other approaches.
- **Versatility:** Dynamic programming can be applied to a wide range of problems, including but not limited to optimization problems such as the knapsack problem, shortest path problems, and sequence alignment. It's a powerful tool for solving various combinatorial, probabilistic, and deterministic problems.
- **Space-time tradeoff:** Dynamic programming often allows a tradeoff between space and time complexity. For example, by storing intermediate results, time complexity is reduced at the expense of increased space usage. In some cases, space optimization techniques can also be applied to reduce space requirements.

However, these benefits come with certain drawbacks and limitations:

- **High space complexity:** One of the major drawbacks of dynamic programming is the potentially high space complexity. Storing solutions to all subproblems can require significant memory, especially for problems with large input sizes or dimensions, which might be infeasible in memory-constrained environments.
- **Complex implementation:** Dynamic programming can be more complex to implement compared to simpler approaches such as greedy or divide-and-conquer. The need to correctly define subproblems, identify the recursive structure, and manage the dynamic programming table can make implementation challenging, particularly for complex problems.
- **Problem specific:** Dynamic programming is not a universal solution and is only applicable to problems that exhibit both overlapping subproblems and optimal substructure. For problems that do not meet these criteria, dynamic programming might not provide any advantages or might even be inefficient.
- **Difficulty in identifying subproblems:** In some cases, identifying the appropriate subproblems and constructing the recurrence relation for a dynamic programming solution can be nontrivial. This requires a deep understanding of the problem, which can be a barrier to applying dynamic programming effectively.

- **Overhead of table management:** Managing the dynamic programming table, especially in complex problems with multiple dimensions or states, can add additional overhead and complexity to the implementation. This can also lead to increased computational overhead if not managed efficiently.

Dynamic programming is a powerful technique that can significantly reduce time complexity and guarantee optimal solutions for a wide range of problems. However, it comes with tradeoffs, particularly in terms of space complexity and implementation complexity. Understanding when and how to apply dynamic programming is key to leveraging its advantages while mitigating its limitations. In the next and final section of this chapter, we will introduce greedy algorithms. While they may not fully address all the limitations of dynamic programming, they provide greater flexibility in solving a broader range of problems.

Greedy algorithms – an introduction

At the beginning of this chapter, we highlighted a key distinction between divide-and-conquer algorithms and dynamic programming: while both strategies leverage optimal substructure, divide-and-conquer algorithms do not typically involve overlapping subproblems. Dynamic programming is particularly effective when overlapping subproblems are present because it avoids redundant calculations by storing and reusing the solutions to these subproblems.

But what happens if we cannot define an optimal substructure for the problem at hand? In such cases, we turn to a different category of algorithms known as Greedy Algorithms. **Greedy algorithms** adopt a fundamentally different approach to problem-solving. Instead of incrementally building solutions by optimally solving subproblems, as in dynamic programming, a greedy algorithm makes a series of decisions based on what appears to be the best choice at each step, with the expectation that these locally optimal decisions will collectively lead to a globally optimal solution.

Key characteristics of greedy algorithms are as follows:

- **Local optimal choice:** Greedy algorithms make decisions by selecting the option that seems best at each step, without considering the global consequences of this choice.
- **No overlapping subproblems:** Greedy algorithms do not require overlapping subproblems. Instead, they work best on problems where each choice is independent of others.
- **Simple implementation:** Because greedy algorithms often involve straightforward, sequential decision-making, they tend to be easier to implement and more efficient in terms of time complexity compared to dynamic programming.

Let's explain the greedy algorithm in a classical optimization problem.

Traveling salesman problem

The **Traveling Salesman Problem (TSP)** is a classic challenge in computer science and operations research. Given a set of locations and the distances between them, the goal is to determine the shortest possible route that visits each location once and returns to the starting point. The TSP is known for its computational complexity, as finding the exact optimal solution requires checking all possible permutations of city visits, which becomes infeasible for a large number of cities. However, a greedy algorithm offers a simpler, though not necessarily optimal, approach to solving the problem.

The core concept behind greedy algorithms is the use of heuristics. In the next section, we will discuss the details of heuristics. For now, let's pause and focus on solving the TSP using the greedy approach.

One of the common greedy heuristics for solving TSP is the nearest neighbor algorithm. The steps are as follows:

1. **Start at a random city:** Choose an arbitrary city as the starting point.
2. **Visit the nearest unvisited city:** From the current city, visit the nearest city that has not yet been visited. This decision is made based on the shortest distance between the current city and the potential next cities.
3. **Repeat until all cities are visited:** Continue the process of moving to the nearest unvisited city until all cities have been visited.
4. **Return to the starting city:** Once all cities have been visited, return to the starting city to complete the tour.

Example 10.4:

Let's consider a simplified example with four cities: A, B, C, and D. These are the distances between cities:

- A to B = 10
- A to C = 15
- A to D = 20
- B to C = 35
- B to D = 25
- C to D = 30

The nearest neighbor algorithm steps are as follows:

1. Start at City A.
2. Visit the nearest city: From A, the nearest city is B (distance = 10).
3. Move to City B: From B, the nearest unvisited city is D (distance = 25).
4. Move to City D: From D, the nearest unvisited city is C (distance = 30).

5. Move to City C: Now all cities have been visited. Finally, return to the starting city A (distance = 15).
6. The resulting tour using the greedy algorithm is $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$ and the total distance is $10 + 25 + 30 + 15 = 80$ units.

Heuristics and their role in greedy algorithms

The term *heuristic* originates from the Latin word *heuristicus*, which itself is derived from the Greek word *heuriskein*, meaning to *find*. Heuristics are utilized across various domains. Here, we will focus on their application in problem-solving and artificial intelligence.

Heuristics are problem-solving strategies or techniques that are designed to produce a solution quickly and efficiently, though not necessarily optimally. Heuristics are particularly useful in complex problems where finding the exact solution would require too much time or computational power. Instead of exhaustively exploring all possible solutions, a heuristic approach uses rules of thumb, educated guesses, or intuitive strategies to generate a solution that is *good enough* within a reasonable amount of time.

Greedy algorithms are a class of algorithms that often rely on heuristics to make decisions. In a greedy algorithm, the strategy is to make the best possible choice at each step based on current information, with the hope that this will lead to an optimal or near-optimal solution. The *best possible choice* in each step is determined by a heuristic.

Let's discuss how heuristics work in greedy algorithms. In a greedy algorithm, a heuristic helps the algorithm decide which option to choose at each step. This decision is based on local information, meaning the algorithm does not consider the entire problem but focuses on the current step. In the TSP using the nearest neighbor algorithm, the heuristic is *choosing the nearest unvisited city*. This decision is made based on the distance from the current city to other cities, without considering the overall tour.

The heuristic guides the algorithm by providing a simple and fast rule to follow. This rule is based on the specific problem's characteristics and is designed to lead to a good solution. Let's examine this in the coin change problem. The coin change problem is a classic optimization problem where the goal is to determine the minimum number of coins needed to make a specific amount of money using a given set of coin denominations. For example, if we have coins of denominations 1, 5, and 10 and need to make 12, the challenge is to find the combination of these coins that minimizes the total number of coins used. This problem can be solved using various approaches, including greedy algorithms, dynamic programming, or recursion, depending on the specific set of denominations. For this problem, a greedy heuristic might *always take the largest denomination coin first*. This simple rule is easy to follow and often leads to a solution quickly.

Heuristics also allow greedy algorithms to find solutions quickly, even in complex problems. However, this speed comes at the cost of potentially missing the optimal solution. The heuristic provides a balance between finding a solution quickly and finding the best possible solution. Let's discuss this behavior in the Huffman Coding Algorithm. Huffman coding is a greedy algorithm used for lossless data compression. It assigns variable-length codes to characters based on their frequencies in the

input data, with more frequent characters receiving shorter codes. The algorithm builds a binary tree, called the Huffman tree, where each leaf node represents a character and its frequency. By traversing the tree, a unique binary code is generated for each character, minimizing the overall length of the encoded data. Huffman coding is widely used in applications like file compression and encoding. In the Huffman coding algorithm, a greedy approach uses the heuristic of always merging the two least frequent symbols first, which leads to an efficient, though not necessarily optimal, encoding.

Heuristics provide significant advantages to greedy algorithms. They make these algorithms fast by enabling decisions without the need to explore every possible option. Moreover, heuristic-based decisions are typically easy to understand and implement, making greedy algorithms straightforward. In many real-world scenarios, particularly in artificial intelligence, a quickly found approximate solution is often more valuable than an exact solution that requires excessive computation time.

However, heuristics have their own limitations. Because heuristics rely on local information, they can sometimes result in suboptimal solutions; what seems like the best choice in the short term may not be the best choice in the long run. Additionally, heuristics are often problem-specific, meaning a heuristic that works well for one problem may not be effective for another. Greedy algorithms, guided by heuristics, can also become trapped in local optima – solutions that are better than nearby alternatives but not the best possible overall. This can cause the heuristic to miss the globally optimal solution.

Greedy algorithms don't always yield the optimal solution. In the TSP example discussed earlier, the greedy approach may overlook shorter routes that require more strategic decisions rather than just choosing the nearest neighbor at each step. Since the greedy algorithm is heuristic, it offers a fast, approximate solution but cannot guarantee that the solution is optimal. Additionally, the greedy approach can get trapped in local optima, where a seemingly best decision at the moment hinders achieving the global optimum.

The greedy algorithm is best used in the following scenarios:

- Optimal substructure doesn't exist. If a problem lacks a clear optimal substructure, dynamic programming might not be applicable. Greedy algorithms can provide a more straightforward solution in such cases.
- Greedy algorithms are tailored to specific types of problems, such as those involving scheduling, shortest paths, or resource allocation, where making the locally optimal choice at each step leads to an overall optimal solution.
- A quick, approximate solution is acceptable, data is small, and computational resources are limited.
- The problem context allows for the potential trade-off between solution quality and computational efficiency.

While greedy algorithms offer a simpler and often more efficient alternative to dynamic programming, they are not a one-size-fits-all solution. The absence of overlapping subproblems and the reliance on local optimization means that greedy algorithms can only be applied to certain types of problems.

Understanding when to use a greedy approach versus dynamic programming is crucial for effective problem-solving in algorithm design.

Table 10.4 demonstrates a comparison between divide-and-conquer, dynamic programming, and greedy algorithms.

Feature	Divide-and-conquer	Dynamic programming	Greedy algorithms
Problem-solving strategy	Breaks problem into independent subproblems, solves recursively, combines solutions	Breaks problem into overlapping subproblems, solves once, stores results	Builds solution piece by piece with locally optimal choices
Optimal substructure	Uses optimal substructure implicitly	Relies heavily on optimal substructure	Assumes local optimum leads to global optimum
Overlapping subproblems	Subproblems are usually independent; solving one subproblem does not affect the solution of another	Handles problems with overlapping subproblems by storing and reusing results to avoid redundant calculations	Typically, it does not involve overlapping subproblems. Each decision is made based on the current state, independent of previous decisions
Use cases	Sorting, searching, numerical problems	Optimization problems, complex problems	Minimum spanning tree, shortest path, scheduling
Computational efficiency	Depends on problem nature, recursive overhead	More space intensive, avoids redundant calculations	Generally faster, less space, may not always be optimal
Example problems	Merge sort, quick sort, binary search	Knapsack, Longest Common Subsequence, Floyd-Warshall	Kruskal's, Dijkstra's shortest path, activity selection

Table 10.4: Comparison of divide-and-conquer, dynamic programming, and greedy algorithms

Summary

In this chapter, we explored the key concepts and differences among these algorithmic strategies, highlighting how each approach solves problems with optimal substructure. We discussed how divide-and-conquer algorithms break problems into smaller, non-overlapping subproblems, and how dynamic programming efficiently handles overlapping subproblems by storing and reusing their solutions. This chapter also covered greedy algorithms, emphasizing their reliance on heuristics to make locally optimal choices at each step, even though this may not always lead to a globally optimal solution.

Throughout the chapter, we provided examples such as the 0/1 knapsack problem and the TSP to illustrate the strengths and limitations of each approach. We also examined the role of heuristics in greedy algorithms, noting how they enable quick, approximate solutions but can sometimes lead to suboptimal results. As we concluded the discussion, we acknowledged the importance of choosing the right algorithmic strategy based on the problem at hand.

In the next chapter, we will transition into an introduction to data structures, the foundational elements that support efficient algorithm design and implementation.

References and further reading

- Introduction to Algorithms. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022.
 - *Chapter 15, Dynamic Programming*
 - *Chapter 16, Greedy Algorithms*
 - *Chapter 34, NP-Completeness (for a comparison of algorithm complexities)*
- Algorithm Design. By J. Kleinberg and É. Tardos. Pearson. 2006.
 - *Chapter 4, Greedy Algorithms*
 - *Chapter 5, Divide-and-Conquer*
 - *Chapter 6, Dynamic Programming*
- Algorithms. By S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani. McGraw-Hill. 2008
 - *Chapter 2, Divide-and-Conquer*
 - *Chapter 5, Greedy Algorithms*
 - *Chapter 6, Dynamic Programming*

Part 3:

Fundamental Data Structures

This part explores the core data structures that underpin algorithm performance. We examine both linear and non-linear structures, exploring their operations, benefits, and trade-offs. Understanding these data structures is crucial for designing efficient algorithms and optimizing solutions.

This part includes the following chapters:

- *Chapter 11, Landscape of Data Structures*
- *Chapter 12, Linear Data Structures*
- *Chapter 13, Non-Linear Data Structures*

Landscape of Data Structures

The relationship between algorithms and data is fundamental to designing efficient software programs. The choice of data structures directly impacts the performance of algorithms, as basic data structures might require resource-intensive operations such as search, insertion, and deletion, potentially leading to inefficiencies in software execution. In contrast, more advanced data structures can simplify these operations, reducing complexity and significantly enhancing overall algorithm performance. Understanding this relationship is key to optimizing both the speed and resource usage of software programs.

In this chapter, we will explore various aspects of data structures, including their classifications as linear and non-linear types, as well as the distinctions between static and dynamic data allocations. We also discuss the fundamental operations supported by data structures, such as searching, insertion, and deletion, and discuss how the efficiency of these operations varies with different data structures. By examining these characteristics, this chapter aims to provide a comprehensive understanding of how to select and implement data structures that best meet the needs of specific algorithms, ultimately leading to more efficient and effective software design.

We'll cover the following main topics in this chapter:

- Taxonomy of data structures
- Abstract data types
- Dictionaries

Taxonomy of data structures

Understanding and designing algorithms without considering the data they operate on is an incomplete endeavor. In *Chapter 1*, we explored the unique relationship between computing hardware and algorithms. Similarly, there is a significant connection between algorithms and data. An efficient algorithm relies heavily on the use of an appropriate and efficient data structure. For this reason, in this chapter and the next two, we will focus on data structures from the perspective of algorithm design. While data structures are a broad and complex field that warrants detailed study, our focus here and in the next two chapters will be on their critical role in algorithm efficiency.

First, it is crucial to grasp the key aspects used to evaluate and assess the efficiency and characteristics of data structures. Understanding these aspects is important, as the choice of an appropriate data structure directly impacts the performance and effectiveness of an algorithm. When designing an algorithm, selecting the right data structure is not merely a matter of preference but also a necessity dictated by the algorithm's specific requirements.

Let's begin by defining what a data structure is within the context of algorithms. A data structure is a systematic way of organizing, managing, and storing data in a computer so that it can be accessed and modified efficiently. The organization of data is not arbitrary; it is intentionally designed to support specific types of operations that are critical for the performance of algorithms and computer programs.

In the domain of algorithms, a data structure serves as the foundation upon which algorithms operate. It determines how data is stored, how it is retrieved, and how it can be manipulated during the execution of a program. The efficiency of an algorithm often hinges on the effectiveness of the underlying data structure. Whether the task involves searching, sorting, inserting, or deleting data, the choice of data structure can dramatically impact the speed and resource usage of the algorithm.

In essence, a data structure is more than just a method of data organization; it is a vital component that influences the efficiency, scalability, and overall performance of algorithms in computer science.

The key question now is, which data structure is best suited for specific computational tasks and algorithms? This section is dedicated to addressing that question. In this section, we will introduce the criteria that define the efficiency of data structures. These criteria will guide our exploration of various data structures and their suitability for different types of algorithms. Following this, the discussion will shift to a comprehensive examination of how data structures can be categorized. This categorization, or taxonomy, of data structures will help us understand the different types and their appropriate use cases, providing a structured approach to selecting the most suitable data structure for a given algorithmic challenge.

Physical versus logical data structures

In data structures, it's important to distinguish between physical and logical data structures. A **physical data structure** refers to the actual organization and arrangement of data in the memory of a computer. In this context, the data items are physically related to each other, based on their location in memory. When an algorithm is executed, the data it processes is stored in a computer's memory in a manner that reflects its physical structure.

Physical data structures are directly concerned with how data is laid out in memory, including aspects such as memory allocation, pointers, and how data is retrieved or modified at the hardware level. The way data is physically organized can significantly affect the performance of an algorithm, particularly in terms of speed and memory efficiency.

Conversely, a **logical data structure** refers to the abstract organization of data as perceived by the algorithm or the programmer. This is a conceptual model that defines how data is related to each other, regardless of how it is physically stored. Examples of logical data structures include arrays, linked lists,

trees, and graphs. These structures are defined by the operations that can be performed on them and the relationships between data items, rather than their physical location in memory.

When an algorithm is executed, it operates on these logical data structures. However, in order for the algorithm to function, there must be a mapping between the logical structure and the physical memory where the data resides. This mapping is managed by a combination of system programs and the memory management subsystem, which are integral components of the operating system. These systems ensure that the logical data structures used by the algorithm are efficiently translated into the appropriate physical data structures in memory.

As we move to higher levels of abstraction in computer algorithms and models, the dependence on the physical representation of data diminishes. At these advanced levels, the focus shifts from how data is physically stored in memory to how it is logically structured and manipulated to achieve the desired outcomes.

In essence, higher-level algorithms are designed to operate on data without needing to concern themselves with the underlying physical data representation. This abstraction allows for greater flexibility and generality in algorithm design, as the same logical structure can be implemented on various physical systems without altering the algorithm itself.

In this book, we concentrate on studying data structures at the logical level. This approach allows us to focus on the abstract organization of data and the operations that can be performed on it, independent of the specifics of physical storage. By understanding data structures in this way, we can develop more versatile and robust algorithms that are applicable across different computing environments, regardless of the underlying hardware or memory architecture.

In summary, physical data structures are concerned with the actual storage of data in memory, while logical data structures provide an abstract way to organize and manipulate data. The efficiency of an algorithm can depend heavily on how well these two aspects are managed and aligned.

Primitive versus composite data structures

When discussing data structures, it is essential to differentiate between primitive and composite data structures. *Primitive data structures* are the most basic forms of data representation. They include fundamental types, such as integers, floats, characters, and pointers. These structures are typically directly supported by computer hardware or the programming language, and they form the building blocks for more complex data structures. Primitive data structures are straightforward in terms of both their implementation and their operations, such as assigning values, performing arithmetic, and basic input/output operations.

Conversely, *composite data structures* are more complex and are composed of multiple primitive data elements. They are designed to handle more sophisticated data organization and operations. Examples of composite data structures include arrays, linked lists, trees, and graphs. These structures allow you to group multiple primitive data types into a single, more powerful entity, enabling more complex operations such as searching, sorting, and hierarchical organization.

Composite data structures are essential for implementing algorithms that require the handling of more complex data relationships. For instance, a tree structure can represent hierarchical relationships between data items, while a graph structure can represent more complex networks of relationships. Unlike primitive data structures, composite data structures require careful design and management to ensure efficient data processing. In this book (*Chapter 12* and *Chapter 13*), our focus is on understanding and utilizing composite data structures from a logical perspective.

Linear versus non-linear data structures

One important way to classify logical data structures is by their linearity. Logical data structures can be categorized as either linear or non-linear, with each type serving distinct purposes and enabling different types of operations.

Linear data structures are those in which data elements are arranged in a sequential order, where each element is connected to its previous and next elements, forming a straight line. Examples of linear data structures include arrays, linked lists, stacks, and queues. In a linear data structure, operations such as traversal, insertion, and deletion typically follow a straightforward, sequential pattern. This ordered nature makes linear data structures ideal for tasks where data needs to be processed in a specific order, such as in managing queues or performing simple searches.

In terms of memory, linear data structure has the following advantages:

- **Contiguous memory allocation:** Linear data structures, such as arrays, use contiguous blocks of memory, making memory access faster and more predictable due to the locality of reference. This results in efficient utilization of cache memory and reduces the overhead of accessing elements.
- **Memory management simplicity:** Because linear data structures often involve fixed sizes (e.g., arrays) or sequential pointers (e.g., linked lists), managing memory is straightforward. Memory allocation and deallocation are easier to implement compared to more complex structures.
- **Low memory overhead:** For structures such as arrays, there is minimal memory overhead because no additional pointers or links are required to connect the elements, leading to lower memory usage compared to non-linear structures such as trees or graphs.

Conversely, linear data structures come with limitations in terms of memory:

- **Wasted memory (a fixed-size limitation):** In structures such as arrays, memory is allocated for a fixed number of elements at the time of creation. If the number of elements is fewer than the allocated size, there will be unused memory space, leading to inefficient utilization.
- **Memory reallocation:** Expanding a linear data structure such as an array requires reallocation of memory and copying of elements to a new, larger memory block, which is costly in terms of time and space. This reallocation can lead to memory fragmentation and inefficiency.

- **Pointer overhead in linked lists:** In linked lists, each element stores an additional pointer to the next element, which increases the overall memory usage, especially when dealing with a large number of elements. This overhead can negate some of the memory advantages gained by dynamic sizing.
- **Sequential memory allocation for contiguous structures:** Linear structures such as arrays require contiguous blocks of memory. If sufficient contiguous space is not available, memory allocation may fail or become inefficient, leading to potential performance bottlenecks.

The simplicity of linear data structures often leads to ease of implementation and predictability in performance. However, they may not always be the most efficient choice to represent more complex relationships between data elements, as their sequential nature can limit flexibility in certain scenarios.

Non-linear data structures, conversely, do not organize data in a linear sequence. Instead, the data elements are connected in a more complex way, forming structures such as trees, graphs, and heaps. In non-linear data structures, each element may be connected to multiple elements, allowing for the representation of hierarchical or networked relationships between data points.

Non-linear data structures are particularly powerful for representing data that has an inherent hierarchical organization, such as a family tree or a file directory, or for modeling networks, such as social connections or transportation routes. They offer greater flexibility and efficiency for tasks that require managing and querying complex relationships between data items. However, their complexity also requires more sophisticated algorithms for traversal, searching, and manipulation.

In this book, we will examine both linear (*Chapter 12*) and non-linear data structures (*Chapter 13*) from a logical standpoint, focusing on how they can be utilized to support different types of algorithms.

Static versus dynamic memory allocation

Another important classification is based on how memory is allocated to store data. This classification divides data structures into those with static data allocation and those with dynamic data allocation.

Static data structures are those where the size of the data structure is fixed at the time of its creation. This means that the amount of memory allocated for the data structure remains constant throughout its lifetime. An example of a static data structure is an array. When we declare an array, we specify the number of elements it can hold, and this size cannot be changed during runtime. Static data structures are generally simpler to implement and access because their memory layout is predetermined, which can lead to efficient memory usage when the data size is known in advance.

However, the main drawback of static data structures is their inflexibility. If the actual data size exceeds the predefined capacity, there is no way to expand the structure without reallocating memory and potentially moving data, which can be inefficient and cumbersome.

Dynamic data structures, conversely, allow memory allocation to be adjusted at runtime. This means that the size of the data structure can grow or shrink as needed, depending on the operations being performed. Examples of dynamic data structures include linked lists, trees, and graphs. In a dynamic data structure, memory is allocated as elements are added and deallocated as they are removed, providing greater flexibility and efficient use of memory when dealing with data of unpredictable size or varying over time.

Dynamic data structures are especially useful in scenarios where the amount of data cannot be determined beforehand, or when the data structure needs to accommodate varying amounts of data efficiently. However, this flexibility comes at the cost of additional overhead in managing memory allocation and pointers, which can lead to more complex implementations and potentially slower access times compared to static data structures. In *Chapters 12 and 13*, we will explore both static and dynamic data structures, focusing on their characteristics, advantages, and trade-offs.

Sequential versus random access

The final classification of data structures is based on how data elements are accessed and retrieved. This classification divides data structures into those with sequential access and those with random access.

Sequential access data structures are those where data elements are accessed in a specific, linear order. To retrieve a particular element, you must first traverse through the preceding elements. Examples of sequential access data structures include linked lists and queues. In these structures, accessing a specific element often requires iterating through a sequence of nodes or elements, which can make retrieval times longer, especially if the desired element is deep within a structure.

Sequential access is often used in scenarios where the order of data elements is important or where data is processed in a linear fashion, such as in streaming applications or when data needs to be processed one element at a time. However, the primary limitation is that direct access to a specific element is not possible without traversing the sequence, which can be inefficient for large datasets.

Random access data structures, conversely, allow direct access to any data element without needing to traverse through others. This means that you can instantly retrieve or modify any element if you know its index or key. Examples of random-access data structures include arrays and hash tables. In an array, for instance, you can access any element directly by its index, making operations such as retrieval and updating very fast.

Random access is highly beneficial when you need quick access to individual elements, especially in applications where speed is critical, such as in databases or real-time systems. However, this convenience comes with trade-offs, such as the need for continuous memory allocation in the case of arrays, which can lead to inefficient memory usage if the data structure is sparsely populated.

In *Chapter 12*, we will examine both sequential and random-access data structures, focusing on their use cases, advantages, and limitations. Understanding the access patterns of different data structures is crucial for designing algorithms that are both efficient and appropriate for the specific type of data they handle. The choice between sequential and random access can significantly impact the performance of an algorithm, particularly in terms of speed and resource utilization.

Abstract data types

An **Abstract Data Type (ADT)** is a mathematical model for data types where the data type is defined by its behavior (operations), rather than its implementation. An ADT encapsulates data and the operations that can be performed on that data, abstracting away the implementation details. In other words, an ADT specifies what operations are possible and what their behavior is, but not how these operations are implemented.

ADTs are fundamental in computer science because they allow developers to work with data in a more flexible and modular way. By defining operations without specifying implementation details, ADTs enable code to be more maintainable and adaptable to different contexts.

ADTs can be broadly categorized into several types, based on the operations they support and the use cases they address. Here are some common types:

- **List:** A list is an ADT that represents an ordered collection of elements, where each element has a specific position within the sequence. The primary operations for lists include insertion, deletion, access, and traversal of elements. Common examples of list implementations are arrays and linked lists. An array is a straightforward implementation in which elements are stored in contiguous memory locations, providing fast access by index. In contrast, a linked list is a more flexible implementation where each element (or node) points to the next, allowing dynamic resizing and easier insertion or deletion of elements. Lists are often used to manage ordered collections of items, such as a list of students, tasks in a to-do list, or a playlist of songs. Both arrays and linked lists will be discussed in *Chapter 12*.
- **Stack:** A stack is an ADT that operates on the **Last-In-First-Out (LIFO)** principle, where the most recently added element is the first to be removed. The primary operations associated with a stack are `push` (adding an element to the top of the stack) and `pop` (removing the top element). One common example of a stack is the call stack, used by programming languages to track function calls and returns. Another prevalent use case is the undo mechanism in text editors, where a stack is used to revert to previous states. Stacks are also essential in parsing expressions in compilers, implementing backtracking algorithms such as solving mazes or puzzles, and managing nested function calls in recursion. Stacks will be reviewed in *Chapter 12*.
- **Queue:** A queue is an ADT that operates on the **First-In-First-Out (FIFO)** principle, meaning the first element added is the first one to be removed. The primary operations associated with queues are `enqueue` (adding an element to the queue) and `dequeue` (removing an element from the queue). Queues have numerous applications in software programming and computer systems, such as managing print jobs in a print queue, where documents are processed in the order they are received, and in task scheduling within operating systems, where processes are handled in the order they are queued for execution. In *Chapter 12*, we will review the major characteristics of Queues.

- **Deque (Double-Ended Queue):** A deque is an ADT that allows elements to be inserted and deleted from both ends of the sequence, effectively generalizing both stacks and queues. For example, the deque implementation in the **Standard Template Library (STL)** of C++ provides a flexible sequence container that can dynamically grow and shrink at either end. Another example is the circular buffer, a deque implementation commonly used in scenarios where data is cyclically added and removed. Deques are particularly useful in implementing features such as undo/redo in software applications such as text editors. They also play a crucial role in more advanced applications, such as managing sliding window problems in algorithms. Deque will be reviewed in detail in *Chapter 12*.
- **Set:** A set is an ADT that represents a collection of unique elements, where the order of elements is not important. The key operations associated with sets include insertion, deletion, membership checks, and set operations such as union, intersection, and difference. Examples of set implementations include `HashSet` in Java, which ensures that a collection contains no duplicate elements, and `Set` in Python, which supports various mathematical operations such as union and intersection. Sets are widely used in scenarios such as managing collections of unique items (e.g., a list of students enrolled in a course), implementing operations that require uniqueness (e.g., removing duplicates from a list), and performing mathematical set operations.
- **Dictionary:** A dictionary, also referred to as a map or associative array, is an ADT that stores data as key-value pairs, with each key being unique and associated with a specific value. The primary operations include inserting new key-value pairs, deleting pairs, looking up values based on keys, and sometimes iterating over keys or values. Examples of dictionary implementations include `HashMap` in Java, which enables fast retrieval of values using unique keys, and `Dictionary` in Python, which offers a versatile way to map keys to values. Dictionaries are widely used in various applications, such as implementing lookup tables, managing configuration settings where each one is identified by a unique key, and storing and retrieving data by key, such as user profiles indexed by user ID. Detailed discussions on dictionaries are found in the next section.
- **Graph:** A graph is an ADT that represents a collection of nodes, known as vertices, which are connected by edges. Graphs can be either directed, where edges have a specific direction, or undirected, where edges have no direction. They can also contain cycles. Common operations on graphs include adding vertices, adding edges, and traversing the structure. Graphs are typically implemented using an adjacency list, where each node has a list of its neighboring nodes, or an adjacency matrix, a 2D array that indicates the presence or absence of edges between nodes. Graphs are widely used in various fields of science and engineering, including modeling networks such as social networks, communication networks, and transportation systems. Graphs are discussed further in *Chapter 13*.
- **Tree:** A tree is a hierarchical ADT where elements are arranged in nodes, starting with a single root node that branches out into child nodes, forming subtrees. Trees can be of various types, such as binary trees, where each node has at most two children, or more complex structures such as B-trees. Examples include the **Binary Search Tree (BST)** and the Heap. In a BST, each

node can have up to two children, with the left child being smaller than the parent and the right child being larger. The Heap is a specialized tree structure that adheres to the `heap` property, commonly used to implement priority queues. We will discuss trees in *Chapter 13*.

ADTs allow programmers to focus on what operations can be performed on data, rather than how these operations are implemented. This abstraction leads to more flexible, maintainable, and reusable code. ADTs such as lists, stacks, queues, sets, maps, graphs, and trees are foundational to computer science, each serving specific use cases based on the nature of the data and operations required.

Now that we have an understanding of data structure classifications and abstract data types, the next section will focus on the major operations that can be performed on data structures. We will begin by introducing an abstract data structure known as a *dictionary*.

Dictionaries

Dictionaries are abstract data structures designed to store data in a key-value format. In a dictionary, each element consists of a key that is unique and used to access the associated value. The key-value pairing allows for rapid lookups, which is one of the primary advantages of using dictionaries. Dictionaries are also commonly referred to as maps, associative arrays, or symbol tables.

While our focus has been on logical data structures, it's worth noting that dictionaries can also be implemented directly in hardware through what is known as associative memory, or **Content-Addressable Memory (CAM)**.

Associative memories are used for ultra-high-speed search operations. As you might expect, with associative memory, any search operation can be executed in constant time, $O(1)$. Although employing associative memories may seem costly and impractical for general algorithmic use, a more feasible approach in software is the use of hashing. Hashing, which was discussed in *Chapter 7*, provides an algorithmic method to implement a content-addressable data structure, allowing for efficient lookups and other operations.

The key properties of dictionaries include the following:

- In most implementations, dictionaries do not maintain any particular order of the key-value pairs. The elements are stored in an arbitrary order based on the hash function used internally.
- Each key in a dictionary is unique. If you attempt to insert a new key-value pair with a key that already exists in the dictionary, the existing value is typically overwritten with the new value.
- The primary operations on a dictionary – such as inserting, deleting, and accessing elements – are typically performed in average-case $O(1)$ time due to the underlying hash table implementation. This makes dictionaries extremely efficient for large datasets where fast lookups are required.

The question is how to implement and maintain dictionaries. Fortunately, many programming languages provide built-in support for dictionaries. Here's a simple example of how dictionaries can be used in Python:

```
# Creating a dictionary to store information about a student
student_info = {
    "name": "John Doe",
    "age": 21,
    "major": "Computer Science",
    "GPA": 3.8
}
# Accessing values using keys
print("Name:", student_info["name"]) # Output: Name: John Doe
print("Age:", student_info["age"])   # Output: Age: 21
# Adding a new key-value pair
student_info["graduation_year"] = 2024
print("Graduation Year:", student_info["graduation_year"]) # Output:
Graduation Year: 2024
# Updating an existing value
student_info["GPA"] = 3.9
print("Updated GPA:", student_info["GPA"]) # Output: Updated GPA: 3.9
# Deleting a key-value pair
del student_info["major"]
print("After deletion:", student_info) # Output: {'name': 'John Doe',
'age': 21, 'GPA': 3.9, 'graduation_year': 2024}
# Iterating over the dictionary
for key, value in student_info.items():
    print(f"{key}: {value}")
```

This example demonstrates the basic operations of a dictionary in Python, showcasing how easy it is to create, manipulate, and access data using this powerful data structure. Let's briefly explain the code:

- **Creating a dictionary:** We define a dictionary, `student_info`, with key-value pairs representing various attributes of a student.
- **Accessing values:** We access the values associated with the `name` and `age` keys using square brackets.
- **Adding a key-value pair:** We add a new key, `graduation_year`, with its corresponding value.
- **Updating a value:** We update the value associated with the `GPA` key.
- **Deleting a key-value pair:** We remove the `major` key and its value using the `del` statement.
- **Iterating over the dictionary:** We iterate over the dictionary using a `for` loop, printing each key-value pair.

As we illustrated in the previous example, dictionaries are more than just collections of key-value pairs; they also support a range of fundamental operations that allow for efficient data management. These basic operations include insertion, search, update (or edit), and deletion. Let's explore each of these functions in detail with examples in Python.

Insertion

Insertion involves adding a new key-value pair to the dictionary. If the key already exists, the value associated with that key is updated; otherwise, a new entry is created. The following is an insertion function for a simple dictionary in Python:

```
# Creating an empty dictionary
student_grades = {}
# Inserting key-value pairs into the dictionary
student_grades["Alice"] = 85
student_grades["Bob"] = 90
print(student_grades) # Output: {'Alice': 85, 'Bob': 90}
```

In this example, we create an empty dictionary called `student_grades` and insert two key-value pairs – Alice: 85 and Bob: 90. The dictionary now holds these values, and new entries can be added similarly.

Search

Search is the operation of retrieving a value associated with a specific key. This is one of the most common and efficient operations on a dictionary, typically performed in constant time. Let's look at a simple search function in a dictionary:

```
# Searching for a value by its key
alice_grade = student_grades.get("Alice")
print("Alice's grade:", alice_grade) # Output: Alice's grade: 85
# Searching for a non-existent key
charlie_grade = student_grades.get("Charlie", "Not Found")
print("Charlie's grade:", charlie_grade) # Output: Charlie's grade:
Not Found
```

Here, the `get` method is used to search for the value associated with the Alice key. If the key exists, the corresponding value is returned. If the key is not found, as in the case of Charlie, a default value ("Not Found") can be returned instead.

Update

Update involves changing the value associated with an existing key in the dictionary. If the key exists, its value is modified; if the key does not exist, a new key-value pair is added. The following is one example of updating:

```
# Updating an existing key-value pair
student_grades["Alice"] = 88
print(student_grades)  # Output: {'Alice': 88, 'Bob': 90}
# Adding a new key-value pair through update
student_grades["Charlie"] = 92
print(student_grades)  # Output: {'Alice': 88, 'Bob': 90, 'Charlie': 92}
```

In this example, the value associated with the `Alice` key is updated from 85 to 88. Additionally, a new key-value pair, `Charlie: 92`, is added to the dictionary.

Deletion

Deletion removes a key-value pair from the dictionary. Once removed, the key and its associated value no longer exist in the dictionary. Take a look at the following example:

```
# Deleting a key-value pair by key
del student_grades["Bob"]
print(student_grades)  # Output: {'Alice': 88, 'Charlie': 92}
# Attempting to delete a non-existent key (optional approach)
removed_grade = student_grades.pop("David", "Key not found")
print(removed_grade)  # Output: Key not found
```

In this example, the `del` statement is used to remove the `"Bob"` key and its associated value from the dictionary. Additionally, the `pop` method is used to attempt to remove the `David` key, which does not exist in the dictionary, returning a default message, `Key not found`.

These basic operations – insertion, search, update, and deletion – are essential for working with dictionaries in any programming language. They allow efficient management of data and enable you to create dynamic and adaptable programs. Whether we are adding new data, retrieving existing information, modifying values, or removing entries, these functions make dictionaries a versatile and powerful tool in algorithm design and everyday programming tasks.

In *Chapters 12* and *13*, as we introduce various data structures, we will evaluate their performance as they relate to these fundamental operations. Additionally, some data structures include extended functions and specific operations, such as finding successor and predecessor in BSTs.

Summary

In this chapter, the foundational concepts of data structures were explored, with a focus on their classification and the essential operations that can be performed on them. The chapter discussed the distinctions between physical and logical data structures, as well as between primitive and composite structures. It also covered the differences between linear and non-linear data structures, along with the implications of static versus dynamic data allocation, and sequential versus random access. Through these discussions, the importance of selecting the appropriate data structure for specific algorithmic tasks was emphasized.

The chapter also provided insights into the workings of dictionaries, highlighting their key operations, such as insertion, search, update, and deletion, with examples to demonstrate their practical usage. The discussion underscored the role of dictionaries as a versatile and efficient tool in algorithm design.

As we move forward, the next chapter will delve into linear data structures, including arrays and linked lists, and we will examine their properties and use cases in greater detail.

References and further reading

- *Introduction to Algorithms* by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022:
 - *Chapter 10, Elementary Data Structures*
- *Algorithms* by R. Sedgewick, K. Wayne. Fourth Edition. Addison-Wesley. 2011:
 - *Chapter 1, Fundamentals*
- *Data Structures and Algorithm Analysis in C++* by Mark A. Weiss. Fourth Edition. Pearson. 2012:
 - *Chapter 3, Lists, Stacks, and Queues*

Linear Data Structures

In this chapter, we will explore the foundational concepts of **linear data structures**, which play a critical role in computer science and algorithm design. We will start by understanding the basics of arrays and linked lists and learning how these structures store and manage data. The chapter will guide you through the key operations performed on these structures, such as insertion, deletion, and searching, and you will examine their time complexities to understand their efficiency. By comparing arrays and linked lists, you will gain insight into the trade-offs involved in choosing the right data structure for specific applications.

As we progress, we will discover more advanced linear data structures such as stacks, queues, and **double-ended queues (deques)**. We will learn how these structures extend the functionality of basic lists and how they are applied in real-world scenarios, such as task scheduling and resource management. Additionally, the chapter will introduce you to **skip lists**, a probabilistic data structure that offers a balance between the efficiency of arrays and the flexibility of linked lists. By the end of this chapter, you will be well-equipped with the knowledge needed to implement and utilize these linear data structures effectively.

Essentially, this chapter will cover the following topics:

- Lists
- Skip lists
- Stacks
- Queue
- Deque

Lists

A **list** is an ordered collection of items that can hold elements of the same or different types, where each element is indexed and has a specific position within the list. Lists are commonly used to store sequences of data that can be easily accessed, inserted, or removed. They can hold elements of different

types, though in some programming languages, lists are more often homogeneous, meaning all elements are of the same type.

Lists are generally implemented using either arrays or linked structures, and these two approaches result in distinct characteristics regarding performance and memory usage. When elements in a list are stored in contiguous memory locations, the list is known as an **array**. In this case, accessing elements by index is very efficient, typically taking constant time ($O(1)$), since the memory location of any element can be directly computed. However, arrays have a fixed size once they are created, which can lead to inefficiencies if the number of elements changes frequently, requiring the creation of new arrays and copying of data.

On the other hand, if the list is implemented using linked structures, it is referred to as a **linked list**. In a linked list, each element, known as a **node**, contains a reference (or link) to the next node in the sequence. This structure allows the list to grow and shrink dynamically as elements are added or removed, without the need for large blocks of contiguous memory. However, because the nodes are scattered throughout memory, accessing an element by index requires traversing the list from the beginning, which can be time-consuming ($O(n)$ in the worst case).

The choice between using an array or a linked list depends on the specific needs of the application, such as the frequency of insertions and deletions versus the need for quick access to elements. Arrays are generally preferred when fast access and predictable memory usage are crucial, while linked lists are more suitable for scenarios where dynamic resizing and frequent modifications are needed. In the following subsections, we will explore the key characteristics of arrays, particularly in relation to algorithm efficiency and the various operations that can be performed on them.

Arrays

An **array** is a fundamental data structure that consists of a collection of elements, each identified by at least one array index or key. Arrays are one of the simplest and most widely used data structures in computer science. They are typically used to store a fixed-size sequence of elements of the same type. The position of each element is defined by its index, which usually starts from zero.

Arrays possess the following defining characteristics that influence their behavior and performance:

- **Fixed size:** Once an array is created, its size is set and cannot be altered. This means that the number of elements that an array can hold is predetermined at the time of its creation. For example, in most programming languages, we must specify the size of an array when it is declared, such as `int [] a = new int [10]` ; in Java, which creates an array that can hold 10 integers. The following is a simple array declaration in Python:

```
# Define an array (list) of integers
a = [10, 20, 30, 40, 50]
# Print the array
print(a) # Outputs: [10, 20, 30, 40, 50]
```

- **Contiguous memory allocation:** The elements of an array are stored in contiguous memory locations. This enables efficient access to any element by calculating its memory address using a simple mathematical formula. For instance, in a one-dimensional array a of size n , the address of the element $a[i]$ can be calculated as $base + i * length$, where $base$ is the base memory address, i is the index of the element in the array a , and $length$ is the size of each element in the array. For example, it is 1 for 1 byte, 2 for 16 bits or words, and so on. Table 12.1 illustrates a simple example of an array.

Memory address	FF01	FF02	FF03	FF04	FF05	FF06
Content	23	123	54	67	34	87
Index	0	1	2	3	4	5

Table 12.1: An example array

- **Homogeneous elements:** All elements in an array must be of the same data type, ensuring that the array is a uniform collection. For example, an array of integers `int []` can only store integer values, and an array of strings `String []` can only store string values.
- **Indexed access:** Arrays allow direct access to any element using its index, providing constant-time access, which is one of the key advantages of this data structure. Accessing the third element in an array a is as simple as $a[2]$.

Arrays support a variety of operations, each with its own performance implications. Here are the major operations commonly performed on arrays, along with their time complexities:

- **Insertion:** This refers to adding a new element to the array. For example, consider `a = [1, 2, 3, 4]`. Inserting 5 at the end of the array is straightforward if there is space. However, inserting 5 at index 1 requires shifting all elements from index 1 to the right. The time complexity of insertion in arrays is $O(1)$ in the best case and $O(n)$ in the worst case. The best case is inserting at the end in a partially filled array. If inserting at the beginning or middle, requires shifting of elements, it is the worst case. The following is a Python example:

```
a = [1, 2, 3, 4]
a.insert(1, 5) # a becomes [1, 5, 2, 3, 4]
print(a)
```

Since Python uses zero-based indexing, the `a.insert(1, 5)` operation inserts the value into the second position of the array.

- **Deletion:** This involves removing an element from the array. Given `a = [1, 2, 3, 4]`, deleting element 2 at index 1 requires shifting all elements after index 1 to the left to fill the gap. In the best case, deleting the last element has a time complexity of $O(1)$. However, if we delete an element from the beginning or middle of the array, it requires shifting the subsequent elements, resulting in a worst-case time complexity of $O(n)$:

```
a = [1, 2, 3, 4]
a.pop(1) # a becomes [1, 3, 4]
```

- **Edit or update:** Editing is modifying an existing element in the array. Given `a = [1, 2, 3, 4]`, changing the element at index 2 from 3 to 5 is a direct operation. We can directly access the element by its index and update it, which makes its time complexity $O(1)$:

```
a = [1, 2, 3, 4]
a[2] = 5 # a becomes [1, 2, 5, 4]
```

- **Search:** Searching involves finding a specific element within an array. This topic was covered extensively in *Chapter 7*, where most of the search algorithms discussed were based on arrays as the underlying data structure. In *Chapter 13*, we will explore how searching can be performed on non-linear data structures, such as trees.
- **Access:** Access refers to retrieving the value of an element at a specific index in an array. One of the key advantages of arrays is that their access time is constant ($O(1)$), allowing any element to be directly retrieved by its index without the need for traversal.

Arrays can be implemented in more sophisticated structures. One example is **dynamic arrays** (such as `list` in Python). These arrays can resize themselves when elements are added beyond the initial capacity. However, the underlying principle remains the same, with arrays providing efficient access and traversal. Another example is **multidimensional arrays**. These arrays can be extended into multiple dimensions, such as 2D arrays (matrices) or 3D arrays, which are particularly useful in applications such as image processing, scientific simulations, and game development. The following is an example 2D array in Python:

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[1][2]) # Outputs: 6 (element at second row, third
column)
```

Arrays are a fundamental and highly efficient data structure for storing and managing collections of elements, especially when the size of the collection is known and remains constant. Their contiguous memory allocation allows for fast access and straightforward implementation of various operations. However, their fixed size and potential inefficiencies in insertion and deletion make them less flexible compared to other data structures such as linked lists. Understanding the trade-offs and appropriate use cases for arrays is essential for effective algorithm design and implementation.

Linked lists

A **linked list** is a linear data structure in which elements, called nodes, are arranged sequentially. Unlike arrays, linked lists do not store their elements in contiguous memory locations. Instead, each node in a linked list contains at least two parts: the data and a reference (or pointer) to the next node in the sequence. This structure allows linked lists to be dynamic in size, easily growing or shrinking as elements are added or removed.

Linked lists have several key characteristics that distinguish them from other data structures such as arrays:

- **Dynamic size:** Linked lists can dynamically increase or decrease in size, as nodes can be added or removed as needed without reallocating or reorganizing the entire data structure. For example, we can continue adding nodes to a linked list without worrying about a predefined size.
- **Non-contiguous memory allocation:** Unlike arrays, linked lists do not require contiguous memory locations. Each node is stored independently in memory and linked together using pointers. For instance, in a singly linked list, each node contains a pointer to the next node, allowing elements to be scattered throughout memory.
- **Sequential access:** Linked lists must be accessed sequentially from the beginning, as there is no direct way to access a specific element by index. For example, to access the third element in a linked list, we must traverse the first two nodes.
- **Variants:** Linked lists come in different forms, including singly linked lists (where each node points to the next), doubly linked lists (where each node points to both the next and previous nodes), and circular linked lists (where the last node points back to the first). For example, in a doubly linked list, traversal can happen in both directions due to the pointers to both the previous and next nodes.

Linked lists support various operations, each with specific performance characteristics. Let's review the major operations, along with their time complexities and examples.

Insertion in a linked list

Consider this linked list: $24 \rightarrow 3 \rightarrow 12 \rightarrow 17$. If we want to insert the value 8 between 3 and 12, the process involves creating a new node with the value 3 and updating the pointers accordingly. Here's how we can do it step by step:

1. **Create the new node:** First, we create a new node that contains the value 8. Initially, the pointer of this new node is set to `null`, as it does not point to anything yet.
2. **Update the pointer of the new node:** Next, set the pointer of the new node to point to the node that comes after 3, which is the node containing 12. Now, the new node 3 is connected to node 4.
3. **Update the pointer of the previous node:** Finally, update the pointer of the node containing 3 to point to the new node 8. This completes the insertion, resulting in the linked list $24 \rightarrow 3 \rightarrow 8 \rightarrow 12 \rightarrow 17$.

Figure 12.1 illustrates the process of inserting a new node in linked lists.

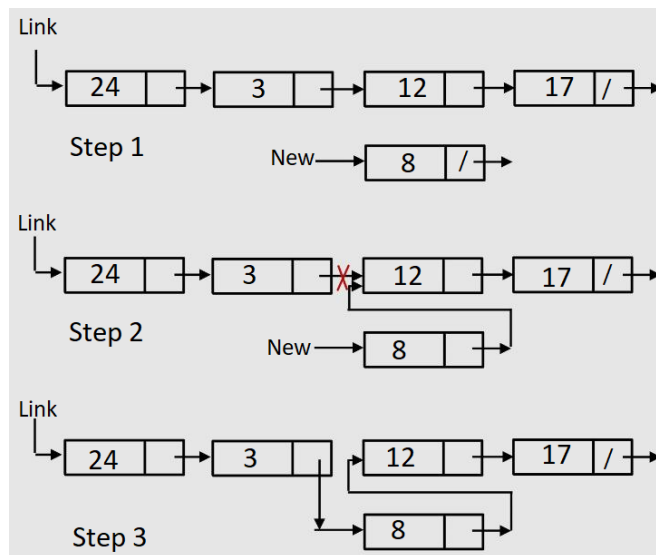


Figure 12.1: The process of adding a new node to a linked list in three steps

In Figure 12.1, *Link* represents the pointer to the **head** of the linked list (node with value 24). The last node, referred to as the **tail** (the node with value 17), points to `null`, indicating the end of the list. We represent a pointer to `null` with the / symbol. The address of the newly created node is labeled as *New*, and initially, it points to `null`.

Here is a simple Python implementation of the preceding process:

```
class Node:
    def __init__(self, data):
        self.data = data # Store data
        self.next = None # Initialize next as null (None in Python)
class LinkedList:
    def __init__(self):
        self.head = None # Initialize the head of the list as None
    def insert_after(self, prev_node, new_data):
        if prev_node is None:
            print("The given previous node must be in the
LinkedList.")
            return
        new_node = Node(new_data) # Create a new node with the
provided data
        new_node.next = prev_node.next # Point the new node to the
next node (e.g., 4)
        prev_node.next = new_node # Point the previous node (e.g., 2)
to the new node (e.g., 3)
    def print_list(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

To test the functionality of both the Node class and the LinkedList class, we can use the following example:

```
if __name__ == "__main__":
    llist = LinkedList()
    # Creating the initial linked list 1 -> 2 -> 4
    llist.head = Node(1)
    second = Node(2)
    third = Node(4)
    llist.head.next = second
    second.next = third
    # Insert 3 between 2 and 4
    llist.insert_after(second, 3)
    # Print the updated linked list
    llist.print_list()
```

Let's briefly explain the code:

- **Node:** Each Node object stores a data value and a next pointer in the next node in the linked list.
- **LinkedList:** The LinkedList class manages the linked list, including the insertion operation.
- **insert_after:** This method inserts a new node after a given node (prev_node). The new node is created with the new_data value, and pointers are updated to insert it correctly into the list.
- **print_list:** This method traverses the linked list and prints the data of each node.

The time complexity of insertion in a linked list is $O(1)$ in the best case, such as when inserting at the beginning or end of the list if the position is already known. In the worst case, when inserting at a specific position requires traversal through the list, the time complexity is $O(n)$.

Deletion in linked lists

Deletion from a linked list involves removing a specific node. For instance, given the $24 \rightarrow 3 \rightarrow 12 \rightarrow 17$ linked list, deleting the node with the value 3 requires bypassing it by updating the pointer of the previous node (2) to point to the next node (4). Like insertion, the time complexity of the deletion operation in linked lists depends on the location of the node to be deleted. In the worst case, when traversal is needed to find the node, the time complexity is $O(n)$. Figure 12.2 illustrates the deleting process in the example link list.

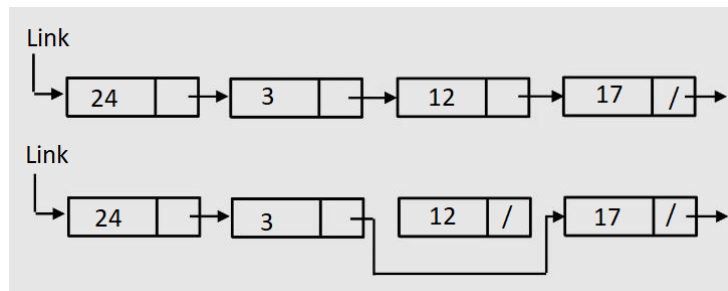


Figure 12.2: The process of deleting a node in a linked list

In Figure 12.3, the top part displays the linked list before the deletion of the node with the value 12, while the bottom part shows the linked list after the deletion. The following is an example Python code for deleting a node in a linked list:

```
def delete_node(self, key):
    temp = self.head
    if (temp is not None):
        if (temp.data == key):
            self.head = temp.next
```

```

        temp = None
        return
    while(temp is not None):
        if temp.data == key:
            break
        prev = temp
        temp = temp.next
    if(temp == None):
        return
    prev.next = temp.next
    temp = None
# Example usage:
l1list.delete_node(3)  # Deletes the node with value 3
l1list.print_list()

```

The `delete_node` function should be added to the `LinkedList` class in the previous section.

Edit in link lists

Editing involves modifying the data within an existing node in the linked list. For instance, if we want to change the value of the second node from 2 to 5 in the $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ linked list, we would update the node's data accordingly. The time complexity for this operation is $O(n)$ because we may need to traverse the list to locate the node to be updated. Here is a piece of sample Python code for the edit/update operation:

```

def update_node(self, old_data, new_data):
    temp = self.head
    while temp is not None:
        if temp.data == old_data:
            temp.data = new_data
            return
        temp = temp.next
# Example usage:
l1list.update_node(2, 5)  # Updates node with value 2 to 5

```

The `update_node` function should be added to the `LinkedList` class in the previous section.

Searching in linked lists

In *Chapter 7*, we explored a variety of search algorithms in detail, all of which are based on arrays as the underlying data structure. For instance, in binary search, we can directly access an element at a specific index, such as the middle of the array. However, when using a linked list, finding a node with a specific value becomes more challenging due to the sequential nature of the data structure. This means

that regardless of the search algorithm used, when applied to a singly linked list, the search essentially becomes a sequential, linear search. As a result, searching in a linked list has a time complexity of $O(n)$.

Here is the Python code example using a singly linked list:

```
def search_node(self, key):
    temp = self.head
    while temp is not None:
        if temp.data == key:
            return True
        temp = temp.next
    return False
# Example usage:
found = llist.search(3) # Returns True if 3 is found
```

The `search_node` function should be added to the `LinkedList` class in the previous section.

Access in link lists

This operation involves retrieving the value of a node at a specific position in the linked list. For example, accessing the fourth node in the $17 \rightarrow 12 \rightarrow 3 \rightarrow 24 \rightarrow 6$ linked list requires sequential traversal to reach the desired node. The time complexity for this operation is $O(n)$. The following is a simple Python code implementing access in a linked list:

```
def get_nth(self, index):
    temp = self.head
    count = 0
    while (temp):
        if (count == index):
            return temp.data
        count += 1
        temp = temp.next
    return None
# Example usage:
value = llist.get_nth(2) # Returns the value of the third node
```

The `get_nth` function should be added to the `LinkedList` class in the previous section.

In practice, linked lists are commonly used as the underlying data structure for implementing stacks and queues due to their ability to dynamically resize. They are also employed in operating systems for managing memory allocation, where free memory blocks are linked together in a list. Another significant application of linked lists is in representing adjacency lists in graphs or trees, where each vertex points to a linked list of adjacent vertices.

Linked lists offer a flexible and dynamic alternative to arrays, especially in scenarios where the size of the data structure needs to change frequently. Their non-contiguous memory allocation allows for efficient insertions and deletions without the need to shift elements, unlike arrays. However, this flexibility comes with the drawback of sequential access times, making linked lists less suitable for applications that require frequent random access. Understanding the trade-offs between linked lists and other data structures, such as arrays, is essential for selecting the most appropriate structure for a given problem. In the next section, we will briefly explore other variants of linked lists.

Doubly and circular linked lists

A **doubly linked list** is a type of linked list in which each node contains two pointers: one pointing to the next node and another pointing to the previous node (see *Figure 12.3*). This bidirectional structure allows traversal of the list in both forward and backward directions, making certain operations more efficient. Although doubly linked lists allow bidirectional access, their sequential nature means that the time complexity for major operations remains similar to that of singly linked lists.

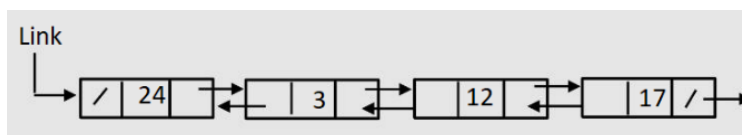


Figure 12.3: An example doubly linked list

Another variation of the linked list is the **circular linked list** where the last node points back to the first node, forming a circular structure (see *Figure 12.4*). This can be applied to both singly and doubly linked lists. Since the last node connects back to the first, we can traverse the list in a loop without encountering a null reference. This characteristic is ideal for applications where data needs to be looped through continuously, such as in round-robin scheduling or buffer management. Like doubly linked lists, circular linked lists do not provide any improvement in time complexity compared to non-circular linked lists.

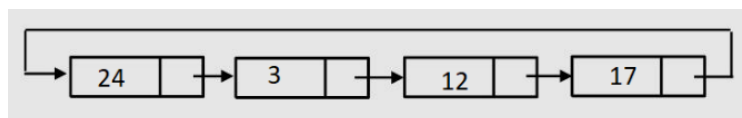


Figure 12.4: A circular linked list

Both doubly linked lists and circular linked lists offer distinct advantages depending on the use case. Doubly linked lists provide more flexibility in navigation and easier node removal, while circular linked lists enable efficient cyclic traversal of data. In terms of performance, both structures exhibit similar time complexities to singly linked lists but with added benefits in specific operations. In the next section, we will explore a data structure that combines the advantages of both linked lists and arrays.

Skip lists

As we discussed in the previous chapter, when studying data structures, it's important to evaluate their performance in three key operations: insertion, deletion, and search. Arrays excel in searching, particularly when the data is already sorted, thanks to their direct (or random) access capabilities, which allow for sublinear time complexity. However, due to their static nature, arrays can present challenges when it comes to inserting new data or deleting existing data. On the other hand, linked lists exhibit the opposite behavior. Their dynamic allocation allows for easy insertion and deletion, but the lack of direct access means that searching, even in sorted data, becomes a sequential process with linear time complexity.

This question then arises: Is it possible to combine the benefits of both arrays and linked lists? In other words, can we achieve faster-than-sequential access, similar to arrays, while also enjoying the dynamic memory allocation of linked lists? The answer lies in the **skip list**. A **skip list** is a probabilistic data structure that extends the basic linked list by adding multiple levels of linked lists, allowing for faster search, insertion, and deletion operations. Each level of a skip list contains a subset of the elements from the level below, with the bottommost level containing all elements in a simple linked list. Higher levels act as “express lanes” to skip over multiple elements at once, hence the name *skip list*. *Figure 12.6* illustrates an example of a skip list. Level 1 functions as a standard linked list, while all the upper levels serve as express lanes.

Let's explain the concept of a skip list using an analogy. Imagine we are traveling to meet a friend at a specific station in a bus system. The bus stops at every station, and each stop takes a considerable amount of time, so we want to minimize the number of stops on our way to our friend's station. There are n stations in total, and we know the exact station number where our friend is waiting. The bus system has two types of lines:

- The regular bus stops at every single station
- The express line stops only at four stations, specifically at $\frac{n}{4}$, $\frac{n}{2}$, $3\frac{n}{4}$, and n

Taking the regular bus is the safest option because it ensures we will reach our destination, but it requires stopping at every station, which can be inefficient, taking n stops in total. Alternatively, we could take the express bus, which only makes four stops. If our friend's station falls within one of the intervals between these express stops (for example, between 1 and $\frac{n}{4}$, or between $\frac{n}{4} + 1$ and $\frac{n}{2}$), we can get off the bus at the nearest express stop and switch to the regular bus to reach our friend's exact station. By doing this, we reduce the number of stops from a maximum of n to around $\frac{n}{4} + 1$, making our journey much more efficient. The skip list is similar to this bus system.

Skip lists have several defining characteristics that influence their behavior and performance:

- **Multiple levels:** Skip lists consist of multiple levels, where each level above the base level contains a subset of elements from the previous level. The bottommost level is a standard linked list containing all elements. In a skip list, the first level might contain all elements, the second level might contain half, the third level might contain a quarter, and so on.

- **Probabilistic balancing:** Skip lists use randomization to determine the level at which each element will appear. This leads to an average-case time complexity similar to balanced binary search trees without the need for complex balancing algorithms. For example, when inserting an element, a random number of levels (up to a maximum) is chosen for the element to be included in.
- **Efficient searching:** Skip lists allow for faster searching by skipping over large portions of the list using the higher levels, reducing the number of comparisons needed. For instance, searching for an element in a skip list can quickly bypass multiple nodes by moving through the upper levels.
- **Dynamic resizing:** Skip lists can dynamically adjust their structure as elements are added or removed, maintaining efficient operations without needing to rebuild the entire structure. For example, as new elements are inserted, they may be added to multiple levels depending on the randomization process.

Skip lists support various operations, each with specific performance characteristics. The following section outlines the major operations, along with their time complexities and examples.

Insertion in skip lists

Inserting numbers into a skip list involves a few steps. For this example, we will insert the numbers 3, 4, 5, 7, 8, 9, and 10 into a skip list with 4 levels. Each number will be inserted one by one, and the levels will be assigned based on randomization. Here's a detailed step-by-step description of how this process works:

- **Step 1: Initialize the skip list:** Start with an empty skip list that has four levels: *Level 4* (topmost), *Level 3*, *Level 2*, and *Level 1* (bottommost). Initially, the list has only a head node at each level pointing to `null`.
- **Step 2: Insert 3:**
 - **Determine levels for 3:** Randomly determine the levels at which 3 should appear. Let's assume it appears at all levels.
 - **Insertion:** At *Levels 1* to 4, there are no other nodes; so, 3 is simply inserted, and the head node at *Levels 1* to 4 now points to 3.

Here is the skip list:

- Level 4: 3 --> null
- Level 3: 3 --> null
- Level 2: 3 --> null
- Level 1: 3 --> null

- **Step 3: Insert 4:**

- **Determine levels for 4:** Randomly determine the levels at which 4 should appear. Suppose it appears only at *Level 1*. The higher levels remain unchanged as 3 does not appear there.
- **Insertion:** Traverse *Level 1* from the head to the appropriate position. 4 should be inserted after 3. Update the pointers so that 3 now points to 4 at *Level 1*.

Here is the skip list:

- Level 4: 3 --> null
- Level 3: 3 --> null
- Level 2: 3 --> null
- Level 1: 3 --> 4 --> null

- **Step 4: Insert 5:**

Determine levels for 5: Let's assume 5 is assigned randomly to *Levels 1, 2, and 3*.

- **Insertion:** In *Levels 2 and 3*, element 5 is the next element to 3. Then, pointers of 3 at these levels are updated to point to 5. In *Level 1*, insert 5 after 4 by updating the pointers accordingly.

Here is the skip list:

- Level 4: 3 --> null
- Level 3: 3 --> 5 --> null
- Level 2: 3 --> 5 --> null
- Level 1: 3 --> 4 --> 5 --> null

- **Step 5: Insert 7:**

- **Determine levels for 7:** Suppose 7 is assigned randomly to *Level 1*. All upper levels remain unchanged.
- **Insertion:** In *Level 1* insert 7 after 5.

Here is the skip list:

- Level 4: 3 --> null
- Level 3: 3 --> 5 --> null
- Level 2: 3 --> 5 --> null
- Level 1: 3 --> 4 --> 5 --> 7 --> null

- **Step 6: Insert 8:**

- **Determine levels for 8:** Let's assume 8 is assigned randomly to *Levels 1* and 2.
- **Insertion:** In *Level 2*, insert 8 after 5 by updating the pointers. In *Level 1*, insert 8 after 7.

Here is the skip list:

- Level 4: 3 --> null
 - Level 3: 3 --> 5 --> null
 - Level 2: 3 --> 5 --> 8 --> null
 - Level 1: 3 --> 4 --> 5 --> 7 --> 8 --> null
- For *Steps 9* and *10*, we do similar to what we did with element 7. The final skip list after inserting all elements is illustrated in *Figure 12.5*.

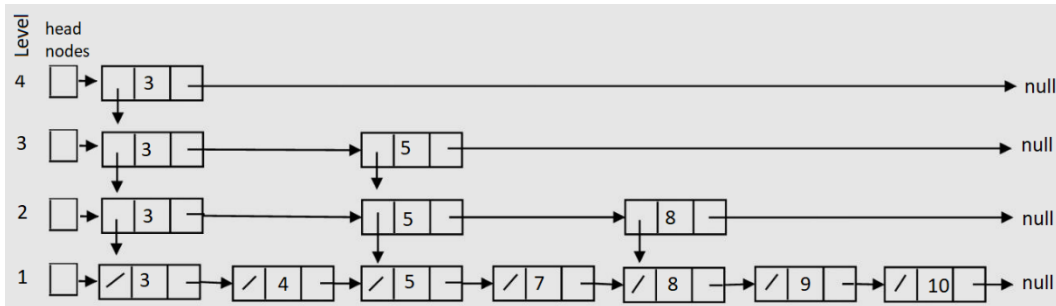


Figure 12.5: An example skip list with four levels

The insertion process in a skip list involves determining the levels at which a new element will appear and updating the pointers at each relevant level to maintain the order. The random assignment of levels ensures that the skip list remains balanced, providing efficient search, insertion, and deletion operations. The preceding steps illustrate how each number was inserted into the skip list, taking advantage of the express lanes created by the higher levels.

The following Python code is an implementation of insertion into a skip list:

```
import random
class Node:
    def __init__(self, value, level):
        self.value = value
        self.forward = [None] * (level + 1)
class SkipList:
    def __init__(self, max_level):
        self.max_level = max_level
```

```
        self.head = Node(-1, max_level) # Head node with value -1
        (acts as a sentinel)
        self.level = 0
    def random_level(self):
        level = 0
        while random.random() < 0.5 and level < self.max_level:
            level += 1
        return level
    def insert(self, value):
        update = [None] * (self.max_level + 1)
        current = self.head
        for i in range(self.level, -1, -1):
            while current.forward[i] and current.forward[i].value <
value:
                current = current.forward[i]
            update[i] = current
        level = self.random_level()
        if level > self.level:
            for i in range(self.level + 1, level + 1):
                update[i] = self.head
            self.level = level
        new_node = Node(value, level)
        for i in range(level + 1):
            new_node.forward[i] = update[i].forward[i]
            update[i].forward[i] = new_node
    def print_skiplist(self):
        print("Skip List:")
        for i in range(self.level, -1, -1):
            print(f"Level {i}: ", end="")
            node = self.head.forward[i]
            while node:
                print(node.value, end=" -> ")
                node = node.forward[i]
            print("None")
# Example usage
if __name__ == "__main__":
    skiplist = SkipList(3)
    # Insert elements into the skip list
    skiplist.insert(3)
    skiplist.insert(4)
    skiplist.insert(5)
    skiplist.insert(7)
    skiplist.insert(8)
    skiplist.insert(9)
```

```
skiplist.insert(10)
# Print the skip list
skiplist.print_skiplist()
```

Let's look inside the code and explain the main classes of the algorithm:

- **Node:** Each Node object stores a value and a list of forward pointers that are 4 in level 1, 5 in level 2, 5 in level 3 and null in level 4
- **SkipList:** This contains the following functions:
 - `random_level()`: This generates a random level for each new node based on a probabilistic model
 - `insert()`: This inserts a new value into the skip list at the appropriate levels
 - `print_skiplist()`: This prints the skip list, displaying the nodes at each level from the topmost level down to the bottommost level

As an example, we create a skip list with up to three levels. The values 3, 4, 5, 7, 8, 9, and 10 are inserted into the skip list. Finally, we print the skip list, displaying how the nodes are organized across the various levels, which looks something like this:

```
Skip List:
Level 2: 7 --> 10 --> None
Level 1: 5 --> 7 --> 8 --> 9 --> 10 --> None
Level 0: 3 --> 4 --> 5 --> 7 --> 8 --> 9 --> 10 --> None
```

After insertion, let's discuss the behavior of a skip list during search operations.

Search in skip lists

Searching for an item in a skip list involves starting at the highest level and moving forward through the list until the target is found or the search needs to move down a level. Here's a step-by-step explanation of how to search for the number 8 in the skip list we previously constructed.

- **Step 1:** Start at the highest level (*Level 4*):
 - Start at the head node at *Level 3*.
 - Is 3 (the first value at *Level 3*) less than 7? Yes; the next node is null.
- **Step 2:** Move down to *Level 3*:
 - Now, drop down to node 3 at *Level 3* and the next node is 5.
 - Is 5 less than 9? Yes. Move to node 5. The next node which in null.

- **Step 3:** Move down to *Level 2*:
 - Now, drop down to node 5 at *Level 3* and the next node is 8.
 - Is 8 less than 9? No.
- **Step 4:** Move down to *Level 1*:
 - The next node is 7. The target is found.

The total number of comparisons is three, demonstrating a clear improvement over the sequential search in *Level 1* that would be required in a standard linked list. Skip lists are significantly more efficient than linked lists, especially when dealing with large datasets.

The following is a search method that must be added to the previously described `SkipList` class:

```
def search(self, value):
    current = self.head
    for i in range(self.level, -1, -1):
        while current.forward[i] and current.forward[i].value <
value:
            current = current.forward[i]
        current = current.forward[0]
        if current and current.value == value:
            return True
    return False
```

The following is an example usage of search in a `SkipList` class:

```
if __name__ == "__main__":
    skiplist = SkipList(3)
    skiplist.insert(3)
    skiplist.insert(4)
    skiplist.insert(5)
    skiplist.insert(7)
    skiplist.insert(8)
    skiplist.insert(9)
    skiplist.insert(10)
    skiplist.print_skiplist()
    value_to_search = 7
    found = skiplist.search(value_to_search)
    print(f"\nSearch for {value_to_search}: {'Found' if found else
'Not Found'}")
    value_to_search = 6
    found = skiplist.search(value_to_search)
    print(f"Search for {value_to_search}: {'Found' if found else 'Not
Found'}")
```

By inserting the example data, it produces the following results:

```
Skip List:
Level 1: 3 --> 8 --> 9 --> 10 --> None
Level 0: 3 --> 4 --> 5 --> 7 --> 8 --> 9 --> 10 --> None
Search for 7: Found
Search for 6: Not Found
```

To conclude our discussion on skip lists, we will skip the detailed explanation of deletion. Deletion in a skip list involves first performing a search, followed by updating the necessary pointers.

Skip lists offer efficient average-case time complexities for search, delete, and insert operations, typically performing at $O(\log n)$. This efficiency is achieved through the use of multiple levels of linked lists, allowing operations to skip over large portions of data, similar to a binary search tree. In the best case, operations can approach $O(1)$ when the target element is close to the starting point. However, due to the probabilistic nature of skip lists, there is a small chance that the structure could degenerate, leading to a worst-case time complexity of $O(n)$ for these operations. Despite this, the average-case performance remains robust, making skip lists a practical choice for dynamic data structures where balanced search, insertion, and deletion times are crucial.

Stacks

A **stack** is a linear data structure that follows the LIFO principle, meaning that the last element added to the stack is the first one to be removed. Think of it as a stack of plates: we add new plates on top, and when we need a plate, we take the top one off first. Stacks are used in various applications, including expression evaluation, function call management, and undo mechanisms in software.

Stacks have several defining characteristics that influence their behavior and performance:

- **LIFO order:** The last element inserted into the stack is the first one to be removed. For example, if we push the numbers 1, 2, and 3 onto a stack, we will pop them off in the reverse order: 3, 2, 1.
- **Operations performed at one end:** All insertions (push) and deletions (pop) are performed at the top of the stack. There is no direct access to elements in the middle or bottom of the stack. If we push several elements onto a stack, we can only access the most recently added element directly. It also means there is no random access to the stack. Unlike arrays, we cannot directly access elements at a specific index in a stack. Access is restricted to the top element only.
- **Dynamic size:** Stacks can dynamically grow or shrink as elements are pushed or popped. As we push more elements onto a stack, its size increases, and as we pop elements off, its size decreases.

Stacks support a few fundamental operations, each with specific performance characteristics. The following are these major operations, along with their time complexities and examples:

- **push:** It adds a new element to the top of the stack. Its time complexity is $O(1)$, as the operation involves simply adding an element to the top of the stack. The following is a simple Python example:

```
stack = []
stack.append(3)  # Stack is now [3]
stack.append(5)  # Stack is now [3, 5]
print(stack)
```

- **pop:** This removes the top element from the stack. It performs the removal in constant time ($O(1)$). The following is a Python example:

```
top_element = stack.pop()
print(stack)
```

- **peek:** It is for retrieving the value of the top element without removing it from the stack. For the stack `[1, 2, 3]`, peeking would return 3 without altering the stack. The time complexity is $O(1)$, as it only involves accessing the top element. A simple Python instruction is as follows:

```
top_element = stack[-1]
```

- **search:** As usual, this is for finding an element in the stack. For example, if we search for element 2 in the stack `[1, 2, 3]`, we would find it at position 1 from the top. Obviously, the time complexity of `search` in stacks is $O(n)$, as it may require scanning through the entire stack from top to bottom. Let's see a simple Python example:

```
element_to_find = 2
position = stack.index(element_to_find)  # Finds the position of
2 in the stack
```

- **edit:** This modifies the value of the top element in the stack. If the top element of the stack `[1, 2, 3]` is 3 and we want to change it to 5, the stack would become `[1, 2, 5]`. It does this operation in constant time ($O(1)$) since the operation only affects the top element. A Python code example is as follows:

```
stack[-1] = 5  # Changes the top element to 5; Stack becomes [3,
5]
```

Some programming languages also support additional operations such as `isFull`, which checks whether the stack is full, and `isEmpty`, which determines whether the stack is empty.

Stacks are extensively used in computer programming for several key applications. Here are a few important use cases:

- Stacks store function calls, pushing each invocation onto the stack and popping it when the function completes
- They are used for converting and evaluating expressions, particularly for changing infix expressions to postfix or prefix notation
- Stacks implement undo features in software, where each action is pushed onto the stack and can be undone by popping it off

Stacks are a fundamental and versatile data structure used in many algorithms and applications. They provide efficient push and pop operations with a time complexity of $O(1)$ and are particularly useful in scenarios where the last element added needs to be the first one removed. However, their lack of random access and LIFO nature can make them less suitable for situations where elements need to be accessed in a different order. Understanding the trade-offs and appropriate use cases for stacks is essential for effective algorithm design and implementation. In the next section, we will explore queues, which operate in a manner opposite to stacks.

Queue

A **queue** operates on the FIFO principle, meaning it has a behavior opposite to stacks: The first element added is the first to be removed. This structure is analogous to a line of people waiting for service: The first person in line is the first to be served. Queues are used in various scenarios, including task scheduling, buffering, and managing resources in computer systems.

Queues have several characteristics that influence their behavior and performance:

- **FIFO order:** The first element inserted into the queue is the first one to be removed.
- **Operations at opposite ends:** Elements are added at the rear (end) of the queue and removed from the front (beginning) of the queue.
- **Dynamic size:** Queues can dynamically grow or shrink as elements are enqueued or dequeued. As we enqueue more elements, the size of the queue increases, and as we dequeue elements, the size decreases.
- **No random access:** Like stacks and unlike arrays, we cannot directly access elements at a specific index in a queue. Access is restricted to the front element only.

Queues support some fundamental operations, each with specific performance characteristics. The following are the major operations:

- **enqueue:** This involves adding a new element to the rear of the queue. Consider a queue that currently holds the elements 1 and 2. Enqueueing 3 into the queue would place it at the rear, resulting in the queue [1, 2, 3]. This operation is performed in constant time or $O(1)$, as the operation involves simply adding an element to the end of the queue. A simple Python example is as follows:

```
queue = []
queue.append(1)  # Queue is now [1]
queue.append(2)  # Queue is now [1, 2]
queue.append(3)  # Queue is now [1, 2, 3]
print(queue)
```

- **dequeue:** It removes the front element from the queue. If we dequeue the queue [1, 2, 3], the front element 1 is removed, leaving us with the queue [2, 3]. Similar to enqueue, the time complexity here is $O(1)$, as it involves removing the front element. Here is a simple Python instruction:

```
front_element = queue.pop(0)
```

- **peek:** This is for retrieving the value of the front element without removing it from the queue and it performs in constant time. For the queue [1, 2, 3], peeking would return 1 without altering the queue. The Python instruction for peeking can be like the following:

```
front_element = queue[0]
```

- **search:** A queue conducts a search in linear time, as it may require scanning through the entire queue from front to rear. The following is a Python code implementing a search in a queue:

```
target = 2
position = queue.index(target)
```

Some programming languages also support additional operations such as `isFull`, which checks whether the queue is full, and `isNull`, which determines whether the queue is empty.

Similar to stacks, queues have numerous applications in computer programming. A well-known example is **task scheduling**, where queues are used in operating systems to manage processes. Processes are enqueued when they are ready to run and dequeued once they are executed. Another example is buffering, where queues temporarily store data before it is processed, such as in print queues or streaming services. Lastly, queues play a crucial role in resource management, ensuring that access to shared resources is handled in the order requests are received.

Queues are a fundamental data structure used in many algorithms and applications, particularly in scenarios that require managing tasks or resources in a FIFO manner. They provide efficient enqueue and dequeue operations with a time complexity of $O(1)$ but do not allow random access to elements. Understanding the appropriate use cases for queues is essential for effective algorithm design and implementation. In the final section of this chapter, we expand on the concept of a queue by introducing a structure that has two ends, effectively combining the functionalities of both a stack and a queue.

Deque

A **deque** is a versatile data structure that allows the insertion and deletion of elements from both ends of the sequence, making it a generalization of both stacks and queues. Deques can operate as both a stack and a queue, providing greater flexibility in how elements are added or removed.

Let's highlight the properties of deques:

- **Insertions and deletions at both ends:** Elements can be added or removed from either the front or the rear of the deque. For example, we can push elements onto the front or rear of the deque and pop them off from either end as well.
- **No fixed direction:** Deques do not enforce a strict LIFO or FIFO order; instead, they allow operations at both ends. For instance, we can treat a deque as a stack by only using one end or as a queue by using both ends.
- **Dynamic size:** Deques can dynamically grow or shrink as elements are added or removed from either end.
- **No random access:** Like stacks and queues, deques do not allow direct access to elements at a specific index.

Deques support a wide range of operations, each with specific performance characteristics. The following are the major operations, along with their time complexities and examples:

- **Add to front (insert at the front):** This involves adding a new element to the front of the deque. For example, if the deque currently holds `[2, 3]`, adding 1 to the front would result in `[1, 2, 3]`. This is done in constant time, as the operation involves simply adding an element to the front. Here is a simple piece of Python code:

```
from collections import deque
d = deque([2, 3])
d.appendleft(1) # Deque is now [1, 2, 3]
print(d)
```

- **Add to rear (insert at the rear):** This adds a new element to the rear of the deque. If the deque currently holds `[1, 2]`, adding 3 to the rear would result in `[1, 2, 3]`. Similar to *add to front*, the time complexity is $O(1)$. The following is a simple Python instruction to do this operation:

```
d.append(3) # Deque is now [1, 2, 3]
```

- **Remove from front (delete from the front):** This operation removes the front element from the deque in constant time. If we remove the front element from the deque `[1, 2, 3]`, it becomes `[2, 3]`. A simple Python instruction to execute the *remove from front* operation is as follows:

```
front_element = d.popleft
```

- **Remove from rear (delete from the rear):** Similar to the previous operation, only from the rear of the deque. The Python code for this operation is as follows:

```
rear_element = d.pop()
```

- **Peek (access the front or rear element):** Retrieve the value of the front or rear element without removing it from the deque in constant time. For the deque `[1, 2, 3]`, peeking the front would return 1, and peeking the rear would return 3:

```
front_element = d[0] #Returns the front element w/o removing  
it; Deque remains [1, 2]  
rear_element = d[-1] #Returns the rear element w/o removing it;  
Deque remains [1, 2]
```

It's important to note that many of the functions used for a deque may or may not be supported by different programming languages. Fortunately, Python provides built-in support for these functions.

Deques have several important use cases in computer programming. One key example is task scheduling, where deques are used in scheduling algorithms to add or remove tasks from either end of the queue as needed. Another use case is in sliding window algorithms, where deques are employed to efficiently manage elements as they are added or removed from the window while it moves across a dataset. Lastly, deques are also used in implementing undo/redo functionality in applications, allowing actions to be added or removed from either end of the deque.

Deques are a flexible data structure that generalizes both stacks and queues, allowing insertion and deletion at both ends. They provide efficient operations with a time complexity of $O(1)$ for adding or removing elements at either end. Deques are particularly useful in scenarios where elements need to be managed from both ends, such as task scheduling, sliding window algorithms, and undo/redo functionality. Understanding the appropriate use cases for deques is essential for effective algorithm design and implementation.

Summary

In this chapter, we explored various linear data structures, focusing on their definitions, characteristics, and operations. We began with an in-depth look at arrays and linked lists, examining how they handle fundamental operations such as insertion, deletion, and searching. We discussed the trade-offs between these structures, noting the efficiency of arrays in accessing elements and the flexibility of linked lists in dynamic memory allocation. The chapter also covered more advanced linear structures such as stacks, queues, and deques, illustrating their practical applications in computer programming, from task scheduling to expression evaluation and resource management.

We then introduced the skip list, a probabilistic data structure that combines the advantages of both arrays and linked lists, offering efficient search, insertion, and deletion operations. Through detailed examples, we demonstrated how skip lists improve upon the limitations of traditional linked lists. As we concluded the chapter, we emphasized the importance of understanding the appropriate use cases for each data structure to optimize algorithm design. You now have a thorough understanding of linear data structures, which play a crucial role in algorithm design. In the next chapter, we will shift our focus to non-linear data structures, exploring their unique properties and applications.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022:
 - *Chapter 10, Elementary Data Structures*
- *Algorithms*. By R. Sedgewick, K. Wayne. Fourth Edition. Addison-Wesley. 2011:
 - *Chapter 1, Fundamentals*
- *Data Structures and Algorithm Analysis in C++*. By Mark A. Weiss. Fourth Edition. Pearson. 2012:
 - *Chapter 3, Lists, Stacks, and Queues*

Non-Linear Data Structures

Non-linear data structures form a crucial class of data structures with extensive applications in designing efficient algorithms. Unlike linear data structures, such as arrays and linked lists, non-linear structures allow data elements to be stored and accessed in a more complex, hierarchical manner. These structures enable the efficient handling of relationships, dependencies, and hierarchical data, making them vital for solving a wide range of computational problems.

In this chapter, we begin by exploring the general properties and characteristics that define non-linear data structures. Following this, we discuss two major groups: graphs and trees. **Graphs** are versatile structures used to model relationships between objects, while **trees** represent hierarchical relationships in a more structured form. Finally, we examine a special case of binary trees, known as **heaps**, which are essential for implementing efficient algorithms such as heapsort. Learning about non-linear data structures is essential for algorithm design, as they play a key role in many algorithms, including those for sorting and searching. Additionally, graphs, trees, and other forms of non-linear data structures are extensively used in advanced fields such as AI, machine learning, and optimization, where efficient data management and processing are critical.

This chapter covers the following topics:

- Introduction to non-linear data structures
- Graphs
- Trees
- Heaps

Introduction to non-linear data structures

In *Chapter 11*, we introduced the concept of **abstract data types (ADTs)**, classifying them into two main categories: linear and non-linear. We followed this with an in-depth discussion of linear data structures in *Chapter 12*, where we examined their relevance to our core focus – designing and analyzing efficient algorithms. While we touched on many essential aspects of linear data structures,

it is worth noting that this area of study is vast and could easily warrant its own in-depth exploration. For those interested in deeper exploration of data structures, we have included references at the end of both *Chapters 11* and *12*.

In this chapter, our focus shifts to non-linear data structures. Like the previous chapter, we will approach this topic with an eye on its relationship with efficient algorithm design. Our goal is not merely to present the various types of non-linear data structures but to highlight their roles and applications in improving the performance of algorithms.

Let's begin by briefly discussing what defines non-linear data structures, their key characteristics, and some of the most commonly used types.

Unlike linear data structures, where elements are arranged in a sequential manner (e.g., arrays, linked lists), non-linear data structures organize data in a hierarchical or interconnected way. In non-linear structures, each element may be connected to multiple other elements, forming complex relationships that enable more efficient data processing for certain operations.

The following are the key features of non-linear data structures:

- **Hierarchical relationships:** Elements are structured in a way that reflects a hierarchy, meaning some elements may act as **parents** while others are **children**. This is particularly true for structures such as **trees** and **graphs**.
- **Complex traversal patterns:** Unlike linear structures, where traversal is relatively straightforward, moving through non-linear data structures requires more sophisticated techniques, often specific to the structure being used.
- **Variable access time:** The time it takes to search, insert, or delete elements can vary greatly depending on the structure and the way it is implemented. In many cases, non-linear data structures allow for more efficient operations compared to their linear counterparts.

Non-linear data structures are composed of several key elements that define their structure and functionality. The most important components include nodes, edges, parents, children, roots, leaves, and subtrees. Let's understand these components in detail:

- **Nodes:** A node or vertex is the fundamental building block of most non-linear data structures. Each node typically contains data and may also have connections to other vertices, depending on the type of structure. In trees, for example, vertices represent individual elements within the hierarchy. For example, in a social network graph, each node represents a user, and the data stored in the node might be the user's profile information.
- **Edges:** An edge or arrow is a link or connection between two nodes. In non-linear structures, edges play a critical role in defining the relationships between nodes. In a binary tree, edges define the relationship between a parent and its children. For example, if a parent node represents a manager, the edges would connect to child nodes that represent their employees. In a graph, an edge represents a relationship between two entities. In a transportation network, for instance, an edge might represent a direct flight between two cities.

- **Parents and children:** In hierarchical non-linear data structures such as trees, nodes are organized in levels, where parent nodes are directly connected to the child nodes below them. The parent-child relationship is a fundamental concept in trees:

- **Parent:** A node that has one or more child nodes directly beneath it
- **Child:** A node that is directly connected to another node above it (its parent)

For example, in a corporate hierarchy tree, a manager is a parent node, and their subordinates are the child nodes.

- **Root:** The root is the topmost node in a tree structure and acts as the starting point for traversing the tree. A tree can only have one root, and all other nodes are descendants of this root. If a node has no parent, it is considered the root. In a filesystem, the root directory is the topmost folder, and all other directories or files branch out from it.
- **Leaf:** A leaf is a node that has no children. Leaves represent the endpoints of a tree structure, where no further branching occurs. These are crucial in many algorithms, as they often signal completion points in traversals or searches.
- **Subtree:** A subtree is a smaller portion of a tree that includes a node and all its descendants. Subtrees allow trees to be treated recursively, where each node, along with its children, can be considered a tree on its own. In a decision tree, each node and its branches form a subtree, representing a subset of possible decisions.

There are several types of non-linear data structures, each suited to different kinds of algorithmic problems. The most common non-linear data structures include graphs, trees, and heaps. In the following sections, we will examine these three fundamental non-linear data structures in detail. Each of the structures has unique characteristics and applications, and understanding them is crucial for designing efficient algorithms. We will explore their properties, how they are implemented, and the roles they play in solving various computational problems.

Graphs

Graphs are one of the most versatile and widely used non-linear data structures. They are used to represent relationships between entities, where entities are represented as nodes (also called **vertices**) and relationships as edges. Graphs can model a wide range of real-world problems, from social networks to transportation systems and communication protocols.

Graphs come in various types, each with specific properties that make them suitable for different tasks. Here are the most common types of graphs:

- **Undirected graphs:** In undirected graphs, edges have no direction. The relationship between two nodes is bidirectional (see *Figure 13.1*). If there is an edge between node A and node B, you can traverse from A to B or from B to A without any restrictions. An example of an undirected graph is the social network of Facebook friends, where connections are mutual.

This means if A is friends with B, then B is also friends with A, reflecting the bidirectional nature of the relationship.

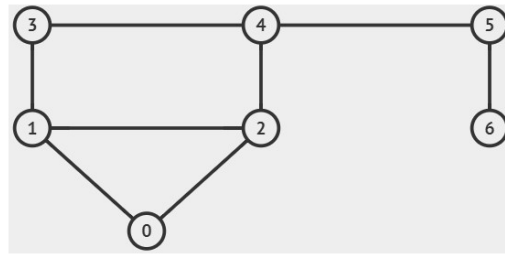


Figure 13.1: An undirected graph

- **Directed graphs (digraphs):** In digraphs, edges have a direction. The relationship between nodes is one-way, meaning that if there is a directed edge from node A to node B, you can only traverse from A to B, not the other way around, for example, a website where pages have links pointing to other pages, forming a digraph. *Figure 13.2* shows an example of digraphs. As illustrated, graphs do not have to be fully connected.

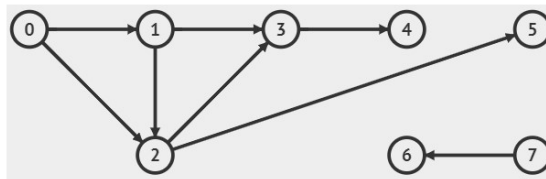


Figure 13.2: A digraph

- **Weighted graphs:** In a weighted graph, each edge is assigned a numerical value or **weight**. This weight often represents a cost, distance, or time associated with the connection between nodes. An example of a weighted graph is a road network where the weight of an edge represents the distance or travel time between cities. *Figure 13.3* depicts a weighted graph, where the weights are assigned to the edges.

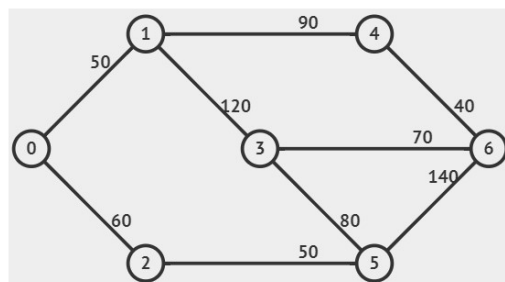


Figure 13.3: A weighted graph

- **Unweighted graphs:** In unweighted graphs, all edges have equal significance, meaning that there is no particular cost or distance associated with traveling between nodes. In *Figure 13.1*, the graph is unweighted.
- **Cyclic and acyclic graphs:** A cyclic graph contains at least one cycle, meaning you can start from a node, traverse edges, and return to the same node. An acyclic graph has no such cycles, making them essential for applications such as task scheduling. The graph in *Figure 13.1* has cycles.
- **Signed graphs:** In signed graphs, edges are labeled with either positive or negative signs, typically representing favorable or unfavorable relationships. These graphs are useful in scenarios where relationships can have polarities, such as in social networks, where edges could represent friendship (positive) or conflict (negative). Social networks in which positive edges represent friendships and negative edges represent rivalries is an example of a signed graph.
- **Hypergraphs:** A hypergraph generalizes the concept of a graph by allowing edges (called **hyperedges**) to connect more than two nodes at a time. This type of graph is particularly useful in representing complex relationships where a single connection might involve multiple entities. For example, in a research collaboration network, a hyperedge might represent a publication authored by three or more researchers, connecting all of them simultaneously.

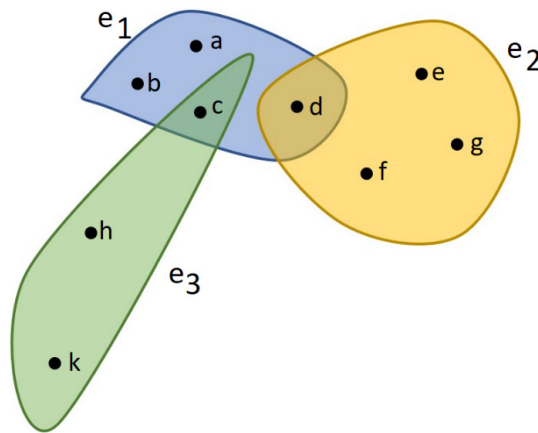


Figure 13.4: A hypergraph

Figure 13.4 illustrates an example of a hypergraph with the following components: The set of vertices is $V = \{a, b, c, d, e, f, g, h, k\}$, and the set of hyperedges is $E = \{e_1, e_2, e_3\}$. Each hyperedge connects multiple vertices as follows: $e_1 = \{a, b, c, d\}$, $e_2 = \{d, e, f, g\}$, and $e_3 = \{c, h, k\}$.

In the following section, we will explore how to represent some of the most commonly used graphs in algorithms and discuss the complexities associated with each representation method.

Graphs representation

Graphs can be represented in several ways, each method being well suited to particular use cases depending on the graph structure and the type of operations needed. In the following subsections, we will examine three common methods for graph representation and their key properties. While evaluating each method, we will focus on three critical aspects of their performance:

- **Space complexity:** How much memory is required to store the graph using the chosen representation
- **The time complexity for accessing all neighbors of a node:** The efficiency of retrieving all nodes that are directly connected (adjacent) to a given node
- **The time complexity for checking edge existence:** The time required to determine whether an edge exists between two specific nodes

By analyzing these complexities, we can better understand the strengths and limitations of each graph representation and how they apply to different algorithmic tasks.

Adjacency matrix

An **adjacency matrix** represents a graph using a $V \times V$ matrix, where V is the number of nodes or vertices in the graph. Each cell in the matrix at position (i, j) indicates the presence (and possibly the weight) of an edge between node i and node j . In an undirected graph, the adjacency matrix D is symmetric, meaning that $D = D'$, where D' represents the transpose of matrix D . This symmetry arises because the edges in an undirected graph have no direction, so if there is an edge between node i and node j , the relationship is mutual. Therefore, both $D(i, j)$ and $D(j, i)$ will have the same value.

For unweighted, unsigned graphs, the adjacency matrix is a binary matrix, where each entry is either 0 or 1. A value of 1 at $D(i, j)$ indicates that there is an edge between vertices i and j , and a 0 means no edge exists between them.

Example 13.1:

Here is a matrix representing a digraph with weights:

A --> B (2)

B --> C (3)

A --> C (4)

The adjacency matrix representing this simple graph is a 3×3 matrix as follows:

$$\begin{array}{c|cc} & A & B & C \\ \hline A & 0 & 2 & 4 \\ B & 0 & 0 & 3 \\ C & 0 & 0 & 0 \end{array}$$

It is obvious that the space complexity of this representation is $O(V^2)$. Regardless of the number of edges, the adjacency matrix requires space proportional to the square of the number of vertices because it must account for every possible edge between all pairs of vertices. This can be inefficient for large graphs, especially if the graph is sparse (i.e., has relatively few edges compared to the number of possible edges).

In graphs, one of the operations is accessing all neighbors of a node in the graph. Using adjacency matrix representation, the time complexity is $O(V)$. To find all neighbors of a given node, we need to inspect all entries in the corresponding row (or column) of the matrix. For a node i , you scan the i^{th} row to check which vertices have a direct edge from i , which reminds us of a linear search algorithm in which we want to report all non-zero entries. This operation requires examining V entries, resulting in a time complexity of $O(V)$, regardless of the number of actual neighbors the node has. In contrast, checking whether an edge exists in the adjacency matrix representation has a time complexity of $O(1)$, as we can directly access any edge in constant time. Since the adjacency matrix is a two-dimensional array, checking whether an edge exists between node i and node j is a direct access operation to the cell at position (i, j) . Simply check whether the value at (i, j) is non-zero (for weighted graphs) or *true* or 1 (for unweighted graphs).

When should we use an adjacency matrix?

It is particularly useful when the graph is dense (i.e., the number of edges is close to V^2), as the high space requirement becomes less significant in such cases. Additionally, if our algorithm needs constant-time edge existence checks, the adjacency matrix offers a clear advantage. Lastly, the adjacency matrix is simple to implement, making it a practical choice for certain algorithms where ease of use is a priority.

Adjacency list

In an **adjacency list**, each node stores a list of its neighboring nodes (or nodes it is connected to). This is a more space-efficient representation, especially for sparse graphs. The adjacency list for *Example 13.1* is as follows:

A: B(2), C(4)

B: C(3)

C: -

The adjacency list representation uses $O(V + E)$ space, where E is the number of edges. This is generally more efficient for sparse graphs.

Accessing the neighbors of a node (i.e., adjacent vertices) takes $O(k)$, where k is the degree of the node (the number of edges connected to the node). In the worst case, this could be $O(V)$, but for most practical applications, it's much smaller. In the preceding adjacency list, the nodes listed after the colon represent the neighbors, or adjacent nodes, of the node on the left side.

To check whether there is an edge between two specific vertices, you need to traverse the list of adjacent vertices. This takes $O(k)$ time, since we may need to scan through all neighbors of the source node. In the worst case, this is $O(V)$, but typically it is smaller for sparse graphs.

When to use an adjacency list

An adjacency list is most effective when the graph is sparse – that is when the number of edges is much smaller than V^2 . Since the space complexity is $O(V + E)$, it is more memory-efficient for graphs with fewer edges. Additionally, if our algorithm frequently needs to access all neighbors of a node, the adjacency list provides efficient $O(k)$ access. It is also beneficial for dynamic graphs where nodes and edges are frequently added or removed, as updating an adjacency list is straightforward and less costly in terms of memory.

Edge list

An **edge list** explicitly stores all the edges in a graph along with their weights (if any). This is useful when the graph is sparse and you need to work primarily with the edges. The edge list in the *Example 13.1* graph is as follows:

(A, B, 2)

(A, C, 4)

(B, C, 3)

The edge list representation uses $O(E)$ space. This is efficient for very sparse graphs. To access all neighbors of a node, the edge list does not directly store them, so finding all neighbors requires scanning the entire edge list, which has a worst-case time complexity of $O(E)$. Likewise, checking whether an edge exists between two vertices also involves scanning the entire edge list, resulting in a time complexity of $O(E)$.

When to use an edge list

An edge list is ideal for very sparse graphs where we primarily need to work with the edges themselves rather than frequently accessing neighbors or checking edge existence. It uses $O(E)$ space, making it highly memory-efficient for graphs with few edges. However, its primary limitation is that it is not efficient for operations such as finding all neighbors of a node or checking whether a specific edge exists, both of which require scanning through the entire list. Therefore, edge lists are best suited for algorithms where working directly with the edges is the focus, such as in certain edge-centric algorithms such as Kruskal's minimum spanning tree algorithm.

Traversing graphs

Traversal is a fundamental operation in graph algorithms, where the goal is to visit all nodes in a specific sequence. The two most widely used graph traversal techniques are **depth-first search (DFS)** and **breadth-first search (BFS)**. In the sections that follow, we will explore both of these methods in detail, highlighting their processes, use cases, and complexities.

DFS graph traversal

DFS is a graph traversal technique that explores as far along each branch or path as possible before backtracking. It is typically implemented using recursion or a stack, and it works well in exploring deep structures or discovering specific paths in a graph.

The idea behind DFS is to start at an arbitrary node (usually called the *root*) and explore each branch of the graph as deeply as possible before moving on to the next branch. DFS follows these basic steps:

1. Visit the starting node.
2. For each unvisited neighbor of the current node, perform a DFS on that neighbor.
3. Repeat this process until all nodes reachable from the starting node are visited.

Let's consider the graph in *Figure 13.5* with six nodes.

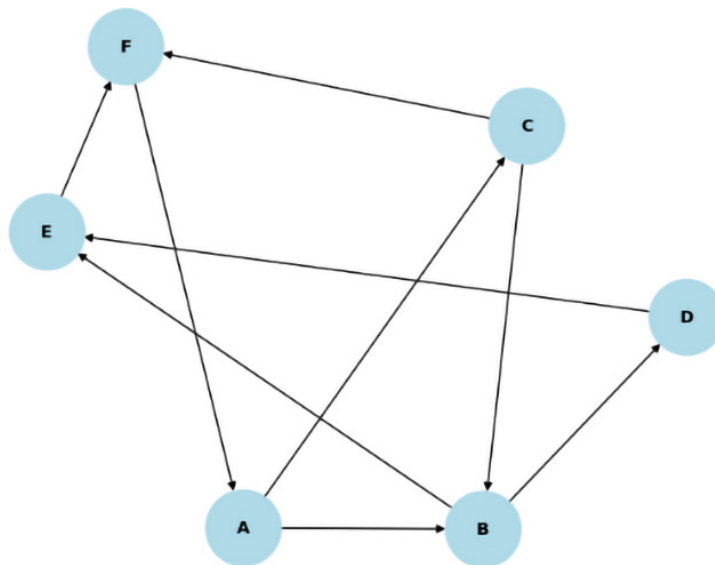


Figure 13.5: An example graph

Starting the DFS at *node A*, the traversal order would be this: A --> B --> D --> E --> F --> C. This traversal explores one path (branch) completely before moving on to the next.

Here is a Python implementation of DFS using recursion. First, we define a simple graph:

```
# You will need first: pip install networkx matplotlib
import networkx as nx
import matplotlib.pyplot as plt
# Define the graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['B', 'F'],
    'D': ['E'],
    'E': ['F'],
    'F': ['A']
}
```

The following code visualizes the example graph:

```
# Visualize the graph
visualize_graph(G)
# Create a directed graph using NetworkX
G = nx.DiGraph()
# Add edges to the graph
for node, neighbors in graph.items():
    for neighbor in neighbors:
        G.add_edge(node, neighbor)
# Visualize the graph using NetworkX and Matplotlib
def visualize_graph(G):
    pos = nx.spring_layout(G) # Positions for all nodes
    nx.draw(G, pos, with_labels=True, node_color='lightblue', node_
size=2000, font_size=10, font_weight='bold')
    plt.title("Graph Visualization")
    plt.show()
```

Now, we implement the DFS traversal algorithm:

```
# DFS function (optional, same as before)
visited = set()
def dfs(node):
    if node not in visited:
        print(node)
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)
# Start DFS at node 'A'
dfs('A')
```

Let's examine the code:

- The graph is represented as an adjacency list
- We use a `visited` set to ensure that nodes are not revisited
- The `dfs` function prints the current node, marks it as visited, and recursively calls itself on all unvisited neighbors

For the given graph, the output will be as follows:

```
A, B, D, E, F, C
```

DFS is more memory efficient compared to BFS, particularly in deep graphs. This is because DFS only needs to keep track of the current path and backtracking information, while BFS must store all nodes at each level of the graph. DFS is also beneficial for pathfinding, especially in scenarios where all possible paths need to be explored, such as maze-solving algorithms. Additionally, DFS is frequently used for topological sorting in **directed acyclic graphs (DAGs)**, a technique useful in scheduling tasks and resolving dependencies.

However, DFS has some limitations. One major drawback is that DFS may not find the shortest path in an unweighted graph, as it could explore a deep path before reaching the solution, which might not be optimal. Additionally, in very large or infinite graphs, DFS can get stuck exploring long paths or cycles unless precautions, such as cycle detection, are in place. In languages such as Python, which have a limit on recursion depth, using a recursive DFS can lead to stack overflow errors if the graph is particularly deep. To avoid this, DFS can be implemented iteratively using an explicit stack instead of relying on recursion.

The time complexity of DFS is $O(V + E)$, where V is the number of vertices and E is the number of edges. This is because DFS visits every node and every edge in the graph once. In a sparse graph where $E \approx V$, the time complexity is close to $O(V)$. On the other side, in a dense graph where $E \approx V^2$, the time complexity approaches $O(V^2)$.

The space complexity of DFS is $O(V)$ in the worst case due to the depth of the recursion stack or the explicit stack used in the iterative version. In a worst-case scenario, where the graph is a long linear chain, the stack could hold all V nodes.

There are two common variants of DFS: **pre-order DFS**, which visits a node before exploring its neighbors (as seen in the earlier example), and **post-order DFS**, which visits a node only after visiting all its neighbors. In the same graph (*Figure 13.5*), if we perform a post-order DFS, the traversal will result in the following order: $D \rightarrow F \rightarrow E \rightarrow B \rightarrow C \rightarrow A$. These variants are useful in different scenarios, such as tree traversal and algorithms that require specific orderings of node processing.

DFS has numerous applications in algorithm design, including its use in AI search algorithms. For instance, DFS is employed in problems that require exploring all possible paths between nodes, such as solving puzzles or finding paths through mazes. It is also highly effective for cycle detection in both directed and undirected graphs, helping to identify loops within the graph structure. Additionally,

DFS can be used to find all connected components in a graph, particularly in undirected graphs. In DAGs, DFS plays a crucial role in topological sorting, which is essential for tasks such as scheduling and dependency resolution. These diverse applications highlight the importance of DFS in various computational problems.

In summary, DFS is a powerful and efficient graph traversal technique, especially useful for pathfinding and solving problems that require exploring all possibilities. Although it doesn't guarantee the shortest path, it excels in scenarios where memory efficiency is critical or when dealing with deep structures. The key trade-offs include potentially long search times for large graphs and the risk of stack overflow in deep recursion. In the next section, we will explore the BFS traversal approach.

BFS graph traversal

BFS is another graph traversal algorithm that explores nodes level by level, starting from a given source node. Unlike DFS, which goes as deep as possible into one branch before backtracking, BFS explores all neighbors of a node before moving on to the next level of neighbors. This makes BFS particularly effective for finding the shortest path in unweighted graphs.

The BFS algorithm starts at the root (or any arbitrary starting node) and explores all its neighbors first. After visiting all neighbors at the current level, it proceeds to the next level, visiting the neighbors of those neighbors, and so on. The traversal continues until all nodes reachable from the starting node have been visited.

The core of BFS relies on using a queue data structure, which ensures that nodes are explored in the correct order (**first-in first-out (FIFO)**). Let's consider the graph in *Figure 13.6* with six nodes. Starting the BFS at *node A*, the traversal order would be A --> B --> C --> D --> E --> F. This traversal explores one path (branch) completely before moving on to the next.

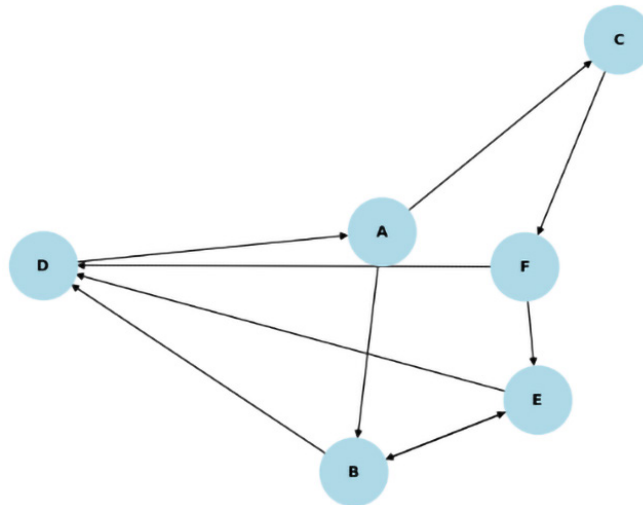


Figure 13.6: An example graph for BFS traversal

Here is a Python implementation of BFS using a queue:

```
from collections import deque
# Graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': ['A'],
    'E': ['B', 'D'],
    'F': ['E', 'D']
}
# BFS function
def bfs(start_node):
    visited = set() # Set to track visited nodes
    queue = deque([start_node]) # Initialize the queue with the
    starting node
    while queue:
        node = queue.popleft() # Dequeue a node
        if node not in visited:
            print(node)
            visited.add(node) # Mark it as visited
            queue.extend(graph[node]) # Enqueue all unvisited
neighbors
# Start BFS at node 'A'
bfs('A')
```

This is how the code works. First, we represent the graph using an adjacency list. Next, we implement the queue data structure, which is essential for the BFS algorithm. Finally, we implement the BFS traversal algorithm itself:

- **Graph representation:** The graph is represented as an adjacency list
- **Queue:** deque is used to efficiently handle queue operations (enqueueing and dequeueing nodes)
- **bfs:** Nodes are processed in the order they are dequeued, and their neighbors are added to the queue for further exploration

For the given graph, the output will be as follows:

```
A, B, C, D, E, F
```

BFS offers several advantages over DFS. In an unweighted graph, BFS guarantees that the first time a node is reached, it is via the shortest path from the source node, making it ideal for pathfinding problems. Additionally, BFS explores all nodes at the same level before moving on to the next level, which is particularly useful when all immediate neighbors need to be visited first, such as in finding

the shortest route in transportation systems. Furthermore, BFS is highly effective in identifying all connected components in an undirected graph, as it systematically explores all reachable nodes.

However, BFS has its drawbacks. It requires storing all nodes at the current level in memory, which can lead to significant memory consumption, particularly in graphs with large branching factors. Additionally, BFS can be inefficient for graphs with deep structures, as it explores level by level. In cases where the graph is deep but not very broad, DFS may be a more efficient alternative since it focuses on depth rather than breadth in its exploration.

The time complexity of BFS is $O(V + E)$, as each vertex and edge is processed exactly once during the traversal. In contrast, the space complexity of BFS is $O(V)$, since in the worst case, all nodes at a particular level could be stored in the queue simultaneously. This becomes particularly significant in graphs with large branching factors or wide levels, where the memory requirements can grow substantially.

BFS has two notable variants: **bidirectional BFS** and **multi-source BFS**. Bidirectional BFS is used to find the shortest path between two nodes. It runs two BFS traversals simultaneously – one from the source node and one from the destination node – until the two searches meet in the middle. This approach reduces the search space significantly, making it faster than a traditional BFS in scenarios where the source and destination are far apart. Multi-source BFS involves multiple starting points. BFS is initiated from all the source nodes at the same time, allowing exploration from multiple origins simultaneously. This variant is useful in scenarios where you need to explore paths from several locations, such as finding the shortest distance from multiple sources to a destination in a graph.

BFS has a wide range of applications, including its use in AI search strategies. One of its key strengths is its ability to find the shortest path in unweighted graphs. Since BFS explores all nodes at the same level before moving deeper, it guarantees that the first time a node is reached, it is via the shortest possible path. This makes BFS ideal for pathfinding algorithms, such as navigation systems or solving puzzles where all moves have equal cost.

Let's modify the BFS algorithm to find the shortest path between two nodes:

```
def bfs_shortest_path(start_node, target_node):
    visited = set()
    queue = deque([start_node]) # Queue stores paths
    while queue:
        path = queue.popleft() # Dequeue the first path
        node = path[-1] # Get the last node from the path
        if node == target_node:
            return path # Return the path when target is reached
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path) # Enqueue the new path
```

```
    return None # Return None if there is no path
# Find the shortest path between 'A' and 'F'
print(bfs_shortest_path('A', 'F'))
```

For the given graph, the shortest path between A and F is as follows:

```
['A', 'C', 'F']
```

Another notable application of BFS is in undirected graphs. BFS is particularly useful for identifying all nodes that are reachable from a given starting point. By doing this, BFS can efficiently detect and label connected components, which is essential in network analysis and social network mapping.

Additionally, BFS is widely used for level-order traversal in trees. In this scenario, BFS visits all nodes at each depth level before progressing to the next, making it an ideal approach when the hierarchy or levels of the nodes matter, such as in organization charts, filesystem structures, or hierarchical clustering.

Beyond these core applications, BFS is widely used in AI search algorithms, alongside DFS, as two major search techniques. Additionally, BFS is employed in finding the minimum spanning tree and the shortest path in various graph-related problems:

- **BFS in AI:** BFS serves as the foundation for many AI search strategies, particularly those that require exploration of all possible states level by level, such as in game trees or puzzle solving
- **Finding the minimum spanning tree:** In unweighted graphs, BFS can be used as a building block to find the minimum spanning tree by ensuring all nodes are visited in the shortest path order from the source
- **Network broadcast:** In computer networks, BFS is used to simulate broadcast routing, where information must be sent to all nodes in the shortest possible time, making it crucial for network discovery protocols such as **Open Shortest Path First (OSPF)**

These diverse applications highlight the versatility of BFS, making it a fundamental tool in algorithm design and practical implementations across various fields.

In summary, BFS is an essential graph traversal technique, particularly useful when the goal is to explore all nodes level by level or to find the shortest path in unweighted graphs. While it is efficient in terms of time complexity, its higher memory requirements can be a drawback for graphs with large branching factors. BFS's guaranteed shortest-path property and its versatility in various algorithmic tasks make it a powerful tool in many real-world applications.

Before moving on to the next non-linear data structure, let's conclude our discussion on graphs by summarizing their significance and applications in algorithm design. Graphs are incredibly versatile structures that allow us to model and solve a wide range of problems in areas such as networking, social analysis, and AI. They can represent complex relationships between entities, and through various algorithms such as BFS and DFS, we can efficiently traverse, search, and process graph data. Graphs play a central role in critical applications such as pathfinding, network routing, cycle detection, and even hierarchical problem-solving.

Their importance in algorithm design cannot be overstated, as they form the foundation for solving problems that involve connectivity, optimization, and search strategies in both theoretical and practical fields.

Trees

A **tree** is a hierarchical, non-linear data structure that consists of nodes connected by edges. Trees are widely used in various applications, such as organizing data, databases, network structures, and more. A tree has a single root node and all other nodes are connected in a parent-child relationship. The tree structure ensures that there are no cycles, and each child has exactly one parent.

In this section, we will explore different types of trees, their properties, and how to represent trees and discuss two important types: **binary search trees (BSTs)** and **red-black trees**.

Different types of trees and their properties

Trees come in many types, each with unique characteristics suited for different applications. Here are some of the most common types:

- **General tree:** A general tree is a type of tree where any node can have an arbitrary number of children. This type of tree can be used to represent hierarchical data, such as filesystems or organization charts. *Figure 13.7* illustrates an example of a general tree.

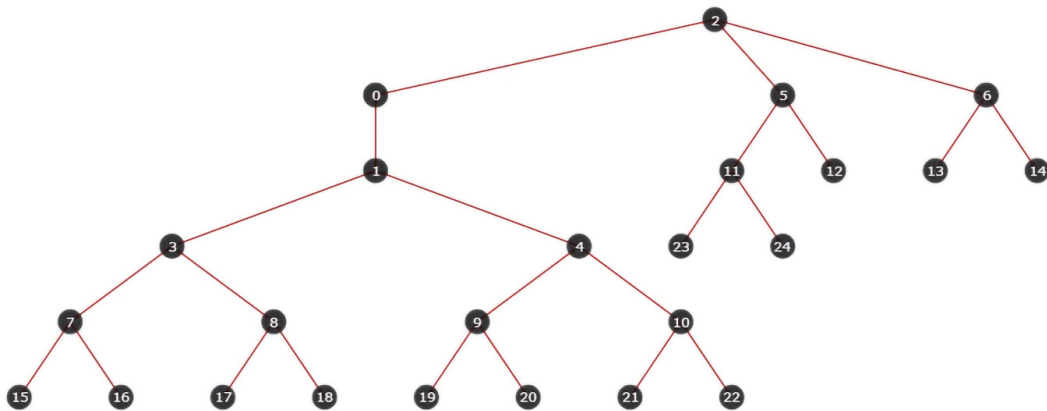


Figure 13.7: An example general tree

- **Binary tree:** A binary tree is a type of tree where each node has a maximum of two children, known as the **left child** and the **right child**. It is one of the most commonly used tree structures in computer science. In *Figure 13.7*, all subtrees rooted at *nodes 1, 5, and 6* are examples of binary trees.

- **Full binary tree:** A binary tree is considered full if every node has either zero or two children. No nodes have only one child. In *Figure 13.7*, the subtree rooted at *node 5* is a full binary tree.
- **Complete binary tree:** A complete binary tree is a binary tree where all levels are fully filled except possibly the last, which must be filled from left to right. A well-known complete binary tree is a heap structure, which will be discussed at the end of this chapter.
- **Balanced binary tree:** A tree is considered balanced if the heights of the left and right subtrees of any node differ by no more than one. Balanced trees are preferred because they ensure optimal performance for search, insertion, and deletion operations. In *Figure 13.8*, the binary tree is fully balanced.

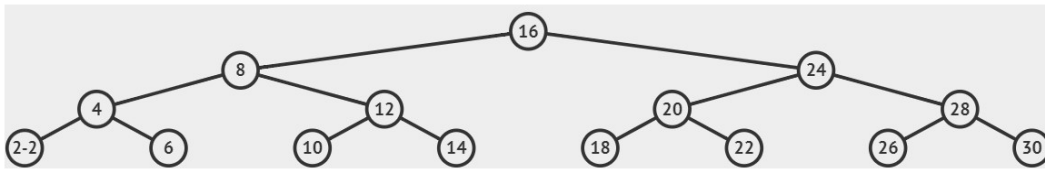


Figure 13.8: A perfectly balanced binary tree

Figure 13.9 illustrates a counterexample of a non-balanced binary tree. In this tree, the height of the subtrees differs significantly, violating the balanced binary tree property, where the heights of the left and right subtrees of any node should differ by no more than one. This imbalance can lead to inefficient operations such as searching, insertion, and deletion, as the tree structure begins to resemble a linear chain.

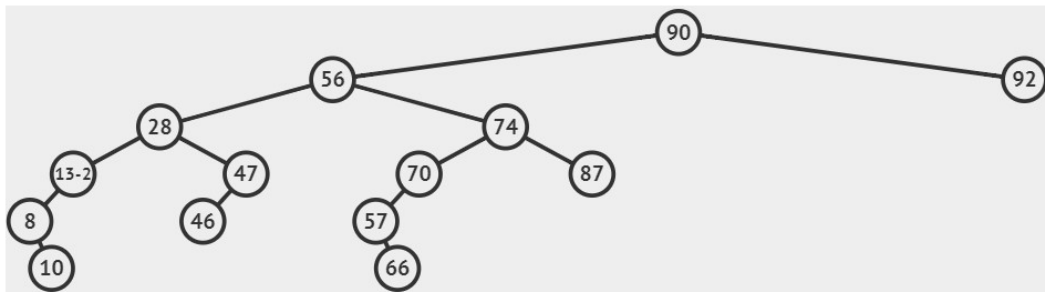


Figure 13.9: A counterexample of a non-balanced binary tree

- **AVL tree:** An AVL tree (named after inventors Adelson-Velsky and Landis) is a self-balancing binary search tree (BST). It maintains a balance factor (the difference between the heights of the left and right subtrees) for every node, ensuring that the tree remains balanced after insertions and deletions.
- **B-tree:** A B-tree is a self-balancing tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. B-trees are commonly used in databases and filesystems.

- **Red-black tree:** A red-black tree is another type of self-balancing BST. Each node in the tree is assigned a color (red or black) to ensure that the tree remains balanced, which guarantees better worst-case time complexity for insertions and deletions.

Next, we will discuss two common tree representation methods: **linked representation** and **array representation**. Each method offers different advantages and is suited for specific types of operations and tree structures. By understanding both, we can choose the most efficient representation for a given application.

Tree representation

Trees can be represented in several ways, each depending on the use case and complexity of the operations. Let's explore this in detail.

Linked representation

In the linked representation of a binary tree, each node contains data and pointers (or references) to its left and right children. This is a more flexible representation than the array representation, as it doesn't require the tree to be a complete binary tree. Instead, each node directly references its children, allowing for irregular structures.

This representation is commonly used for binary trees and BST, where the nodes can have arbitrary arrangements. Each node in the tree is defined as a class or structure that holds a value and two pointers to its left and right child nodes.

The following code is an implementation of a node in Python using the linked representation:

```
# Definition of a TreeNode class in Python
class TreeNode:
    def __init__(self, key):
        self.key = key          # Node value
        self.left = None       # Pointer to left child
        self.right = None      # Pointer to right child
# Creating nodes and linking them to form a binary tree
root = TreeNode(10)           # Root node
root.left = TreeNode(5)       # Left child of root
root.right = TreeNode(20)     # Right child of root
# Adding more nodes to the tree
root.left.left = TreeNode(3)  # Left child of node with value 5
root.left.right = TreeNode(7) # Right child of node with value 5
root.right.left = TreeNode(15) # Left child of node with value 20
root.right.right = TreeNode(25) # Right child of node with value 20
# Function to perform an in-order traversal of the tree
def inorder_traversal(node):
    if node:
```

```

        inorder_traversal(node.left)
        print(node.key, end=' ')
        inorder_traversal(node.right)
# In-order traversal of the tree
print("In-order Traversal:")
inorder_traversal(root)

```

Let's explain the many components of the code:

- **TreeNode:** Each node contains a value (key) and two pointers (left and right), which reference the left and right children, respectively
- **Creating nodes:** We create nodes and link them to form the structure of a binary tree
- **In-order traversal:** The `inorder_traversal` function recursively visits the left subtree, the root, and then the right subtree, printing the nodes in a sorted order for a BST

The linked representation works for any type of binary tree, whether it is complete, balanced, or irregular. This approach is especially memory efficient for sparse trees, as memory is only allocated for the nodes that are actually present. This eliminates the need for continuous memory allocation for non-existent nodes, unlike array representation.

Additionally, the linked representation is more flexible for dynamic operations such as insertion, deletion, and traversal. Since each node directly references its children, modifying the tree structure is straightforward and doesn't require reordering or shifting elements, as would be needed in an array-based representation. This makes it ideal for trees that grow or change frequently, such as in BSTs.

Array representation

This method is commonly used for representing complete binary trees, such as heap structures. In this approach, a binary tree is stored as an array or list, where the root node is located at index 0. For any node at index i , its children are positioned as follows:

- The left child is located at index $2i + 1$
- The right child is located at index $2i + 2$

This array-based representation is highly efficient for complete binary trees because it avoids the need for pointers to track the parent-child relationships. It also allows for fast access to children or parents directly by calculating their indices. Here's a simple Python example demonstrating the array representation of a small complete binary tree:

```

# Array representation of a complete binary tree
binary_tree = [10, 5, 20, 3, 7, 15, 25]
# Accessing elements
root = binary_tree[0]
left_child_of_root = binary_tree[2 * 0 + 1] # index 1

```



```
right_child_of_root = binary_tree[2 * 0 + 2] # index 2
# Display the values
print(f"Root: {root}")
print(f"Left Child of Root: {left_child_of_root}")
print(f"Right Child of Root: {right_child_of_root}")
```

This array-based structure is ideal for heaps, where insertion and deletion operations require efficient reordering to maintain the heap property. The simplicity of the index-based parent-child relationships makes this representation fast and memory-efficient for complete binary trees.

Parent array representation

Another way to represent a tree is by storing the parent of each node in an array. In this approach, each index of the array corresponds to a node, and the value at that index represents the parent of that node. The root node is assigned a special value (commonly -1) to indicate that it has no parent.

This representation is particularly useful when we need to reconstruct the tree from parent-child relationships or when the tree is stored in such a way that direct access to child nodes is not necessary.

Here is a simple Python implementation for representing a tree using a parent array. The first part is a function to build the tree from a parent array:

```
def build_tree(parent_array):
    n = len(parent_array)
    nodes = [None] * n
    root = None
    # Create tree nodes for each index
    for i in range(n):
        nodes[i] = TreeNode(i)
    # Assign parents to each node
    for i in range(n):
        if parent_array[i] == -1:
            root = nodes[i] # This is the root node
        else:
            parent_node = nodes[parent_array[i]]
            if parent_node.left is None:
                parent_node.left = nodes[i]
            else:
                parent_node.right = nodes[i]
    return root
```

Next, we define the `TreeNode` class:

```
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

Finally, we build an example parent array, where -1 represents the root:

```
parent_array = [-1, 0, 0, 1, 1, 2, 2]
# Build the tree from the parent array
root = build_tree(parent_array)
# Function to perform an in-order traversal of the tree
def inorder_traversal(node):
    if node:
        inorder_traversal(node.left)
        print(node.key, end=' ')
        inorder_traversal(node.right)
# In-order traversal of the tree
print("In-order Traversal:")
inorder_traversal(root)
```

Let's explain the many components of the code:

- **Parent array:** The array represents the parent of each node. For example, if `parent_array[3] = 1`, it means *node 3* has *node 1* as its parent. The root node has the value -1, indicating it has no parent.
- **build_tree:** We first create an array of nodes and then link each node to its parent using the parent array. The nodes are connected either as the left or right child, depending on availability.
- **inorder_traversal:** We perform an in-order traversal of the tree to visit the nodes in their sorted order.

The parent array representation offers several advantages. One significant benefit is its space efficiency. Since this method only stores the parent-child relationships, it eliminates the need for additional pointers to left and right children, making it a compact structure. This feature makes it particularly suitable for environments where memory is limited.

Another advantage is its usefulness in reconstructing a tree from given parent-child relationships, such as those found in filesystems, organization charts, or other hierarchical structures. This method allows for easy and efficient tree reconstruction.

Additionally, it is efficient in applications where direct access to parent nodes is required. Since each index in the array corresponds to a specific node and stores its parent, retrieving the parent of any node can be done in constant time.

Lastly, the parent array representation is well suited for cases where trees are stored in external memory. It requires minimal data storage, which makes it particularly useful in databases or large-scale systems where the tree structure needs to be reconstructed on demand without taking up excessive space.

This representation is helpful when working with static tree structures, especially in cases where storing the parent relationships alone suffices for the intended operations.

BSTs

A **BST** is a type of binary tree where the nodes are organized in a way that for every node, the left subtree contains only nodes with values less than the node, and the right subtree contains only nodes with values greater than the node. This property makes BSTs efficient for search operations. *Figure 13.10* illustrates a simple BST. The following are the properties of a BST:

- The left child contains values less than the parent node
- The right child contains values greater than the parent node
- In-order traversal of a BST produces a sorted sequence

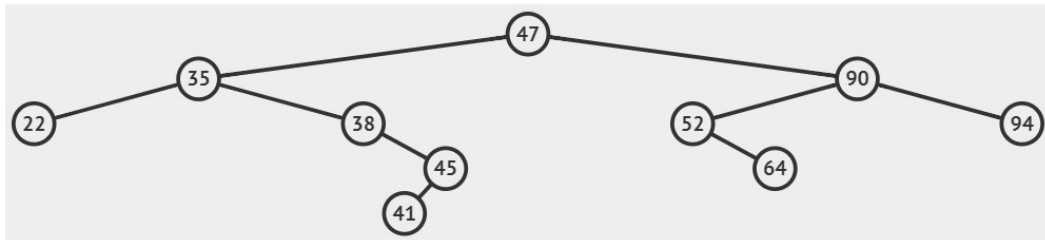


Figure 13.10: An example BST

Before exploring the operations of a BST, it is important to first understand how to traverse a BST. In a BST, similar to graphs, traversal refers to the process of visiting and processing each node in the tree. There are three common methods of traversal (or tree-walk): in-order, pre-order, and post-order. Each traversal method follows a specific order to visit the nodes, and all of them have the same time complexity of $O(n)$ since each node is visited exactly once.

In-order traversal

In-order traversal visits the nodes in the following order: left subtree, root, right subtree. In a BST, an in-order traversal will visit the nodes in sorted order (ascending). The in-order traversal steps are as follows:

1. Traverse the left subtree.
2. Visit the root node.
3. Traverse the right subtree.

For the BST shown in *Figure 13.11*, we aim to perform an in-order traversal of the tree.

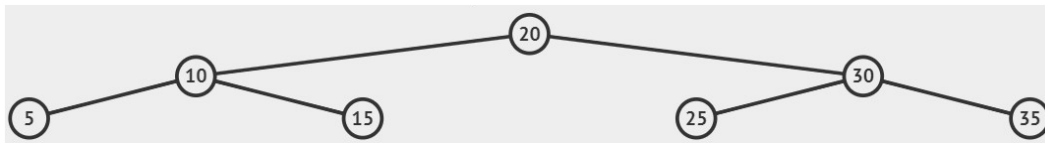


Figure 13.11: An example BST

Here is the Python code for in-order traversal:

```
# Definition of TreeNode class
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
# Function for in-order traversal
def inorder_traversal(node):
    if node:
        inorder_traversal(node.left)
        print(node.key, end=' ')
        inorder_traversal(node.right)
```

First, we build the BST using the following Python code:

```
# Example: Build the BST
root = TreeNode(22)
root.left = TreeNode(35)
root.right = TreeNode(30)
root.left.left = TreeNode(5)
root.left.right = TreeNode(15)
root.right.left = TreeNode(25)
root.right.right = TreeNode(35)
```

Then, we call `inorder_traversal` to perform in-order BST traversal:

```
# Perform in-order traversal
print("In-Order Traversal:")
inorder_traversal(root)
```

In-order traversal generates the following:

```
5 10 15 20 25 30 35
```

Pre-order traversal

Pre-order traversal visits the nodes in the following order: root, left subtree, right subtree. This method is useful for creating a copy of the tree or for printing the tree structure. The pre-order traversal steps are as follows:

1. Visit the root node.
2. Traverse the left subtree.
3. Traverse the right subtree.

The pre-order traversal will visit the nodes in BST of *Figure 13.11* in the following order: 20, 10, 5, 15, 30, 25, 35.

The following is the Python code for pre-order traversal:

```
# Function for pre-order traversal
def preorder_traversal(node):
    if node:
        print(node.key, end=' ')
        preorder_traversal(node.left)
        preorder_traversal(node.right)
# Perform pre-order traversal
print("Pre-Order Traversal:")
preorder_traversal(root)
```

Post-order traversal

Post-order traversal visits the nodes in the following order: left subtree, right subtree, root. This traversal is often used in applications such as tree deletion, where you must delete the children before deleting the parent node. The post-order traversal steps are as follows:

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the root node.

For the example in *Figure 13.11*, the post-order traversal will visit the nodes in the following order: 5, 15, 10, 25, 35, 30, 20.

Here is the Python code for post-order traversal:

```
# Function for post-order traversal
def postorder_traversal(node):
    if node:
        postorder_traversal(node.left)
        postorder_traversal(node.right)
        print(node.key, end=' ')
# Perform post-order traversal
print("Post-Order Traversal:")
postorder_traversal(root)
```

In summary, in-order traversal visits nodes in the sorted order (left, root, right) and is commonly used for extracting sorted data from BSTs. Pre-order traversal visits nodes in the order (root, left, right), making it useful for copying the tree structure or printing it. Post-order traversal visits nodes in the order (left, right, root) and is helpful for tasks such as tree deletion, where processing the child before the parent is necessary.

In a BST, the main operations – insertion, deletion, and searching – rely on the tree structure and properties. The efficiency of these operations depends heavily on whether the tree is balanced or not. Let's explore these operations in detail.

The search operation in a BST

Searching in a BST leverages the property that the left subtree contains values smaller than the current node and the right subtree contains values larger than the current node. This allows us to effectively halve the search space with each step, similar to binary search:

- **Average case (balanced tree):** In a balanced BST (such as the one shown in *Figure 13.6*), the search operation takes $O(\log n)$ time. This is because the tree height is logarithmic relative to the number of nodes, and we reduce the search space at each level.
- **Worst case (unbalanced tree):** If the BST is unbalanced, the search time complexity can approach $O(n)$, where the tree starts resembling a linked list. In such cases, the tree height grows linearly with the number of nodes, making searches inefficient.

For extreme cases, such as the skewed tree in *Figure 13.12*, the search complexity reaches $O(n)$, where every node has only one child, and the tree degenerates into a linear structure.



Figure 13.12: An example of extreme unbalanced BST

The following is a Python implementation of searching in a BST. The first part is the definition of a `TreeNode` class for the BST:

```
class TreeNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

To build the BST, we implement the insertion operation using the `insert` function as follows:

```
Function to insert a node in the BST
def insert(node, key):
    # If the tree is empty, return a new node
    if node is None:
        return TreeNode(key)
    # Otherwise, recur down the tree
    if key < node.key:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right, key)
    return node
```

The search operation is implemented as follows:

```
# Function to search a key in the BST
def search(node, key):
    # Base case: the node is None (key not found) or the key matches
    # the current node's key
    if node is None or node.key == key:
        return node
    # If the key is smaller than the node's key, search the left
```

```

subtree
    if key < node.key:
        return search(node.left, key)
    # Otherwise, search the right subtree
    return search(node.right, key)

```

Let's create the root node and insert elements into the BST:

```

root = None
keys = [20, 10, 30, 5, 15, 25, 35]
for key in keys:
    root = insert(root, key)

```

The following shows how to search a key in BST:

```

search_key = 25
found_node = search(root, search_key)
# Output the result
if found_node:
    print(f"Key {search_key} found in the BST.")
else:
    print(f"Key {search_key} not found in the BST.")

```

Let's explain the important parts of the algorithm:

- **TreeNode:** Each node in the BST contains a key (the value of the node), and references (`left` and `right`) to its left and right children.
- **insert:** The insert function inserts values into the BST. It recursively traverses the tree and inserts the new node in the correct position based on the BST property.
- **search:** The search function recursively looks for a given key. If the current node's key matches the key being searched, it returns that node. Otherwise, it continues searching in the left or right subtree based on whether the key is smaller or larger than the current node's key.

The time complexity of searching in a BST depends on how balanced the tree is. In the worst case, the complexity is $O(L)$, where L is the depth of the BST. In an extremely unbalanced case, where the BST essentially forms a linear structure, the time complexity becomes $O(n)$. However, in a fully balanced BST, the time complexity is $O(\log n)$, ensuring more efficient searches.

The insertion operation in a BST

The **insertion operation** in a BST follows the same logic as searching. Starting from the root, we compare the value to be inserted with the current node and move left if it's smaller or right if it's larger. Once we find an appropriate empty spot (`null`), the new node is inserted there.

Let's briefly discuss the complexity of insertion in a BST: Similar to searching, the time complexity of insertion in a BST depends on its balance. In the worst case, when the tree is highly unbalanced and resembles a linear structure, the time complexity is $O(n)$, where n is the number of nodes. In contrast, in a balanced BST, the time complexity is $O(\log n)$, as the insertion operation involves traversing the tree's height:

- **Average case (balanced tree):** In a balanced BST, insertion takes $O(\log n)$ time on average, as we are essentially performing a search to find the appropriate location to insert the new node
- **Worst case (unbalanced tree):** Similar to searching, if the BST is unbalanced, insertion time can degrade to $O(n)$, particularly if the values being inserted cause the tree to become skewed

In the Python code provided for the search algorithm, the `insert` function is responsible for inserting nodes into the BST while maintaining the BST property. It works recursively to find the correct position for the new node.

Let's see how the `insert` function works. If the tree is empty (i.e., the current node is `None`), the function creates a new node with the given key and returns it, effectively making it the root or a leaf. If the key to be inserted is smaller than the current node's key, the function recursively moves to the left subtree to find the appropriate position. If the key to be inserted is larger than the current node's key, it recursively moves to the right subtree. Once the appropriate position is found, the new node is added as either the left or right child.

Here is a simple Python code implementing insertion in a BST:

```
# Function to insert a node in the BST
def insert(node, key):
    # If the tree is empty, return a new node
    if node is None:
        return TreeNode(key)
    # Otherwise, recur down the tree
    if key < node.key:
        node.left = insert(node.left, key)
    else:
        node.right = insert(node.right, key)
    return node
```

For example, if we insert the values 20, 10, 30, 5, 15, 25, and 35 using this `insert` function, it will create the BST depicted in *Figure 13.13*.



Figure 13.13: The final BST representing [20, 10, 30, 5, 15, 25, 35]

The insertion of the value 35 is highlighted in the tree. When inserting 35, first, it is compared with the root node, 20. Since 35 is greater than 20, we move to the right subtree. Next, 35 is compared with the node 30. Since 35 is greater than 30, it will be inserted as the right child of node 30. The key comparison process ensures that 35 is placed correctly in the BST according to the BST property, where values in the right subtree are greater than the parent node.

As a summary, the `insert` function ensures that each new value is placed in the correct position in the tree by comparing it to the existing node values. It maintains in the BST property that all values in the left subtree are smaller than the parent node, and all values in the right subtree are larger.

The deletion operation in a BST

In a BST, the deletion operation is more complex than the insertion operation because we need to maintain the BST property after removing a node. There are three possible cases to consider when deleting a node:

- **Deleting a leaf node:** A node with no children can be directly removed.
- **Deleting a node with one child:** The node is replaced by its child.
- **Deleting a node with two children:** The node is replaced by either its in-order predecessor (the largest node in the left subtree) or its in-order successor (the smallest node in the right subtree). After replacement, the node from which the replacement came must also be removed.

The following is the Python code that implements the deletion operation in a BST. First, we must define the `TreeNode` class (refer to previous examples). Next, we need to construct the tree using the `insert` function that was discussed earlier. Next, we implement the `min_value_node` function to find the successor when a node is deleted:

```
# Function to find the minimum value node in the right subtree (in-
order successor)
def min_value_node(node):
    current = node
    while current.left is not None:
        current = current.left
    return current
```

Lastly, we have `delete_node` to implement the three cases of the deletion operation in BSTs:

```
# Function to delete a node from the BST
def delete_node(root, key):
    # Base case: the tree is empty
    if root is None:
        return root

    # If the key to be deleted is smaller than the root's key, go to
    the left subtree
```

```
    if key < root.key:
        root.left = delete_node(root.left, key)
    # If the key to be deleted is greater than the root's key, go to
    the right subtree
    elif key > root.key:
        root.right = delete_node(root.right, key)
    # If key is equal to the root's key, this is the node to be
    deleted
    else:
        # Case 1: Node with only one child or no child
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        # Case 2: Node with two children
        # Get the in-order successor (smallest in the right subtree)
        temp = min_value_node(root.right)
        # Replace the current node's key with the in-order successor's
key
        root.key = temp.key
        # Delete the in-order successor
        root.right = delete_node(root.right, temp.key)
    return root
```

The following is an example of constructing a BST using the insert function and then deleting the node with the value 30:

```
# Create a BST and insert values into it
root = None
keys = [20, 10, 30, 5, 15, 25, 35]
for key in keys:
    root = insert(root, key)
# Delete a node from the BST
delete_key = 30
root = delete_node(root, delete_key)
# Function to perform in-order traversal
def inorder_traversal(node):
    if node:
        inorder_traversal(node.left)
        print(node.key, end=' ')
        inorder_traversal(node.right)
# Perform in-order traversal after deletion
print("In-order Traversal after Deletion:")
inorder_traversal(root)
```

Let's explain the example. We first compare *node 30* with the root *node 20*. Since 30 is greater than 20, we move to the right subtree. We find *node 30* and notice that it has two children (*nodes 25* and *35*). We replace *node 30* with its in-order successor, *node 35*, and then remove *node 35* from its original position. *Figure 13.14* demonstrates the BST after removing *node 35*.

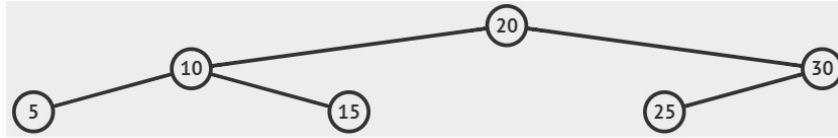


Figure 13.14: The BST in Figure 13.9 after removing 35

The time complexity of deletion in BSTs is $O(\log n)$ on average, which occurs when the tree is balanced since we only need to traverse the tree's height to locate and delete the node. However, in the worst case, the time complexity is $O(n)$, particularly when the tree is unbalanced and takes the form of a linked list.

In the next section, we will explore heap structures, which play a crucial role in sorting algorithms and other applications requiring efficient data management.

Heaps

A **heap** is a special type of binary tree that satisfies the heap property. In a heap, the parent node always follows a specific order relation with respect to its children. Heaps are commonly used in various algorithms, especially in sorting and priority queues, due to their efficient access to the minimum or maximum element.

There are two main types of heaps, based on the order property they follow:

- **Max-heap:** In a max-heap, each node's value is greater than or equal to the values of its children, with the largest element positioned at the root. Max-heaps are commonly used in algorithms that need efficient access to the maximum element, such as heapsort and priority queue implementations. In max-heap, the heap property is as follows: Type equation here.
 - $A[i] \geq A[2i + 1]$ (left child)
 - $A[i] \geq A[2i + 2]$ (right child), where A is the array representation of the heap
- **Min-heap:** In a min-heap, the value of each node is less than or equal to the values of its children. The smallest element is always at the root. Min-heaps are commonly used in algorithms such as Dijkstra's shortest path and Prim's minimum spanning tree. In this case, the heap property is as follows: For each node i , this is the case:
 - $A[i] \leq A[2i + 1]$ (left child)
 - $A[i] \leq A[2i + 2]$ (right child)

Heaps are typically represented as complete binary trees stored in an array. In a complete binary tree, this is the case:

- The root node is at index 0
- For a node at index i , the left child is at index $2i + 1$, and the right child is at index $2i + 2$
- The parent of a node at index i is located at $\text{floor}((i - 1)/2)$

This array representation allows heaps to be efficiently stored in memory without using pointers for the child nodes (see Figure 13.15).

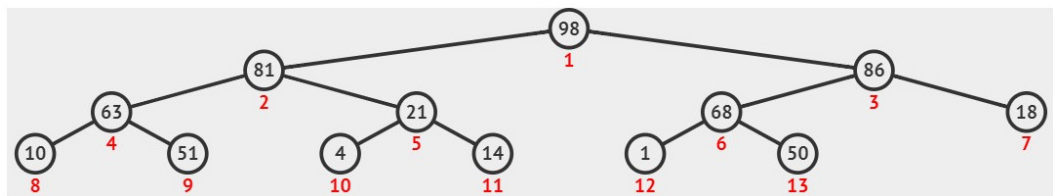


Figure 13.15: An example max-heap

Index	1	2	3	4	5	6	7	8	9	10	11	12	13
node	98	81	86	63	21	68	18	10	51	4	14	1	50

Figure 13.16: The array representation of the max-heap in Figure 13.15

Now that we have learned about the heap property and heap representation, let’s explore the operation on heaps.

Heap operations

The primary operations on heaps are *insertion*, *deletion*, and *heapify* (used for maintaining the heap property). Each of these operations relies on the heap property to ensure that the structure remains a valid heap.

Insertion in a heap

To insert an element into a heap, we first add the element at the last position in the array (the end of the heap). Then, we perform a *heapify-up* operation, which involves comparing the inserted element with its parent. If the heap property is violated, we swap the two. This process is repeated until the heap property is restored.

Here is the Python code for insertion in a max-heap:

```
def heapify_up(heap, index):
    parent = (index - 1) // 2
    if index > 0 and heap[parent] < heap[index]:
        # Swap the parent and current node
        heap[parent], heap[index] = heap[index], heap[parent]
        # Recursively heapify the parent node
        heapify_up(heap, parent)
def insert_max_heap(heap, element):
    heap.append(element)
    heapify_up(heap, len(heap) - 1)
```

The following is an example of building a max-heap:

```
# Example usage
heap = []
insert_max_heap(heap, 20)
insert_max_heap(heap, 15)
insert_max_heap(heap, 30)
insert_max_heap(heap, 5)
insert_max_heap(heap, 40)
print("Heap after insertions:", heap)
```

The time complexity for insertion is $O(\log n)$, as we may need to swap elements up the tree to restore the heap property.

Deletion in a heap

In a heap, deletion typically involves removing the root element (the maximum in a max-heap or the minimum in a min-heap). The deletion process is performed in three steps:

1. Replace the root element with the last element in the array.
2. Remove the last element from the array.
3. The process of heapifying down the root element involves comparing it with its children. In cases where the heap property is violated, a swap occurs between the root and the largest child for a max-heap or the smallest child for a min-heap. This continues until the heap property is restored.

Here is a simple Python code for deletion in a max-heap:

```
def heapify_down(heap, index):
    largest = index
    left = 2 * index + 1
    right = 2 * index + 2
```

```
    if left < len(heap) and heap[left] > heap[largest]:
        largest = left
    if right < len(heap) and heap[right] > heap[largest]:
        largest = right
    if largest != index:
        heap[index], heap[largest] = heap[largest], heap[index]
        heapify_down(heap, largest)
def delete_max_heap(heap):
    if len(heap) == 0:
        return None
    if len(heap) == 1:
        return heap.pop()
    root = heap[0]
    heap[0] = heap.pop() # Move last element to the root
    heapify_down(heap, 0) # Restore heap property
    return root
```

Let's have an example max-heap and delete the root using `delete_max_heap`:

```
heap = [40, 30, 20, 5, 15]
deleted = delete_max_heap(heap)
print("Heap after deletion of max element:", heap)
```

The heap after the deletion of a max element is `[30, 15, 20, 5]`.

The time complexity for deletion is $O(\log n)$, as we may need to swap elements down the tree to restore the heap property.

Heapify (building a heap)

To build a heap from an arbitrary array, we use the heapify process. Starting from the first non-leaf node, we move up to the root, ensuring the heap property is maintained. Consider the following Python implementation of heapify:

```
def heapify(heap, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and heap[left] > heap[largest]:
        largest = left
    if right < n and heap[right] > heap[largest]:
        largest = right
    if largest != i:
        heap[i], heap[largest] = heap[largest], heap[i]
        heapify(heap, n, largest)
```

As we can see, the `heapify` function recursively calls itself to ensure that the heap property is maintained throughout the tree. If a violation of the heap property is detected at any node, `heapify` continues down the tree, correcting the structure by comparing and swapping nodes as needed, until the property is fully restored.

The following is a Python code to build a max-heap using the `heapify` recursive algorithm:

```
def build_max_heap(a):
    n = len(a)
    # Start from the first non-leaf node and heapify each node
    for i in range(n // 2 - 1, -1, -1):
        heapify(a, n, i)
```

The following is a simple example of a `build_max_heap` usage:

```
arr = [5, 15, 20, 30, 40]
build_max_heap(arr)
print("Array after building max-heap:", arr)
```

The array representing the max-heap after building the max-heap is `[40, 30, 20, 5, 15]`.

The time complexity for building a heap is $O(n)$ since each node requires at most $O(\log n)$ swaps, but most nodes are near the bottom of the tree and need fewer swaps.

Let's explore a major application of heap structures: heapsort.

Heapsort

Heapsort is an efficient comparison-based sorting algorithm that leverages the heap data structure, particularly the max-heap, to sort elements in $O(n \log n)$ time. It works by first converting the input array into a max-heap and then repeatedly extracting the largest element (the root of the heap) to build the sorted output. This process ensures that the elements are sorted in ascending order. The following outlines the steps involved in performing heapsort:

1. Build a max-heap from the input array.
2. Swap the root (maximum element) with the last element.
3. Reduce the heap size by one and `heapify` the root element.
4. Repeat the process until the heap is empty.

Here is the Python implementation of heapsort:

```
def heapsort(arr):
    n = len(arr)
    build_max_heap(arr) # Step 1: Build a max-heap
    for i in range(n - 1, 0, -1):
```



```
        arr[0], arr[i] = arr[i], arr[0] # Step 2: Swap root with last
element
        heapify(arr, i, 0) # Step 3: Heapify the reduced heap
```

Let's examine the heapsort algorithm in an example:

```
a = [5, 15, 20, 30, 40]
heapsort(arr)
print("Sorted array:", arr)
```

The output will be as follows:

```
Sorted array: [5, 15, 20, 30, 40]
```

The time complexity of heapsort is $O(n \log n)$, as building the heap takes $O(n)$, and each extraction of the maximum (in a max-heap) or minimum (in a min-heap) requires $O(\log n)$ over n elements. As discussed and demonstrated in *Chapter 6*, the time complexity of heapsort cannot be better than $O(n \log n)$ because it is a comparison-based sorting algorithm, which imposes a lower bound on the time complexity for such algorithms. Heapsort is an in-place sorting algorithm and its space complexity is $O(1)$.

With the heap structure, we conclude our discussion on non-linear data structures. This chapter has provided key highlights of this important class of data structures, but it is not meant to serve as a comprehensive replacement for a dedicated book on data structures. It offers an overview, emphasizing their role in algorithm design, while more detailed exploration can be found in specialized texts.

Summary

In this chapter, we explored the key concepts and applications of non-linear data structures, which are essential for designing efficient algorithms. We began by discussing the general properties of non-linear structures, highlighting how they differ from linear data structures in terms of organization and access patterns. Two major categories were covered in detail: graphs and trees. Graphs were presented as versatile structures for modeling relationships, while trees provided a more hierarchical organization of data. We examined different types of trees, such as BSTs, discussing their properties, operations, and use cases in algorithm design.

The chapter concluded by focusing on heaps, a special form of binary tree that is commonly used in priority queues and sorting algorithms such as heapsort. We covered how heaps are constructed, how the heap property is maintained through insertion, deletion, and heapify operations, and the role heaps play in sorting. Overall, this chapter provided a foundational understanding of non-linear data structures and emphasized their significance in the efficient processing and manipulation of complex data relationships. By the end of this chapter, our discussion of algorithms comes to a close. However, in the next chapter, we will explore emerging trends and future directions in algorithm development.

References and further reading

- *Introduction to Algorithms*. By Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Fourth Edition. MIT Press. 2022:
 - *Chapter 6, Heapsort*
 - *Chapter 12, Binary Search Trees*
 - *Chapter 22, Elementary Graph Algorithms*
- *Data Structures and Algorithm Analysis in C++*. By Mark A. Weiss. Fourth Edition. Pearson. 2012:
 - *Chapter 4, Trees*
 - *Chapter 5, Binary Search Trees*
 - *Chapter 6, Heaps*
 - *Chapter 9, Graph Algorithms*
- *Algorithms*. By R. Sedgewick, K. Wayne. Fourth Edition. Addison-Wesley. 2011.
 - *Chapter 3, Searching (Binary Search Trees)*
 - *Chapter 4, Sorting (Heapsort)*
 - *Chapter 5, Graphs*

Part 4:

Next Steps

This final part looks ahead to the future of algorithms, exploring emerging trends and the evolving landscape of computing. It discusses the challenges and opportunities in scaling algorithms, building context-aware systems, and ensuring ethical decision-making in algorithm design.

This part includes the following chapter:

- *Chapter 14, Tomorrow's Algorithms*

Tomorrow's Algorithms

In this chapter, we explore advanced and emerging topics in algorithm design that are shaping the future of computing. As technology evolves at an unprecedented pace, algorithms are at the forefront of this transformation, driving innovations in fields such as quantum computing, machine learning, and big data processing. We outline various trends that are influencing the development of scalable, context-aware, and ethically conscious algorithms. These include new approaches for handling large-scale data, the integration of biologically inspired techniques, the increasing importance of memory-efficient and environmentally sustainable algorithms, and the critical need for ethical considerations in algorithm deployment. This chapter aims to provide an overview of these cutting-edge areas, offering insights into how they are redefining the landscape of computer science and software engineering.

The following topics will be covered in this chapter:

- Learning from the past
- Scalability
- Context awareness
- Moral responsibility
- Final words

Learning from the past

Since the advent of computers, there have been numerous groundbreaking ideas, concepts, and technological advancements that have significantly shaped the way we interact with machines and process information. Here's a brief overview of the key stages in the evolution of computing technology:

- **Early computers:** The journey began with the development of early computers, such as the ENIAC and UNIVAC in the mid-20th century. These machines were large, room-sized devices that used vacuum tubes and punch cards to perform basic calculations. They marked the beginning of automated computing, setting the foundation for future innovations.

- **The first wave of artificial intelligence (AI):** The concept of AI emerged alongside early computers, with researchers exploring the idea of machines that could mimic human thought. Early AI focused on symbolic reasoning and problem-solving, laying the groundwork for what would become a rapidly evolving field.
- **Mainframe computers:** As technology progressed, mainframe computers came into prominence. These powerful machines were used primarily by large organizations for complex data-processing tasks. Mainframes introduced the concept of centralized computing, allowing multiple users to access a single, powerful computer system.
- **Expert systems and the second wave of AI:** The second wave of AI brought about the development of expert systems, which were designed to emulate the decision-making abilities of human experts in specific domains. These systems relied on rule-based logic and were used in fields such as medical diagnosis and financial analysis.
- **Personal computers (PCs):** The invention of the personal computer in the 1970s and 1980s marked a significant shift in computing. PCs made computing accessible to the general public, transforming it from a tool for large institutions to a household necessity. This era saw the rise of user-friendly interfaces, software applications, and the democratization of computing power.
- **Internet and web:** The advent of the Internet and the World Wide Web in the 1990s revolutionized how we access and share information. It connected computers globally, enabling instant communication, data exchange, and the emergence of online services and digital economies. This period also saw the rise of e-commerce, social media, and cloud computing.
- **Mobile computing:** The early 2000s brought about the era of mobile computing, with smartphones and tablets becoming ubiquitous. These devices combined computing power with portability, allowing users to access information, communicate, and perform a wide range of tasks on the go. Mobile apps and wireless technologies further expanded the capabilities of these devices.
- **The third and current wave of AI:** We are now in the midst of the third wave of AI, characterized by advances in machine learning, deep learning, and natural language processing. This wave focuses on building systems that can learn from vast amounts of data, recognize patterns, and make autonomous decisions. AI is being integrated into various aspects of daily life, from virtual assistants and autonomous vehicles to medical diagnostics and financial services.

Each of these stages represents a leap in technological capability and a shift in how we interact with and utilize computers, shaping the modern digital landscape.

Despite the rapid and transformative changes brought by each of these technological revolutions, one constant has remained: the fundamental need for algorithms. Regardless of the massive leaps in hardware and software capabilities, algorithms have always been at the core of every advancement, driving the efficiency and functionality of each new development.

Just as in mathematical induction, where a principle holds true at each step of a progression, we can infer that the necessity for designing, analyzing, and optimizing algorithms will continue to be vital in the future. In fact, as technology grows more complex and the amount of data we handle increases exponentially, the importance of robust and efficient algorithms will only become more pronounced.

Algorithms are the underlying frameworks that enable systems to process information, solve problems, and make decisions. As we venture further into realms such as AI, quantum computing, and big data, the challenges become increasingly sophisticated. Efficient algorithms will be crucial to harnessing the full potential of these technologies, ensuring that they can operate at scale, in real time, and with optimal resource utilization.

Moreover, as we continue to integrate technology into critical aspects of society, such as healthcare, finance, and infrastructure, the need for algorithms that are not only efficient but also secure, fair, and interpretable becomes even more essential. The future will likely demand algorithms that can adapt to dynamic environments, manage uncertainty, and operate autonomously, all while maintaining ethical standards and safeguarding privacy.

In conclusion, we can say that while the tools and platforms we use may evolve, the foundational role of algorithms in shaping the future of technology is here to stay. Their design, analysis, and optimization will remain a cornerstone of progress, and their significance will only grow as we confront more complex and critical challenges.

Numerous emerging trends in algorithms are being shaped by the current AI revolution and other technological advancements. This chapter does not aim to predict the future but rather to provide a framework for understanding these trends through the lens of three crucial aspects: scalability, context awareness, and moral responsibility.

By exploring these dimensions, we aim to initiate a conversation about the evolving landscape of algorithm design and its broader implications. The goal is to encourage readers to engage with these topics actively, considering how these trends might influence the future of technology and society.

In the following sections, we will delve into these aspects in greater detail, discussing how they shape the current trends and the future of algorithms. This discussion is not meant to be exhaustive but rather a starting point for deeper exploration. We invite readers to join this ongoing conversation, reflecting on how these trends might influence their work and the broader societal implications of algorithm design.

Scalability

The explosion of data and the increasing complexity of modern systems demand algorithms that can operate effectively at an unprecedented scale. Whether it's handling massive datasets, managing large-scale distributed systems, or optimizing processes for millions of users, scalability has become a central concern. Algorithms must not only be efficient but also adaptable to growing demands and diverse application contexts.

The current AI revolution, more than being driven by purely algorithmic advancements, has been fueled by three major developments:

- **An exponential increase in processing power:** The continuous rise in processing capabilities, to the point where even **Moore's law** (the prediction that the number of transistors on a microchip doubles every two years) might no longer hold, has allowed for much more complex and resource-intensive algorithms to be executed (see *Figure 14.1*). The emergence of specialized hardware, such as **graphical processing units (GPUs)** and **tensor processing units (TPUs)**, further accelerated the ability to handle massive computations, particularly those required for machine learning models. GPUs have been originally designed to accelerate the rendering of images and video in computer graphics. They have evolved to handle complex mathematical computations required for tasks such as deep learning, scientific simulations, and data analysis. Due to their highly parallel structure, GPUs are well suited for processing large blocks of data simultaneously, making them particularly effective for training and running machine learning models.

TPU is a specialized hardware accelerator designed by Google specifically for machine learning tasks, particularly those involving neural networks and deep learning models. TPUs are optimized for running large-scale computations efficiently, providing high performance for training and inference in machine learning applications. They are designed to handle the computational demands of algorithms used in AI, especially those using TensorFlow, Google's open source machine learning framework (see *Figure 14.1*).

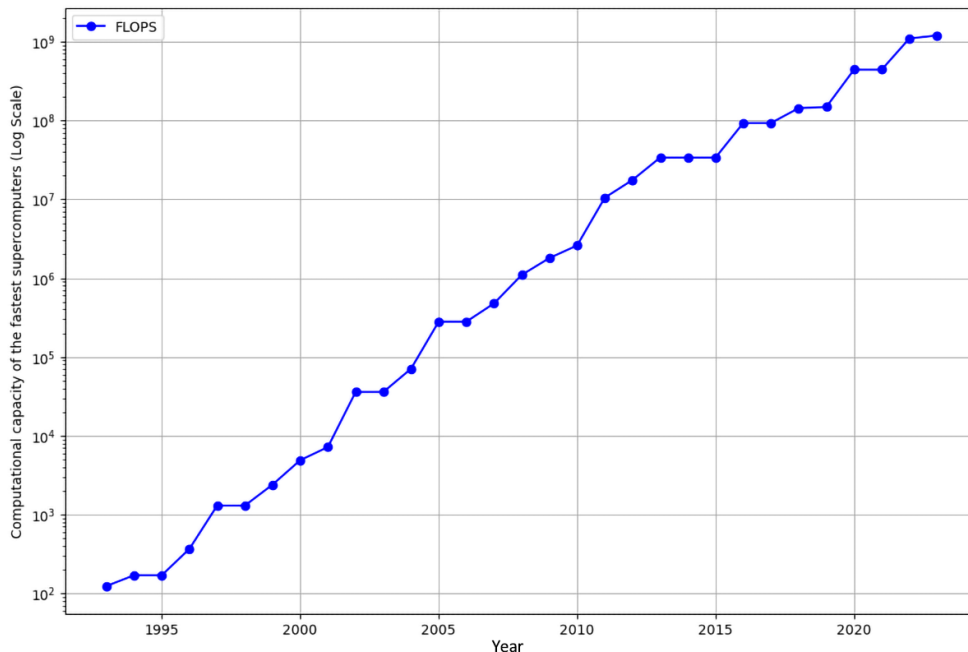


Figure 14.1: Computation capacity of the fastest computers in the last 20 years (log scale)

- Decreasing cost and increasing capacity of storage:** While storage and memory costs have dropped dramatically, their capacity has surged (see *Figure 14.2*). This means that vast amounts of data can now be stored cheaply, enabling the retention and processing of massive datasets. These trends allow for data-driven algorithms to thrive, as the constraints on storage that once limited the size of models or the amount of data that could be used no longer exist (see *Figure 14.2*).

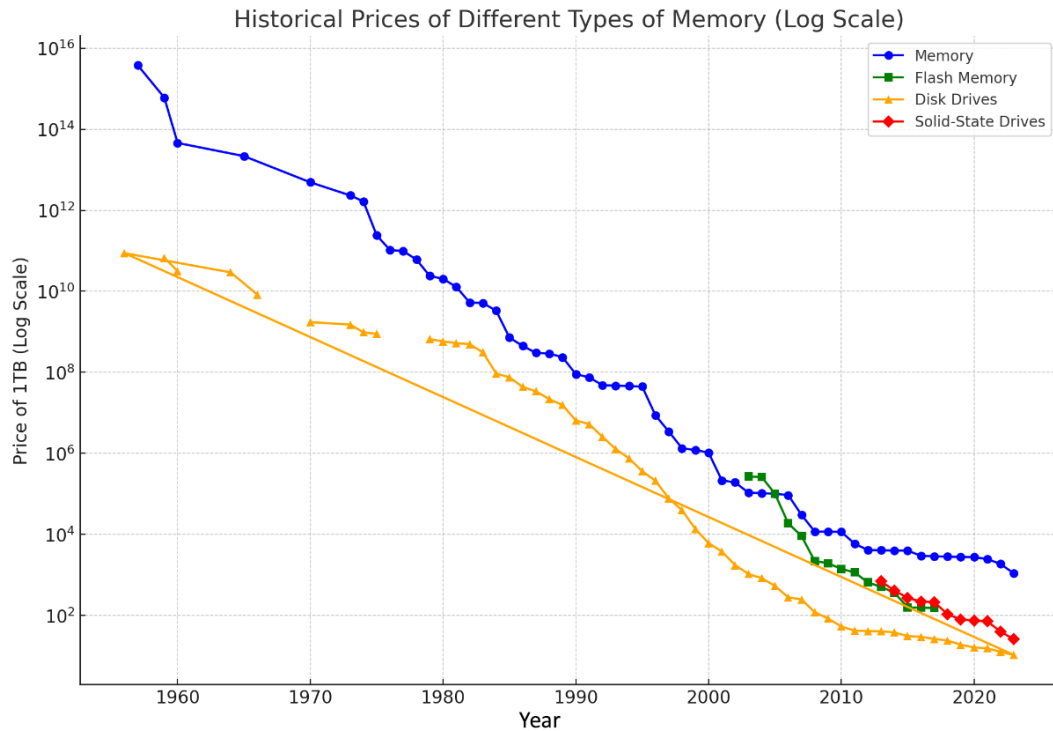


Figure 14.2: Historical prices of different types of 1 TB memory (log scale)

- Data explosion fueled by the internet:** Since the inception of the internet in 1991, the volume of available data has exploded. The freedom and openness of the web have unleashed a tidal wave of information, ranging from user-generated content to transactional data, providing an unparalleled dataset for algorithms to leverage. Social networks, online platforms, and IoT devices further contribute to this ever-growing ocean of data.

These three factors have transformed algorithms, particularly those used in AI and machine learning. The availability of vast amounts of data has allowed algorithms to rely more heavily on statistical learning and large-scale pattern recognition. As a result, algorithms such as neural networks have evolved into models of staggering size and complexity, such as deep learning architectures with billions of parameters.

In turn, these models have driven the need for even greater processing power and storage, reinforcing a cycle of ever-increasing scale. Concurrent computing and parallelism have become essential strategies to handle the demands of training these massive models. Techniques such as distributed computing, cloud-based machine learning, and parallel processing frameworks, such as MapReduce, have been developed to cope with this growing need.

Thus, the AI revolution today is as much about scaling up infrastructure as it is about designing smarter algorithms. Without the advances in processing power, storage, and the abundance of data, many of the breakthroughs in AI would not have been possible. This focus on scale continues to drive the development of algorithms capable of learning from, processing, and acting upon massive datasets.

In parallel with designing and analyzing algorithms, it's essential to consider the issue of scale and how algorithms can leverage and support emerging hardware technologies. As data and computational needs continue to grow, scalable algorithms have become increasingly important to ensure efficient processing and problem-solving in large-scale systems. Here are some emerging trends that support the development of scalable algorithms:

- **Quantum algorithms:** Quantum computing is poised to revolutionize how we tackle large-scale problems. Traditional computers struggle with certain types of computations, such as factoring large numbers or simulating quantum systems. Quantum algorithms, such as **Shor's algorithm** for factorization and **Grover's algorithm** for search, can provide exponential speedups, making them ideal for large-scale problems that are currently intractable for classical computers.
- **Approximation algorithms:** In many large-scale problems, finding an exact solution is computationally expensive or even impossible. Approximation algorithms provide a way to obtain near-optimal solutions within a reasonable time frame. These algorithms are crucial in scenarios where a perfect solution isn't necessary, but a good enough solution is valuable and must be found quickly, such as in routing, scheduling, and optimization problems.
- **Parameterized complexity:** This is a framework in computational complexity theory that provides a more nuanced approach to analyzing the complexity of problems by focusing on specific aspects or parameters within the input data, rather than the overall input size alone. In traditional complexity analysis, problems are classified based on how their running time grows with the size of the input. However, for many real-world problems, certain parameters of the input may have a greater influence on the algorithm's performance than the total size of the data. By identifying and isolating these critical parameters, parameterized complexity allows algorithms to be designed more efficiently for practical use cases, even if the problem remains computationally hard in the general case.

The primary goal in parameterized complexity is to develop algorithms whose running time is not necessarily optimal for every instance of the problem but is manageable when certain parameters remain small, even if the overall input size is large. The central concept is **fixed-parameter tractability (FPT)**, where a problem is considered fixed-parameter tractable if it can be solved in time $O(f(k) \cdot n^c)$, where n is the input size, k is the parameter, and $f(k)$ is a function that depends only on the parameter, not the overall input size. If k is small, even though n is

large, the algorithm can still run efficiently. Real-world applications of parameterized complexity are in bioinformatics, scheduling problems, and database systems.

- **Sublinear algorithms:** As datasets grow to sizes where even reading the entire input is impractical, sublinear algorithms become essential. These algorithms aim to produce useful results by examining only a small portion of the input, making them particularly suitable for applications such as property testing, where we seek to determine the properties of the dataset without fully processing it.
- **Streaming algorithms:** In scenarios where data arrives in a continuous stream, such as network traffic monitoring or financial tickers, algorithms must process information on the fly with limited memory. Streaming algorithms are designed to work within these constraints, making a single or limited number of passes over the data to extract meaningful insights or maintain summaries, even as the data volume grows.
- **Parallel and distributed algorithms:** To harness the full power of modern hardware, algorithms must be designed to operate efficiently on multi-core processors and distributed systems. Parallel and distributed algorithms break down large problems into smaller subproblems that can be solved concurrently, reducing computation time and enabling the processing of large-scale data in real time or near real time.
- **Smoothed analysis:** Traditional algorithm analysis often focuses on worst-case or average-case scenarios, which may not accurately reflect real-world performance. The smoothed analysis provides a more nuanced view by analyzing how algorithms perform under slight random perturbations of the input. This approach yields more realistic performance metrics for algorithms in practical, large-scale settings, bridging the gap between theoretical analysis and empirical performance.
- **Algorithms for large-scale data:** Big data applications require algorithms that can efficiently process and analyze massive datasets. These algorithms are designed to handle high-dimensional data, distributed across different storage systems while ensuring scalability and performance. They include methods for data mining, clustering, and machine learning that can operate effectively on vast amounts of data.
- **Graph neural networks (GNNs):** In many real-world scenarios, data is naturally represented as graphs, such as social networks, biological systems, and communication networks. GNNs are an emerging class of algorithms that operate directly on graph-structured data, allowing them to scale with the complexity of the relationships within the data. GNNs are particularly powerful for tasks that involve large, interconnected datasets, making them valuable tools for applications ranging from network analysis to molecular chemistry.

Each of these trends addresses the challenges of scaling in different ways, from leveraging new computing paradigms such as quantum computing to optimizing how algorithms interact with large datasets. Biologically inspired algorithms and memory-efficient algorithms, in particular, offer innovative approaches to problem-solving by harnessing natural processes and making the most of limited resources. Together, these advancements form a critical foundation for the future of algorithm design in an era of ever-growing data and computational demands.

While computational power is essential, it alone is insufficient for effective decision-making in volatile, chaotic, and dynamic environments. Beyond sheer processing capability, we must invest in algorithms that can adapt to diverse or even hostile conditions. This requires making algorithms more context-aware, enabling them to respond intelligently to the complexities of their surroundings.

Context awareness

Modern algorithms are increasingly required to operate in diverse, dynamic, and often unpredictable environments. These environments range from real-time data streams, which demand rapid processing and decision-making, to embedded systems with strict limitations on computational resources. Additionally, algorithms must navigate complex ecosystems that include cloud infrastructures, edge computing environments, and the integration of **internet of things (IoT)** devices. In such varied contexts, algorithms must be resilient, adaptive, and capable of performing under different conditions and constraints. The following are emerging trends that support the development of context-aware algorithms:

- **Biologically inspired algorithms:** Nature serves as a rich source of inspiration for developing algorithms that can adapt to changing environments. Biologically inspired algorithms, such as genetic algorithms, ant colony optimization, and neural-inspired computing, leverage mechanisms found in natural processes. Genetic algorithms, for instance, mimic the process of natural selection, evolving solutions over successive generations to optimize complex problems. Ant colony optimization is inspired by the behavior of ants in finding optimal paths to resources, providing efficient solutions to routing and scheduling issues. These algorithms excel in situations where traditional methods may struggle, particularly in complex, large-scale environments that require adaptability and robustness.
- **Memory-efficient algorithms:** In environments where memory is a limited resource, such as embedded systems and IoT devices, the ability to minimize memory usage is crucial. Memory-efficient algorithms are designed to operate effectively within these constraints, using techniques such as data compression, in-place computation, and space-efficient data structures. By reducing the memory footprint, these algorithms enable complex computations on devices with limited storage and processing power, broadening the range of applications that can benefit from advanced algorithmic solutions. This is particularly important in contexts where devices need to process data locally and cannot rely on cloud resources due to latency, bandwidth, or privacy concerns.

- **Online algorithms:** These algorithms are designed to make decisions in real time with incomplete information. In many real-world applications, data arrives continuously, and decisions must be made on the fly without the luxury of knowing the entire input in advance. Online algorithms excel in such dynamic environments, making them suitable for use cases such as stock trading, network routing, and load balancing, where immediate responses are necessary.
- **Adaptive and dynamic algorithms:** To function effectively in changing environments, some algorithms are designed to be adaptive and capable of adjusting to new input data or shifting conditions. Adaptive algorithms can modify their behavior based on the observed data, learning and improving over time. Dynamic algorithms, on the other hand, can handle changes in the problem structure itself, such as updates in a graph or alterations in a scheduling problem. These capabilities are crucial for applications such as autonomous systems, which must continuously adapt to their surroundings to function effectively.
- **Machine learning algorithms:** Machine learning algorithms inherently possess a level of context awareness due to their ability to learn from data. These algorithms can adapt to various environments, whether they are dealing with static datasets or processing real-time data streams. Techniques such as reinforcement learning allow algorithms to learn optimal behaviors through interaction with their environment, making them suitable for complex decision-making tasks in dynamic settings. Furthermore, machine learning models can be deployed in environments such as edge devices, where they process and analyze data locally, reducing the need for constant cloud connectivity.

These emerging trends illustrate the increasing demand for algorithms that are not only efficient but also context-aware. In today's rapidly evolving technological landscape, algorithms must be capable of adapting to new data, operating within resource constraints, and making real-time decisions. Whether drawing inspiration from natural processes, optimizing for limited memory, or learning from data, context-aware algorithms are essential for tackling the complexities of modern computational environments.

While ultra-fast computers, intelligent algorithms, and advanced devices may seem like pathways to a utopian future, they alone cannot deliver such an outcome. The painful lessons of the past century – marked by endless wars, environmental destruction, and the erosion of personal privacy—have shown us that unchecked promotion of technological progress can cause more harm than good. This is why it is crucial to remain morally vigilant when dealing with technology, ensuring that innovation is guided by ethical considerations and serves the greater good.

Moral responsibility

Algorithms have become deeply embedded in almost every aspect of our lives, influencing decisions from healthcare and finance to the media we consume and the products we buy. While they have greatly enhanced the quality of life for many, providing efficiency, convenience, and new capabilities, the rise of algorithms also brings significant ethical challenges and responsibilities.

One of the pressing concerns is the risk of misuse and the unintentional negative side effects that algorithms can produce. Large corporations design algorithms to maximize user engagement and drive profit, often by steering users toward advertisements or content that increases shareholder value. While these practices are legal and fall within the boundaries of business models, they raise ethical questions. For example, algorithms that prioritize attention-grabbing content can inadvertently promote sensationalism, misinformation, or polarizing content, which can have a harmful impact on society. A real-world example is the role of social media algorithms in amplifying divisive content during political elections. In 2016, for instance, Facebook's algorithm was criticized for promoting highly emotional and polarizing political content, which contributed to the spread of misinformation and deepened societal divisions during the US presidential election.

Beyond these legitimate yet ethically questionable practices, there are outright unethical and illegal uses of algorithms. These include spreading misinformation, targeting vulnerable individuals with harmful or exploitative content, and reinforcing biases, such as racial or gender discrimination. Algorithms can perpetuate stereotypes and systemic inequalities if they are not carefully designed and monitored. For instance, biased training data can lead to unfair outcomes in areas such as hiring processes, loan approvals, and law enforcement.

Another critical issue is the environmental impact of algorithms. The computational power required to train giant language models support large-scale data centers, and mine cryptocurrencies is immense. These activities consume vast amounts of electricity, contributing to a significant carbon footprint. The environmental cost of algorithms extends beyond energy consumption to encompass the entire life cycle of technology, including the production and disposal of electronic devices, which involve the extraction of raw materials and the generation of electronic waste.

Moreover, the societal implications of algorithm-driven systems are profound. The global shift toward automation and AI has led to changes in employment patterns, with certain jobs becoming obsolete while others require increasingly specialized skills. This transition can result in economic displacement and inequality, particularly in regions where there is a lack of infrastructure to support retraining and adaptation to new job markets. Additionally, the brain drain phenomenon, where talent from developing countries migrates to more developed nations to work on advanced algorithms, can exacerbate global inequalities, leaving some regions without the expertise needed to drive local innovation.

Given these complexities, the moral responsibility associated with algorithm design and deployment is immense. It calls for a thoughtful approach that goes beyond technical considerations to include ethical, social, and environmental factors. Developers, businesses, and policymakers must work together to establish guidelines and frameworks that promote the responsible use of algorithms. The guidelines and frameworks can be built around the following considerations:

- **Ethical design and implementation:** Ensuring that algorithms are designed with fairness, transparency, and accountability in mind. This involves using diverse datasets, implementing bias detection and mitigation strategies, and providing clear explanations of how algorithms make decisions.

- **Regulation and oversight:** Governments and regulatory bodies need to establish and enforce standards that prevent the misuse of algorithms, protect user privacy, and ensure that AI systems operate within ethical boundaries.
- **Environmental considerations:** Developing and adopting energy-efficient algorithms and practices that minimize environmental impact. This includes optimizing data centers, investing in renewable energy sources, and considering the full environmental life cycle of technological infrastructure.
- **Public awareness and education:** Increasing public understanding of how algorithms influence everyday life and empowering individuals to make informed choices. This also involves educating future generations of developers about the ethical implications of their work.
- **Risk management of algorithms:** Algorithms can introduce various types of risks, one of the most prominent being algorithmic bias, where certain groups of users may be favored or disadvantaged based on factors such as gender, race, or social class. These biases often arise from biased data used to train machine learning models, leading to unfair outcomes. For instance, algorithms used in hiring, loan approval, or criminal justice systems have been shown to perpetuate or even amplify societal biases when not properly audited or monitored. The responsibility lies in ensuring that algorithms are trained on diverse, representative datasets and are regularly tested for fairness and accuracy.

Another category of risk is associated with the misuse or abuse of code, particularly in embedded systems. Embedded systems are integrated into countless devices – from household appliances to medical equipment and industrial control systems – making them vulnerable to malicious exploitation. It is often said that any code in any device could become a liability if breached. This is particularly concerning in systems where safety and security are paramount, such as autonomous vehicles, medical devices, and defense technologies. If the embedded code in these systems is compromised, the consequences could be catastrophic, ranging from loss of human life to large-scale financial and infrastructural damage.

While the preceding considerations involve various stakeholders, as engineers and computer scientists, we bear a unique responsibility to uphold moral principles, particularly those related to human rights and the public good. Beyond relying on regulatory bodies, we must cultivate our own ethical compass, guided by a deep sense of moral consciousness. This requires actively safeguarding ethical standards in our work, ensuring that our contributions benefit society in a fair and responsible manner.

Moral consciousness in software practice

Beyond the recognized responsibilities in moral, environmental, and societal domains, software practitioners must cultivate a heightened level of ethical awareness, or what we refer to as **moral consciousness**. This goes beyond simply adhering to ethical guidelines or legal requirements. It involves a deep, proactive consideration of the broader impact and potential indirect uses of their innovations, even when those innovations are created with the best intentions and for ethical applications.

A morally conscious practitioner recognizes that algorithms, once developed, can take on lives of their own, being used in ways that the original creator may not have anticipated or intended. For instance, an algorithm designed to enhance user engagement on a social media platform could inadvertently contribute to the spread of misinformation or addictive behaviors. Similarly, a facial recognition algorithm developed for security purposes might be repurposed for surveillance in ways that infringe on privacy or civil liberties.

To mitigate such risks, practitioners can adopt several strategies:

- **Intellectual property rights and licensing:** One way to control the use of an algorithm is to protect it under intellectual property rights. By securing patents or employing specific licensing agreements, developers can place restrictions on how their work is used, ensuring that it aligns with ethical standards. This approach, however, is not always foolproof, as enforcing intellectual property rights can be challenging, especially in a global context where different countries have varying legal frameworks.
- **Ethical safeguards and impact assessments:** Practitioners should conduct thorough ethical impact assessments during the development process. This involves analyzing not only the immediate application of the algorithm but also its potential secondary effects and unintended consequences. By considering the various ways an algorithm could be misused, developers can implement safeguards, such as usage constraints or built-in monitoring systems, to prevent unethical applications.
- **Dual-use consideration:** Before releasing an algorithm, it is crucial to consider its dual-use potential – how it might be used legally and beneficially, but also how it could be co-opted for illegal or harmful purposes. If an algorithm has a high risk of being misused, practitioners need to think carefully about whether it should be made publicly available or whether additional safeguards are necessary to limit access.
- **Transparency and collaboration:** Engaging with stakeholders, including ethicists, legal experts, and the broader community, can provide valuable insights into the potential impacts of an algorithm. Transparency in development practices, coupled with an open dialog about the ethical considerations involved, can lead to more informed decisions and help identify potential risks before they become problematic.
- **Continual ethical education:** Staying informed about the evolving ethical landscape and being open to revisiting the moral implications of their work is essential for practitioners. This involves continual education on emerging ethical issues, as well as reflecting on how societal norms and values may change over time.

Moral consciousness is about maintaining vigilance and responsibility beyond the technical creation and deployment of algorithms. It calls for a mindset that constantly questions not just the immediate benefits of an algorithm, but also its broader societal implications, indirect effects, and potential for misuse. By adopting this conscientious approach, software practitioners can contribute to a technological landscape that prioritizes ethical considerations and works toward the greater good.

In summary, while algorithms have the potential to bring about significant positive change, they also carry a moral responsibility that cannot be overlooked. Addressing the ethical, social, and environmental challenges associated with algorithm use is crucial to ensuring that their impact on society is positive and sustainable.

Final words

As we conclude this book, our aim has been to provide a comprehensive overview of the most important topics in algorithm design and analysis that computer science students and software engineers will encounter both in their academic journey and in practical applications. We've endeavored to strike a balance between the theoretical foundations and the hands-on problem-solving skills essential for building efficient and effective algorithms.

Throughout this book, we have aimed for accuracy and clarity in presenting complex concepts. However, given the vastness and depth of the field, errors and omissions are inevitable. We encourage readers to reach out with any issues, errors, or ambiguities they encounter so they can be addressed and improved in future editions. Your feedback is invaluable in refining the content and ensuring it remains a reliable resource for learners and practitioners alike.

One topic we deliberately chose not to discuss in this final chapter is the role of generative AI in software engineering and algorithm design. The reason for this omission is simple: the impact and direction of generative AI on the field of algorithms are still unfolding. We are in the midst of a technological evolution, and it is difficult to predict with certainty how generative AI will reshape the landscape of algorithm design.

However, one thing is certain: **Large language models (LLMs)** and generative AI are rapidly becoming the driving force behind the demand for more efficient algorithms. Their complexity requires sophisticated analysis and evaluation. Among all the emerging use cases, LLMs and generative AI stand out as domains where scalability, context awareness, and moral responsibility intersect.

The pressure for scalable AI algorithms is already evident. The size of these models and the number of users engaging with them is increasing at an astronomical rate. For instance, OpenAI's ChatGPT broke records in technological mass adoption, reaching 200 million weekly users in a short time. This unprecedented scale demands highly efficient, scalable algorithms to support such massive infrastructures.

Additionally, conversational AI and LLMs are rapidly transitioning from groundbreaking innovations to standard features in everyday products. This shift means AI systems must perform effectively in highly diverse contexts, often operating under uncertainty and volatility, all while involving human interaction. Designing algorithms that can adapt to these challenging environments is critical for their long-term success.

Perhaps the most pressing concern with generative AI and LLMs is their societal and environmental impact. The carbon footprint of training massive models such as GPT-4 is already significant and will only grow unless we develop more efficient algorithms and adopt stricter industry regulations. Moreover, the moral risks are equally troubling. These systems can propagate misinformation on a large scale, widen socio-economic disparities, and potentially undermine democratic processes by manipulating public opinion.

Looking ahead to the role of AI and LLMs in algorithm design, I firmly believe that while the field will undoubtedly be transformed by generative AI, the core principles of problem-solving will endure. We may gain access to new tools and methodologies, but the need for human insight in algorithm design and analysis will persist. Generative AI can automate certain aspects of the development process, speeding up cycles and expanding algorithmic capabilities. However, the creative and critical thinking that humans bring to the process will remain indispensable, particularly in areas that demand ethical judgment, context sensitivity, and thoughtful decision-making.

In closing, I hope this book serves as a valuable guide and companion for those venturing into the world of algorithms. The journey of learning and innovation in this field is ongoing, and I encourage readers to continue exploring, questioning, and contributing to the ever-evolving landscape of computer science and software engineering.

References and further reading

- *What is Moore's Law?* By Max Roser, Hannah Ritchie, and Edouard Mathieu (2023). Published online at [OurWorldinData.org](https://www.ourworldindata.org).
- *Quantum Computation and Quantum Information*. By M. A. Nielsen and I. L. Chuang. Cambridge University Press. 2010.
- *Approximation Algorithms*. V. V. Vazirani. Springer. 2001.
- *Parameterized Algorithms*. By M. Cygan, F. V. Fomin, Ł. Kowalik, D. Lokshtanov, D. Marx, M. Pilipczuk and M., A. Saurabh. Springer. 2016.
- *Sublinear-Time Algorithms*. In Handbook of Data Structures and Applications. By A. Czumaj and C. Sohler. Chapman & Hall/CRC. 2004.
- *Data Streams: Algorithms and Applications*. By S. Muthukrishnan. Now Foundations and Trends. 2005.
- *Introduction to Parallel Computing*. By A. Grama, A. Gupta, G. Karypis, and V. Kumar. Pearson Education. 2003.
- *Distributed Systems: Principles and Paradigms*. By A. S. Tanenbaum and M. V. Steen. Pearson Education. 2007.
- *Smoothed Analysis of Algorithms: Why The Simplex Algorithm Usually Takes Polynomial Time*. By D. A. Spielman and S.-H. Teng. Journal of the ACM, 51(3), 385-463. 2004.

-
- *Mining of Massive Datasets*. By J. Leskovec, A. Rajaraman, and J. D. Ullman. Cambridge University Press. 2014.
 - *Graph Neural Networks: A Review of Methods and Applications*. By J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, and M. Sun. *AI Open*, 1, 57-81. 2020.
 - *Introduction to Evolutionary Computing*. By A. E. Eiben and J. E. Smith. Springer. 2003.
 - *Ant Colony Optimization*. By M. Dorigo and T. Stützle. MIT Press. 2004.
 - *Online Computation and Competitive Analysis*. By A. Borodin and R. El-Yaniv. Cambridge University Press. 2005.
 - *The Ethics of Algorithms: Mapping the Debate*. By B. D. Mittelstadt, P. Allo, M. Taddeo, S. Wachter, and L. Floridi. *Big Data & Society*, 3(2). 2016.
 - *Growth in Data Center Electricity Use 2005 to 2010*. By J. G. Koomey. Analytics Press. 2011.

Index

A

Abstract Data Type (ADT) 235

- deque 236
- dictionary 236
- graph 236
- list 235
- queue 235
- set 236
- stack 235
- tree 236

acyclic graph 273

adaptability 102

adaptive and dynamic algorithms 317

adaptive sorting algorithm 103

adjacency list 275, 276

adjacency matrix 274, 275

AI search algorithms 134

Akra-Bazzi method 73, 85, 89

- using 90, 91

algorithm analysis 6, 7

- dual dimensions 8-10
- problem-solving methods 7, 8

algorithmic search 134

- binary search 134
- hashing 134

- linear search 134

- tree traversal 134

algorithms 4, 311

- context awareness 316
- for large-scale data 315
- key properties 5
- moral responsibility 317-319
- problem-solving 4-6
- scalability 311-314

approximation algorithms 314

array 244, 247

- access 246
- characteristics 244, 245
- deletion 246
- dynamic arrays 246
- edit 246
- insertion 245
- multidimensional arrays 246
- operations 245
- search 246
- update 246

array representation 287, 288

Artificial Intelligence (AI) 6

A* search algorithm 134

asymptotic analysis 9

asymptotic bounds 34

asymptotic lower bound 34-38

asymptotic notations 32

 simplification rules 33, 34

asymptotic tight bound 34, 39, 40

asymptotic upper bound 34-37

AVL tree 285

B

balanced binary tree 285

BFS graph traversal 280

 advantages 281

 applications 282, 283

 bidirectional BFS 282

 example 281

 multi-source BFS 282

binary recursion 50

binary search 24, 62, 139, 174

 implementation 139, 140

binary search trees (BSTs) 136, 290

 balanced BST 293

 deletion operation 297-299

 example 290

 in-order traversal 291, 292

 insertion operation 295-297

 post-order traversal 292, 293

 pre-order traversal 292

 properties 290

 search operation 293-295

 unbalanced tree 293

binary tree 284

 left child 284

 right child 284

biologically inspired algorithms 316

birthday paradox 188-190

Bounded Knapsack Problem (BKP) 215

breadth-first search (BFS) 134, 176, 277

B-tree 285

bubble sort 106

 complexity analysis 109

 example 107, 108

 proof of correctness 109

bucket sort 127

 complexity analysis 129

 example 128

 proof of correctness 129

C

case studies

 closest parking spot, finding 197-199

 optimal selection, in online

 dating app 195-197

chaining 159

 example 160

 for collision handling 160, 161

circular linked list 253

collision handling 159

 chaining 159-162

 cuckoo hashing 167-169

 open addressing 162-167

comparison-based sorting algorithms

 bubble sort 98

 heap sort 98

 insertion sort 98

 merge sort 98

 quick sort 98

complete binary tree 285

complex traversal patterns 270

composite data structures 231

compound commands 14

computing technology evolution, stages

 early computers 309

 expert systems 310

 first wave, of AI 310

Internet 310
 mainframe computers 310
 mobile computing 310
 personal computers (PCs) 310
 second wave, of AI 310
 third and current wave, of AI 310
 World Wide Web 310
constant 23
constant-time search algorithms 136
constant time search, with hashing 150, 151
 collision handling 159
 hash function, using 151
constraint-satisfaction problems 174
Content-Addressable Memory (CAM) 237
context-aware algorithms, emerging trends
 adaptive and dynamic algorithms 317
 biologically inspired algorithms 316
 machine learning algorithms 317
 memory-efficient algorithms 316
 online algorithms 317
counting sort 121
 complexity analysis 124, 125
 example 121-123
 proof of correctness 123
cuckoo hashing 167
 for handling collisions 167-169
current AI revolution
 major developments 312, 313
cyclic graph 273

D

data structures
 linear, versus non-linear data
 structures 232, 233
 physical, versus logical data
 structures 230, 231
 primitive, versus composite
 data structures 231
 sequential, versus random access 234
 static, versus dynamic memory
 allocation 233
 taxonomy 229, 230
depth-first search (DFS) 134, 176, 277
 post-order DFS 279
 pre-order DFS 279
 space complexity 279
 time complexity 279
DFS graph traversal 277-279
dictionaries 237
 deletion 240
 implementing 238, 239
 insertion 239
 properties 237
 search 239
 update 240
directed acyclic graphs (DAGs) 279
directed graphs (digraphs) 272
direct proof methods 9
direct recursion 49
 binary recursion 50
 head recursion 50
 linear recursion 50
 multiple recursion 51
 nested recursion 52
 tail recursion 49
 tree recursion 50
 use cases 49
divide-and-conquer recurrence
 functions 57, 60
 binary search 62
 merge sort 61
 properties 61
 versus subtractive recurrence
 functions 63, 64

- divide and conquer strategy** 46
- divide, conquer, and combine framework**
 - in merge sort algorithm 54
- division-remainder (modulo) method** 151
 - example 151, 152
- DJB2 hash function** 158
 - implementing 158, 159
- double-ended queues (deque)** 243, 265
 - operations 265, 266
 - properties 265
- double hashing** 165
 - advantages 167
 - example 165-167
- doubling search** 143
- doubly linked list** 253
- driving function** 63
- dynamic data structures** 234
- dynamic programming** 201
 - exploring 208
 - greedy algorithms 220
 - knapsack problem, solving with 215-218
 - limitations 219, 220
 - optimal substructure 202-207
 - overlapping subproblems 202, 207
 - top-down, versus bottom-up
 - approaches 211-214
 - versus divide-and-conquer
 - strategies 202, 203
 - with memoization 210, 211
 - without memoization 209

E

- edge list** 276
- exponential search** 143
 - implementation 144
- exponential time algorithms** 28

F

- factorial calculation** 49
- factorial function** 59
- factorial time algorithms** 28, 29
- Fibonacci sequence** 59, 60
- Fibonacci series** 49
- First-In-First-Out (FIFO) principle** 235
- fixed-parameter tractability (FPT)** 314
- Floyd-Warshall shortest path** 27
- folding hash function** 154
 - example 154
 - limitations 155
 - properties 155
- full binary tree** 285

G

- galloping search** 143
- general tree** 284
- graphical processing units (GPUs)** 6, 312
- graph neural networks (GNNs)** 315
- graphs** 269-271
 - acyclic graph 273
 - cyclic graph 273
 - directed graphs (digraphs) 272
 - hypergraphs 273
 - signed graphs 273
 - undirected graphs 271
 - unweighted graphs 273
 - weighted graphs 272
- graphs representation** 274
 - adjacency list 275, 276
 - adjacency matrix 274, 275
 - edge list 276

greedy algorithms 220
 characteristics 220
 heuristic role 222, 223
 Traveling Salesman Problem (TSP) 221, 222
greedy search 134
Grover's algorithm 314

H

halting problem 5, 41
hash function 149
 applying 150
 division-remainder (modulo)
 method 151, 152
 DJB2 hash function 158, 159
 folding hash method 154, 155
 mid-square method 153, 154
 multiplication hash method 152
 polynomial hashing 157, 158
 properties 149
 universal hashing 155, 156
hashing 136, 148
 address 149
 index 148
 key 148
 used, for constant time search 150, 151
hash value 149
head recursion 50
heap operations 300
 deletion 301, 302
 heap building 302, 303
 insertion 300, 301
heaps 269, 299
 max-heap 299
 min-heap 299, 300
heap sort 27, 303
 performing 303, 304

heuristic 222
 role in greedy algorithms 222, 223
hierarchical relationships 270
hiring secretary problem 191, 192
 example 193-195
 immediate decision 191
 objective 191
 observation phase 191
 selection phase 191
 sequential interviews 191
hyperedges 273
hypergraph 273

I

indirect proof 9, 12
indirect recursion 52
 use cases 52
in-place sorting algorithms 104
insertion sort 111
 complexity analysis 113
 example 112
 proof of correctness 113
internet of things (IoT) devices 316
interpolation search 25, 141, 174
 implementation 141-143
inversions 103
 maximum inversions 103
 zero inversions 103
iteration block 15, 16
iterative alternatives 55
iterative sorting algorithms 106
 bubble sort 106
 insertion sort 111, 112
 selection sort 109, 110

J

jump search 24, 145
implementation 145-147

K

knapsack problem 215
solving, with dynamic
programming 215-218

L

Large language models (LLMs) 321, 322
Last-In-First-Out (LIFO) principle 235
linear data structures 232, 243
advantages 232
limitations 232
linear growth 25, 26
linear or sequential search 137, 138
applications 138
linear probing 162-164
example 162-164
linear recursion 50
linear-time search algorithms 136
linked list 244, 247
access 252
characteristics 247
circular linked list 253
deletion 250
doubly linked list 253
editing 251
insertion 248-250
searching 251, 252
linked representation 286, 287
list 243
array 244-247
linked list 247

load factor

limitations 162
properties 162

logical data structure

 230

log-linear time algorithms

 27

heap sort 27
merge sort 27
quick sort 27

Longest Common Subsequence (LCS)

 204

loop condition

 18

loop invariant

 16-19

example 17

loops

 23, 46

M

machine learning algorithms

 317

master theorem

 65, 78

alternative approaches 85
balanced growth or balanced
recursion trees 80
critical exponent (w) 79
dominance, of non-recursive work 81, 82
dominance, of recursive calls 79
limitations 84
limitations, examples 85-88
modified master theorem, for handling
subtracting recurrence function 82, 83
watershed function 79

Matcher app

 195-197

mathematical induction

 12

base case (initial step) 13
example 13, 14
induction step 13

matrix multiplication

 27

memoization

 55, 208

memory-efficient algorithms

 316

merge sort 27, 61, 114, 174
 complexity analysis 116, 117
 example 114-116
 proof of correctness 116
mid-square hash function 153
 example 153
 limitations 154
 properties 153
Monty Hall problem 187, 188
Moore's law 312
moral consciousness 319, 320
multiple recursion 51
multiplication hash method 152
 example 152

N

nested recursion 52
node 244
non-comparison-based sorting algorithms 98, 121
 bucket sort 127, 128
 counting sort 121
 properties 98
 Radix sort 125
non-computable problem 5
non-in-place sorting algorithms 104, 105
non-linear algorithms 29, 30
 $O(n \log n)$ algorithms 27
non-linear data structures 233, 269
 child node 271
 components 270
 edges 270
 features 270
 leaf node 271
 nodes 270
 parent node 271

root node 271
 subtree 271
non-linear growth 26
non-recursive component 63
non-recursive sorting algorithms 101
 characteristics 101
 examples 101, 102
NP-complete problems 42
NP-hard problems 41, 42
NP problems 42

O

online algorithms 317
open addressing 162
optimization 174

P

parallel and distributed algorithms 315
parameterized complexity 314
parent array representation 288-290
physical data structure 230
polynomial hashing 157
 implementing 157, 158
polynomial time algorithms 27
 Floyd-Warshall shortest path 27
 matrix multiplication 27
primitive data structures 231
probabilistic analysis 186

Q

quadratic probing 164
 example 164, 165
quantum algorithms 314
quasi-linear time algorithms 27

queue 263

- characteristics 263
- dequeue 264
- enqueue 264
- peek 264
- search 264

quick sort 27, 117

- complexity analysis 119, 120
- example 118
- partitioning 118
- pivot selection 117
- proof of correctness 119
- recursion 118

R

Rabin-Karp rolling hash 157**Radix sort 125**

- advantages 125
- complexity analysis 127
- example 125, 126
- proof of correctness 126, 127

random access data structures 234**randomized algorithms 185**

- analysis 186, 187

rate of growth 21, 22, 30-32**recurrence functions 45, 56, 57, 63**

- divide-and-conquer recurrence 57, 60-62
- subtractive recurrence 57-60

recurrence relations 56**recursion 23, 56, 99**

- advantages 54, 55
- basics 46-48
- challenges 55
- direct recursion 49
- indirect recursion 52
- types 48

recursion tree method 75

- as visualization technique 76-78

recursive algorithms 46

- base case 47
- recursive calls 47
- recursive case 48

recursive component 63**recursive problem-solving 53**

- combine step 53
- conquer step 53
- divide step 53

recursive sorting algorithms 99, 114

- examples 100
- merge sort 114, 116
- properties 99
- quick sort 117, 118

recursive tree method 85**red-black tree 286**

S

scalable algorithms

- emerging trends 315

scalable algorithms, emerging trends

- algorithms, for large-scale data 315
- approximation algorithms 314
- graph neural networks (GNNs) 315
- parallel and distributed algorithms 315
- parameterized complexity 314
- quantum algorithms 314
- smoothed analysis 315
- streaming algorithms 315
- sub-linear algorithms 315

scale-free commands 14**search algorithms 134**

- constant-time search algorithms 136
- linear-time search algorithms 136
- logarithmic search algorithms 136

- properties 134-136
 - scenarios 135
 - sub-linear-time search algorithms 136
 - search in AI 134**
 - selection block 14-16**
 - selection sort 109**
 - complexity analysis 111
 - example 110, 111
 - proof of correctness 111
 - sequential access data structures 234**
 - Shor's algorithm 314**
 - signed graphs 273**
 - simple commands 14, 15**
 - skip lists 243, 254**
 - characteristics 254, 255
 - insertion 255-259
 - searching 259-261
 - smoothed analysis 315**
 - sorting algorithms**
 - adaptive sorting algorithm 103
 - comparison-based sorting 97, 98
 - in-place sorting algorithms 104
 - inversion 103
 - memory usage 104
 - non-comparison-based sorting 98, 99
 - non-in-place sorting algorithms 104, 105
 - non-recursive sorting algorithms 101
 - recursion 99
 - recursive sorting algorithms 99
 - stability 105, 106
 - sorting and searching**
 - design, of efficient algorithms 177-180
 - symbiotic link 174-177
 - space complexity 22**
 - stability 105**
 - multiple key sorts 106
 - preserving input order 106
 - stack 261**
 - characteristics 261-263
 - Standard Template Library (STL) 236**
 - static data structures 233**
 - streaming algorithms 315**
 - sub-linear algorithms 315**
 - sub-linear growth 23**
 - sub-linear search algorithms 139**
 - binary search 139, 140
 - exponential search 143, 144
 - interpolation search 141-143
 - jump search 145-147
 - sub-linear time algorithms 23**
 - binary search 24
 - interpolation search 25
 - jump search 24
 - subproblem 53**
 - substitution method 68, 85**
 - guessing and induction approach 71-73
 - iteration approach 68-71
 - variable change approach 73-75
 - subtractive recurrence functions 57, 58**
 - factorial function 59
 - Fibonacci sequence 59, 60
 - properties 58
 - versus divide-and-conquer
 - recurrence functions 63, 64
 - symbiosis**
 - commensalism 175
 - mutualism 174
 - parasitism 175
- ## T
- tail recursion 49, 55**
 - task scheduling 264**
 - Tensor Processing Units (TPUs) 312**
 - time complexity 22**

Tower of Hanoi problem 28

Traveling Salesman Problem (TSP) 28, 221

traversal 277

BFS graph traversal 280-283

DFS graph traversal 277-280

tree recursion 50

tree representation 286

array representation 287, 288

linked representation 286, 287

parent array representation 288-290

trees 269, 284

AVL tree 285

balanced binary tree 285

binary search trees (BSTs) 284

binary tree 284

B-tree 285

complete binary tree 285

full binary tree 285

general tree 284

red-black tree 284-286

tree traversals 49

U

Unbounded Knapsack Problem (UKP) 215

undirected graphs 271

universal hashing 155

example 156

unrolling the recurrence function 68

unweighted graphs 273

V

variable access time 270

vertices 271

W

weighted graphs 272



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

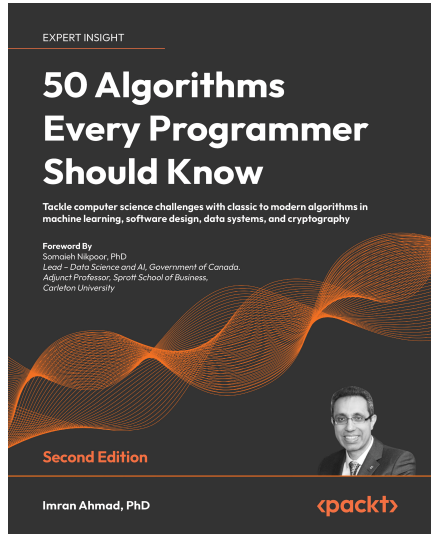
- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at packtpub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:

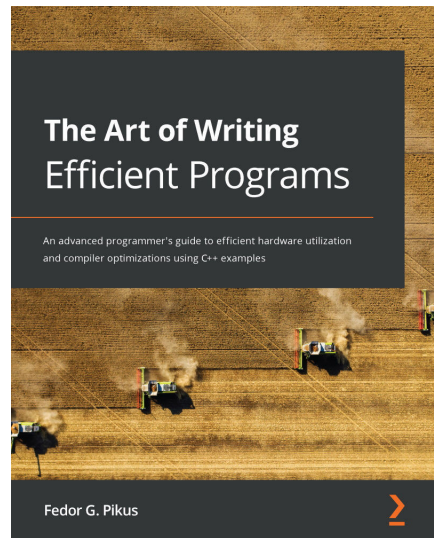


50 Algorithms Every Programmer Should Know

Imran Ahmad

ISBN: 978-1-80324-776-2

- Design algorithms for solving complex problems
- Become familiar with neural networks and deep learning techniques
- Explore existing data structures and algorithms found in Python libraries
- Implement graph algorithms for fraud detection using network analysis
- Delve into state-of-the-art algorithms for proficient Natural Language Processing illustrated with real-world examples
- Create a recommendation engine that suggests relevant movies to subscribers
- Grasp the concepts of sequential machine learning models and their foundational role in the development of cutting-edge LLMs



The Art of Writing Efficient Programs

Fedor G. Pikus

ISBN: 978-1-80020-811-7

- Discover how to use the hardware computing resources in your programs effectively
- Understand the relationship between memory order and memory barriers
- Familiarize yourself with the performance implications of different data structures and organizations
- Assess the performance impact of concurrent memory accessed and how to minimize it
- Discover when to use and when not to use lock-free programming techniques
- Explore different ways to improve the effectiveness of compiler optimizations
- Design APIs for concurrent data structures and high-performance data structures to avoid inefficiencies

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share Your Thoughts

Now you've finished *Efficient Algorithm Design*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Download a free PDF copy of this book

Thanks for purchasing this book!

Do you like to read on the go but are unable to carry your print books everywhere?

Is your eBook purchase not compatible with the device of your choice?

Don't worry, now with every Packt book you get a DRM-free PDF version of that book at no cost.

Read anywhere, any place, on any device. Search, copy, and paste code from your favorite technical books directly into your application.

The perks don't stop there, you can get exclusive access to discounts, newsletters, and great free content in your inbox daily

Follow these simple steps to get the benefits:

1. Scan the QR code or visit the link below



<https://packt.link/free-ebook/9781835886823>

2. Submit your proof of purchase
3. That's it! We'll send your free PDF and other benefits to your email directly